

从 C++ 到 AI 计算：第三册源码卷

LCQI 本地 CPU 推理引擎完整源码与阅读路径

CPU Performance Study

本卷把第三册贯穿项目 LCQI 的构建入口、模型资产、公共头文件、核心实现、工具脚本、benchmark、trace、报告生成器和 CTest 验收源码独立成册。原理卷负责解释机制，实践卷负责安排阶段任务；源码卷负责给出一份可完整审阅、可构建、可回归的代码证据。

前言

第三册原理卷讲 AI 推理系统的机器事实，实践卷把任务拆成阶段作业。本源码卷只处理一件事：把贯穿项目 [books/cpu-volume-3/labs/linux_cpu_inference](https://books.cpu-volume-3/labs/linux_cpu_inference) 以及相关工具脚本单独成册，让读者能从构建入口一路读到测试和报告，而不是在原理正文或实践任务后面翻一段过长的源码附录。

核心理想

源码卷不是“代码仓库的打印版”。每章先说明该组文件承担的工程责任，再给完整源码。阅读时要把函数签名、错误边界、测试断言和报告字段连起来看：一个推理引擎是否可信，不取决于某个单独函数看起来像模型，而取决于资产能校验、shape 能拒绝错误、kernel 能对照、trace 能定位、benchmark 能说明口径、CTest 能长期守住回归。

本卷收录的主线代码是 LCQI。它从 tiny MLP 教学闭环开始，逐步覆盖 tokenizer、safetensors manifest gate、int8 kernel、packed/AVX2 路径、reference decoder、KV cache trace、GPT-2 reference path、7B compute ledger、serving SLO probe、真实 small 开源模型 smoke、自研 GPT-2 smoke 和 benchmark compare。它不是生产级推理引擎，但每个能力都要求有源码、命令、测试或报告证据。

阅读顺序建议如下。第一章先看构建入口和模型资产，明确哪些目标一定构建、哪些对标依赖外部库。第二章读公共头文件，先把数据结构和函数合同固定下来。第三章读 tiny MLP、int8 kernel 和基础 CLI，建立最小推理闭环。第四章读 reference decoder、safetensors、tokenizer 和 GPT-2 代码，理解真实模型路径为什么比模板 demo 多出许多边界检查。第五章读工具脚本、benchmark、trace、ledger、serving probe 和测试，把代码和可复查证据连起来。

源码卷的 LaTeX 文件通过 `lstinputlisting` 直接读取第三册工程源码。也就是说，源码不在书中复制第二份；代码更新后重新构建本卷，PDF 和 EPUB 会读取当前仓库里的真实文件。

缩写与读码约定

LCQI Local CPU Quantized Inference，本书第三册的本地 CPU 量化推理贯穿项目。

CLI Command Line Interface，命令行入口。源码里通常只负责参数解析和结果打印，不承载核心推理逻辑。

MLP Multi-Layer Perceptron，多层感知机。tiny MLP 用来建立可手算、可测试的最小推理闭环。

KV Cache Key/Value Cache，decoder-only 模型跨 token 复用 attention key/value 的状态缓存。

BPE Byte Pair Encoding，GPT-2 使用的 byte-level 子词编码方法。

GGUF GPT-Generated Unified Format，llama.cpp 常用的模型文件格式。

TTFT Time To First Token，服务请求从进入系统到输出第一个 token 的时间。

TPOT Time Per Output Token，生成阶段每个输出 token 的平均或分位耗时。

SLO Service Level Objective，服务等级目标。本书用它描述 TTFT、TPOT、拒绝率和取消回收等运行时约束。

PMU Performance Monitoring Unit，硬件性能计数器单元。没有 PMU 权限时，性能报告不能伪装成硬件级结论。

AVX2 Advanced Vector Extensions 2，x86-64 SIMD 指令集扩展。源码存在不等于当前机器可用，报告要看运行时可用性。

读码时先看头文件中的数据结构和入口，再看实现文件中的检查、循环和错误分支，最后回到测试文件确认哪些行为已经被锁成回归合同。遇到 benchmark 或 smoke 脚本时，要区分“同模型同精度对比”和“成熟引擎参考口径”，不能把不同模型、不同量化格式的速度直接写成质量排名。

目录

前言	ii
缩写与读码约定	iii
第 1 章 构建入口、模型资产与项目边界	1
1.1 源码卷覆盖范围	1
1.2 根构建入口	1
1.3 LCQI 目标定义	4
1.4 项目说明与模型资产	8
第 2 章 源码阅读路线：从入口到调用链	18
2.1 为什么要先讲调用链	18
2.2 一、全工程入口：哪些文件决定能不能复现	18
2.3 二、Tiny MLP 调用链：最短闭环怎样跑通	19
2.4 三、Int8 Kernel 调用链：reference、packed、AVX2 和 benchmark 怎样对照	19
2.5 四、Tokenizer、Safetensors 与 Reference Decoder：真实模型前的三道门	20
2.6 五、GPT-2 调用链：从 HuggingFace 资产到生成 token	20
2.7 六、报告、服务探针、外部对标和测试	21
第 3 章 公共合同：模型、Tokenizer、Kernel 与 Decoder	23
3.1 为什么先读头文件	23
3.2 Tiny MLP 与推理入口	23
3.3 Int8 Kernel、Tokenizer 与 Safetensors	24
3.4 Reference Decoder 与 GPT-2 合同	28
第 4 章 Tiny MLP、Int8 Kernel 与基础 CLI	37
4.1 最短闭环：从模型文件到 logits	37
4.2 模型加载与基础推理	37
4.3 Int8 Kernel 基础实现	44
第 5 章 Reference Decoder、GPT-2 与模型导入	54
5.1 从 tiny MLP 走向 decoder-only 模型	54
5.2 Tiny Reference Decoder	54
5.3 Tokenizer 与 Safetensors 实现	71
5.4 GPT-2 Reference Path	87
第 6 章 工具、报告、Benchmark 与回归测试	150
6.1 工具不是附属品	150
6.2 Benchmark、Trace、Ledger 与 Serving Probe	150
6.3 外部模型 Smoke 与对比脚本	165
6.4 完整测试	189
读码检查清单	234

第 1 章

构建入口、模型资产与项目边界

1.1 源码卷覆盖范围

LCQI 的源码边界由三个层次组成。第一层是仓库构建入口，它决定 CMake 怎样把第三册的实验代码纳入构建和测试。第二层是 `labs/linux_cpu_inference`，它包含真正的库、CLI、benchmark、trace、ledger、serving probe 和测试目标。第三层是工具脚本，它们下载或复用外部模型资产，生成 smoke 和 benchmark 报告。

源码阅读任务

先读本章的 CMake 文件，再读 README。读者要能回答三个问题：一是 `lcqi` 静态库包含哪些实现文件；二是哪些可执行文件只依赖自研代码，哪些依赖可选外部库；三是真实 GPT-2 和 SmoLLM2 smoke 为什么不进入默认 CTest。

1.2 根构建入口

第三册根目录的 CMake 文件只做项目级选择：是否构建第三册 labs，以及如何把 `labs/linux_cpu_inference` 接入。它应该保持薄层，避免把具体目标定义分散到多个地方。

Listing 1.1 `cpu-volume-3/README.md`

```
1 # 从 C++ 到 AI 计算：第三册
2
3 第三册用于承接 AI、AI 编译器、算子开发和本地大模型推理引擎。第二册只讨论硬件原
  ↳ 理、多核、高性能计算、并发、锁、无锁数据结构、并行算法和分布式计算，不再
  ↳ 直接进入 AI。
4
5 本册贯穿项目是一台能推理开源 7B 左右 decoder-only 大模型的本地引擎，并把 AI 编
  ↳ 译器作为引擎内部主线：模型图导入、typed tensor IR、算子融合、内存规划、
  ↳ CPU/GPU lowering、kernel selection 和 runtime dispatch 都要能落回编译原理
  ↳ 与底层机器账本。工程路线从 CPU 上可解释的 reference 和量化切片开始，逐步
  ↳ 扩展到 tokenizer、模型加载、Transformer 层、KV cache、CPU SIMD/多线程后
  ↳ 端、GPU 后端、正确性报告和性能对标。
6
7 正式书稿工程位于：
8
9 ```text
10 source/latex/
11 ```
12
13 构建命令：
14
15 ```bash
16 make -C source/latex check
17 make -C source/latex pdf
18 make -C source/latex epub
19 ```
```

```

20
21 生成文件：
22
23 ```text
24 source/latex/main.pdf
25 source/latex/main.epub
26 ```
27
28 当前保留的第一阶段切片：
29
30 ```text
31 labs/linux_cpu_inference/
32 ```
33
34 该目录目前包含最小 MLP/int8 权重教学切片、tiny reference decoder、GPT-2 ↵
    ↵ reference path、int8 scalar baseline、packed layout、AVX2 SIMD 路径和 ↵
    ↵ shape sweep benchmark，用来验证张量、线性层、量化权重、tokenizer、↵
    ↵ safetensors、GPT-2 forward、KV cache、oracle 和 benchmark 的基本边界。当↵
    ↵ 前 x86-64 AVX2 结果见：
35
36 ```text
37 results/lcqi-x86-avx2-benchmark-2026-06-25.csv
38 results/lcqi-x86-avx2-decode4096-2026-06-25.csv
39 ```
40
41 自研 GPT-2 smoke 入口如下：
42
43 ```bash
44 make cpu3-gpt2-smoke
45 make cpu3-gpt2-benchmark-compare
46 ```
47
48 第一条命令下载 `openai-community/gpt2` 的标准 safetensors 资产到用户缓存目录，↵
    ↵ 用 LCQI C++ reference path 生成 1 个 token，并把报告写入：
49
50 ```text
51 results/lcqi-gpt2-smoke.txt
52 ```
53
54 第二条命令用同一个 GPT-2 F32 模型对比 LCQI full-prefix 和 cached-KV 生成路径，↵
    ↵ 并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考，报告写入：
55
56 ```text
57 results/lcqi-gpt2-benchmark-compare.txt
58 ```
59
60 它仍然不是生产级 AI Infra。生产级门禁见：
61
62 ```text
63 reports/ai-infra-maturity-audit.md
64 ```
65

```

66 第三册后续必须把 lab 推进到“小型 CPU 推理 runtime + 手写高性能算子 + 严格
 ↪ benchmark 报告 + 真实系统对标”，否则只能证明系统学习，不能证明生产级能
 ↪ 力。

Listing 1.2 cpu-volume-3/CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.31)
2 project(CPUVolume3 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
9 option(CPU_VOLUME_3_BUILD_LABS "Build CPU Volume 3 labs" ON)
10
11 if(CPU_VOLUME_3_BUILD_LABS)
12     enable_testing()
13     add_subdirectory(labs/linux_cpu_inference)
14 endif()
```

实践卷也提供一个薄 CMake 入口，它把同一份 LCQI 源码作为实践卷可测试对象接入。这样实践卷不会复制实现，只复用第三册主工程。

Listing 1.3 cpu-volume-3-practice/CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.31)
2 project(CPUVolume3Practice LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
9 enable_testing()
10 add_subdirectory(..cpu-volume-3/labs/linux_cpu_inference linux_cpu_inference)
```

源码卷自身的 Makefile 负责构建 PDF/EPUB，并通过 `test` 目标回到第三册真实 LCQI 工程做 CMake/CTest 验证。源码卷如果只排版、不测试，就不能宣称“完整源码可运行”。

Listing 1.4 cpu-volume-3-source/Makefile

```
1 LATEX_DIR := source/latex
2
3 .PHONY: all check pdf epub test clean
4
5 all: pdf epub
6
7 check:
8     $(MAKE) -C $(LATEX_DIR) check
9
```

```

10 pdf:
11     $(MAKE) -C $(LATEX_DIR) pdf
12
13 epub:
14     $(MAKE) -C $(LATEX_DIR) epub
15
16 test:
17     cmake -S ../cpu-volume-3 -B ../cpu-volume-3/build/lcqi-debug -<
18     ↪ DCMMAKE_BUILD_TYPE=Debug
19     cmake --build ../cpu-volume-3/build/lcqi-debug
20     ctest --test-dir ../cpu-volume-3/build/lcqi-debug --output-on-failure
21
22 clean:
23     $(MAKE) -C $(LATEX_DIR) clean

```

Listing 1.5 cpu-volume-3-source/source/latex/Makefile

```

1 LATEXMK ?= latexmk
2 ENGINE ?= -xelatex
3 MAIN ?= main.tex
4 EPUB_BUILDER ?= ../../cpu-volume-1/source/latex/scripts/build_epub.py
5
6 .PHONY: all pdf epub clean check
7
8 all: pdf epub
9
10 pdf:
11     $(LATEXMK) $(ENGINE) -interaction=nonstopmode -halt-on-error $(MAIN)
12
13 epub:
14     @BOOK_TITLE="从 C++ 到 AI 计算：第三册源码卷" \
15     BOOK_IMPORT_TITLE="CPU Volume 3 Source" \
16     BOOK_ID_SEED="cpu-volume-3-source-weread-20260629" \
17     EPUB_ASCII_TITLES=1 \
18     BOOK_SUBTITLE="LCQI 本地 CPU 推理引擎完整源码与阅读路径" \
19     BOOK_AUTHOR="CPU Performance Study" \
20     python3 $(EPUB_BUILDER) --book-dir . --output main.epub --no-cover-page --<
21     ↪ forbid-images --linearize-tables --wechat-compatible --no-embed-fonts
22
23 clean:
24     $(LATEXMK) -C
25
26 check:
27     @python3 scripts/check_inputs.py
28     @command -v xelatex >/dev/null 2>&1 || { echo "missing xelatex"; exit 1; }
29     @command -v latexmk >/dev/null 2>&1 || { echo "missing latexmk"; exit 1; }

```

1.3 LCQI 目标定义

linux_cpu_inference/CMakeLists.txt 是源码卷的第一份核心文件。这里能看到 lcqi 库、lcqi_cli、lcqi_bench、lcqi_trace、lcqi_gpt2、lcqi_ledger、lcqi_

serving_probe、可选 lcqi_onednn_bench 和 lcqi_tests 的完整关系。

Listing 1.6 linux_cpu_inference/CMakeLists.txt

```

1 find_package(Threads REQUIRED)
2
3 add_library(lcqi STATIC
4     src/f32_kernels.cpp
5     src/ggml_matvec.cpp
6     src/ggml_tensors.cpp
7     src/gguf.cpp
8     src/gpt2_reference.cpp
9     src/inference.cpp
10    src/int8_kernels.cpp
11    src/llama_gguf.cpp
12    src/model.cpp
13    src/q4_k.cpp
14    src/reference_decoder.cpp
15    src/safetensors.cpp
16    src/tokenizer.cpp
17 )
18
19 if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64|AMD64|i[3-6]86")
20     if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
21         target_sources(lcqi PRIVATE src/f32_kernels_avx2.cpp)
22         target_sources(lcqi PRIVATE src/ggml_matvec_avx2.cpp)
23         target_sources(lcqi PRIVATE src/int8_kernels_avx2.cpp)
24         target_sources(lcqi PRIVATE src/q4_k_avx2.cpp)
25         set_source_files_properties(src/f32_kernels_avx2.cpp PROPERTIES ↵
↵ COMPILE_OPTIONS "-mavx2;-mfma")
26         set_source_files_properties(src/ggml_matvec_avx2.cpp PROPERTIES ↵
↵ COMPILE_OPTIONS "-mavx2")
27         set_source_files_properties(src/int8_kernels_avx2.cpp PROPERTIES ↵
↵ COMPILE_OPTIONS "-mavx2;-mfma")
28         set_source_files_properties(src/q4_k_avx2.cpp PROPERTIES ↵
↵ COMPILE_OPTIONS "-mavx2")
29         target_compile_definitions(lcqi PRIVATE LCQI_ENABLE_AVX2=1)
30     endif()
31 endif()
32
33 if(NOT CMAKE_SYSTEM_NAME STREQUAL "Linux")
34     message(WARNING "LCQI is written for the book's local Linux CPU target; ↵
↵ current system is ${CMAKE_SYSTEM_NAME}.")
35 endif()
36
37 target_include_directories(lcqi
38     PUBLIC
39     ${CMAKE_CURRENT_SOURCE_DIR}/include
40 )
41
42 target_compile_features(lcqi PUBLIC cxx_std_20)
43 target_link_libraries(lcqi PUBLIC Threads::Threads)
44

```

```
45 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
46     target_compile_options(lcqi PRIVATE -Wall -Wextra -Wpedantic)
47 endif()
48
49 add_executable(lcqi_cli
50     src/main.cpp
51 )
52
53 target_link_libraries(lcqi_cli PRIVATE lcqi)
54
55 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
56     target_compile_options(lcqi_cli PRIVATE -Wall -Wextra -Wpedantic)
57 endif()
58
59 add_executable(lcqi_bench
60     src/bench.cpp
61 )
62
63 target_link_libraries(lcqi_bench PRIVATE lcqi)
64
65 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
66     target_compile_options(lcqi_bench PRIVATE -Wall -Wextra -Wpedantic)
67 endif()
68
69 add_executable(lcqi_gguf_q4_bench
70     src/gguf_q4_bench.cpp
71 )
72
73 target_link_libraries(lcqi_gguf_q4_bench PRIVATE lcqi)
74
75 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
76     target_compile_options(lcqi_gguf_q4_bench PRIVATE -Wall -Wextra -Wpedantic)
77 endif()
78
79 add_executable(lcqi_trace
80     src/trace.cpp
81 )
82
83 target_link_libraries(lcqi_trace PRIVATE lcqi)
84
85 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
86     target_compile_options(lcqi_trace PRIVATE -Wall -Wextra -Wpedantic)
87 endif()
88
89 add_executable(lcqi_gpt2
90     src/gpt2_cli.cpp
91 )
92
93 target_link_libraries(lcqi_gpt2 PRIVATE lcqi)
94
95 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
96     target_compile_options(lcqi_gpt2 PRIVATE -Wall -Wextra -Wpedantic)
```

```

97 endif()
98
99 add_executable(lcqi_llama_gguf
100     src/llama_cli.cpp
101 )
102
103 target_link_libraries(lcqi_llama_gguf PRIVATE lcqi)
104
105 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
106     target_compile_options(lcqi_llama_gguf PRIVATE -Wall -Wextra -Wpedantic)
107 endif()
108
109 add_executable(lcqi_ledger
110     src/ledger.cpp
111 )
112
113 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
114     target_compile_options(lcqi_ledger PRIVATE -Wall -Wextra -Wpedantic)
115 endif()
116
117 add_executable(lcqi_serving_probe
118     src/serving_probe.cpp
119 )
120
121 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
122     target_compile_options(lcqi_serving_probe PRIVATE -Wall -Wextra -Wpedantic)
123 endif()
124
125 find_package(dnnl QUIET)
126 if(dnnl_FOUND)
127     add_executable(lcqi_onednn_bench
128         src/onednn_bench.cpp
129     )
130     target_link_libraries(lcqi_onednn_bench PRIVATE DNNL::dnnl)
131     target_compile_features(lcqi_onednn_bench PRIVATE cxx_std_20)
132     if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
133         target_compile_options(lcqi_onednn_bench PRIVATE -Wall -Wextra ↵
134             ↵ Wpedantic)
135     endif()
136 endif()
137
138 add_executable(lcqi_tests
139     tests/lcqi_tests.cpp
140 )
141
142 target_link_libraries(lcqi_tests PRIVATE lcqi)
143
144 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
145     target_compile_options(lcqi_tests PRIVATE -Wall -Wextra -Wpedantic)
146 endif()
147
148 add_test(NAME lcqi_tests COMMAND ${TARGET_FILE:lcqi_tests})

```

```

148 add_test(NAME lcqi_gpt2_tiny COMMAND ${TARGET_FILE:lcqi_gpt2})
149 set_tests_properties(lcqi_gpt2_tiny PROPERTIES
150     PASS_REGULAR_EXPRESSION "generated_ids 1 2 3 3 3"
151 )
152 add_test(NAME lcqi_trace COMMAND ${TARGET_FILE:lcqi_trace})
153 set_tests_properties(lcqi_trace PROPERTIES
154     PASS_REGULAR_EXPRESSION "tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4"
155 )
156 add_test(NAME lcqi_ledger COMMAND ${TARGET_FILE:lcqi_ledger})
157 set_tests_properties(lcqi_ledger PROPERTIES
158     PASS_REGULAR_EXPRESSION "bandwidth_limit,int4_at_100_gbps,23.602,tokens/s"
159 )
160 add_test(NAME lcqi_serving_probe COMMAND ${TARGET_FILE:lcqi_serving_probe})
161 set_tests_properties(lcqi_serving_probe PROPERTIES
162     PASS_REGULAR_EXPRESSION "request,req_too_long,0,memory_budget_exceeded"
163 )

```

1.4 项目说明与模型资产

README 是复现实验的入口。它写清楚当前能力、后续边界、构建命令、tiny 推理命令、benchmark 字段、reference trace 字段、GPT-2 smoke、7B ledger、serving SLO probe 和 SmolLM2 small smoke。读源码前先读 README，可以避免把 tiny fixture、GPT-2 reference path 和外部 llama.cpp smoke 混成同一个能力等级。

Listing 1.7 linux_cpu_inference/README.md

```

1 # Linux CPU Quantized Inference
2
3 这是第三册贯穿项目的第一阶段切片：Linux 本地 CPU 量化线性层和最小推理流程。第三
    ↳ 册的贯穿目标不是这个 MLP 本身，而是一台能推理开源 7B 左右 decoder-only 大
    ↳ 模型的本地引擎，并逐步覆盖 tokenizer、模型加载、Transformer 层、KV cache
    ↳ 、CPU/GPU 后端、正确性报告和性能对标。
4
5 目标平台是 x86-64 Linux 实验环境。完整性能报告应记录 CPU 型号、内核版本、编译器
    ↳ 版本、是否能使用 PMU、NUMA 和线程亲和能力。若运行环境缺少 perf、NUMA、线
    ↳ 程亲和性或硬件性能计数器权限，报告必须把相关结论降级为“未验证”，不能把源
    ↳ 码路径存在当作实测性能证据。后续 GPU 后端会作为独立 backend 接入，不改变
    ↳ 当前切片的语义合同。
6
7 当前版本支持：
8
9 - 教学文本模型格式 `LCQI_MODEL_V1`
10 - 最小 tokenizer 合同格式 `LCQI_TOKENIZER_V1`
11 - safetensors header manifest 解析：metadata、tensor name、dtype、shape、data
    ↳ ↳ offsets、offset 边界和 dtype/shape 字节数校验
12 - 两层 MLP 教学切片
13 - int8 权重加 per-layer scale
14 - float 输入和激活
15 - 量化线性层、ReLU、分类 argmax
16 - int8 linear scalar baseline、packed layout、AVX2 路径和 shape sweep benchmark
17 - GPT-2 F32 dot kernel: cached generation 热路径使用可替换 dot 边界，x86-64/GNU
    ↳ ↳ /Clang 构建接入 AVX2/FMA 实现，其他平台保留标量 fallback

```

```

18 - tiny reference decoder: RMSNorm、float linear、RoPE、GQA KV cache、decode ↵
    ↵ attention、SwiGLU、lm head
19 - GPT-2 reference path: byte-level BPE tokenizer、`config.json`、F32/F16/BF16 ↵
    ↵ safetensors 张量读取、HuggingFace GPT-2 name mapping、LayerNorm、绝对位置 ↵
    ↵ 嵌入、packed QKV、causal attention、GELU、tied lm head、full-prefix ↵
    ↵ greedy generation、KV cache greedy generation 和分阶段 benchmark
20 - reference trace CSV: 从 prompt/tokenizer 合同到 logits/sampler/token, 逐 ↵
    ↵ checkpoint 输出 shape、stride、dtype、layout、checksum、max_abs、values ↵
    ↵ 摘要
21 - 7B ledger CSV: 输出典型 7B shape 的 FLOPs、KV 容量、权重/KV 字节和 bandwidth-↵
    ↵ only tokens/s 上限
22 - serving SLO probe CSV: 输出 admission、batch state、KV 预算、queue wait、TTFT↵
    ↵ 、TPOT、取消回收和拒绝率
23 - 自研 GPT-2 冒烟: 加载 `openai-community/gpt2` 的 `config.json`、`vocab.json`↵
    ↵ 、`merges.txt`、`model.safetensors`, 在 LCQI C++ reference path 上生成 1 ↵
    ↵ 个 token, 并把文件 hash、prompt ids、generated ids 和文本输出写入报告
24 - 自研 GPT-2 benchmark 对比: 同一个 GPT-2 F32 safetensors 模型分别跑 full-↵
    ↵ prefix 和 cached-KV, 并在可用时附带 llama.cpp/SmolLM2 Q4_K_M 作为成熟引擎↵
    ↵ 参考
25 - 自研 GPT-2 优化 A/B: 从指定 baseline commit 建临时 worktree, 当前实现和 ↵
    ↵ baseline 都通过 `books/cpu-volume-3-practice` 的 Release CMake 入口构建, ↵
    ↵ 再用同一个 GPT-2 F32 模型多轮测量 full-prefix 与 cached-KV
26 - 真实 small 开源模型冒烟: 通过 llama.cpp 加载 SmolLM2-135M-Instruct 的 GGUF 量↵
    ↵ 化文件, 在本地 CPU 上生成一段 assistant 文本, 并把模型 hash、命令、输出和↵
    ↵ 性能行写入报告
27 - 自研 SmolLM2/GGUF reference decode: 读取同一个 GGUF 内的 tokenizer tokens/↵
    ↵ merges 和 `F32/Q8_0/Q5_0/Q6_K/Q4_K` 混合权重, 加载 30 层 SmolLM2, 执行 ↵
    ↵ RMSNorm、normal RoPE、GQA KV cache、SwiGLU 和 tied lm head; 默认把 shape ↵
    ↵ 兼容的 `Q4_K` linear tensor 留在 GGUF block bytes 中直接执行, 其余格式走 ↵
    ↵ f32 fallback; `LCQI_LLAMA_GGML_DIRECT=1` 可打开 `Q5_0/Q6_K/Q8_0` 实验直算↵
    ↵ 路径用于分析
28 - CLI 推理和简单 benchmark
29 - CTest 正确性测试
30
31 后续扩展方向:
32
33 - 真实 7B 级模型 metadata、tokenizer 和量化权重导入
34 - SentencePiece tokenizer、safetensors 分片加载和更多模型方言 name mapping
35 - GPT-2 reference path 的 Transformers/PyTorch golden logits 对齐报告
36 - KV cache 分页、内存规划和可选量化
37 - CPU packed weight、SIMD、NUMA 和线程亲和优化
38 - GPU backend adapter 和 device kernel
39 - 与现有框架的 prefill/decode/内存/误差对标报告
40
41 构建:
42
43 ```bash
44 cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↵
    ↵ DCMMAKE_BUILD_TYPE=Debug
45 cmake --build books/cpu-volume-3/build/lcqi-debug
46 ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
47 ```

```

```

48
49 运行：
50
51 ```bash
52 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_cli \
53   books/cpu-volume-3/labs/linux_cpu_inference/models/tiny_mlp_i8.txt \
54   1.0 2.0 -1.0 0.5 1000
55 ```
56
57 kernel benchmark:
58
59 ```bash
60 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_bench 200
61 ```
62
63 输出 CSV 字段：
64
65 ```text
66 target_arch,input_size,output_size,output_block,repeat,backend,available,↵
67   ↵ average_us,max_abs_diff,checksum
68 ```
69 当前 benchmark 按 backend 逐行输出：`scalar`、`packed_scalar`、`avx2`。x86-64 ↵
70   ↵ 构建会编译 AVX2 路径；不支持 AVX2/FMA 的机器会把该 backend 标为 `↵
71   ↵ available=0`，避免把源码存在误报成当前环境实测性能。
72
73 这还不是生产级高性能 kernel。当前 benchmark 建立 scalar baseline、packed layout↵
74   ↵ 、AVX2 基线正确性和 shape sweep 口径。要达到完整 AI Infra 证据，还必须继↵
75   ↵ 续补反汇编分析、AVX-512、perf counter、oneDNN/OpenBLAS/llama.cpp/ggml 对↵
76   ↵ 照、sanitizer、fuzz 和 CI。
77
78 safetensors manifest gate:
79
80 `lcqi_tests` 会运行时生成一个最小 safetensors fixture，验证 header 长度、`↵
81   ↵ __metadata__`、tensor name、dtype、shape、data offsets、payload 边界和 ↵
82   ↵ dtype/shape 推导出的字节数。它还会生成 offset 超出 payload、dtype/shape ↵
83   ↵ 字节数不匹配的坏文件，确认加载器会拒绝。这个 gate 只覆盖模型资产目录表，↵
84   ↵ 不等于已经支持完整 LLaMA/Qwen 权重映射、分片合并或 mmap 执行；后续真实模↵
85   ↵ 型加载必须在这个 manifest 上继续接 name mapping、shape verifier、quant ↵
86   ↵ descriptor 和 first token trace。
87
88 reference decoder trace:
89
90 ```bash
91 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_trace
92 ```
93
94 输出 CSV 字段：
95
96 ```text
97 name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values
98 ```

```

```

88
89 这条 trace 固定 prompt fixture `tok1 tok2 tok3`, tokenizer 输出完整 `tokens`
    ↪ `=[0,1,2,3,4]`, 其中 `0/4` 是 BOS/EOS; decoder trace 会剥离 BOS/EOS 后进入
    ↪ tiny decoder。这样报告能同时固定 tokenizer 合同和 decoder 输入合同, 避免
    ↪ 把二者混成一个不可解释的 token 数组。随后 trace 把 embedding、RMSNorm、Q/
    ↪ K/V、RoPE、KV cache slot、attention、SwiGLU、layer output、final norm、
    ↪ logits、argmax sampler 和 generated token 串成可回归检查的硬链路。`
    ↪ lcqi_tests` 会检查 tokenizer contract hash、关键 checkpoint 的 shape、
    ↪ stride、dtype、layout 和 logits golden, CTest 也会直接运行 `lcqi_trace`。

90
91 当前 tokenizer fixture 的关键输出:
92
93 ```text
94 tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4
95 tokenizer_contract,0,-1,scalar,scalar,u64,fnv1a,13452902845388333734,0,tiny-
    ↪ chat-template-v1
96 ```
97
98 GPT-2 自研 reference smoke:
99
100 ```bash
101 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
102 make cpu3-gpt2-smoke
103 make cpu3-gpt2-benchmark-compare
104 make cpu3-gpt2-hotspot-profile
105 python3 books/cpu-volume-3/tools/run_gpt2_optimization_ab.py --rounds 3
106 python3 books/cpu-volume-3/tools/run_gpt2_hotspot_profile.py --token-counts
    ↪ 4,16 --rounds 3
107 ```
108
109 第一条命令不需要下载模型, 直接运行仓库内 tiny GPT-2 fixture, 输出固定 prompt
    ↪ ids、logits、predicted token 和 greedy generated ids。`lcqi_tests` 会覆盖
    ↪ tiny GPT-2 forward/generation、byte-level BPE 编码解码、F32/F16/BF16
    ↪ safetensors 读取、HuggingFace 风格 tiny GPT-2 checkpoint 加载和坏 shape
    ↪ 拒绝。

110
111 第二条命令显式依赖网络, 只下载 `openai-community/gpt2` 的 `config.json`、`vocab-
    ↪ .json`、`merges.txt`、`model.safetensors` 到 `~/cache/lcqi-gpt2-smoke/`
    ↪ `openai-community--gpt2`, 模型文件不进入 Git。脚本会校验四个文件的 bytes
    ↪ 和 SHA256, 构建 `lcqi_gpt2`, 运行:

112
113 ```text
114 Hello, my name is
115 ```
116
117 当前验证报告写到:
118
119 ```text
120 books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
121 ```
122
123 最近一次本机报告的关键行是:

```



```

124
125 ```text
126 prompt_ids 15496 11 616 1438 318
127 generated_ids 15496 11 616 1438 318 1757
128 generated_text Hello, my name is John
129 validation=PASS
130 ```
131
132 这说明 LCQI 自研 C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer↵
    ↵ 、权重导入、forward 和 greedy generation。默认真实模型路径现在使用 `↵
    ↵ cached_kv`，也可以用 `--engine full` 复现旧的 full-prefix 重算路径：
133
134 ```bash
135 books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_gpt2 \
136 --benchmark --engine cached --threads 0 \
137 ~/.cache/lcqi-gpt2-smoke/openai-community--gpt2 \
138 "Hello, my name is" 4
139 ```
140
141 `--threads 0` 表示自动选择 cached decoder worker 数；`--threads 1` 强制串行；↵
    ↵ `--threads N` 固定 worker 数。当前 auto 上限是 16，避免在短 decode 上把线↵
    ↵ 程调度开销放大到超过收益。
142
143 `make cpu3-gpt2-benchmark-compare` 会生成：
144
145 ```text
146 books/cpu-volume-3/results/lcqi-gpt2-benchmark-compare.txt
147 ```
148
149 `run_gpt2_optimization_ab.py` 会生成：
150
151 ```text
152 books/cpu-volume-3/results/lcqi-gpt2-optimization-ab.txt
153 ```
154
155 这个 A/B 报告专门回答“当前 LCQI 代码相对指定 baseline 改快了吗”。脚本默认 ↵
    ↵ baseline 是 `4feb3f`，会临时创建 baseline worktree，分别构建 baseline 和↵
    ↵ 当前工作区的 Release `lcqi_gpt2`，再多轮运行同一条 prompt。上一轮工作区复↵
    ↵ 用优化的 2 轮 smoke 摘要保存在 `books/cpu-volume-3/reports/lcqi-gpt2-↵
    ↵ optimization-ab-summary.md`；本轮线程优化的正式 raw 报告保存在 `books/cpu↵
    ↵ -volume-3/reports/lcqi-gpt2-threaded-manual-ab-caw.txt`，汇总保存在 `↵
    ↵ books/cpu-volume-3/reports/lcqi-gpt2-threaded-optimization-summary.md`。↵
    ↵ 以 `73f40c7` 作为 baseline，在 `caw` 上 5 轮中位数显示：4 token cached-KV↵
    ↵ 生成阶段从 `395.397 ms` 降到 `76.4141 ms`，16 token 从 `989.060 ms` 降到↵
    ↵ `185.479 ms`，生成吞吐分别提升到 `52.3464 token/s` 和 `86.2633 token/s`↵
    ↵ 。随后的 F32 dot kernel 优化以 `cb4d89f` 为 baseline，raw 报告保存在 `↵
    ↵ books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-ab-caw.txt`，汇总保存在 `↵
    ↵ books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-optimization-summary.md`。在↵
    ↵ 同一台 `caw`、同一 GPT-2 F32 模型、同样 `--threads 0` 自动 16 workers ↵
    ↵ 下，4 token 生成阶段中位数从 `74.4247 ms` 到 `62.3628 ms`，16 token 从 ↵
    ↵ `181.257 ms` 到 `157.230 ms`；decode token/s 分别提升到 `130.073` 和 ↵
    ↵ `127.405`。再往后，row-range kernel 和去 `std::function` 调度开销的 raw ↵

```


↪ 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-rowrange-ab-caw.txt`
 ↪ , 汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-rowrange-
 ↪ optimization-summary.md`。这一轮相对 `ab72ccb` 的端到端提升很小, 16 token
 ↪ 生成中位数约 `153.354 ms` 到 `152.574 ms`; 但直接比较 row-range 与 4-row
 ↪ AVX2 row-range 时, 4 token 从 `62.0243 ms` 到 `60.3965 ms`, 16 token 从
 ↪ `154.087 ms` 到 `151.222 ms`。本轮 prefill-skip 优化的 raw 报告保存在
 ↪ `books/cpu-volume-3/reports/lcqi-gpt2-prefill-skip-ab-caw.txt`, 汇总保存在
 ↪ `books/cpu-volume-3/reports/lcqi-gpt2-prefill-skip-optimization-summary.
 ↪ md`。它跳过 prompt 前缀 token 上不会被使用的 `lm_head` 预测: 4 token 生成
 ↪ 中位数从 `60.5154 ms` 到 `52.1949 ms`, 16 token 从 `153.574 ms` 到
 ↪ `147.424 ms`, 生成文本和 token id 全部一致。端到端 GPT-2 数字受调度、温度
 ↪ 和共享机器状态影响很大, 因此正式结论必须看多轮中位数、min/max 和生成文本
 ↪ 一致性, 不能只挑一次最快结果。

156

157 `run\_gpt2\_hotspot\_profile.py` 专门回答“现在热点到底在哪”。它通过 `--profile-
 ↪ hotspots` 打开 cached decode 内部计时, 字段包括 `hotspot_lm_head_pct`、`
 ↪ hotspot_mlp_fc_pct`、`hotspot_mlp_projection_pct`、`
 ↪ hotspot_qkv_projection_pct` 和 `hotspot_attention_pct`。`caw` 上的 3 轮
 ↪ Release profile 摘要保存在 `books/cpu-volume-3/reports/lcqi-gpt2-hotspot-
 ↪ profile-summary.md`, 原始报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-
 ↪ -hotspot-profile-caw.txt`。关键结论是: 4 token 与 16 token 两个口径下, `
 ↪ lm_head` 中位数约 `30.2%`, MLP 两个线性投影合计约 `46%`, QKV 投影约 `17%`
 ↪ , cached attention 本体只有 `0.06%` 到 `0.11%`。所以当前短上下文 GPT-2 的
 ↪ 慢点不是 KV attention, 而是标量 F32 matvec 和 `50257 x 768` vocab 扫描。

158

159 当前优化点集中在 cached-KV generation: `Gpt2CachedGreedyDecoder` 把模型级 shape
 ↪ validation、KV cache、临时 workspace 和 worker pool 的生命周期提升到整段
 ↪ 生成; `Gpt2ForwardWorkspace` 复用 hidden、normed、Q/K/V、packed QKV、
 ↪ attention、MLP、scores 和 logits 缓冲区; 生成路径只需要 greedy token 时走
 ↪ logits-free argmax, 避免把完整 logits vector 作为 API 结果反复分配;
 ↪ prompt prefill 前缀通过 `advance\_without\_prediction()` 只更新 KV cache,
 ↪ 不跑 final norm 和 `lm\_head`, 因为这些预测结果不会被用作输出; QKV、
 ↪ attention output projection、MLP 两个 projection 和 tied `lm\_head` 都在
 ↪ cached session 内按输出行并行; 每个线程 chunk 再通过
 ↪ `linear\_f32\_rows\_unchecked` 或 `max\_dot\_f32\_rows\_unchecked` 进入 F32 row-
 ↪ range kernel, x86-64 可用时使用 AVX2/FMA, 其他平台走标量 fallback。KV
 ↪ cache 的 checked public API 仍保留, 内部 cached attention 在一次边界校验
 ↪ 后使用私有 unchecked span 扫描热路径。它不改变 GPT-2 数学语义, 只改变会话
 ↪ 生命周期、任务分片、无用工作剔除和热循环调度。

160

161 `make cpu3-gpt2-benchmark-compare` 仍用于和外部成熟引擎放在同一报告里看工程差
 ↪ 距。同机 llama.cpp/SmolLM2 Q4_K_M 是成熟引擎参照, 不是同模型质量排名:
 ↪ LCQI 跑的是 GPT-2 F32 reference path, llama.cpp 跑的是 GGUF 量化 small
 ↪ instruct 模型。它揭示的是工程差距: LCQI 已经消除了“每步重算前缀”的主要算
 ↪ 法错误, 补上 cached F32 row parallelism, 并把热路径接到 AVX2/FMA row-
 ↪ range kernel; 但还缺 packed/quantized weight、GGUF 量化 block、mmap/lazy
 ↪ load、采样器、分页 KV cache、NUMA/亲和和成熟调度。

162

163 它仍然不是生产级推理引擎: 当前 GPT-2 路径还没有把 logits 和 Transformers/
 ↪ PyTorch 逐数值对齐成外部 golden 报告, 也没有 GGUF 自研导入、分页 KV cache
 ↪ 、量化权重执行和高性能 SIMD/packed kernel。

164

```

165 7B ledger:
166
167 ```bash
168 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_ledger \
169   > books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv
170 ```
171
172 输出 CSV 字段:
173
174 ```text
175 section,item,value,unit,notes
176 ```
177
178 这份账本固定 `L=32,H=4096,n_q=32,n_kv=8,head_dim=128,context=4096`，报告 decode↵
    ↵ FLOPs、KV 容量、fp16/int8/int4 每 token 字节和不同有效带宽下的 tokens/s ↵
    ↵ 上限。CTest 会检查 `int4_at_100_gbps=23.602 tokens/s` 这一行，防止账本公↵
    ↵ 式漂移。
179
180 serving SLO probe:
181
182 ```bash
183 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe↵
    ↵ \
184   > books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
185 ```
186
187 输出 CSV 字段:
188
189 ```text
190 row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,↵
    ↵ kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,↵
    ↵ tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,↵
    ↵ metric_value
191 ```
192
193 probe 固定四个请求：短请求、RAG 请求、取消请求和超长请求。它演示 admission 先按↵
    ↵ `prompt_tokens + max_new_tokens` 预留 KV，取消请求释放预算，超长请求返回↵
    ↵ `memory_budget_exceeded`，并输出 `ttft_p95_ms`、`queue_wait_p95_ms`、↵
    ↵ `rejection_rate` 等服务化指标。
194
195 真实 small 开源模型 smoke:
196
197 ```bash
198 make cpu3-smolllm2-smoke
199 ```
200
201 这条命令显式依赖网络和外部构建，因此不放进默认 CTest。脚本会把 llama.cpp 和模型↵
    ↵ 文件缓存到 `~/cache/lcqi-smolllm2-small-smoke`，模型不进入 Git。固定模型↵
    ↵ 是 `HuggingFaceTB/SmolLM2-135M-Instruct` 的 GGUF 量化版本，来源仓库为 `↵
    ↵ bartowski/SmolLM2-135M-Instruct-GGUF`，文件为 `SmolLM2-135M-Instruct-↵
    ↵ Q4_K_M.gguf`。脚本会校验文件大小 `105454432` 字节和 SHA256:
202

```

```

203 ```text
204 2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
205 ```
206
207 脚本构建固定 commit 的 `llama-simple-chat`，用 `-ngl 0` 强制 CPU 路径，输入短
    ↪ prompt，检查 assistant response 非空，并生成：
208
209 ```text
210 books/cpu-volume-3/results/lcqi-smolllm2-small-model-smoke.txt
211 ```
212
213 这份报告只能证明 llama.cpp 成熟外部引擎上的真实 GGUF small 模型已经完成加载、
    ↪ tokenizer/chat template、prefill、decode 和文本输出。它不能证明 LCQI 自研
    ↪ C++ 引擎已经达到 llama.cpp 的性能，也不能替代 tiny fixture 的逐层 golden
    ↪ trace。
214
215 LCQI 自研 SmolLM2/GGUF reference decode：
216
217 ```bash
218 books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_llama_gguf
    ↪ \
219 ~/.cache/lcqi-smolllm2-small-smoke/models/SmolLM2-135M-Instruct-Q4_K_M.gguf \
220 --prompt "Hello" --max-new 2 --benchmark --decode-text
221 ```
222
223 caw 上的最新报告写到：
224
225 ```text
226 books/cpu-volume-3/results/lcqi-smolllm2-gguf-reference-decode-caw.txt
227 ```
228
229 关键行是：
230
231 ```text
232 weight_execution gguf_mixed_q4_k_direct
233 generated_text ... <|im_start|>assistant
234 Hello!
235 benchmark_prompt_tokens 31
236 benchmark_generated_tokens 2
237 benchmark_decode_tokens_per_second 64.8586
238 benchmark_quantized_weight_bytes 103668480
239 benchmark_f32_weight_bytes 481436928
240 benchmark_direct_quantized_weight_bytes 7962624
241 benchmark_fallback_dequantized_weight_bytes 481436928
242 benchmark_q4_k_direct_tensors 16
243 benchmark_ggml_direct_tensors 0
244 benchmark_f32_fallback_tensors 256
245 benchmark_hotspot_w_down_ms 63.5371
246 benchmark_hotspot_q4_k_direct_calls 512
247 benchmark_hotspot_ggml_direct_calls 0
248 benchmark_hotspot_f32_fallback_calls 6208
249 ```

```

250

251 这条路径证明 LCQI 自研代码已经能读取同一个 SmolLM2 GGUF、解析 tokenizer arrays
 ↳ 、处理混合量化权重并端到端生成文本。默认执行路径不再只是“加载时全部解成
 ↳ f32”：shape 兼容的 `Q4\_K` linear tensor 会保留 GGUF block bytes，并在真实
 ↳ decoder 里通过 `matvec\_q4\_k\_q8` 直接执行；`Q5\_0/Q6\_K/Q8\_0/F32` tensor 仍
 ↳ 然 materialize 成 f32 fallback。当前只有约 `7.68%` 的加载权重字节进入默认
 ↳ direct Q4_K 路径，prefill 仍逐 token 执行；与 llama.cpp 的差距主要来自更
 ↳ 完整的低比特权重覆盖、prompt batching、packed kernel、线程调度、prefetch
 ↳ 、KV cache 和整图执行。

252

253 实验路径 `LCQI\_LLAMA\_GGML\_DIRECT=1` 会尝试让 shape 兼容的 `Q5\_0/Q6\_K/Q8\_0`
 ↳ linear tensor 也保留 GGUF block bytes，并通过 `matvec\_ggml\_quantized\_q8\_0`
 ↳ 执行。第一版单线程/整矩阵实验曾把 direct 量化字节覆盖率从 `0.076809` 提
 ↳ 高到 `0.708479`，但 prefill 从默认 `370.847 ms` 退化到 `1211.710 ms`，热
 ↳ 点 `benchmark\_hotspot\_ggml\_direct\_ms` 达到 `1207.13 ms`。当前版本已经给
 ↳ GGML direct 补上 row-range unchecked 入口，并接入同一个
 ↳ LlamaParallelWorkerPool：caw 同输入 5 轮中位数显示，实验路径降到 prefill
 ↳ `242.149 ms`、decode step `8.98937 ms`、
 ↳ benchmark_hotspot_ggml_direct_ms=228.300。这说明 row-range 解决了“所有行
 ↳ 串行扫”的一大块问题，但它仍慢于默认 Q4_K+f32 fallback 的 `162.492 ms`、
 ↳ `6.41487 ms`，所以继续保留为 opt-in，不默认启用。

254

255 LCQI 与 llama.cpp 同输入对比：

256

257 ```bash

```
258 python3 books/cpu-volume-3/tools/run_smolllm2_same_input_compare.py \
259 --cache-dir ~/.cache/lcqi-smolllm2-same-input-compare \
260 --model-path ~/.cache/lcqi-smolllm2-small-smoke/models/SmolLM2-135M-Instruct-
↳ Q4_K_M.gguf \
261 --build-dir books/cpu-volume-3/build/lcqi-release \
262 --rounds 5 --max-new 2
263 ```
```

264

265 这个脚本会构建固定 commit 的 llama.cpp，运行 `llama-tokenize`、`llama-simple`
 ↳ 和 `llama-bench`，并确认 LCQI 和 llama.cpp 看到的 prompt token ids 完全
 ↳ 致。caw 上的同输入报告保存在：

266

267 ```text

```
268 books/cpu-volume-3/reports/lcqi-smolllm2-ggml-threaded-compare-caw.txt
269 ```
```

270

271 关键结论：同一个模型、同一个展开 chat prompt、同样 31 个 token id、同样 `max`
 ↳ new=2` 下，当前默认 LCQI 使用 `8` 个 worker，对大行数 f32 fallback、Q4_K
 ↳ direct row-range 和实验 GGML direct row-range 都能做输出行切分。默认 Q4_K
 ↳ direct prefill 中位数从串行 `364.282 ms` 降到 `162.492 ms`，decode step
 ↳ 从串行 `14.9794 ms` 降到 `6.41487 ms`；f32 fallback hotspot 从 `338.779`
 ↳ ms` 降到 `148.132 ms`。相对同输入 llama.cpp，LCQI prefill 仍慢 `7.089529x`
 ↳ ，decode step 仍慢 `2.250832x`。同一二进制里关闭 `LCQI\_LLAMA\_Q4\_DIRECT`
 ↳ =0` 后，LCQI prefill 为 `174.378 ms`，decode step 为 `6.78061 ms`，
 ↳ w_down` 热点为 `36.4924 ms`；默认 Q4\_K direct 后分别为 `162.492 ms`、
 ↳ `6.41487 ms`、`22.2228 ms`。所以这轮默认优化的 A/B 证据是：线程池带来
 ↳ prefill `2.241846x`、decode step `2.335106x`、f32 hotspot `2.287008x`；

↪ Q4_K direct 在并行后带来 prefill `1.073148x`、decode step `1.057014x`、`↪
 ↪ w_down` `1.642115x`，并减少 `56,623,104` 字节 f32 materialized 权重。实验↪
 ↪ GGML direct row-range 相比最初 `1211.710 ms` 已明显改善到 `242.149 ms`，↪
 ↪ 但仍慢于默认路径，下一步需要 packed multi-row layout、更完整 SIMD 覆盖和 ↪
 ↪ prefetch，而不是只扩大格式覆盖。

tiny MLP 模型文件是最短闭环的资产。它决定输入维度、hidden 维度、输出维度、权重量化值、scale 和 bias；测试中的 logits golden 依赖这份文件。

Listing 1.8 models/tiny_mlp_i8.txt

```

1 LCQI_MODEL_V1
2 input_size 4
3 hidden_size 3
4 output_size 2
5 hidden_scale 0.1
6 hidden_weights
7 5 -3 2 1
8 -4 6 1 -2
9 3 2 -5 4
10 hidden_bias
11 0.1 -0.2 0.0
12 output_scale 0.05
13 output_weights
14 4 -6 2
15 -3 5 8
16 output_bias
17 -0.5 0.4

```

tiny tokenizer fixture 固定 BOS、EOS、UNK、词表和模板 hash。tokenizer 合同如果漂移，后面的 decoder trace 和测试都会被污染。

Listing 1.9 models/tiny_tokenizer.txt

```

1 LCQI_TOKENIZER_V1
2 bos_id 0
3 eos_id 4
4 unk_id -1
5 chat_template_hash tiny-chat-template-v1
6 vocab_size 5
7 vocab
8 0 <bos>
9 1 tok1
10 2 tok2
11 3 tok3
12 4 <eos>

```

第 2 章

源码阅读路线：从入口到调用链

2.1 为什么要先讲调用链

源码卷如果只把 *.cpp 和 *.hpp 全部贴出来，读者仍然不知道从哪里下手。LCQI 的代码同时包含 tiny MLP、int8 kernel、tokenizer、safetensors、reference decoder、GPT-2、benchmark、trace、ledger、serving probe 和外部模型脚本；这些文件不是平铺关系，而是几条调用链。先讲调用链，再给完整源码，读者才能把每个文件放到正确位置。

核心思想

读 LCQI 的顺序不是“哪个文件长先读哪个”，而是“入口、合同、资产、执行、证据”。入口说明目标怎样构建，合同说明类型怎样连接，资产说明模型从哪里来，执行说明 token 或张量怎样流动，证据说明测试和报告守住了哪些边界。

2.2 一、全工程入口：哪些文件决定能不能复现

文件	负责什么	读源码时要确认什么
第三册README	第三册 AI 计算工程路线、当前 LCQI 能力、GPT-2 smoke 和结果报告位置。	能力声明必须能在 labs/linux_cpu_inference、tools 和 results 中找到证据。
第三册CMake	第三册根 CMake，负责把 linux_cpu_inference 接入。	根 CMake 应保持薄层；真正 target 定义必须在 lab 内部，避免入口文件承担实现细节。
实践卷CMake	实践卷复用同一份 LCQI lab。	实践卷不复制源码，只把同一工程接入自己的测试构建，防止两份代码漂移。
源码卷Makefile	源码卷的书籍构建入口。	它构建 PDF/EPUB，也提供 test 目标去构建第三册 LCQI；源码卷不能只排版不验证。
LCQICMake	lcqi 库、CLI、benchmark、trace、GPT-2、ledger、serving probe、CTest 目标。	读这里能看到哪些文件进入库，哪些是可执行程序，哪些依赖可选外部库。
LCQIREADME	本地构建、tiny 推理、benchmark、trace、GPT-2、SmolLM2 smoke 的命令。	先按 README 跑命令，再读源码；否则不知道某个函数是否真的能从命令行触达。

上表里的短标签对应这些真实路径：cpu-volume-3/README.md、cpu-volume-3/CMakeLists.txt、cpu-volume-3-practice/CMakeLists.txt、cpu-volume-3-source/Makefile、linux_cpu_inference/CMakeLists.txt 和 linux_cpu_inference/README.md。

这些入口文件回答“代码在哪里”和“怎样运行”。读者打开源码卷时，先不要直接跳到 gpt2_reference.cpp。先确认 lcqi 静态库包含 model.cpp、inference.cpp、int8_kernels.cpp、reference_decoder.cpp、tokenizer.cpp、safetensors.cpp 和 gpt2_reference.cpp；再确认 lcqi_cli、lcqi_bench、lcqi_trace、lcqi_gpt2、lcqi_ledger、lcqi_serving_probe 和 lcqi_tests 分别从哪条路径进入。

2.3 二、Tiny MLP 调用链：最短闭环怎样跑通

Tiny MLP 是最小推理闭环，用来证明“模型资产、shape 检查、int8 linear、激活函数、logits、argmax、CLI 输出”可以完整连起来。它不是大模型能力本身，但它是后面所有复杂路径的可手算底座。

文件	关键类型或函数	这段代码的意思
<code>models/tiny_mlp_i8.txt</code>	输入维度、hidden 维度、输出维度、int8 权重、scale、bias。	这是模型资产。测试中的 golden logits 来自这里，资产改动必须带着测试改动。
<code>include/lcqi/model.hpp</code>	<code>QuantizedLinearLayer</code> 、 <code>TinyMlpModel</code> 、 <code>load_model</code> 。	定义 tiny MLP 如何在内存中表示；只负责模型结构和加载入口，不做推理。
<code>src/model.cpp</code>	<code>read_i8_vector</code> 、 <code>validate_layer</code> 、 <code>load_model</code> 。	解析文本模型，拒绝坏 key、坏维度、权重数量不匹配和非法 scale，把错误挡在执行前。
<code>include/lcqi/inference.hpp</code>	<code>InferenceResult</code> 、 <code>linear_i8</code> 、 <code>relu_inplace</code> 、 <code>run_inference</code> 。	定义推理输出要包含 hidden、logits 和 predicted class，方便测试定位错误层。
<code>src/inference.cpp</code>	输入 shape 检查、两层 int8 linear、ReLU、argmax。	最短执行路径：模型加输入变成 logits。这里的代码应保持清楚，不能混入 CLI 或文件解析。
<code>src/main.cpp</code>	解析命令行输入并打印结果。	CLI 是薄层；它只接模型路径和输入，不复制 <code>run_inference</code> 逻辑。

这条链路按下面顺序读：`main()` 取得模型路径和输入向量，调用 `load_model()` 把 `tiny_mlp_i8.txt` 变成 `TinyMlpModel`，再调用 `run_inference()`。在 `run_inference()` 内部，第一层 `linear_i8()` 把 float 输入和 int8 权重相乘，乘以 scale 后加 bias；`relu_inplace()` 把负 hidden 截断；第二层 `linear_i8()` 得到 logits；最后 `argmax()` 给出 predicted class。

读这个路径时要抓住边界条件。输入长度不等于 `input_size` 要立即拒绝；权重数量不是 `rows*columns` 要在加载时拒绝；logits 为空不能取 `argmax`。这些错误如果拖到 SIMD kernel 或 benchmark 里才暴露，定位成本会高很多。

2.4 三、Int8 Kernel 调用链：reference、packed、AVX2 和 benchmark 怎样对照

文件	关键代码	这段代码的意思
<code>include/lcqi/int8_kernels.hpp</code>	<code>PackedLinearI8</code> 、 <code>KernelBenchmarkCase</code> 、 <code>benchmark_linear_i8_case</code> 。	定义 kernel 对标口径：shape、repeat、backend、平均耗时、checksum、max diff。
<code>src/int8_kernels.cpp</code>	<code>pack_linear_i8</code> 、 <code>linear_i8_packed</code> 、 <code>linear_i8_packed_with_workspace</code> 、 <code>deterministic fixture</code> 。	scalar 和 packed 路径是正确性基线，workspace 路径减少重复分配，fixture 让 benchmark 和测试可复现。

src/int8_kernels_avx2.cpp	linear_i8_packed_avx2_available、linear_i8_packed_avx2。	平台优化路径。必须先判断机器能力，再执行 AVX2/FMA；不可用时报告 unavailable，而不是假装测过。
src/bench.cpp	shape 列表、repeat 参数、backend 输出。	benchmark 程序只负责测 kernel 入口。性能结论必须和 max_abs_diff、CPU、编译器、构建类型一起报告。

kernel 文件要按“可信基线到优化路径”的顺序读。先看 deterministic layer 和 input 怎么生成，保证每次运行输入相同；再看 linear_i8_packed() 怎样把 int8 权重、scale、bias 和 float 输入算成输出；再看 workspace 路径怎样复用临时缓冲；最后看 AVX2 路径怎样在支持平台上替换内层循环。测试用 packed 与 scalar 的 max_abs_diff 对照，benchmark 再比较耗时。

2.5 四、Tokenizer、Safetensors 与 Reference Decoder：真实模型前的三道门

文件	关键类型或函数	这段代码的意思
models/tiny_tokenizer.txt	BOS、EOS、UNK、词表、模板 hash。	固定 tokenizer 合同，避免 prompt 到 token id 的规则漂移。
include/lcqi/tokenizer.hpp 与 src/tokenizer.cpp	TokenizerModel、load_tokenizer、encode_prompt、tokenizer_contract_hash。	把 prompt 切成稳定 token id；未知 token 是否允许，必须由 UNK 合同决定。
include/lcqi/safetensors.hpp 与 src/safetensors.cpp	SafeTensorManifest、SafeTensorFile、manifest parser、dtype 转换。	解析模型字节之前先验证 header、dtype、shape、offset 和 payload 边界，防止坏资产进入数学层。
include/lcqi/reference_decoder.hpp 与 src/reference_decoder.cpp	embedding、RMSNORM、RoPE、KV cache、attention、SwiGLU、trace checkpoint。	Tiny decoder-only reference path。它用 checkpoint 解释每一步张量状态，不只返回最终 token。
src/trace.cpp	trace CSV 输出。	把 reference decoder 的 checkpoint 打印成可复查表格，定位错误发生在哪个阶段。

这三道门分别解决三个问题。Tokenizer 解决“人写的 prompt 如何变成 token id”；Safetensors 解决“磁盘里的模型字节如何变成带 shape 的张量”；Reference Decoder 解决“token 和权重如何经过 decoder-only 层生成 logits”。任何一层含糊，后面的 GPT-2 路径都会变成看似能跑、实则无法定位的黑盒。

2.6 五、GPT-2 调用链：从 HuggingFace 资产到生成 token

GPT-2 路径是源码卷里最长的一条链，因为它要处理真实模型格式。读 gpt2_reference.cpp 时按模块读，不要从第一行一路滚到最后。

文件	关键代码	这段代码的意思
<code>include/lcqi/gpt2_reference.hpp</code>	<code>Gpt2Config</code> 、 <code>Gpt2ReferenceModel</code> 、 <code>Gpt2KvCache</code> 、 <code>Gpt2Tokenizer</code> 、 <code>forward/generate API</code> 。	公开 GPT-2 reference path 的合同：配置、权重、KV cache、tokenizer、full-prefix 和 cached generation。
<code>src/gpt2_reference.cpp</code>	JSON parser、byte-level BPE、safetensors loader、LayerNorm、QKV、causal attention、GELU、lm head、KV cache。	正确性优先实现。它把 HuggingFace GPT-2 权重名、Conv1D 转置、tokenizer 和 forward 规则全部显式写出来。
<code>src/gpt2_cli.cpp</code>	tiny mode、真实模型目录模式、generation、benchmark 输出。	命令行入口。无参数跑 tiny fixture；传模型目录时加载真实 GPT-2 资产；benchmark 对比 full-prefix 和 cached-KV。
<code>tools/run_gpt2_smoke.py</code>	下载标准 GPT-2 资产，调用 <code>lcqi_gpt2</code> 。	证明自研 safetensors、BPE、forward、KV cache 和 greedy generation 能跑真实开源模型。
<code>tools/run_gpt2_benchmark_compare.py</code>	运行 LCQI full-prefix、cached-KV，并可附带成熟引擎参考。	用同一模型口径比较算法差距和 kernel 差距，不把不同模型的速度混成一个结论。

GPT-2 代码按五段读。第一段是配置和权重加载：`load_gpt2_config()` 读 `config.json`，`load_gpt2_reference_model()` 从 safetensors 中取 embedding、每层 LayerNorm、QKV、MLP 和 final norm。HuggingFace GPT-2 的 Conv1D 权重形状和普通线性层方向不同，所以代码显式调用转置函数，不能跳过。

第二段是 tokenizer：`load_gpt2_tokenizer()` 读 `vocab.json` 和 `merges.txt`，`gpt2_encode()` 执行 byte-level BPE，`gpt2_decode()` 把 id 还原为文本。这里的重点是字节到 Unicode 映射、BPE merge rank 和 unknown token 边界。

第三段是 full-prefix forward：`run_gpt2_forward()` 每生成一个新 token 都用完整 token 前缀重新跑所有层。它慢，但容易验证。流程是 token embedding 加 position embedding，逐层执行 LayerNorm、packed QKV、causal attention、残差、MLP、残差，最后 final LayerNorm 和 tied lm head 得到 logits。

第四段是 cached-KV forward：`Gpt2KvCache` 保存每层每个 position 的 key/value，`run_gpt2_forward_cached()` 每次只处理最新 token，并让 attention 读取过去 cache。这里要特别看 `validate_written()`：读取未写入 cache slot 必须报错，否则生成错误会很难定位。

第五段是 greedy generation：`gpt2_generate_greedy()` 用 full-prefix 路径，`gpt2_generate_greedy_cached()` 用 KV cache 路径。两个函数都要检查 prompt 非空、`max_new_tokens` 非负、不会超过 `max_positions`，并在生成 EOS 时停止。

2.7 六、报告、服务探针、外部对标和测试

文件	关键代码	这段代码的意思
<code>src/ledger.cpp</code>	7B 参数量、KV cache、bandwidth-only 上限。	输出的是工程账本，不是实测速度；它帮助读者估算权重和 KV 内存压力。

<code>src/serving_probe.cpp</code>	短请求、RAG 请求、取消请求、超长请求，TTFT/TPOT 字段。	把服务层边界变成字段：哪些请求接受，哪些拒绝，取消后资源是否回收。
<code>src/onednn_bench.cpp</code>	oneDNN matmul benchmark。	可选外部库对标入口。没有 oneDNN 环境时不能声称已验证。
<code>tools/run_smollm2_small_smoke.py</code>	下载 SmolLM2-135M-Instruct GGUF，构建固定 commit 的 llama.cpp smoke。	这是成熟外部引擎上的真实 small 模型 smoke，用来建立外部参照，不代表 LCQI 已具备同等能力。
<code>tools/run_lcqi_benchmark.sh</code>	收集 LCQI 本地 benchmark。	在固定机器上重复运行同一组 shape，形成可复查 CSV 或文本报告。
<code>tests/lcqi_tests.cpp</code>	tiny MLP、tokenizer、safetensors、GPT-2 tiny、HF-style checkpoint、bad shape、decoder、trace、kernel。	这是源码卷最重要的证据。新增功能如果没有进入这里，就不能写成已经守住的能力。

读测试文件时，不要只看最后是否返回 0。它里面的 fixture 本身就是源码讲解的一部分：测试会手写 safetensors header，构造 F16/BF16/F32 payload，生成 tiny GPT-2 权重，验证 bad shape 会被拒绝，比较 packed int8 kernel 和 scalar 路径。这些测试告诉读者“代码真正承诺了什么”。如果一个能力只出现在 README 或 CLI 输出里，却没有进入测试或 smoke 报告，就只能算实验入口，不算稳定能力。

源码阅读任务

读完本章后，再去读后面的完整源码。每看到一个文件，都把它归到五类之一：入口、合同、资产解析、执行路径、证据。归不进去的文件，就说明工程边界需要重新解释或重构。

第 3 章

公共合同：模型、Tokenizer、Kernel 与 Decoder

3.1 为什么先读头文件

头文件是系统对外承诺的最小合同。实现可以重写，kernel 可以替换，benchmark 可以补充平台路径，但头文件里暴露的数据结构、函数入口和返回字段决定了测试、CLI、工具脚本和章节讲解怎样调用这套工程。

核心思想

读 LCQI 时不要从最长的 `gpt2_reference.cpp` 开始。先读公共头文件，明确“模型如何表示”“推理返回什么”“trace 记录什么”“KV cache 怎样寻址”“tokenizer 的坏输入怎样处理”，再进入实现。这样读到长循环和 shape 检查时，知道它们在维护哪份合同。

3.2 Tiny MLP 与推理入口

`model.hpp` 定义 tiny MLP 的模型对象、权重、bias、scale 和加载入口。它的责任是把文本模型资产解释成 C++ 对象，不负责执行推理。

Listing 3.1 `include/lcqi/model.hpp`

```
1 #pragma once
2
3 #include <cstdint>
4 #include <filesystem>
5 #include <string>
6 #include <vector>
7
8 namespace lcqi {
9
10 struct QuantizedLinearLayer {
11     std::int32_t input_size = 0;
12     std::int32_t output_size = 0;
13     float scale = 1.0F;
14     std::vector<std::int8_t> weights;
15     std::vector<float> bias;
16 };
17
18 struct TinyMlpModel {
19     std::int32_t input_size = 0;
20     std::int32_t hidden_size = 0;
21     std::int32_t output_size = 0;
22     QuantizedLinearLayer hidden;
23     QuantizedLinearLayer output;
24 };
25
26 TinyMlpModel load_model(const std::filesystem::path& path);
```

```

27
28 } // namespace lcqi

```

`inference.hpp` 定义推理结果和 `run_inference` 入口。返回值包含 `hidden`、`logits` 和 `predicted class`，测试可以用这些字段定位是加载错、kernel 错，还是 `argmax` 错。

Listing 3.2 `include/lcqi/inference.hpp`

```

1 #pragma once
2
3 #include <cstdint>
4 #include <span>
5 #include <vector>
6
7 #include <lcqi/model.hpp>
8
9 namespace lcqi {
10
11 struct InferenceResult {
12     std::vector<float> logits;
13     std::int32_t predicted_class = 0;
14 };
15
16 void linear_i8(const QuantizedLinearLayer& layer,
17               std::span<const float> input,
18               std::span<float> output);
19
20 void relu_inplace(std::span<float> values);
21
22 InferenceResult run_inference(const TinyMlpModel& model,
23                               std::span<const float> input);
24
25 } // namespace lcqi

```

3.3 Int8 Kernel、Tokenizer 与 Safetensors

`int8` kernel 头文件定义 `packed linear`、`deterministic fixture`、`scalar/workspace/AVX2` 路径和误差比较。注意 `AVX2` 是运行时能力，不是源码存在就等于当前机器已验证。

Listing 3.3 `include/lcqi/int8_kernels.hpp`

```

1 #pragma once
2
3 #include <cstdint>
4 #include <span>
5 #include <vector>
6
7 #include <lcqi/model.hpp>
8
9 namespace lcqi {

```

[illegible]

```

62 std::vector<float> make_deterministic_input(std::int32_t input_size);
63
64 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs);
65
66 KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ↵
    ↵ benchmark_case);
67
68 } // namespace lcqi

```

tokenizer 头文件固定 BOS/EOS/UNK、词表和 template hash。prompt 进入模型前，必须先变成稳定 token id；未知 token 行为也必须被写进合同。

Listing 3.4 include/lcqi/tokenizer.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <filesystem>
5 #include <string>
6 #include <string_view>
7 #include <vector>
8
9 namespace lcqi {
10
11 struct TokenEntry {
12     std::string text;
13     std::int32_t id = 0;
14 };
15
16 struct TokenizerModel {
17     std::int32_t bos_id = -1;
18     std::int32_t eos_id = -1;
19     std::int32_t unk_id = -1;
20     std::string chat_template_hash;
21     std::vector<TokenEntry> vocab;
22 };
23
24 [[nodiscard]] TokenizerModel load_tokenizer(const std::filesystem::path& path);
25
26 [[nodiscard]] std::vector<std::int32_t> encode_prompt(
27     const TokenizerModel& tokenizer,
28     std::string_view prompt);
29
30 [[nodiscard]] std::uint64_t tokenizer_contract_hash(const TokenizerModel& ↵
    ↵ tokenizer);
31
32 } // namespace lcqi

```

safetensors 头文件固定 dtype、shape、offset 和 payload 边界。它负责模型字节怎样被解释，不应该和数学推理逻辑混在一起。

Listing 3.5 include/lcqi/safetensors.hpp

```

1  #pragma once
2
3  #include <cstdint>
4  #include <filesystem>
5  #include <span>
6  #include <string>
7  #include <string_view>
8  #include <utility>
9  #include <vector>
10
11 namespace lcqi {
12
13 struct SafeTensorEntry {
14     std::string name;
15     std::string dtype;
16     std::vector<std::int64_t> shape;
17     std::uint64_t data_begin = 0;
18     std::uint64_t data_end = 0;
19
20     [[nodiscard]] std::uint64_t byte_size() const;
21 };
22
23 struct SafeTensorManifest {
24     std::uint64_t header_size = 0;
25     std::uint64_t data_start_offset = 0;
26     std::vector<std::pair<std::string, std::string>> metadata;
27     std::vector<SafeTensorEntry> tensors;
28
29     [[nodiscard]] const SafeTensorEntry* find_tensor(std::string_view name) ↵
        ↵ const;
30 };
31
32 struct SafeTensorFile {
33     std::filesystem::path path;
34     SafeTensorManifest manifest;
35     std::vector<std::uint8_t> bytes;
36 };
37
38 [[nodiscard]] SafeTensorManifest load_safetensors_manifest(
39     const std::filesystem::path& path);
40
41 [[nodiscard]] SafeTensorFile load_safetensors_file(const std::filesystem::path& ↵
        ↵ path);
42
43 [[nodiscard]] std::vector<float> read_safetensor_f32_tensor(
44     const SafeTensorFile& file,
45     std::string_view name);
46
47 [[nodiscard]] std::span<const std::uint8_t> read_safetensor_tensor_bytes(
48     const SafeTensorFile& file,
49     std::string_view name);
50

```



```
51 } // namespace lcqi
```

3.4 Reference Decoder 与 GPT-2 合同

reference decoder 头文件定义张量、层配置、trace checkpoint、采样结果和 decode 函数。它是定位推理错误的主要合同，不只返回最终 token。

Listing 3.6 include/lcqi/reference_decoder.hpp

```
1 #pragma once
2
3 #include <stdint>
4 #include <span>
5 #include <string>
6 #include <vector>
7
8 namespace lcqi {
9
10 struct LinearWeightsF32 {
11     std::int32_t input_size = 0;
12     std::int32_t output_size = 0;
13     std::vector<float> weights;
14     std::vector<float> bias;
15 };
16
17 struct DecoderConfig {
18     std::int32_t hidden_size = 0;
19     std::int32_t query_heads = 0;
20     std::int32_t kv_heads = 0;
21     std::int32_t head_dim = 0;
22     std::int32_t intermediate_size = 0;
23     std::int32_t vocab_size = 0;
24     std::int32_t max_context = 0;
25     float rms_epsilon = 1.0e-5F;
26     float rope_base = 10000.0F;
27 };
28
29 struct DecoderLayerWeightsF32 {
30     std::vector<float> rms_attention_weight;
31     LinearWeightsF32 wq;
32     LinearWeightsF32 wk;
33     LinearWeightsF32 wv;
34     LinearWeightsF32 wo;
35     std::vector<float> rms_mlp_weight;
36     LinearWeightsF32 w_gate;
37     LinearWeightsF32 w_up;
38     LinearWeightsF32 w_down;
39 };
40
41 struct ReferenceDecoderModel {
42     DecoderConfig config;
43     std::vector<float> token_embedding;
```

[illegible]

```

96     void validate_address(std::int32_t layer_id,
97                           std::int32_t model_position,
98                           std::int32_t kv_head) const;
99
100     DecoderConfig config_;
101     std::int32_t layer_count_ = 0;
102     std::int32_t filled_tokens_ = 0;
103     std::vector<float> keys_;
104     std::vector<float> values_;
105     std::vector<std::uint8_t> written_;
106 };
107
108 void rms_norm(std::span<const float> input,
109               std::span<const float> weight,
110               float epsilon,
111               std::span<float> output);
112
113 void linear_f32(const LinearWeightsF32& layer,
114                 std::span<const float> input,
115                 std::span<float> output);
116
117 void apply_rope(std::span<float> heads,
118                 std::int32_t head_count,
119                 std::int32_t head_dim,
120                 std::int32_t model_position,
121                 float rope_base);
122
123 void attention_decode(const DecoderConfig& config,
124                       const ReferenceKVCache& cache,
125                       std::int32_t layer_id,
126                       std::int32_t model_position,
127                       std::span<const float> query,
128                       std::span<float> output);
129
130 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
131                                           std::span<const std::int32_t> ↵
132                                           ↵ token_ids);
133
134 ReferenceDecodeTraceResult run_reference_decode_with_trace(
135     const ReferenceDecoderModel& model,
136     std::span<const std::int32_t> token_ids);
137
138 ReferenceDecoderModel make_tiny_reference_decoder_model();
139 } // namespace lcqi

```

GPT-2 reference 头文件定义 GPT-2 配置、byte-level BPE tokenizer、HuggingFace safetensors 加载、forward、cached forward 和 greedy generation。它单独存在，是因为 GPT-2 使用 LayerNorm、绝对位置嵌入、packed QKV、GELU 和 tied lm head，不能直接塞进 LLaMA-like reference decoder。

Listing 3.7 include/lcqi/gpt2_reference.hpp

```

1 #pragma once
2
3 #include <stdint>
4 #include <filesystem>
5 #include <memory>
6 #include <span>
7 #include <string>
8 #include <string_view>
9 #include <unordered_map>
10 #include <vector>
11
12 namespace lcqi {
13
14 struct Gpt2Config {
15     std::int32_t hidden_size = 0;
16     std::int32_t head_count = 0;
17     std::int32_t layer_count = 0;
18     std::int32_t max_positions = 0;
19     std::int32_t vocab_size = 0;
20     std::int32_t intermediate_size = 0;
21     std::int32_t bos_token_id = -1;
22     std::int32_t eos_token_id = -1;
23     float layer_norm_epsilon = 1.0e-5F;
24     std::string activation_function = "gelu_new";
25 };
26
27 struct Gpt2LinearF32 {
28     std::int32_t input_size = 0;
29     std::int32_t output_size = 0;
30     std::vector<float> weights;
31     std::vector<float> bias;
32 };
33
34 struct Gpt2LayerWeightsF32 {
35     std::vector<float> ln_1_weight;
36     std::vector<float> ln_1_bias;
37     Gpt2LinearF32 c_attn;
38     Gpt2LinearF32 c_proj;
39     std::vector<float> ln_2_weight;
40     std::vector<float> ln_2_bias;
41     Gpt2LinearF32 c_fc;
42     Gpt2LinearF32 mlp_c_proj;
43 };
44
45 struct Gpt2ReferenceModel {
46     Gpt2Config config;
47     std::vector<float> token_embedding;
48     std::vector<float> position_embedding;
49     std::vector<Gpt2LayerWeightsF32> layers;
50     std::vector<float> final_ln_weight;
51     std::vector<float> final_ln_bias;

```

```

52     std::vector<float> lm_head_weight;
53     bool tie_lm_head_to_embedding = true;
54 };
55
56 struct Gpt2ForwardResult {
57     std::vector<float> logits;
58     std::int32_t predicted_token = 0;
59 };
60
61 struct Gpt2HotspotProfile {
62     std::int64_t decoder_steps = 0;
63     std::int64_t layer_steps = 0;
64     double total_step_ms = 0.0;
65     double embedding_ms = 0.0;
66     double layer_norm_ms = 0.0;
67     double final_norm_ms = 0.0;
68     double qkv_projection_ms = 0.0;
69     double kv_append_ms = 0.0;
70     double attention_ms = 0.0;
71     double attention_projection_ms = 0.0;
72     double residual_add_ms = 0.0;
73     double mlp_fc_ms = 0.0;
74     double gelu_ms = 0.0;
75     double mlp_projection_ms = 0.0;
76     double lm_head_ms = 0.0;
77     double logits_result_ms = 0.0;
78 };
79
80 struct Gpt2ExecutionOptions {
81     std::int32_t worker_count = 0;
82     std::int32_t parallel_min_rows = 512;
83 };
84
85 class Gpt2KvCache;
86
87 namespace detail {
88     class Gpt2ParallelWorkerPool;
89
90     void attend_cached_position(const Gpt2KvCache& cache,
91                               std::int32_t layer_id,
92                               std::int32_t model_position,
93                               std::span<const float> query,
94                               std::span<float> scores,
95                               std::span<float> output);
96 } // namespace detail
97
98 class Gpt2ForwardWorkspace {
99 public:
100     explicit Gpt2ForwardWorkspace(const Gpt2Config& config);
101
102     void reset_for_config(const Gpt2Config& config);
103

```

[illegible]

```

156         std::int32_t model_position,
157         std::span<const float> query,
158         std::span<float> scores,
159         std::span<float> output);
160
161     [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
162                                           std::int32_t model_position,
163                                           std::int32_t head) const;
164     [[nodiscard]] std::size_t base_offset_unchecked(std::int32_t layer_id,
165                                                    std::int32_t model_position↵
166     ↵ ,
167                                                    std::int32_t head) const ↵
168     ↵ noexcept;
169     [[nodiscard]] std::span<const float> key_unchecked(std::int32_t layer_id,
170                                                       std::int32_t ↵
171     ↵ model_position,
172                                                       std::int32_t head) const↵
173     ↵ noexcept;
174     [[nodiscard]] std::span<const float> value_unchecked(std::int32_t layer_id,
175                                                         std::int32_t ↵
176     ↵ model_position,
177                                                         std::int32_t head) ↵
178     ↵ const noexcept;
179     void validate_address(std::int32_t layer_id,
180                          std::int32_t model_position,
181                          std::int32_t head) const;
182     void validate_written(std::int32_t layer_id,
183                          std::int32_t model_position) const;
184
185     Gpt2Config config_;
186     std::int32_t filled_tokens_ = 0;
187     std::vector<float> keys_;
188     std::vector<float> values_;
189     std::vector<std::uint8_t> written_;
190 };
191
192 class Gpt2CachedGreedyDecoder {
193 public:
194     explicit Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model);
195     Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model,
196                             Gpt2HotspotProfile* hotspot_profile);
197     Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model,
198                             Gpt2HotspotProfile* hotspot_profile,
199                             const Gpt2ExecutionOptions& execution_options);
200     ~Gpt2CachedGreedyDecoder();
201
202     Gpt2CachedGreedyDecoder(const Gpt2CachedGreedyDecoder&) = delete;
203     Gpt2CachedGreedyDecoder& operator=(const Gpt2CachedGreedyDecoder&) = delete↵
204     ↵ ;
205     Gpt2CachedGreedyDecoder(Gpt2CachedGreedyDecoder&&) noexcept;
206     Gpt2CachedGreedyDecoder& operator=(Gpt2CachedGreedyDecoder&&) noexcept;
207

```



```

201 void advance_without_prediction(std::int32_t token_id);
202 [[nodiscard]] std::int32_t step(std::int32_t token_id);
203 [[nodiscard]] Gpt2ForwardResult step_with_logits(std::int32_t token_id);
204 [[nodiscard]] const Gpt2KvCache& cache() const noexcept;
205 [[nodiscard]] std::int32_t filled_tokens() const noexcept;
206 [[nodiscard]] std::size_t kv_cache_bytes() const noexcept;
207 [[nodiscard]] std::int32_t worker_count() const noexcept;
208
209 private:
210     const Gpt2ReferenceModel* model_;
211     Gpt2HotspotProfile* hotspot_profile_;
212     Gpt2ExecutionOptions execution_options_;
213     Gpt2KvCache cache_;
214     Gpt2ForwardWorkspace workspace_;
215     std::unique_ptr<detail::Gpt2ParallelWorkerPool> worker_pool_;
216 };
217
218 struct Gpt2Tokenizer {
219     std::int32_t bos_token_id = -1;
220     std::int32_t eos_token_id = -1;
221     std::unordered_map<std::string, std::int32_t> vocab;
222     std::vector<std::string> id_to_token;
223     std::unordered_map<std::string, std::int32_t> bpe_ranks;
224 };
225
226 [[nodiscard]] Gpt2Tokenizer load_gpt2_tokenizer(
227     const std::filesystem::path& vocab_json,
228     const std::filesystem::path& merges_txt,
229     std::int32_t bos_token_id,
230     std::int32_t eos_token_id);
231
232 [[nodiscard]] std::vector<std::int32_t> gpt2_encode(
233     const Gpt2Tokenizer& tokenizer,
234     std::string_view text);
235
236 [[nodiscard]] std::string gpt2_decode(
237     const Gpt2Tokenizer& tokenizer,
238     std::span<const std::int32_t> token_ids);
239
240 [[nodiscard]] Gpt2Config load_gpt2_config(const std::filesystem::path& ↵
    ↵ config_json);
241
242 [[nodiscard]] Gpt2ReferenceModel load_gpt2_reference_model(
243     const std::filesystem::path& config_json,
244     const std::filesystem::path& safetensors_path);
245
246 [[nodiscard]] Gpt2ReferenceModel load_gpt2_from_directory(
247     const std::filesystem::path& model_directory);
248
249 [[nodiscard]] Gpt2ForwardResult run_gpt2_forward(
250     const Gpt2ReferenceModel& model,
251     std::span<const std::int32_t> token_ids);

```

```
252
253 [[nodiscard]] Gpt2ForwardResult run_gpt2_forward_cached(
254     const Gpt2ReferenceModel& model,
255     Gpt2KvCache& cache,
256     std::int32_t token_id);
257
258 [[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy(
259     const Gpt2ReferenceModel& model,
260     std::span<const std::int32_t> prompt_token_ids,
261     std::int32_t max_new_tokens);
262
263 [[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy_cached(
264     const Gpt2ReferenceModel& model,
265     std::span<const std::int32_t> prompt_token_ids,
266     std::int32_t max_new_tokens);
267
268 [[nodiscard]] Gpt2ReferenceModel make_tiny_gpt2_reference_model();
269
270 } // namespace lcqi
```

第 4 章

Tiny MLP、Int8 Kernel 与基础 CLI

4.1 最短闭环：从模型文件到 logits

tiny MLP 路径不是为了证明 LCQI 已经是完整大模型引擎，而是为了建立最短的、可手算的、可测试的推理闭环。读者要把模型加载、输入检查、int8 linear、ReLU、第二层 logits、argmax 和 CLI 输出连起来看。

4.2 模型加载与基础推理

model.cpp 负责解析 tiny 文本模型、权重、scale、bias 和 shape。坏格式、坏维度和坏数值都应该在这里被拒绝，不能拖到 kernel 内部才崩溃。

Listing 4.1 src/model.cpp

```
1 #include <lcqi/model.hpp>
2
3 #include <fstream>
4 #include <limits>
5 #include <stdexcept>
6 #include <string>
7
8 namespace lcqi {
9 namespace {
10
11 constexpr std::int32_t LCQI_INT8_MIN_VALUE = -128;
12 constexpr std::int32_t LCQI_INT8_MAX_VALUE = 127;
13
14 std::string read_key(std::istream& input) {
15     std::string key;
16     if (!(input >> key)) {
17         throw std::runtime_error("unexpected end of model file");
18     }
19     return key;
20 }
21
22 void expect_key(std::istream& input, const std::string& expected) {
23     const std::string key = read_key(input);
24     if (key != expected) {
25         throw std::runtime_error("expected key '" + expected + "', got '" + key +
26     ↪ + "'");
27     }
28 }
29
30 std::int32_t read_i32(std::istream& input, const std::string& key) {
31     expect_key(input, key);
32     std::int32_t value = 0;
33     if (!(input >> value)) {
34         throw std::runtime_error("invalid int32 value for key '" + key + "'");
35     }
36 }
```

```

34     }
35     return value;
36 }
37
38 float read_float(std::istream& input, const std::string& key) {
39     expect_key(input, key);
40     float value = 0.0F;
41     if (!(input >> value)) {
42         throw std::runtime_error("invalid float value for key '" + key + "'");
43     }
44     return value;
45 }
46
47 std::vector<std::int8_t> read_i8_vector(std::istream& input,
48                                         const std::string& key,
49                                         std::int32_t count) {
50     expect_key(input, key);
51     if (count < 0) {
52         throw std::runtime_error("negative vector size for key '" + key + "'");
53     }
54     std::vector<std::int8_t> values(static_cast<std::size_t>(count));
55     for (std::int32_t i = 0; i < count; ++i) {
56         std::int32_t raw = 0;
57         if (!(input >> raw)) {
58             throw std::runtime_error("invalid int8 payload for key '" + key + "'");
59         }
60         if (raw < LCQI_INT8_MIN_VALUE || raw > LCQI_INT8_MAX_VALUE) {
61             throw std::runtime_error("int8 payload out of range for key '" +
62                                     key + "'");
63         }
64         values[static_cast<std::size_t>(i)] = static_cast<std::int8_t>(raw);
65     }
66     return values;
67 }
68
69 std::vector<float> read_float_vector(std::istream& input,
70                                       const std::string& key,
71                                       std::int32_t count) {
72     expect_key(input, key);
73     if (count < 0) {
74         throw std::runtime_error("negative vector size for key '" + key + "'");
75     }
76     std::vector<float> values(static_cast<std::size_t>(count));
77     for (std::int32_t i = 0; i < count; ++i) {
78         if (!(input >> values[static_cast<std::size_t>(i)])) {
79             throw std::runtime_error("invalid float payload for key '" + key + "'");
80         }
81     }
82     return values;
83 }

```

```

83
84 void validate_layer(const QuantizedLinearLayer& layer, const std::string& name)↵
    ↵ {
85     if (layer.input_size <= 0 || layer.output_size <= 0) {
86         throw std::runtime_error(name + " layer has invalid shape");
87     }
88     const std::size_t expected_weights =
89         static_cast<std::size_t>(layer.input_size) *
90         static_cast<std::size_t>(layer.output_size);
91     if (layer.weights.size() != expected_weights) {
92         throw std::runtime_error(name + " layer weight size mismatch");
93     }
94     if (layer.bias.size() != static_cast<std::size_t>(layer.output_size)) {
95         throw std::runtime_error(name + " layer bias size mismatch");
96     }
97 }
98
99 std::int32_t checked_element_count(std::int32_t rows,
100                                   std::int32_t columns,
101                                   const std::string& name) {
102     if (rows <= 0 || columns <= 0) {
103         throw std::runtime_error(name + " layer has invalid shape");
104     }
105     const std::int64_t count =
106         static_cast<std::int64_t>(rows) * static_cast<std::int64_t>(columns);
107     if (count > static_cast<std::int64_t>(std::numeric_limits<std::int32_t>::↵
    ↵ max())) {
108         throw std::runtime_error(name + " layer element count is too large");
109     }
110     return static_cast<std::int32_t>(count);
111 }
112
113 } // namespace
114
115 TinyMlpModel load_model(const std::filesystem::path& path) {
116     std::ifstream input(path);
117     if (!input) {
118         throw std::runtime_error("cannot open model file: " + path.string());
119     }
120
121     expect_key(input, "LCQI_MODEL_V1");
122
123     TinyMlpModel model;
124     model.input_size = read_i32(input, "input_size");
125     model.hidden_size = read_i32(input, "hidden_size");
126     model.output_size = read_i32(input, "output_size");
127
128     const std::int32_t hidden_weight_count =
129         checked_element_count(model.input_size, model.hidden_size, "hidden");
130     const std::int32_t output_weight_count =
131         checked_element_count(model.hidden_size, model.output_size, "output");
132

```

```

133     model.hidden.input_size = model.input_size;
134     model.hidden.output_size = model.hidden_size;
135     model.hidden.scale = read_float(input, "hidden_scale");
136     model.hidden.weights = read_i8_vector(
137         input,
138         "hidden_weights",
139         hidden_weight_count);
140     model.hidden.bias = read_float_vector(input, "hidden_bias", model.hidden_size);
141
142     model.output.input_size = model.hidden_size;
143     model.output.output_size = model.output_size;
144     model.output.scale = read_float(input, "output_scale");
145     model.output.weights = read_i8_vector(
146         input,
147         "output_weights",
148         output_weight_count);
149     model.output.bias = read_float_vector(input, "output_bias", model.output_size);
150
151     validate_layer(model.hidden, "hidden");
152     validate_layer(model.output, "output");
153     return model;
154 }
155
156 } // namespace lcqi

```

`inference.cpp` 执行 int8 linear、ReLU、第二层 logits 和 predicted class。它是系统中最短的“模型资产到输出”路径。

Listing 4.2 src/inference.cpp

```

1  #include <lcqi/inference.hpp>
2
3  #include <algorithm>
4  #include <cmath>
5  #include <stdexcept>
6
7  namespace lcqi {
8  namespace {
9
10 void validate_input_size(std::span<const float> input, std::int32_t expected) {
11     if (input.size() != static_cast<std::size_t>(expected)) {
12         throw std::runtime_error("input size does not match model shape");
13     }
14 }
15
16 std::int32_t argmax(std::span<const float> values) {
17     if (values.empty()) {
18         throw std::runtime_error("cannot take argmax of empty logits");
19     }
20     std::int32_t best_index = 0;
21     float best_value = values[0];

```

```

22     for (std::int32_t i = 1; i < static_cast<std::int32_t>(values.size()); ++i) ←
    ↪ {
23         const float candidate = values[static_cast<std::size_t>(i)];
24         if (candidate > best_value) {
25             best_value = candidate;
26             best_index = i;
27         }
28     }
29     return best_index;
30 }
31
32 } // namespace
33
34 void linear_i8(const QuantizedLinearLayer& layer,
35               std::span<const float> input,
36               std::span<float> output) {
37     if (input.size() != static_cast<std::size_t>(layer.input_size)) {
38         throw std::runtime_error("linear_i8 input size mismatch");
39     }
40     if (output.size() != static_cast<std::size_t>(layer.output_size)) {
41         throw std::runtime_error("linear_i8 output size mismatch");
42     }
43
44     for (std::int32_t out = 0; out < layer.output_size; ++out) {
45         const std::size_t row_base =
46             static_cast<std::size_t>(out) * static_cast<std::size_t>(layer. ←
    ↪ input_size);
47         float acc = layer.bias[static_cast<std::size_t>(out)];
48         for (std::int32_t in = 0; in < layer.input_size; ++in) {
49             const std::int8_t weight =
50                 layer.weights[row_base + static_cast<std::size_t>(in)];
51             acc += static_cast<float>(weight) * layer.scale *
52                 input[static_cast<std::size_t>(in)];
53         }
54         output[static_cast<std::size_t>(out)] = acc;
55     }
56 }
57
58 void relu_inplace(std::span<float> values) {
59     for (float& value : values) {
60         value = std::max(0.0F, value);
61     }
62 }
63
64 InferenceResult run_inference(const TinyMlpModel& model,
65                               std::span<const float> input) {
66     validate_input_size(input, model.input_size);
67
68     std::vector<float> hidden(static_cast<std::size_t>(model.hidden_size), 0.0F ←
    ↪ );
69     std::vector<float> logits(static_cast<std::size_t>(model.output_size), 0.0F ←
    ↪ );

```



```

70
71     linear_i8(model.hidden, input, hidden);
72     relu_inplace(hidden);
73     linear_i8(model.output, hidden, logits);
74
75     InferenceResult result;
76     result.predicted_class = argmax(logits);
77     result.logits = std::move(logits);
78     return result;
79 }
80
81 } // namespace lcqi

```

`main.cpp` 把模型路径和输入接到 `run_inference`，并打印结果。CLI 应保持薄层，不复制模型解析和推理逻辑。

Listing 4.3 `src/main.cpp`

```

1  #include <lcqi/inference.hpp>
2  #include <lcqi/model.hpp>
3
4  #include <chrono>
5  #include <cstdint>
6  #include <cstdlib>
7  #include <filesystem>
8  #include <iostream>
9  #include <span>
10 #include <stdexcept>
11 #include <string>
12 #include <vector>
13
14 namespace {
15
16 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 1000;
17 constexpr int LCQI_EXPECTED_ARGC_MIN = 2;
18 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
19
20 std::vector<float> parse_input(int argc, char** argv, std::int32_t ↵
    ↵ expected_size) {
21     if (argc < LCQI_EXPECTED_ARGC_MIN + expected_size) {
22         throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats...>↵
    ↵ [repeat]");
23     }
24     std::vector<float> input(static_cast<std::size_t>(expected_size));
25     for (std::int32_t i = 0; i < expected_size; ++i) {
26         input[static_cast<std::size_t>(i)] =
27             std::stof(argv[LCQI_EXPECTED_ARGC_MIN + i]);
28     }
29     return input;
30 }
31
32 std::int32_t parse_repeat(int argc, char** argv, std::int32_t input_size) {
33     const int repeat_index = LCQI_EXPECTED_ARGC_MIN + input_size;

```

```

34     if (argc <= repeat_index) {
35         return LCQI_DEFAULT_REPEAT;
36     }
37     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ repeat_index]));
38     if (repeat <= 0) {
39         throw std::runtime_error("repeat must be positive");
40     }
41     return repeat;
42 }
43
44 } // namespace
45
46 int main(int argc, char** argv) {
47     try {
48         if (argc < LCQI_EXPECTED_ARGC_MIN) {
49             throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats↵
↵ ...> [repeat]");
50         }
51
52         const std::filesystem::path model_path = argv[1];
53         const lcqi::TinyMlpModel model = lcqi::load_model(model_path);
54         const std::vector<float> input = parse_input(argc, argv, model.↵
↵ input_size);
55         const std::int32_t repeat = parse_repeat(argc, argv, model.input_size);
56
57         const lcqi::InferenceResult result = lcqi::run_inference(model, input);
58         std::cout << "predicted_class " << result.predicted_class << "\n";
59         std::cout << "logits";
60         for (const float value : result.logits) {
61             std::cout << ' ' << value;
62         }
63         std::cout << "\n";
64
65         const auto begin = std::chrono::steady_clock::now();
66         std::int32_t checksum = 0;
67         for (std::int32_t i = 0; i < repeat; ++i) {
68             checksum += lcqi::run_inference(model, input).predicted_class;
69         }
70         const auto end = std::chrono::steady_clock::now();
71         const std::chrono::duration<double> elapsed = end - begin;
72         const double average_us =
73             elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double↵
↵ >(repeat);
74         std::cout << "repeat " << repeat << "\n";
75         std::cout << "average_us " << average_us << "\n";
76         std::cout << "checksum " << checksum << "\n";
77         return EXIT_SUCCESS;
78     } catch (const std::exception& error) {
79         std::cerr << "error: " << error.what() << "\n";
80         return EXIT_FAILURE;
81     }

```

82 }

4.3 Int8 Kernel 基础实现

`int8_kernels.cpp` 包含 `scalar`、`packed`、`workspace`、`deterministic fixture` 和误差比较。它是 AVX2 路径的 `reference` 对照，也是 `benchmark` 的正确性底座。

Listing 4.4 `src/int8_kernels.cpp`

```

1  #include <lcqi/int8_kernels.hpp>
2
3  #include <lcqi/inference.hpp>
4
5  #include <algorithm>
6  #include <chrono>
7  #include <cmath>
8  #include <stdexcept>
9  #include <string>
10
11 namespace lcqi {
12 namespace {
13
14 constexpr std::int32_t LCQI_I8_PATTERN_MODULUS = 23;
15 constexpr std::int32_t LCQI_I8_PATTERN_CENTER = 11;
16 constexpr std::int32_t LCQI_INPUT_PATTERN_MODULUS = 17;
17 constexpr std::int32_t LCQI_INPUT_PATTERN_CENTER = 8;
18 constexpr float LCQI_INPUT_PATTERN_SCALE = 0.125F;
19 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
20 constexpr std::int32_t LCQI_SIMD_OUTPUT_BLOCK = 8;
21 constexpr std::size_t LCQI_BENCHMARK_BACKEND_COUNT = 3;
22 constexpr const char* LCQI_BACKEND_SCALAR = "scalar";
23 constexpr const char* LCQI_BACKEND_PACKED_SCALAR = "packed_scalar";
24 constexpr const char* LCQI_BACKEND_AVX2 = "avx2";
25
26 void require_positive(std::int32_t value, const char* name) {
27     if (value <= 0) {
28         throw std::runtime_error(std::string(name) + " must be positive");
29     }
30 }
31
32 std::size_t checked_size(std::int32_t value, const char* name) {
33     if (value < 0) {
34         throw std::runtime_error(std::string(name) + " cannot be negative");
35     }
36     return static_cast<std::size_t>(value);
37 }
38
39 void validate_quantized_layer(const QuantizedLinearLayer& layer) {
40     require_positive(layer.input_size, "input_size");
41     require_positive(layer.output_size, "output_size");
42     const std::size_t expected_weights =
43         checked_size(layer.input_size, "input_size") *

```

```

44     checked_size(layer.output_size, "output_size");
45     if (layer.weights.size() != expected_weights) {
46         throw std::runtime_error("quantized layer weight size mismatch");
47     }
48     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
49         throw std::runtime_error("quantized layer bias size mismatch");
50     }
51 }
52
53 std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
54     require_positive(value, "value");
55     require_positive(block_size, "block_size");
56     return ((value + block_size - 1) / block_size) * block_size;
57 }
58
59 float checksum(std::span<const float> values) {
60     float sum = 0.0F;
61     for (const float value : values) {
62         sum += value;
63     }
64     return sum;
65 }
66
67 void add_unavailable_backend(KernelBenchmarkResult& result, const char* name) {
68     result.backends.push_back(KernelBackendBenchmark{
69         .name = name,
70         .available = false,
71         .average_us = 0.0,
72         .max_abs_diff = 0.0F,
73         .checksum = 0.0F,
74     });
75 }
76
77 template <typename Callable>
78 double measure_average_us(std::int32_t repeat, Callable&& callable) {
79     require_positive(repeat, "repeat");
80     const auto begin = std::chrono::steady_clock::now();
81     for (std::int32_t index = 0; index < repeat; ++index) {
82         callable();
83     }
84     const auto end = std::chrono::steady_clock::now();
85     const std::chrono::duration<double> elapsed = end - begin;
86     return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
87 }
88
89 } // namespace
90
91 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
92                               std::int32_t output_block_size) {
93     validate_quantized_layer(layer);
94     require_positive(output_block_size, "output_block_size");

```

```

95
96     PackedLinearI8 packed;
97     packed.input_size = layer.input_size;
98     packed.output_size = layer.output_size;
99     packed.output_block_size = output_block_size;
100    packed.scale = layer.scale;
101    packed.bias = layer.bias;
102
103    const std::int32_t padded_outputs =
104        round_up_to_block(layer.output_size, output_block_size);
105    packed.packed_weights.assign(
106        checked_size(padded_outputs, "padded_outputs") *
107        checked_size(layer.input_size, "input_size"),
108        0);
109
110    std::size_t packed_index = 0;
111    for (std::int32_t output_block = 0; output_block < padded_outputs;
112         output_block += output_block_size) {
113        for (std::int32_t input_index = 0; input_index < layer.input_size; ++
114    ↪ input_index) {
115            for (std::int32_t offset = 0; offset < output_block_size; ++offset)
116    ↪ {
117                const std::int32_t output_index = output_block + offset;
118                if (output_index < layer.output_size) {
119                    const std::size_t source =
120                        checked_size(output_index, "output_index") *
121                        checked_size(layer.input_size, "input_size") +
122                        checked_size(input_index, "input_index");
123                    packed.packed_weights[packed_index] = layer.weights[source
124    ↪ ];
125                }
126                ++packed_index;
127            }
128        }
129    }
130    return packed;
131 }
132
133 void linear_i8_packed(const PackedLinearI8& layer,
134                     std::span<const float> input,
135                     std::span<float> output) {
136     std::vector<float> accumulators(
137         checked_size(layer.output_block_size, "output_block_size"),
138         0.0F);
139     linear_i8_packed_with_workspace(layer, input, output, accumulators);
140 }
141
142 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
143                                     std::span<const float> input,
144                                     std::span<float> output,
145                                     std::span<float> accumulator_workspace) {
146     require_positive(layer.input_size, "packed input_size");

```

```

144     require_positive(layer.output_size, "packed output_size");
145     require_positive(layer.output_block_size, "packed output_block_size");
146     if (input.size() != checked_size(layer.input_size, "input_size")) {
147         throw std::runtime_error("linear_i8_packed input size mismatch");
148     }
149     if (output.size() != checked_size(layer.output_size, "output_size")) {
150         throw std::runtime_error("linear_i8_packed output size mismatch");
151     }
152     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
153         throw std::runtime_error("linear_i8_packed bias size mismatch");
154     }
155
156     const std::int32_t padded_outputs =
157         round_up_to_block(layer.output_size, layer.output_block_size);
158     const std::size_t expected_packed_size =
159         checked_size(padded_outputs, "padded_outputs") *
160         checked_size(layer.input_size, "input_size");
161     if (layer.packed_weights.size() != expected_packed_size) {
162         throw std::runtime_error("linear_i8_packed packed weight size mismatch" ←
163     ↪ );
164     }
165     if (accumulator_workspace.size() != checked_size(layer.output_block_size, " ←
166     ↪ output_block_size")) {
167         throw std::runtime_error("linear_i8_packed accumulator workspace size ←
168     ↪ mismatch");
169     }
170
171     std::size_t packed_index = 0;
172     for (std::int32_t output_block = 0; output_block < padded_outputs;
173         output_block += layer.output_block_size) {
174         std::fill(accumulator_workspace.begin(), accumulator_workspace.end(), ←
175     ↪ 0.0F);
176         for (std::int32_t offset = 0; offset < layer.output_block_size; ++ ←
177     ↪ offset) {
178             const std::int32_t output_index = output_block + offset;
179             if (output_index < layer.output_size) {
180                 accumulator_workspace[checked_size(offset, "offset")] =
181                 layer.bias[checked_size(output_index, "output_index")];
182             }
183         }
184
185         for (std::int32_t input_index = 0; input_index < layer.input_size; ++ ←
186     ↪ input_index) {
187             const float input_value = input[checked_size(input_index, " ←
188     ↪ input_index")];
189             for (std::int32_t offset = 0; offset < layer.output_block_size; ++ ←
190     ↪ offset) {
191                 const std::int32_t output_index = output_block + offset;
192                 const std::int8_t weight = layer.packed_weights[packed_index ←
193     ↪ ++];
194                 if (output_index < layer.output_size) {
195                     accumulator_workspace[checked_size(offset, "offset")] +=

```

```

187         static_cast<float>(weight) * layer.scale * input_value;
188     }
189 }
190 }
191
192     for (std::int32_t offset = 0; offset < layer.output_block_size; ++↵
↵ offset) {
193         const std::int32_t output_index = output_block + offset;
194         if (output_index < layer.output_size) {
195             output[checked_size(output_index, "output_index")] =
196                 accumulator_workspace[checked_size(offset, "offset")];
197         }
198     }
199 }
200 }
201
202 QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
203                                                  std::int32_t output_size,
204                                                  float scale) {
205     require_positive(input_size, "input_size");
206     require_positive(output_size, "output_size");
207     QuantizedLinearLayer layer;
208     layer.input_size = input_size;
209     layer.output_size = output_size;
210     layer.scale = scale;
211     const std::size_t weight_count =
212         checked_size(input_size, "input_size") * checked_size(output_size, "↵
↵ output_size");
213     layer.weights.resize(weight_count);
214     layer.bias.resize(checked_size(output_size, "output_size"));
215     for (std::int32_t output_index = 0; output_index < output_size; ++↵
↵ output_index) {
216         layer.bias[checked_size(output_index, "output_index")] =
217             static_cast<float>((output_index % LCQI_INPUT_PATTERN_MODULUS) -
218                               LCQI_INPUT_PATTERN_CENTER) *
219             LCQI_INPUT_PATTERN_SCALE;
220         for (std::int32_t input_index = 0; input_index < input_size; ++↵
↵ input_index) {
221             const std::int32_t raw =
222                 ((output_index * input_size + input_index) % ↵
↵ LCQI_I8_PATTERN_MODULUS) -
223                 LCQI_I8_PATTERN_CENTER;
224             layer.weights[checked_size(output_index, "output_index") *
225                           checked_size(input_size, "input_size") +
226                           checked_size(input_index, "input_index")] =
227                 static_cast<std::int8_t>(raw);
228         }
229     }
230     return layer;
231 }
232
233 std::vector<float> make_deterministic_input(std::int32_t input_size) {

```

```

234     require_positive(input_size, "input_size");
235     std::vector<float> input(checkered_size(input_size, "input_size"));
236     for (std::int32_t input_index = 0; input_index < input_size; ++input_index) {
237         input[checked_size(input_index, "input_index")] =
238             static_cast<float>((input_index % LCQI_INPUT_PATTERN_MODULUS) -
239                               LCQI_INPUT_PATTERN_CENTER) *
240             LCQI_INPUT_PATTERN_SCALE;
241     }
242     return input;
243 }
244
245 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
246     if (lhs.size() != rhs.size()) {
247         throw std::runtime_error("max_abs_diff size mismatch");
248     }
249     float diff = 0.0F;
250     for (std::size_t index = 0; index < lhs.size(); ++index) {
251         diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
252     }
253     return diff;
254 }
255
256 KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase&
257     benchmark_case) {
258     require_positive(benchmark_case.input_size, "benchmark input_size");
259     require_positive(benchmark_case.output_size, "benchmark output_size");
260     require_positive(benchmark_case.output_block_size, "benchmark
261     output_block_size");
262     require_positive(benchmark_case.repeat, "benchmark repeat");
263
264     const QuantizedLinearLayer layer =
265         make_deterministic_i8_layer(benchmark_case.input_size,
266                                     benchmark_case.output_size,
267                                     0.015625F);
268     const PackedLinearI8 packed = pack_linear_i8(layer, benchmark_case.
269     output_block_size);
270     const PackedLinearI8 packed_simd = pack_linear_i8(layer,
271     LCQI_SIMD_OUTPUT_BLOCK);
272     const std::vector<float> input = make_deterministic_input(benchmark_case.
273     input_size);
274     std::vector<float> scalar_output(checked_size(benchmark_case.output_size, "
275     output_size"),
276                                     0.0F);
277     std::vector<float> packed_output(scalar_output.size(), 0.0F);
278     std::vector<float> simd_output(scalar_output.size(), 0.0F);
279     std::vector<float> packed_workspace(
280         checked_size(benchmark_case.output_block_size, "output_block_size"),
281         0.0F);
282     linear_i8(layer, input, scalar_output);
283     linear_i8_packed_with_workspace(packed, input, packed_output,

```



```

    ↪ packed_workspace);
279
280 KernelBenchmarkResult result;
281 result.benchmark_case = benchmark_case;
282 result.backends.reserve(LCQI_BENCHMARK_BACKEND_COUNT);
283 result.backends.push_back(KernelBackendBenchmark{
284     .name = LCQI_BACKEND_SCALAR,
285     .available = true,
286     .average_us = measure_average_us(benchmark_case.repeat,
287                                     [&]() { linear_i8(layer, input, ↪
    ↪ scalar_output); })),
288     .max_abs_diff = 0.0F,
289     .checksum = checksum(scalar_output),
290 });
291 result.backends.push_back(KernelBackendBenchmark{
292     .name = LCQI_BACKEND_PACKED_SCALAR,
293     .available = true,
294     .average_us = measure_average_us(
295         benchmark_case.repeat,
296         [&]() { linear_i8_packed_with_workspace(packed,
297                                                  input,
298                                                  packed_output,
299                                                  packed_workspace); })),
300     .max_abs_diff = max_abs_diff(scalar_output, packed_output),
301     .checksum = checksum(packed_output),
302 });
303 if (linear_i8_packed_avx2_available()) {
304     linear_i8_packed_avx2(packed_simd, input, simd_output);
305     result.backends.push_back(KernelBackendBenchmark{
306         .name = LCQI_BACKEND_AVX2,
307         .available = true,
308         .average_us = measure_average_us(
309             benchmark_case.repeat,
310             [&]() { linear_i8_packed_avx2(packed_simd, input, simd_output); ↪
    ↪ }),
311         .max_abs_diff = max_abs_diff(scalar_output, simd_output),
312         .checksum = checksum(simd_output),
313     });
314 } else {
315     add_unavailable_backend(result, LCQI_BACKEND_AVX2);
316 }
317 return result;
318 }
319
320 } // namespace lcqi
321
322 #if !defined(LCQI_ENABLE_AVX2)
323 namespace lcqi {
324
325 bool linear_i8_packed_avx2_available() noexcept {
326     return false;
327 }

```

```

328
329 void linear_i8_packed_avx2(const PackedLinearI8&, std::span<const float>, std::span<float>) {
330     throw std::runtime_error("AVX2 packed kernel is not enabled in this build");
331 }
332
333 } // namespace lcgqi
334 #endif

```

`int8_kernels_avx2.cpp` 是平台优化路径。读者要先看可用性判断，再看向量化实现；如果机器不支持 AVX2/FMA，报告必须写 `unavailable`。

Listing 4.5 `src/int8_kernels_avx2.cpp`

[illegible]

```

39         std::span<float> output) {
40     require_positive(layer.input_size, "packed input_size");
41     require_positive(layer.output_size, "packed output_size");
42     if (layer.output_block_size != LCQI_AVX2_OUTPUT_BLOCK) {
43         throw std::runtime_error("AVX2 packed kernel requires output_block_size ←
↵ == 8");
44     }
45     if (input.size() != checked_size(layer.input_size, "input_size")) {
46         throw std::runtime_error("AVX2 packed kernel input size mismatch");
47     }
48     if (output.size() != checked_size(layer.output_size, "output_size")) {
49         throw std::runtime_error("AVX2 packed kernel output size mismatch");
50     }
51     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
52         throw std::runtime_error("AVX2 packed kernel bias size mismatch");
53     }
54     const std::int32_t padded_outputs =
55         round_up_to_block(layer.output_size, layer.output_block_size);
56     const std::size_t expected_packed_size =
57         checked_size(padded_outputs, "padded_outputs") *
58         checked_size(layer.input_size, "input_size");
59     if (layer.packed_weights.size() != expected_packed_size) {
60         throw std::runtime_error("AVX2 packed kernel packed weight size ←
↵ mismatch");
61     }
62 }
63
64 } // namespace
65
66 bool linear_i8_packed_avx2_available() noexcept {
67     #if defined(__GNUC__) || defined(__clang__)
68         __builtin_cpu_init();
69         return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
70     #else
71         return true;
72     #endif
73 }
74
75 void linear_i8_packed_avx2(const PackedLinearI8& layer,
76     std::span<const float> input,
77     std::span<float> output) {
78     if (!linear_i8_packed_avx2_available()) {
79         throw std::runtime_error("AVX2/FMA is not available on this CPU");
80     }
81     validate_avx2_contract(layer, input, output);
82
83     const std::int32_t padded_outputs =
84         round_up_to_block(layer.output_size, layer.output_block_size);
85     std::size_t packed_index = 0;
86     alignas(32) float block_output[LCQI_AVX2_OUTPUT_BLOCK] = {};
87
88     for (std::int32_t output_block = 0; output_block < padded_outputs;

```

```

89     output_block += LCQI_AVX2_OUTPUT_BLOCK) {
90         __m256 accumulator = _mm256_setzero_ps();
91
92         for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) {
93             const std::int32_t output_index = output_block + offset;
94             block_output[checked_size(offset, "offset")] =
95                 output_index < layer.output_size
96                 ? layer.bias[checked_size(output_index, "output_index")]
97                 : 0.0F;
98         }
99         accumulator = _mm256_load_ps(block_output);
100
101         for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
102             std::uint64_t packed_bytes = 0;
103             std::memcpy(&packed_bytes,
104                 layer.packed_weights.data() + packed_index,
105                 sizeof(packed_bytes));
106             const __m128i packed_i8 =
107                 _mm_cvtsi64_si128(static_cast<long long>(packed_bytes));
108             packed_index += LCQI_AVX2_OUTPUT_BLOCK;
109             const __m256i weights_i32 = _mm256_cvtepi8_epi32(packed_i8);
110             const __m256 weights_f32 = _mm256_cvtepi32_ps(weights_i32);
111             const __m256 input_scaled =
112                 _mm256_set1_ps(input[checked_size(input_index, "input_index")] * layer.scale);
113             accumulator = _mm256_fmadd_ps(weights_f32, input_scaled, accumulator);
114         }
115
116         _mm256_store_ps(block_output, accumulator);
117         for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) {
118             const std::int32_t output_index = output_block + offset;
119             if (output_index < layer.output_size) {
120                 output[checked_size(output_index, "output_index")] =
121                     block_output[checked_size(offset, "offset")];
122             }
123         }
124     }
125 }
126
127 } // namespace lcqi
128
129 #endif

```

第 5 章

Reference Decoder、GPT-2 与模型导入

5.1 从 tiny MLP 走向 decoder-only 模型

Reference decoder 和 GPT-2 reference path 是 LCQI 从教学闭环走向真实模型导入的关键层。这里的代码更长，是因为真实模型不只有矩阵乘法：它还需要 tokenizer、位置、归一化、QKV 拆分、attention mask、KV cache、激活函数、lm head、权重命名映射、dtype 转换和坏资产拒绝。

5.2 Tiny Reference Decoder

reference_decoder.cpp 实现 embedding、RMSNorm、Q/K/V、RoPE、KV cache、attention、SwiGLU、final norm、logits 和 sampler，并在关键位置生成 trace checkpoint。读者应按 checkpoint 顺序读，而不是从最终 logits 倒推。

Listing 5.1 src/reference_decoder.cpp

```
1 #include <lcqi/reference_decoder.hpp>
2
3 #include <algorithm>
4 #include <array>
5 #include <cmath>
6 #include <limits>
7 #include <stdexcept>
8 #include <string>
9
10 namespace lcqi {
11 namespace {
12
13 constexpr float LCQI_SOFTMAX_NEGATIVE_INFINITY = -std::numeric_limits<float>::infinity();
14 constexpr std::int32_t LCQI_ROPE_PAIR_WIDTH = 2;
15 constexpr const char* LCQI_TRACE_DTYPE_F32 = "f32";
16 constexpr const char* LCQI_TRACE_LAYOUT_CONTIGUOUS = "contiguous";
17 constexpr const char* LCQI_TRACE_LAYOUT_ROW_MAJOR = "row_major";
18 constexpr const char* LCQI_TRACE_LAYOUT_KV_CONTIGUOUS = "kv_contiguous";
19
20 void require_positive(std::int32_t value, const char* name) {
21     if (value <= 0) {
22         throw std::runtime_error(std::string(name) + " must be positive");
23     }
24 }
25
26 void validate_config(const DecoderConfig& config) {
27     require_positive(config.hidden_size, "hidden_size");
28     require_positive(config.query_heads, "query_heads");
29     require_positive(config.kv_heads, "kv_heads");
```

```

30     require_positive(config.head_dim, "head_dim");
31     require_positive(config.intermediate_size, "intermediate_size");
32     require_positive(config.vocab_size, "vocab_size");
33     require_positive(config.max_context, "max_context");
34     if (config.query_heads % config.kv_heads != 0) {
35         throw std::runtime_error("query_heads must be divisible by kv_heads");
36     }
37     if (config.hidden_size != config.query_heads * config.head_dim) {
38         throw std::runtime_error("hidden_size must equal query_heads * head_dim↵
↵ ");
39     }
40     if (config.head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
41         throw std::runtime_error("head_dim must be even for RoPE");
42     }
43     if (config.rope_base <= 1.0F) {
44         throw std::runtime_error("rope_base must be greater than 1");
45     }
46 }
47
48 std::size_t checked_size(std::int32_t value, const char* name) {
49     if (value < 0) {
50         throw std::runtime_error(std::string(name) + " cannot be negative");
51     }
52     return static_cast<std::size_t>(value);
53 }
54
55 std::int32_t kv_width(const DecoderConfig& config) {
56     return config.kv_heads * config.head_dim;
57 }
58
59 void validate_linear(const LinearWeightsF32& layer, const char* name) {
60     require_positive(layer.input_size, "linear input_size");
61     require_positive(layer.output_size, "linear output_size");
62     const std::size_t expected_weights =
63         checked_size(layer.input_size, name) * checked_size(layer.output_size, ↵
↵ name);
64     if (layer.weights.size() != expected_weights) {
65         throw std::runtime_error(std::string(name) + " weight size mismatch");
66     }
67     if (!layer.bias.empty() &&
68         layer.bias.size() != checked_size(layer.output_size, name)) {
69         throw std::runtime_error(std::string(name) + " bias size mismatch");
70     }
71 }
72
73 void validate_span_size(std::size_t actual, std::int32_t expected, const char* ↵
↵ name) {
74     if (actual != checked_size(expected, name)) {
75         throw std::runtime_error(std::string(name) + " size mismatch");
76     }
77 }
78

```

```

79 std::int32_t argmax(std::span<const float> values) {
80     if (values.empty()) {
81         throw std::runtime_error("cannot take argmax of empty values");
82     }
83     std::int32_t best_index = 0;
84     float best_value = values[0];
85     for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()
↵ )); ++index) {
86         const float candidate = values[static_cast<std::size_t>(index)];
87         if (candidate > best_value) {
88             best_value = candidate;
89             best_index = index;
90         }
91     }
92     return best_index;
93 }
94
95 float silu(float value) {
96     return value / (1.0F + std::exp(-value));
97 }
98
99 std::span<const float> row_span(std::span<const float> values,
100                                std::int32_t row,
101                                std::int32_t row_width) {
102     const std::size_t begin = checked_size(row, "row") * checked_size(row_width
↵ , "row_width");
103     return values.subspan(begin, checked_size(row_width, "row_width"));
104 }
105
106 void add_inplace(std::span<float> target, std::span<const float> source) {
107     if (target.size() != source.size()) {
108         throw std::runtime_error("residual add size mismatch");
109     }
110     for (std::size_t index = 0; index < target.size(); ++index) {
111         target[index] += source[index];
112     }
113 }
114
115 void swiglu_mlp(const DecoderLayerWeightsF32& layer,
116                std::span<const float> input,
117                std::span<float> output) {
118     std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0
↵ F);
119     std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
120     std::vector<float> hidden(gate.size(), 0.0F);
121     linear_f32(layer.w_gate, input, gate);
122     linear_f32(layer.w_up, input, up);
123     if (gate.size() != up.size()) {
124         throw std::runtime_error("SwiGLU gate/up size mismatch");
125     }
126     for (std::size_t index = 0; index < hidden.size(); ++index) {
127         hidden[index] = silu(gate[index]) * up[index];

```

```

128     }
129     linear_f32(layer.w_down, hidden, output);
130 }
131
132 float checksum(std::span<const float> values) {
133     float sum = 0.0F;
134     for (const float value : values) {
135         sum += value;
136     }
137     return sum;
138 }
139
140 float max_abs(std::span<const float> values) {
141     float result = 0.0F;
142     for (const float value : values) {
143         result = std::max(result, std::fabs(value));
144     }
145     return result;
146 }
147
148 std::vector<std::int32_t> contiguous_stride(std::span<const std::int32_t> shape ←
    ↪ ) {
149     std::vector<std::int32_t> stride(shape.size(), 1);
150     std::int32_t running = 1;
151     for (std::int32_t index = static_cast<std::int32_t>(shape.size()) - 1; ←
    ↪ index >= 0; --index) {
152         stride[checked_size(index, "stride index")] = running;
153         running *= shape[checked_size(index, "shape index")];
154     }
155     return stride;
156 }
157
158 void add_checkpoint(std::vector<ReferenceTensorCheckpoint>* checkpoints,
159                    const char* name,
160                    std::int32_t token_position,
161                    std::int32_t layer_id,
162                    std::span<const std::int32_t> shape,
163                    const char* layout,
164                    std::span<const float> values) {
165     if (checkpoints == nullptr) {
166         return;
167     }
168     ReferenceTensorCheckpoint checkpoint;
169     checkpoint.name = name;
170     checkpoint.token_position = token_position;
171     checkpoint.layer_id = layer_id;
172     checkpoint.shape.assign(shape.begin(), shape.end());
173     checkpoint.stride = contiguous_stride(shape);
174     checkpoint.dtype = LCQI_TRACE_DTYPE_F32;
175     checkpoint.layout = layout;
176     checkpoint.checksum = checksum(values);
177     checkpoint.max_abs = max_abs(values);

```



```

178     checkpoint.values.assign(values.begin(), values.end());
179     checkpoints->push_back(std::move(checkpoint));
180 }
181
182 void swiglu_mlp_with_trace(const DecoderLayerWeightsF32& layer,
183                           std::span<const float> input,
184                           std::span<float> output,
185                           std::int32_t token_position,
186                           std::int32_t layer_id,
187                           std::vector<ReferenceTensorCheckpoint>* checkpoints)↵
188     ↵ {
189     std::vector<float> gate(check_size(layer.w_gate.output_size, "gate"), 0.0↵
190     ↵ F);
191     std::vector<float> up(check_size(layer.w_up.output_size, "up"), 0.0F);
192     std::vector<float> hidden(gate.size(), 0.0F);
193     linear_f32(layer.w_gate, input, gate);
194     linear_f32(layer.w_up, input, up);
195     if (gate.size() != up.size()) {
196         throw std::runtime_error("SwiGLU gate/up size mismatch");
197     }
198     const std::array<std::int32_t, 1> intermediate_shape{layer.w_gate.↵
199     ↵ output_size};
200     add_checkpoint(checkpoints,
201                   "mlp_gate",
202                   token_position,
203                   layer_id,
204                   intermediate_shape,
205                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
206                   gate);
207     add_checkpoint(checkpoints,
208                   "mlp_up",
209                   token_position,
210                   layer_id,
211                   intermediate_shape,
212                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
213                   up);
214     for (std::size_t index = 0; index < hidden.size(); ++index) {
215         hidden[index] = silu(gate[index]) * up[index];
216     }
217     add_checkpoint(checkpoints,
218                   "mlp_hidden",
219                   token_position,
220                   layer_id,
221                   intermediate_shape,
222                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
223                   hidden);
224     linear_f32(layer.w_down, hidden, output);
225 }
226
227 void forward_layer(const DecoderConfig& config,
228                   const DecoderLayerWeightsF32& layer,
229                   std::int32_t layer_id,

```

```

227         std::int32_t model_position,
228         ReferenceKVCache& cache,
229         std::span<float> hidden,
230         std::vector<ReferenceTensorCheckpoint>* checkpoints = ↵
↵ nullptr) {
231     validate_span_size(hidden.size(), config.hidden_size, "hidden");
232
233     std::vector<float> normed(check_size(config.hidden_size, "hidden_size"), ↵
↵ 0.0F);
234     std::vector<float> query(check_size(config.hidden_size, "hidden_size"), ↵
↵ 0.0F);
235     std::vector<float> key(check_size(kv_width(config), "kv_width"), 0.0F);
236     std::vector<float> value(check_size(kv_width(config), "kv_width"), 0.0F);
237     std::vector<float> attention(check_size(config.hidden_size, "hidden_size" ↵
↵ ), 0.0F);
238     std::vector<float> attention_projected(check_size(config.hidden_size, " ↵
↵ hidden_size"), 0.0F);
239     std::vector<float> mlp(check_size(config.hidden_size, "hidden_size"), 0.0 ↵
↵ F);
240
241     const std::array<std::int32_t, 1> hidden_shape{config.hidden_size};
242     const std::array<std::int32_t, 2> query_shape{config.query_heads, config. ↵
↵ head_dim};
243     const std::array<std::int32_t, 2> kv_shape{config.kv_heads, config.head_dim ↵
↵ };
244     const std::array<std::int32_t, 4> kv_slot_shape{
245         1, 1, config.kv_heads, config.head_dim};
246
247     rms_norm(hidden, layer.rms_attention_weight, config.rms_epsilon, normed);
248     add_checkpoint(checkpoints,
249         "attn_norm",
250         model_position,
251         layer_id,
252         hidden_shape,
253         LCQI_TRACE_LAYOUT_CONTIGUOUS,
254         normed);
255     linear_f32(layer.wq, normed, query);
256     linear_f32(layer.wk, normed, key);
257     linear_f32(layer.wv, normed, value);
258     add_checkpoint(checkpoints,
259         "q_before_rope",
260         model_position,
261         layer_id,
262         query_shape,
263         LCQI_TRACE_LAYOUT_CONTIGUOUS,
264         query);
265     add_checkpoint(checkpoints,
266         "k_before_rope",
267         model_position,
268         layer_id,
269         kv_shape,
270         LCQI_TRACE_LAYOUT_CONTIGUOUS,

```

```

271         key);
272     add_checkpoint(checkpoints,
273         "v",
274         model_position,
275         layer_id,
276         kv_shape,
277         LCQI_TRACE_LAYOUT_CONTIGUOUS,
278         value);
279     apply_rope(query, config.query_heads, config.head_dim, model_position, ↵
↵ config.rope_base);
280     apply_rope(key, config.kv_heads, config.head_dim, model_position, config.↵
↵ rope_base);
281     add_checkpoint(checkpoints,
282         "q_after_rope",
283         model_position,
284         layer_id,
285         query_shape,
286         LCQI_TRACE_LAYOUT_CONTIGUOUS,
287         query);
288     add_checkpoint(checkpoints,
289         "k_after_rope",
290         model_position,
291         layer_id,
292         kv_shape,
293         LCQI_TRACE_LAYOUT_CONTIGUOUS,
294         key);
295
296     cache.append(layer_id, model_position, key, value);
297     add_checkpoint(checkpoints,
298         "kv_cache_key_slot",
299         model_position,
300         layer_id,
301         kv_slot_shape,
302         LCQI_TRACE_LAYOUT_KV_CONTIGUOUS,
303         key);
304     attention_decode(config, cache, layer_id, model_position, query, attention)↵
↵ ;
305     add_checkpoint(checkpoints,
306         "attention_out",
307         model_position,
308         layer_id,
309         hidden_shape,
310         LCQI_TRACE_LAYOUT_CONTIGUOUS,
311         attention);
312     linear_f32(layer.wo, attention, attention_projected);
313     add_inplace(hidden, attention_projected);
314     add_checkpoint(checkpoints,
315         "post_attention_residual",
316         model_position,
317         layer_id,
318         hidden_shape,
319         LCQI_TRACE_LAYOUT_CONTIGUOUS,

```

```

320         hidden);
321
322     rms_norm(hidden, layer.rms_mlp_weight, config.rms_epsilon, normed);
323     add_checkpoint(checkpoints,
324                   "mlp_norm",
325                   model_position,
326                   layer_id,
327                   hidden_shape,
328                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
329                   normed);
330     if (checkpoints == nullptr) {
331         swiglu_mlp(layer, normed, mlp);
332     } else {
333         swiglu_mlp_with_trace(layer, normed, mlp, model_position, layer_id, ←
↵ checkpoints);
334     }
335     add_inplace(hidden, mlp);
336     add_checkpoint(checkpoints,
337                   "layer_out",
338                   model_position,
339                   layer_id,
340                   hidden_shape,
341                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
342                   hidden);
343 }
344
345 LinearWeightsF32 make_linear(std::int32_t input_size,
346                             std::int32_t output_size,
347                             std::vector<float> weights,
348                             std::vector<float> bias = {}) {
349     LinearWeightsF32 layer;
350     layer.input_size = input_size;
351     layer.output_size = output_size;
352     layer.weights = std::move(weights);
353     layer.bias = std::move(bias);
354     validate_linear(layer, "tiny linear");
355     return layer;
356 }
357
358 std::vector<float> zeros(std::int32_t count) {
359     return std::vector<float>(checked_size(count, "zero count"), 0.0F);
360 }
361
362 } // namespace
363
364 ReferenceDecodeTraceResult run_reference_decode_with_trace(
365     const ReferenceDecoderModel& model,
366     std::span<const std::int32_t> token_ids) {
367     validate_config(model.config);
368     if (token_ids.empty()) {
369         throw std::runtime_error("reference decode needs at least one token");
370     }

```

```

371     if (token_ids.size() > checked_size(model.config.max_context, "max_context") {
372         throw std::runtime_error("token_ids exceed max_context");
373     }
374     if (model.token_embedding.size() !=
375         checked_size(model.config.vocab_size, "vocab_size") *
376         checked_size(model.config.hidden_size, "hidden_size")) {
377         throw std::runtime_error("token embedding size mismatch");
378     }
379     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size,
380     ↪ , "hidden_size")) {
381         throw std::runtime_error("final norm weight size mismatch");
382     }
383     validate_linear(model.lm_head, "lm_head");
384
385     ReferenceDecodeTraceResult trace;
386     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers
387     ↪ .size()));
388     std::vector<float> hidden(checked_size(model.config.hidden_size, "
389     ↪ hidden_size"), 0.0F);
390     const std::array<std::int32_t, 1> hidden_shape{model.config.hidden_size};
391     for (std::int32_t position = 0; position < static_cast<std::int32_t>(
392     ↪ token_ids.size());
393         ++position) {
394         const std::int32_t token_id = token_ids[checked_size(position, "
395     ↪ position)];
396         if (token_id < 0 || token_id >= model.config.vocab_size) {
397             throw std::runtime_error("token id out of vocabulary");
398         }
399         const std::span<const float> embedding =
400         ↪ row_span(model.token_embedding, token_id, model.config.hidden_size);
401         std::copy(embedding.begin(), embedding.end(), hidden.begin());
402         add_checkpoint(&trace.checkpoints,
403             "embedding",
404             position,
405             -1,
406             hidden_shape,
407             LCQI_TRACE_LAYOUT_ROW_MAJOR,
408             hidden);
409         for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(
410     ↪ model.layers.size());
411             ++layer_id) {
412             forward_layer(model.config,
413                 model.layers[checked_size(layer_id, "layer_id")],
414                 layer_id,
415                 position,
416                 cache,
417                 hidden,
418                 &trace.checkpoints);
419         }
420     }

```



```

462         std::span<const float> key,
463         std::span<const float> value) {
464     validate_span_size(key.size(), kv_width(this->config_), "KV key");
465     validate_span_size(value.size(), kv_width(this->config_), "KV value");
466     this->validate_address(layer_id, model_position, 0);
467     for (std::int32_t kv_head = 0; kv_head < this->config_.kv_heads; ++kv_head) {
468         const std::size_t target = this->base_offset(layer_id, model_position,
469         kv_head);
470         const std::size_t source =
471             checked_size(kv_head, "kv_head") * checked_size(this->config_.
472             head_dim, "head_dim");
473         std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
474                     this->config_.head_dim,
475                     this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
476         std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
477                     this->config_.head_dim,
478                     this->values_.begin() + static_cast<std::ptrdiff_t>(target));
479     }
480     const std::size_t written_index =
481         checked_size(layer_id, "layer_id") * checked_size(this->config_.
482         max_context, "max_context") +
483         checked_size(model_position, "model_position");
484     this->written_[written_index] = 1;
485     this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
486 }
487
488 std::span<const float> ReferenceKVCache::key(std::int32_t layer_id,
489                                             std::int32_t model_position,
490                                             std::int32_t kv_head) const {
491     this->validate_address(layer_id, model_position, kv_head);
492     const std::size_t offset = this->base_offset(layer_id, model_position,
493     kv_head);
494     return std::span<const float>(this->keys_.data() + offset,
495                                   checked_size(this->config_.head_dim, "
496     head_dim"));
497 }
498
499 std::span<const float> ReferenceKVCache::value(std::int32_t layer_id,
500                                              std::int32_t model_position,
501                                              std::int32_t kv_head) const {
502     this->validate_address(layer_id, model_position, kv_head);
503     const std::size_t offset = this->base_offset(layer_id, model_position,
504     kv_head);
505     return std::span<const float>(this->values_.data() + offset,
506                                   checked_size(this->config_.head_dim, "
507     head_dim"));
508 }
509
510 std::int32_t ReferenceKVCache::layer_count() const noexcept {
511     return this->layer_count_;

```

```

505 }
506
507 std::int32_t ReferenceKVCache::filled_tokens() const noexcept {
508     return this->filled_tokens_;
509 }
510
511 std::size_t ReferenceKVCache::base_offset(std::int32_t layer_id,
512                                           std::int32_t model_position,
513                                           std::int32_t kv_head) const {
514     return (((checked_size(layer_id, "layer_id") *
515                checked_size(this->config.max_context, "max_context")) +
516             checked_size(model_position, "model_position")) *
517            checked_size(this->config.kv_heads, "kv_heads") +
518            checked_size(kv_head, "kv_head")) *
519            checked_size(this->config.head_dim, "head_dim");
520 }
521
522 void ReferenceKVCache::validate_address(std::int32_t layer_id,
523                                         std::int32_t model_position,
524                                         std::int32_t kv_head) const {
525     if (layer_id < 0 || layer_id >= this->layer_count_) {
526         throw std::runtime_error("KV layer_id out of range");
527     }
528     if (model_position < 0 || model_position >= this->config.max_context) {
529         throw std::runtime_error("KV model_position out of range");
530     }
531     if (kv_head < 0 || kv_head >= this->config.kv_heads) {
532         throw std::runtime_error("KV head out of range");
533     }
534 }
535
536 void rms_norm(std::span<const float> input,
537              std::span<const float> weight,
538              float epsilon,
539              std::span<float> output) {
540     if (input.size() != weight.size() || input.size() != output.size()) {
541         throw std::runtime_error("RMSNorm size mismatch");
542     }
543     if (input.empty()) {
544         throw std::runtime_error("RMSNorm input is empty");
545     }
546     float sum_square = 0.0F;
547     for (const float value : input) {
548         sum_square += value * value;
549     }
550     const float mean_square = sum_square / static_cast<float>(input.size());
551     const float scale = 1.0F / std::sqrt(mean_square + epsilon);
552     for (std::size_t index = 0; index < input.size(); ++index) {
553         output[index] = input[index] * scale * weight[index];
554     }
555 }
556

```



```

557 void linear_f32(const LinearWeightsF32& layer,
558               std::span<const float> input,
559               std::span<float> output) {
560     validate_linear(layer, "linear_f32");
561     validate_span_size(input.size(), layer.input_size, "linear input");
562     validate_span_size(output.size(), layer.output_size, "linear output");
563     for (std::int32_t out = 0; out < layer.output_size; ++out) {
564         const std::size_t row_base =
565             checked_size(out, "out") * checked_size(layer.input_size, "↵
↵ input_size");
566         float accumulator = layer.bias.empty() ? 0.0F : layer.bias[checked_size↵
↵ (out, "out")];
567         for (std::int32_t in = 0; in < layer.input_size; ++in) {
568             accumulator += layer.weights[row_base + checked_size(in, "in")] *
569                             input[checked_size(in, "in")];
570         }
571         output[checked_size(out, "out")] = accumulator;
572     }
573 }
574
575 void apply_rope(std::span<float> heads,
576                std::int32_t head_count,
577                std::int32_t head_dim,
578                std::int32_t model_position,
579                float rope_base) {
580     require_positive(head_count, "head_count");
581     require_positive(head_dim, "head_dim");
582     validate_span_size(heads.size(), head_count * head_dim, "RoPE heads");
583     if (head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
584         throw std::runtime_error("RoPE head_dim must be even");
585     }
586     if (model_position < 0) {
587         throw std::runtime_error("RoPE model_position cannot be negative");
588     }
589     if (rope_base <= 1.0F) {
590         throw std::runtime_error("RoPE base must be greater than 1");
591     }
592     for (std::int32_t head = 0; head < head_count; ++head) {
593         for (std::int32_t offset = 0; offset < head_dim; offset += ↵
↵ LCQI_ROPE_PAIR_WIDTH) {
594             const float exponent = static_cast<float>(offset) / static_cast<↵
↵ float>(head_dim);
595             const float theta = 1.0F / std::pow(rope_base, exponent);
596             const float angle = static_cast<float>(model_position) * theta;
597             const float cosine = std::cos(angle);
598             const float sine = std::sin(angle);
599             const std::size_t first =
600                 checked_size(head, "head") * checked_size(head_dim, "head_dim")↵
↵ +
601                 checked_size(offset, "offset");
602             const float x0 = heads[first];
603             const float x1 = heads[first + 1];

```

```

604         heads[first] = x0 * cosine - x1 * sine;
605         heads[first + 1] = x0 * sine + x1 * cosine;
606     }
607 }
608 }
609
610 void attention_decode(const DecoderConfig& config,
611                     const ReferenceKVCache& cache,
612                     std::int32_t layer_id,
613                     std::int32_t model_position,
614                     std::span<const float> query,
615                     std::span<float> output) {
616     validate_config(config);
617     validate_span_size(query.size(), config.hidden_size, "attention query");
618     validate_span_size(output.size(), config.hidden_size, "attention output");
619     if (model_position < 0 || model_position >= cache.filled_tokens()) {
620         throw std::runtime_error("attention model_position has not been written↵
↵ ");
621     }
622     std::fill(output.begin(), output.end(), 0.0F);
623
624     const std::int32_t group_size = config.query_heads / config.kv_heads;
625     const float score_scale = 1.0F / std::sqrt(static_cast<float>(config.↵
↵ head_dim));
626     std::vector<float> scores(check_size(model_position + 1, "score count"),
627                             LCQI_SOFTMAX_NEGATIVE_INFINITY);
628     std::vector<float> probabilities(scores.size(), 0.0F);
629
630     for (std::int32_t query_head = 0; query_head < config.query_heads; ++↵
↵ query_head) {
631         const std::int32_t kv_head = query_head / group_size;
632         const std::size_t query_base =
633             checked_size(query_head, "query_head") * checked_size(config.↵
↵ head_dim, "head_dim");
634         float max_score = LCQI_SOFTMAX_NEGATIVE_INFINITY;
635         for (std::int32_t position = 0; position <= model_position; ++position)↵
↵ {
636             const std::span<const float> key = cache.key(layer_id, position, ↵
↵ kv_head);
637             float dot = 0.0F;
638             for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
639                 dot += query[query_base + checked_size(dim, "dim")] *
640                     key[checked_size(dim, "dim")];
641             }
642             const float score = dot * score_scale;
643             scores[checked_size(position, "position")] = score;
644             max_score = std::max(max_score, score);
645         }
646
647         float probability_sum = 0.0F;
648         for (std::int32_t position = 0; position <= model_position; ++position)↵
↵ {

```

```

649         const std::size_t index = checked_size(position, "position");
650         const float unnormalized = std::exp(scores[index] - max_score);
651         probabilities[index] = unnormalized;
652         probability_sum += unnormalized;
653     }
654     if (probability_sum <= 0.0F) {
655         throw std::runtime_error("attention probability sum is invalid");
656     }
657     for (std::int32_t position = 0; position <= model_position; ++position) {
658         const float probability =
659             probabilities[checked_size(position, "position")] /
660             probability_sum;
661         const std::span<const float> value = cache.value(layer_id, position, kv_head);
662         for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
663             output[query_base + checked_size(dim, "dim")] +=
664                 probability * value[checked_size(dim, "dim")];
665         }
666     }
667 }
668
669 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
670     std::span<const std::int32_t> token_ids) {
671     validate_config(model.config);
672     if (token_ids.empty()) {
673         throw std::runtime_error("reference decode needs at least one token");
674     }
675     if (token_ids.size() > checked_size(model.config.max_context, "max_context")) {
676         throw std::runtime_error("token_ids exceed max_context");
677     }
678     if (model.token_embedding.size() !=
679         checked_size(model.config.vocab_size, "vocab_size") *
680         checked_size(model.config.hidden_size, "hidden_size")) {
681         throw std::runtime_error("token embedding size mismatch");
682     }
683     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size, "hidden_size")) {
684         throw std::runtime_error("final norm weight size mismatch");
685     }
686     validate_linear(model.lm_head, "lm_head");
687
688     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers.size()));
689     std::vector<float> hidden(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
690     for (std::int32_t position = 0; position < static_cast<std::int32_t>(token_ids.size()); ++position) {

```

```

692     const std::int32_t token_id = token_ids[checked_size(position, "↵
↵ position")];
693     if (token_id < 0 || token_id >= model.config.vocab_size) {
694         throw std::runtime_error("token id out of vocabulary");
695     }
696     const std::span<const float> embedding =
697         row_span(model.token_embedding, token_id, model.config.hidden_size)↵
↵ ;
698     std::copy(embedding.begin(), embedding.end(), hidden.begin());
699     for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(↵
↵ model.layers.size());
700         ++layer_id) {
701         forward_layer(model.config,
702             model.layers[checked_size(layer_id, "layer_id")],
703             layer_id,
704             position,
705             cache,
706             hidden);
707     }
708 }
709
710 std::vector<float> normed(checked_size(model.config.hidden_size, "↵
↵ hidden_size"), 0.0F);
711 std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size↵
↵ "), 0.0F);
712 rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed)↵
↵ ;
713 linear_f32(model.lm_head, normed, logits);
714
715 ReferenceDecodeResult result;
716 result.predicted_token = argmax(logits);
717 result.logits = std::move(logits);
718 return result;
719 }
720
721 ReferenceDecoderModel make_tiny_reference_decoder_model() {
722     ReferenceDecoderModel model;
723     model.config.hidden_size = 4;
724     model.config.query_heads = 2;
725     model.config.kv_heads = 1;
726     model.config.head_dim = 2;
727     model.config.intermediate_size = 6;
728     model.config.vocab_size = 5;
729     model.config.max_context = 8;
730     model.config.rms_epsilon = 1.0e-5F;
731     model.config.rope_base = 10000.0F;
732
733     model.token_embedding = {
734         0.20F, -0.10F, 0.05F, 0.30F,
735         -0.40F, 0.10F, 0.25F, -0.20F,
736         0.15F, 0.35F, -0.30F, 0.05F,
737         0.50F, -0.25F, 0.10F, -0.15F,

```

```

738     -0.05F, 0.20F, 0.40F, -0.35F,
739 };
740
741 DecoderLayerWeightsF32 layer;
742 layer.rms_attention_weight = {1.0F, 0.9F, 1.1F, 1.0F};
743 layer.wq = make_linear(4,
744     4,
745     {
746         0.20F, -0.10F, 0.05F, 0.30F,
747         -0.15F, 0.25F, 0.10F, -0.05F,
748         0.05F, 0.10F, -0.20F, 0.15F,
749         0.30F, -0.05F, 0.20F, 0.10F,
750     });
751 layer.wk = make_linear(4,
752     2,
753     {
754         0.10F, 0.20F, -0.10F, 0.05F,
755         -0.05F, 0.15F, 0.25F, -0.20F,
756     });
757 layer.wv = make_linear(4,
758     2,
759     {
760         0.25F, -0.05F, 0.10F, 0.15F,
761         -0.10F, 0.30F, 0.05F, 0.20F,
762     });
763 layer.wo = make_linear(4,
764     4,
765     {
766         0.20F, 0.10F, -0.05F, 0.15F,
767         -0.10F, 0.25F, 0.20F, -0.05F,
768         0.05F, -0.20F, 0.30F, 0.10F,
769         0.15F, 0.05F, -0.10F, 0.25F,
770     });
771 layer.rms_mlp_weight = {0.95F, 1.05F, 1.0F, 0.9F};
772 layer.w_gate = make_linear(4,
773     6,
774     {
775         0.20F, -0.10F, 0.05F, 0.10F,
776         -0.05F, 0.15F, 0.20F, -0.10F,
777         0.10F, 0.05F, -0.15F, 0.25F,
778         -0.20F, 0.10F, 0.15F, 0.05F,
779         0.05F, -0.25F, 0.10F, 0.20F,
780         0.15F, 0.05F, 0.05F, -0.15F,
781     });
782 layer.w_up = make_linear(4,
783     6,
784     {
785         0.10F, 0.20F, -0.05F, 0.05F,
786         -0.15F, 0.05F, 0.25F, 0.10F,
787         0.20F, -0.10F, 0.05F, 0.15F,
788         0.05F, 0.10F, -0.20F, 0.25F,
789         -0.05F, 0.15F, 0.10F, -0.10F,

```

```

790         0.25F, -0.05F, 0.05F, 0.10F,
791     });
792     layer.w_down = make_linear(6,
793         4,
794         {
795             0.10F, -0.05F, 0.15F, 0.20F, -0.10F, 0.05F,
796             -0.20F, 0.10F, 0.05F, -0.05F, 0.15F, 0.20F,
797             0.05F, 0.15F, -0.10F, 0.10F, 0.20F, -0.05F,
798             0.20F, 0.05F, 0.10F, -0.15F, -0.05F, 0.10F,
799         });
800     model.layers.push_back(std::move(layer));
801     model.final_norm_weight = {1.0F, 0.95F, 1.05F, 1.0F};
802     model.lm_head = make_linear(4,
803         5,
804         {
805             0.20F, -0.10F, 0.05F, 0.15F,
806             -0.05F, 0.25F, 0.10F, -0.20F,
807             0.15F, 0.05F, -0.25F, 0.10F,
808             -0.10F, 0.20F, 0.15F, 0.05F,
809             0.05F, -0.15F, 0.20F, 0.25F,
810         },
811         zeros(5));
812     return model;
813 }
814
815 } // namespace lcqi

```

5.3 Tokenizer 与 Safetensors 实现

`tokenizer.cpp` 把 prompt 变成 token id, 并固定 unknown token 行为。读者应检查 BOS/EOS 是否稳定插入, UNK 是否按合同处理。

Listing 5.2 `src/tokenizer.cpp`

```

1  #include <lcqi/tokenizer.hpp>
2
3  #include <cstdint>
4  #include <fstream>
5  #include <stdexcept>
6  #include <string>
7  #include <string_view>
8
9  namespace lcqi {
10 namespace {
11
12 constexpr std::uint64_t LCQI_FNV_OFFSET_BASIS = 1469598103934665603ULL;
13 constexpr std::uint64_t LCQI_FNV_PRIME = 1099511628211ULL;
14 constexpr std::int32_t LCQI_INVALID_TOKEN_ID = -1;
15
16 std::string read_key(std::istream& input) {
17     std::string key;
18     if (!(input >> key)) {

```

```

19     throw std::runtime_error("unexpected end of tokenizer file");
20 }
21 return key;
22 }
23
24 void expect_key(std::istream& input, const std::string& expected) {
25     const std::string key = read_key(input);
26     if (key != expected) {
27         throw std::runtime_error("expected tokenizer key '" + expected + "', ←
↪ got '" + key + "'");
28     }
29 }
30
31 std::int32_t read_i32(std::istream& input, const std::string& key) {
32     expect_key(input, key);
33     std::int32_t value = 0;
34     if (!(input >> value)) {
35         throw std::runtime_error("invalid tokenizer int32 value for key '" + ←
↪ key + "'");
36     }
37     return value;
38 }
39
40 std::string read_string(std::istream& input, const std::string& key) {
41     expect_key(input, key);
42     std::string value;
43     if (!(input >> value)) {
44         throw std::runtime_error("invalid tokenizer string value for key '" + ←
↪ key + "'");
45     }
46     return value;
47 }
48
49 std::int32_t lookup_token_id(const TokenizerModel& tokenizer, std::string_view ←
↪ text) {
50     for (const TokenEntry& entry : tokenizer.vocab) {
51         if (entry.text == text) {
52             return entry.id;
53         }
54     }
55     return LCQI_INVALID_TOKEN_ID;
56 }
57
58 void validate_tokenizer(const TokenizerModel& tokenizer) {
59     if (tokenizer.vocab.empty()) {
60         throw std::runtime_error("tokenizer vocab is empty");
61     }
62     for (std::size_t i = 0; i < tokenizer.vocab.size(); ++i) {
63         const TokenEntry& left = tokenizer.vocab[i];
64         if (left.id < 0) {
65             throw std::runtime_error("tokenizer token id must be non-negative")←
↪ ;

```

```

66     }
67     for (std::size_t j = i + 1; j < tokenizer.vocab.size(); ++j) {
68         const TokenEntry& right = tokenizer.vocab[j];
69         if (left.id == right.id) {
70             throw std::runtime_error("duplicate tokenizer token id");
71         }
72         if (left.text == right.text) {
73             throw std::runtime_error("duplicate tokenizer token text");
74         }
75     }
76 }
77 if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID &&
78     lookup_token_id(tokenizer, "<bos>") != tokenizer.bos_id) {
79     throw std::runtime_error("tokenizer BOS id does not match vocab");
80 }
81 if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID &&
82     lookup_token_id(tokenizer, "<eos>") != tokenizer.eos_id) {
83     throw std::runtime_error("tokenizer EOS id does not match vocab");
84 }
85 }
86
87 std::uint64_t hash_byte(std::uint64_t hash, unsigned char value) {
88     hash ^= static_cast<std::uint64_t>(value);
89     hash *= LCQI_FNV_PRIME;
90     return hash;
91 }
92
93 std::uint64_t hash_string(std::uint64_t hash, std::string_view text) {
94     for (const unsigned char value : text) {
95         hash = hash_byte(hash, value);
96     }
97     return hash;
98 }
99
100 std::uint64_t hash_i32(std::uint64_t hash, std::int32_t value) {
101     const std::uint32_t bits = static_cast<std::uint32_t>(value);
102     for (std::int32_t shift = 0; shift < 32; shift += 8) {
103         hash = hash_byte(hash, static_cast<unsigned char>((bits >> shift) & 0
104 ↪ xFFU));
105     }
106     return hash;
107 }
108 } // namespace
109
110 TokenizerModel load_tokenizer(const std::filesystem::path& path) {
111     std::ifstream input(path);
112     if (!input) {
113         throw std::runtime_error("cannot open tokenizer file: " + path.string()
114 ↪ );
115     }

```



```

116     expect_key(input, "LCQI_TOKENIZER_V1");
117
118     TokenizerModel tokenizer;
119     tokenizer.bos_id = read_i32(input, "bos_id");
120     tokenizer.eos_id = read_i32(input, "eos_id");
121     tokenizer.unk_id = read_i32(input, "unk_id");
122     tokenizer.chat_template_hash = read_string(input, "chat_template_hash");
123     const std::int32_t vocab_size = read_i32(input, "vocab_size");
124     if (vocab_size <= 0) {
125         throw std::runtime_error("tokenizer vocab_size must be positive");
126     }
127
128     tokenizer.vocab.reserve(static_cast<std::size_t>(vocab_size));
129     expect_key(input, "vocab");
130     for (std::int32_t i = 0; i < vocab_size; ++i) {
131         TokenEntry entry;
132         if (!(input >> entry.id >> entry.text)) {
133             throw std::runtime_error("invalid tokenizer vocab entry");
134         }
135         tokenizer.vocab.push_back(entry);
136     }
137
138     validate_tokenizer(tokenizer);
139     return tokenizer;
140 }
141
142 std::vector<std::int32_t> encode_prompt(const TokenizerModel& tokenizer,
143                                         std::string_view prompt) {
144     std::vector<std::int32_t> ids;
145     if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID) {
146         ids.push_back(tokenizer.bos_id);
147     }
148
149     std::size_t begin = 0;
150     while (begin < prompt.size()) {
151         while (begin < prompt.size() && prompt[begin] == ' ') {
152             ++begin;
153         }
154         if (begin >= prompt.size()) {
155             break;
156         }
157         std::size_t end = begin;
158         while (end < prompt.size() && prompt[end] != ' ') {
159             ++end;
160         }
161         const std::string_view token = prompt.substr(begin, end - begin);
162         const std::int32_t token_id = lookup_token_id(tokenizer, token);
163         if (token_id == LCQI_INVALID_TOKEN_ID) {
164             if (tokenizer.unk_id == LCQI_INVALID_TOKEN_ID) {
165                 throw std::runtime_error("tokenizer encountered unknown token")↵
166             ↵ ;
167         }
168     }

```

```

167         ids.push_back(tokenizer.unk_id);
168     } else {
169         ids.push_back(token_id);
170     }
171     begin = end;
172 }
173
174 if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID) {
175     ids.push_back(tokenizer.eos_id);
176 }
177 return ids;
178 }
179
180 std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer) {
181     std::uint64_t hash = LCQI_FNV_OFFSET_BASIS;
182     hash = hash_string(hash, tokenizer.chat_template_hash);
183     hash = hash_i32(hash, tokenizer.bos_id);
184     hash = hash_i32(hash, tokenizer.eos_id);
185     hash = hash_i32(hash, tokenizer.unk_id);
186     for (const TokenEntry& entry : tokenizer.vocab) {
187         hash = hash_string(hash, entry.text);
188         hash = hash_i32(hash, entry.id);
189     }
190     return hash;
191 }
192
193 } // namespace lcqi

```

`safetensors.cpp` 解析 header、metadata、dtype、shape 和 data offsets。坏 dtype、坏 shape、坏 offset 和 payload 太短都应该被拒绝。

Listing 5.3 src/safetensors.cpp

```

1 #include <lcqi/safetensors.hpp>
2
3 #include <algorithm>
4 #include <cctype>
5 #include <cstdint>
6 #include <cmath>
7 #include <cstring>
8 #include <fstream>
9 #include <limits>
10 #include <stdexcept>
11 #include <string>
12 #include <string_view>
13 #include <vector>
14
15 namespace lcqi {
16 namespace {
17
18 constexpr std::size_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
19 constexpr std::uint64_t LCQI_SAFETENSORS_MAX_HEADER_BYTES = 16U * 1024U * 1024U↵
    ↵ ;

```

```

20 constexpr std::size_t LCQI_SAFETENSORS_OFFSET_COUNT = 2;
21 constexpr std::uint64_t LCQI_BYTE_BITS = 8;
22 constexpr std::uint64_t LCQI_DECIMAL_BASE = 10;
23 constexpr std::uint64_t LCQI_BYTE_MASK = 0xFFU;
24 constexpr unsigned char LCQI_JSON_CONTROL_CHAR_LIMIT = 0x20U;
25 constexpr std::uint64_t LCQI_DTYPE_BYTES_F64 = 8;
26 constexpr std::uint64_t LCQI_DTYPE_BYTES_F32 = 4;
27 constexpr std::uint64_t LCQI_DTYPE_BYTES_F16 = 2;
28 constexpr std::uint64_t LCQI_DTYPE_BYTES_I64 = 8;
29 constexpr std::uint64_t LCQI_DTYPE_BYTES_I32 = 4;
30 constexpr std::uint64_t LCQI_DTYPE_BYTES_I16 = 2;
31 constexpr std::uint64_t LCQI_DTYPE_BYTES_I8 = 1;
32 constexpr std::uint64_t LCQI_DTYPE_BYTES_U8 = 1;
33 constexpr std::uint32_t LCQI_F32_EXPONENT_MASK = 0x7F800000U;
34 constexpr std::uint32_t LCQI_F16_SIGN_MASK = 0x8000U;
35 constexpr std::uint32_t LCQI_F16_EXPONENT_MASK = 0x7C00U;
36 constexpr std::uint32_t LCQI_F16_MANTISSA_MASK = 0x03FFU;
37 constexpr std::uint32_t LCQI_F16_EXPONENT_BIAS = 15U;
38 constexpr std::uint32_t LCQI_F32_EXPONENT_BIAS = 127U;
39 constexpr std::uint32_t LCQI_F32_MANTISSA_BITS = 23U;
40 constexpr std::uint32_t LCQI_F16_MANTISSA_BITS = 10U;
41 constexpr std::uint32_t LCQI_F16_TO_F32_SIGN_SHIFT = 16U;
42
43 class HeaderCursor {
44 public:
45     explicit HeaderCursor(std::string_view text) : text_(text) {}
46
47     void expect(char expected) {
48         this->skip_ws();
49         if (this->eof() || this->text_[this->position_] != expected) {
50             throw std::runtime_error("invalid safetensors header syntax");
51         }
52         ++this->position_;
53     }
54
55     [[nodiscard]] bool consume(char expected) {
56         this->skip_ws();
57         if (!this->eof() && this->text_[this->position_] == expected) {
58             ++this->position_;
59             return true;
60         }
61         return false;
62     }
63
64     [[nodiscard]] std::string parse_string() {
65         this->skip_ws();
66         this->expect('"');
67         std::string result;
68         while (!this->eof()) {
69             const char ch = this->text_[this->position_++];
70             if (ch == '"') {
71                 return result;

```

```

72     }
73     if (static_cast<unsigned char>(ch) < LCQI_JSON_CONTROL_CHAR_LIMIT) ↵
↵ {
74         throw std::runtime_error("control character in safetensors ↵
↵ string");
75     }
76     if (ch == '\\') {
77         result.push_back(this->parse_escape());
78     } else {
79         result.push_back(ch);
80     }
81 }
82 throw std::runtime_error("unterminated safetensors string");
83 }
84
85 [[nodiscard]] std::uint64_t parse_u64() {
86     this->skip_ws();
87     if (this->eof() ||
88         !std::isdigit(static_cast<unsigned char>(this->text_[this->↵
↵ position_]))) {
89         throw std::runtime_error("expected unsigned integer in safetensors ↵
↵ header");
90     }
91     std::uint64_t value = 0;
92     while (!this->eof() &&
93         std::isdigit(static_cast<unsigned char>(this->text_[this->↵
↵ position_]))) {
94         const std::uint64_t digit =
95             static_cast<std::uint64_t>(this->text_[this->position_] - '0');
96         if (value >
97             (std::numeric_limits<std::uint64_t>::max() - digit) / ↵
↵ LCQI_DECIMAL_BASE) {
98             throw std::runtime_error("safetensors integer overflow");
99         }
100         value = value * LCQI_DECIMAL_BASE + digit;
101         ++this->position_;
102     }
103     return value;
104 }
105
106 [[nodiscard]] std::vector<std::int64_t> parse_i64_array() {
107     std::vector<std::int64_t> values;
108     this->expect('[');
109     if (this->consume(' ')) {
110         return values;
111     }
112     while (true) {
113         const std::uint64_t raw = this->parse_u64();
114         if (raw > static_cast<std::uint64_t>(std::numeric_limits<std::↵
↵ int64_t>::max())) {
115             throw std::runtime_error("safetensors shape value is too large"↵
↵ );

```

```

116         }
117         values.push_back(static_cast<std::int64_t>(raw));
118         if (this->consume(' ')) {
119             return values;
120         }
121         this->expect(',');
122     }
123 }
124
125 [[nodiscard]] std::vector<std::uint64_t> parse_u64_array() {
126     std::vector<std::uint64_t> values;
127     this->expect('[');
128     if (this->consume(' ')) {
129         return values;
130     }
131     while (true) {
132         values.push_back(this->parse_u64());
133         if (this->consume(' ')) {
134             return values;
135         }
136         this->expect(',');
137     }
138 }
139
140 [[nodiscard]] bool eof_after_ws() {
141     this->skip_ws();
142     return this->eof();
143 }
144
145 private:
146     std::string_view text_;
147     std::size_t position_ = 0;
148
149     [[nodiscard]] bool eof() const {
150         return this->position_ >= this->text_.size();
151     }
152
153     void skip_ws() {
154         while (!this->eof() &&
155             std::isspace(static_cast<unsigned char>(this->text_[this->position_])) {
156             ++this->position_;
157         }
158     }
159
160     [[nodiscard]] char parse_escape() {
161         if (this->eof()) {
162             throw std::runtime_error("unterminated safetensors escape");
163         }
164         const char escaped = this->text_[this->position_++];
165         switch (escaped) {
166             case '\\':

```

```

167         case '\\':
168         case '/':
169             return escaped;
170         case 'b':
171             return '\\b';
172         case 'f':
173             return '\\f';
174         case 'n':
175             return '\\n';
176         case 'r':
177             return '\\r';
178         case 't':
179             return '\\t';
180         default:
181             throw std::runtime_error("unsupported safetensors string escape↵
↵ ");
182     }
183 }
184 };
185
186 std::uint64_t read_le_u64(std::span<const std::uint8_t> bytes) {
187     if (bytes.size() < LCQI_SAFETENSORS_PREFIX_BYTES) {
188         throw std::runtime_error("safetensors file is too small");
189     }
190     std::uint64_t value = 0;
191     for (std::size_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
192         value |= static_cast<std::uint64_t>(bytes[i]) << (LCQI_BYTE_BITS * i);
193     }
194     return value;
195 }
196
197 std::string read_file_bytes(const std::filesystem::path& path) {
198     std::ifstream input(path, std::ios::binary);
199     if (!input) {
200         throw std::runtime_error("cannot open safetensors file: " + path.string↵
↵ ());
201     }
202     input.seekg(0, std::ios::end);
203     const std::streamoff size = input.tellg();
204     if (size < 0) {
205         throw std::runtime_error("cannot determine safetensors file size");
206     }
207     input.seekg(0, std::ios::beg);
208     std::string bytes(static_cast<std::size_t>(size), '\\0');
209     if (!bytes.empty() && !input.read(bytes.data(), static_cast<std::streamsize↵
↵ >(bytes.size())) {
210         throw std::runtime_error("cannot read safetensors file");
211     }
212     return bytes;
213 }
214
215 std::vector<std::uint8_t> read_file_u8(const std::filesystem::path& path) {

```

```

216     std::ifstream input(path, std::ios::binary);
217     if (!input) {
218         throw std::runtime_error("cannot open safetensors file: " + path.string() ←
    ↪ ());
219     }
220     input.seekg(0, std::ios::end);
221     const std::streamoff size = input.tellg();
222     if (size < 0) {
223         throw std::runtime_error("cannot determine safetensors file size");
224     }
225     input.seekg(0, std::ios::beg);
226     std::vector<std::uint8_t> bytes(static_cast<std::size_t>(size), 0);
227     if (!bytes.empty() &&
228         !input.read(reinterpret_cast<char*>(bytes.data()),
229                     static_cast<std::streamsize>(bytes.size()))) {
230         throw std::runtime_error("cannot read safetensors file");
231     }
232     return bytes;
233 }
234
235 void parse_metadata_value(HeaderCursor& cursor,
236                           SafeTensorManifest& manifest,
237                           const std::string& key) {
238     if (key == "__metadata__") {
239         cursor.expect('{');
240         if (cursor.consume('}')) {
241             return;
242         }
243         while (true) {
244             const std::string metadata_key = cursor.parse_string();
245             cursor.expect(':');
246             const std::string metadata_value = cursor.parse_string();
247             manifest.metadata.emplace_back(metadata_key, metadata_value);
248             if (cursor.consume('}')) {
249                 return;
250             }
251             cursor.expect(',');
252         }
253     }
254     throw std::runtime_error("unexpected safetensors metadata value");
255 }
256
257 SafeTensorEntry parse_tensor_entry(HeaderCursor& cursor, const std::string& ←
    ↪ name) {
258     SafeTensorEntry entry;
259     entry.name = name;
260     bool saw_dtype = false;
261     bool saw_shape = false;
262     bool saw_offsets = false;
263
264     cursor.expect('{');
265     if (cursor.consume('}')) {

```

```

266     throw std::runtime_error("empty safetensors tensor entry");
267 }
268 while (true) {
269     const std::string field = cursor.parse_string();
270     cursor.expect(':');
271     if (field == "dtype") {
272         entry.dtype = cursor.parse_string();
273         saw_dtype = true;
274     } else if (field == "shape") {
275         entry.shape = cursor.parse_i64_array();
276         saw_shape = true;
277     } else if (field == "data_offsets") {
278         const std::vector<std::uint64_t> offsets = cursor.parse_u64_array();
279         ↵ ;
280         if (offsets.size() != LCQI_SAFETENSORS_OFFSET_COUNT) {
281             throw std::runtime_error("safetensors data_offsets must contain ↵
282             ↵ two values");
283         }
284         entry.data_begin = offsets[0];
285         entry.data_end = offsets[1];
286         saw_offsets = true;
287     } else {
288         throw std::runtime_error("unsupported safetensors tensor field: " + ↵
289         ↵ field);
290     }
291     if (cursor.consume('}')) {
292         break;
293     }
294     cursor.expect(',');
295 }
296
297 if (!saw_dtype || !saw_shape || !saw_offsets) {
298     throw std::runtime_error("safetensors tensor entry is missing required ↵
299     ↵ fields");
300 }
301 if (entry.data_end < entry.data_begin) {
302     throw std::runtime_error("safetensors tensor offset range is invalid");
303 }
304 return entry;
305 }
306
307 void parse_header(std::string_view header, SafeTensorManifest& manifest) {
308     HeaderCursor cursor(header);
309     cursor.expect('{');
310     if (cursor.consume('}')) {
311         throw std::runtime_error("safetensors header is empty");
312     }
313
314     while (true) {
315         const std::string key = cursor.parse_string();
316         cursor.expect(':');
317         if (key == "__metadata__") {

```



```

314         parse_metadata_value(cursor, manifest, key);
315     } else {
316         manifest.tensors.push_back(parse_tensor_entry(cursor, key));
317     }
318     if (cursor.consume('}')) {
319         break;
320     }
321     cursor.expect(',');
322 }
323
324 if (!cursor.eof_after_ws()) {
325     throw std::runtime_error("trailing data after safetensors header");
326 }
327 }
328
329 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry);
330
331 void validate_manifest(const SafeTensorManifest& manifest,
332                       std::uint64_t file_size) {
333     const std::uint64_t payload_size = file_size - manifest.data_start_offset;
334     for (std::size_t i = 0; i < manifest.tensors.size(); ++i) {
335         const SafeTensorEntry& left = manifest.tensors[i];
336         if (left.name.empty() || left.dtype.empty()) {
337             throw std::runtime_error("safetensors tensor has empty name or ↵
↵ dtype");
338         }
339         if (left.data_end > payload_size) {
340             throw std::runtime_error("safetensors tensor data_offsets exceed ↵
↵ payload size");
341         }
342         const std::uint64_t expected_bytes = expected_tensor_bytes(left);
343         if (left.byte_size() != expected_bytes) {
344             throw std::runtime_error("safetensors tensor byte size does not ↵
↵ match dtype and shape");
345         }
346         for (const std::int64_t dim : left.shape) {
347             if (dim < 0) {
348                 throw std::runtime_error("safetensors shape dimension is ↵
↵ negative");
349             }
350         }
351         for (std::size_t j = i + 1; j < manifest.tensors.size(); ++j) {
352             const SafeTensorEntry& right = manifest.tensors[j];
353             if (left.name == right.name) {
354                 throw std::runtime_error("duplicate safetensors tensor name");
355             }
356             const bool overlaps =
357                 left.data_begin < right.data_end && right.data_begin < left.↵
↵ data_end;
358             if (overlaps) {
359                 throw std::runtime_error("overlapping safetensors tensor data ↵
↵ range");

```

```

360     }
361 }
362 }
363 }
364
365 std::uint64_t dtype_byte_size(std::string_view dtype) {
366     if (dtype == "F64") {
367         return LCQI_DTYPE_BYTES_F64;
368     }
369     if (dtype == "F32") {
370         return LCQI_DTYPE_BYTES_F32;
371     }
372     if (dtype == "F16" || dtype == "BF16") {
373         return LCQI_DTYPE_BYTES_F16;
374     }
375     if (dtype == "I64" || dtype == "U64") {
376         return LCQI_DTYPE_BYTES_I64;
377     }
378     if (dtype == "I32" || dtype == "U32") {
379         return LCQI_DTYPE_BYTES_I32;
380     }
381     if (dtype == "I16" || dtype == "U16") {
382         return LCQI_DTYPE_BYTES_I16;
383     }
384     if (dtype == "I8") {
385         return LCQI_DTYPE_BYTES_I8;
386     }
387     if (dtype == "U8" || dtype == "BOOL") {
388         return LCQI_DTYPE_BYTES_U8;
389     }
390     throw std::runtime_error("unsupported safetensors dtype: " + std::string(↵
↵ dtype));
391 }
392
393 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry) {
394     std::uint64_t element_count = 1;
395     for (const std::int64_t dim : entry.shape) {
396         if (dim < 0) {
397             throw std::runtime_error("safetensors shape dimension is negative")↵
↵ ;
398         }
399         const std::uint64_t extent = static_cast<std::uint64_t>(dim);
400         if (extent != 0 &&
401             element_count > std::numeric_limits<std::uint64_t>::max() / extent)↵
↵ {
402             throw std::runtime_error("safetensors shape element count overflow"↵
↵ );
403         }
404         element_count *= extent;
405     }
406     const std::uint64_t dtype_bytes = dtype_byte_size(entry.dtype);
407     if (dtype_bytes != 0 &&

```

```

408     element_count > std::numeric_limits<std::uint64_t>::max() / dtype_bytes) {
409         throw std::runtime_error("safetensors tensor byte size overflow");
410     }
411     return element_count * dtype_bytes;
412 }
413
414 SafeTensorManifest parse_manifest_from_bytes(std::span<const std::uint8_t> bytes) {
415     const std::uint64_t header_size = read_le_u64(bytes);
416     if (header_size == 0 || header_size > LCQI_SAFETENSORS_MAX_HEADER_BYTES) {
417         throw std::runtime_error("safetensors header size is invalid");
418     }
419     const std::uint64_t data_start_offset =
420         static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) + header_size;
421     if (data_start_offset > bytes.size()) {
422         throw std::runtime_error("safetensors header extends beyond file size");
423     }
424
425     SafeTensorManifest manifest;
426     manifest.header_size = header_size;
427     manifest.data_start_offset = data_start_offset;
428     const char* header_begin =
429         reinterpret_cast<const char*>(bytes.data() +
430         LCQI_SAFETENSORS_PREFIX_BYTES);
431     const std::string_view header(header_begin, static_cast<std::size_t>(
432     header_size));
433     parse_header(header, manifest);
434     validate_manifest(manifest, static_cast<std::uint64_t>(bytes.size()));
435     return manifest;
436 }
437
438 std::uint16_t read_le_u16(std::span<const std::uint8_t> bytes, std::size_t offset) {
439     return static_cast<std::uint16_t>(
440         static_cast<std::uint16_t>(bytes[offset]) |
441         (static_cast<std::uint16_t>(bytes[offset + 1]) << LCQI_BYTE_BITS));
442 }
443
444 std::uint32_t read_le_u32(std::span<const std::uint8_t> bytes, std::size_t offset) {
445     std::uint32_t value = 0;
446     for (std::size_t i = 0; i < LCQI_DTYPE_BYTES_F32; ++i) {
447         value |= static_cast<std::uint32_t>(bytes[offset + i]) << (
448         LCQI_BYTE_BITS * i);
449     }
450     return value;
451 }
452
453 float bits_to_float(std::uint32_t bits) {

```

```

451     float value = 0.0F;
452     std::memcpy(&value, &bits, sizeof(value));
453     return value;
454 }
455
456 float f16_to_f32(std::uint16_t bits) {
457     const std::uint32_t sign = (static_cast<std::uint32_t>(bits) & ↵
↵ LCQI_F16_SIGN_MASK)
458         << LCQI_F16_TO_F32_SIGN_SHIFT;
459     const std::uint32_t exponent = static_cast<std::uint32_t>(bits) & ↵
↵ LCQI_F16_EXPONENT_MASK;
460     const std::uint32_t mantissa = static_cast<std::uint32_t>(bits) & ↵
↵ LCQI_F16_MANTISSA_MASK;
461
462     if (exponent == 0U) {
463         if (mantissa == 0U) {
464             return bits_to_float(sign);
465         }
466         float value = static_cast<float>(mantissa) /
467             static_cast<float>(1U << LCQI_F16_MANTISSA_BITS);
468         value = std::ldexp(value, 1 - static_cast<int>(LCQI_F16_EXPONENT_BIAS))↵
↵ ;
469         return (sign == 0U) ? value : -value;
470     }
471
472     if (exponent == LCQI_F16_EXPONENT_MASK) {
473         const std::uint32_t f32_mantissa =
474             mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
475         return bits_to_float(sign | LCQI_F32_EXPONENT_MASK | f32_mantissa);
476     }
477
478     const std::uint32_t f32_exponent =
479         ((exponent >> LCQI_F16_MANTISSA_BITS) - LCQI_F16_EXPONENT_BIAS +
480          LCQI_F32_EXPONENT_BIAS)
481         << LCQI_F32_MANTISSA_BITS;
482     const std::uint32_t f32_mantissa =
483         mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
484     return bits_to_float(sign | f32_exponent | f32_mantissa);
485 }
486
487 float bf16_to_f32(std::uint16_t bits) {
488     return bits_to_float(static_cast<std::uint32_t>(bits) << 16U);
489 }
490
491 std::span<const std::uint8_t> tensor_payload_span(const SafeTensorFile& file,
492     const SafeTensorEntry& entry)↵
↵ {
493     const std::uint64_t begin = file.manifest.data_start_offset + entry.↵
↵ data_begin;
494     const std::uint64_t end = file.manifest.data_start_offset + entry.data_end;
495     if (begin > end || end > file.bytes.size()) {
496         throw std::runtime_error("safetensors tensor byte range is invalid");

```

```

497     }
498     return std::span<const std::uint8_t>(
499         file.bytes.data() + static_cast<std::size_t>(begin),
500         static_cast<std::size_t>(end - begin));
501 }
502
503 } // namespace
504
505 std::uint64_t SafeTensorEntry::byte_size() const {
506     return this->data_end - this->data_begin;
507 }
508
509 const SafeTensorEntry* SafeTensorManifest::find_tensor(std::string_view name) ↵
    ↵ const {
510     const auto found = std::find_if(
511         this->tensors.begin(),
512         this->tensors.end(),
513         [name](const SafeTensorEntry& entry) {
514             return entry.name == name;
515         });
516     if (found == this->tensors.end()) {
517         return nullptr;
518     }
519     return &(*found);
520 }
521
522 SafeTensorManifest load_safetensors_manifest(const std::filesystem::path& path) ↵
    ↵ {
523     const std::string bytes = read_file_bytes(path);
524     return parse_manifest_from_bytes(
525         std::span<const std::uint8_t>(
526             reinterpret_cast<const std::uint8_t*>(bytes.data()),
527             bytes.size()));
528 }
529
530 SafeTensorFile load_safetensors_file(const std::filesystem::path& path) {
531     SafeTensorFile file;
532     file.path = path;
533     file.bytes = read_file_u8(path);
534     file.manifest = parse_manifest_from_bytes(file.bytes);
535     return file;
536 }
537
538 std::span<const std::uint8_t> read_safetensor_tensor_bytes(
539     const SafeTensorFile& file,
540     std::string_view name) {
541     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
542     if (entry == nullptr) {
543         throw std::runtime_error("missing safetensors tensor: " + std::string(↵
    ↵ name));
544     }
545     return tensor_payload_span(file, *entry);

```

```

546 }
547
548 std::vector<float> read_safetensor_f32_tensor(const SafeTensorFile& file,
549                                             std::string_view name) {
550     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
551     if (entry == nullptr) {
552         throw std::runtime_error("missing safetensors tensor: " + std::string(↵
↵ name));
553     }
554     const std::span<const std::uint8_t> bytes = tensor_payload_span(file, *↵
↵ entry);
555     std::vector<float> values;
556     if (entry->dtype == "F32") {
557         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F32);
558         for (std::size_t index = 0; index < values.size(); ++index) {
559             values[index] = bits_to_float(
560                 read_le_u32(bytes, index * LCQI_DTYPE_BYTES_F32));
561         }
562         return values;
563     }
564     if (entry->dtype == "F16") {
565         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
566         for (std::size_t index = 0; index < values.size(); ++index) {
567             values[index] = f16_to_f32(
568                 read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
569         }
570         return values;
571     }
572     if (entry->dtype == "BF16") {
573         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
574         for (std::size_t index = 0; index < values.size(); ++index) {
575             values[index] = bf16_to_f32(
576                 read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
577         }
578         return values;
579     }
580     throw std::runtime_error("safetensors tensor is not float-compatible: " + ↵
↵ entry->name);
581 }
582
583 } // namespace lcqi

```

5.4 GPT-2 Reference Path

`gpt2_reference.cpp` 包括 JSON 配置读取、byte-level BPE、safetensors F32/F16/BF16 张量读取、HuggingFace 权重名前缀解析、LayerNorm、packed QKV、causal attention、GELU、tied lm head、full-prefix greedy generation 和 cached-KV generation。它是正确性优先的 reference path，暂不追求吞吐。

Listing 5.4 `src/gpt2_reference.cpp`

```
1 #include <lcqi/gpt2_reference.hpp>
```

```

2
3 #include <lcqi/f32_kernels.hpp>
4 #include <lcqi/safetensors.hpp>
5
6 #include <algorithm>
7 #include <array>
8 #include <atomic>
9 #include <chrono>
10 #include <cmath>
11 #include <cstdint>
12 #include <fstream>
13 #include <limits>
14 #include <optional>
15 #include <sstream>
16 #include <stdexcept>
17 #include <string>
18 #include <string_view>
19 #include <thread>
20 #include <unordered_set>
21 #include <utility>
22
23 namespace lcqi {
24 namespace {
25
26 constexpr std::int32_t LCQI_GPT2_DEFAULT_BOS_EOS = 50256;
27 constexpr std::int32_t LCQI_GPT2_ATTENTION_PROJECTIONS = 3;
28 constexpr std::int32_t LCQI_GPT2_PAIR_TOKEN_COUNT = 2;
29 constexpr std::int32_t LCQI_GPT2_BYTE_COUNT = 256;
30 constexpr std::int32_t LCQI_GPT2_UNICODE_PRIVATE_BASE = 256;
31 constexpr std::int32_t LCQI_GPT2_ASCII_EXCLAMATION = 33;
32 constexpr std::int32_t LCQI_GPT2_ASCII_TILDE = 126;
33 constexpr std::int32_t LCQI_GPT2_LATIN_INVERTED_EXCLAMATION = 161;
34 constexpr std::int32_t LCQI_GPT2_LATIN_NOT_SIGN = 172;
35 constexpr std::int32_t LCQI_GPT2_LATIN_REGISTERED = 174;
36 constexpr std::int32_t LCQI_GPT2_LATIN_Y_DIAERESIS = 255;
37 constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_MASK = 0x3F;
38 constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_BITS = 0x80;
39 constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_HEAD = 0xC0;
40 constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_HEAD = 0xE0;
41 constexpr std::int32_t LCQI_GPT2_UTF8_FOUR_BYTE_HEAD = 0xF0;
42 constexpr std::int32_t LCQI_GPT2_UTF8_ONE_BYTE_LIMIT = 0x80;
43 constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_LIMIT = 0x800;
44 constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_LIMIT = 0x10000;
45 constexpr unsigned char LCQI_GPT2_ASCII_APOSTROPHE = 0x27U;
46 constexpr float LCQI_GPT2_GELU_SQRT_2_OVER_PI = 0.7978845608028654F;
47 constexpr float LCQI_GPT2_GELU_CUBIC_COEFF = 0.044715F;
48 constexpr float LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY =
49     -std::numeric_limits<float>::infinity();
50 constexpr std::int32_t LCQI_GPT2_MAX_AUTO_WORKERS = 8;
51 constexpr std::int32_t LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS = 512;
52 constexpr std::int32_t LCQI_GPT2_PARALLEL_MIN_COLUMNS = 512;
53 constexpr std::int64_t LCQI_GPT2_PARALLEL_MIN_OPS = 1'000'000;

```

```

54 constexpr std::size_t LCQI_GPT2_PARALLEL_ROW_BLOCK_MULTIPLIER = 4;
55 constexpr std::size_t LCQI_GPT2_WORKER_SPIN_PAUSES_BEFORE_YIELD = 4096;
56
57 using Gpt2ProfileClock = std::chrono::steady_clock;
58
59 void pause_worker_spin(std::size_t& pause_count) noexcept {
60     #if defined(__GNUC__) && (defined(__x86_64__) || defined(__i386__))
61         __builtin_ia32_pause();
62     #endif
63     ++pause_count;
64     if (pause_count >= LCQI_GPT2_WORKER_SPIN_PAUSES_BEFORE_YIELD) {
65         pause_count = 0;
66         std::this_thread::yield();
67     }
68 }
69
70 } // namespace
71
72 namespace detail {
73
74 class Gpt2ParallelWorkerPool {
75 public:
76     explicit Gpt2ParallelWorkerPool(std::int32_t requested_workers)
77         : worker_count_(Gpt2ParallelWorkerPool::normalize_worker_count(↵
↵ requested_workers)) {
78         if (this->worker_count_ <= 1) {
79             return;
80         }
81         const std::size_t background_count =
82             static_cast<std::size_t>(this->worker_count_ - 1);
83         this->threads_.reserve(background_count);
84         for (std::size_t index = 0; index < background_count; ++index) {
85             this->threads_.emplace_back([this, index]() { this->worker_loop(↵
↵ index); });
86         }
87     }
88
89     ~Gpt2ParallelWorkerPool() {
90         this->stopping_.store(true, std::memory_order_release);
91         this->generation_.fetch_add(1, std::memory_order_release);
92         for (std::thread& thread : this->threads_) {
93             if (thread.joinable()) {
94                 thread.join();
95             }
96         }
97     }
98
99     Gpt2ParallelWorkerPool(const Gpt2ParallelWorkerPool&) = delete;
100     Gpt2ParallelWorkerPool& operator=(const Gpt2ParallelWorkerPool&) = delete;
101
102     [[nodiscard]] std::int32_t worker_count() const noexcept {
103         return this->worker_count_;

```



```

104     }
105
106     template <typename Function>
107     void parallel_for_rows(std::size_t begin,
108                           std::size_t end,
109                           std::size_t grain,
110                           Function&& function) {
111         if (end <= begin) {
112             return;
113         }
114         const std::size_t row_count = end - begin;
115         if (this->worker_count_ <= 1 || row_count <= grain) {
116             function(begin, end);
117             return;
118         }
119
120         const std::size_t task_count =
121             std::min<std::size_t>(static_cast<std::size_t>(this->worker_count_)↵
↵ ,
122                                  (row_count + grain - 1) / grain);
123         if (task_count <= 1) {
124             function(begin, end);
125             return;
126         }
127
128         auto task_context = ParallelTaskContext<Function>{function};
129         // Publish all task metadata before the generation bump; workers use ↵
↵ the
130         // generation acquire load as the hand-off point for this stack context↵
↵ .
131         this->task_begin_.store(begin, std::memory_order_release);
132         this->task_end_.store(end, std::memory_order_release);
133         this->task_count_.store(task_count, std::memory_order_release);
134         this->task_context_.store(&task_context, std::memory_order_release);
135         this->task_invoke_.store(&ParallelTaskContext<Function>::invoke,
136                                 std::memory_order_release);
137         this->active_workers_.store(task_count - 1, std::memory_order_release);
138         this->generation_.fetch_add(1, std::memory_order_release);
139
140         const std::size_t main_worker = task_count - 1;
141         const auto [main_begin, main_end] = this->chunk_bounds(begin, end, ↵
↵ task_count, main_worker);
142         function(main_begin, main_end);
143
144         std::size_t pause_count = 0;
145         while (this->active_workers_.load(std::memory_order_acquire) != 0) {
146             pause_worker_spin(pause_count);
147         }
148         this->task_context_.store(nullptr, std::memory_order_release);
149         this->task_invoke_.store(nullptr, std::memory_order_release);
150     }
151

```

```

152 private:
153     template <typename Function>
154     struct ParallelTaskContext {
155         Function& function;
156
157         static void invoke(void* context, std::size_t begin, std::size_t end) {
158             auto* typed_context = static_cast<ParallelTaskContext*>(context);
159             typed_context->function(begin, end);
160         }
161     };
162
163     std::int32_t worker_count_ = 1;
164     std::vector<std::thread> threads_;
165     std::atomic<bool> stopping_ = false;
166     std::atomic<std::uint64_t> generation_ = 0;
167     std::atomic<std::size_t> task_begin_ = 0;
168     std::atomic<std::size_t> task_end_ = 0;
169     std::atomic<std::size_t> task_count_ = 0;
170     std::atomic<std::size_t> active_workers_ = 0;
171     std::atomic<void*> task_context_ = nullptr;
172     std::atomic<void (*) (void*, std::size_t, std::size_t)> task_invoke_ = ↵
    ↵ nullptr;
173
174     [[nodiscard]] static std::int32_t normalize_worker_count(std::int32_t ↵
    ↵ requested_workers) {
175         if (requested_workers > 0) {
176             return requested_workers;
177         }
178         const unsigned int hardware_workers = std::thread::hardware_concurrency; ↵
    ↵ ();
179         if (hardware_workers == 0) {
180             return 1;
181         }
182         const std::int32_t capped =
183             std::min<std::int32_t>(static_cast<std::int32_t>(hardware_workers),
184                                   LCQI_GPT2_MAX_AUTO_WORKERS);
185         return std::max(capped, 1);
186     }
187
188     [[nodiscard]] std::pair<std::size_t, std::size_t> chunk_bounds(
189         std::size_t begin,
190         std::size_t end,
191         std::size_t task_count,
192         std::size_t worker_index) const {
193         const std::size_t row_count = end - begin;
194         const std::size_t chunk_begin = begin + row_count * worker_index / ↵
    ↵ task_count;
195         const std::size_t chunk_end = begin + row_count * (worker_index + 1) / ↵
    ↵ task_count;
196         return {chunk_begin, chunk_end};
197     }
198

```

```

199 void worker_loop(std::size_t worker_index) {
200     std::uint64_t observed_generation = 0;
201     std::size_t pause_count = 0;
202     while (true) {
203         const std::uint64_t current_generation =
204             this->generation_.load(std::memory_order_acquire);
205         if (current_generation == observed_generation) {
206             if (this->stopping_.load(std::memory_order_acquire)) {
207                 return;
208             }
209             pause_worker_spin(pause_count);
210             continue;
211         }
212
213         observed_generation = current_generation;
214         pause_count = 0;
215         if (this->stopping_.load(std::memory_order_acquire)) {
216             return;
217         }
218
219         const std::size_t task_count = this->task_count_.load(std::memory_order_acquire);
220         if (this->generation_.load(std::memory_order_acquire) != current_generation) {
221             continue;
222         }
223         const std::size_t background_task_count = task_count - 1;
224         if (worker_index >= background_task_count) {
225             continue;
226         }
227         void* task_context = this->task_context_.load(std::memory_order_acquire);
228         void (*task_invoke)(void*, std::size_t, std::size_t) =
229             this->task_invoke_.load(std::memory_order_acquire);
230         if (task_context == nullptr || task_invoke == nullptr) {
231             continue;
232         }
233         const std::size_t task_begin = this->task_begin_.load(std::memory_order_acquire);
234         const std::size_t task_end = this->task_end_.load(std::memory_order_acquire);
235         if (this->generation_.load(std::memory_order_acquire) != current_generation) {
236             continue;
237         }
238         const auto [begin, end] =
239             this->chunk_bounds(task_begin,
240                               task_end,
241                               task_count,
242                               worker_index);
243         task_invoke(task_context, begin, end);
244     }

```

```

245         this->active_workers_.fetch_sub(1, std::memory_order_acq_rel);
246     }
247 }
248 };
249
250 } // namespace detail
251
252 namespace {
253
254 class JsonCursor {
255 public:
256     explicit JsonCursor(std::string_view text) : text_(text) {}
257
258     void expect(char expected) {
259         this->skip_ws();
260         if (this->eof() || this->text_[this->position_] != expected) {
261             throw std::runtime_error("invalid JSON syntax");
262         }
263         ++this->position_;
264     }
265
266     [[nodiscard]] bool consume(char expected) {
267         this->skip_ws();
268         if (!this->eof() && this->text_[this->position_] == expected) {
269             ++this->position_;
270             return true;
271         }
272         return false;
273     }
274
275     [[nodiscard]] std::string parse_string() {
276         this->skip_ws();
277         this->expect('"');
278         std::string result;
279         while (!this->eof()) {
280             const char ch = this->text_[this->position_++];
281             if (ch == '"') {
282                 return result;
283             }
284             if (ch == '\\') {
285                 result += this->parse_escape();
286             } else {
287                 result.push_back(ch);
288             }
289         }
290         throw std::runtime_error("unterminated JSON string");
291     }
292
293     [[nodiscard]] std::int64_t parse_i64() {
294         this->skip_ws();
295         bool negative = false;
296         if (this->consume('-')) {

```

```

297         negative = true;
298     }
299     if (this->eof() || this->text_[this->position_] < '0' ||
300         this->text_[this->position_] > '9') {
301         throw std::runtime_error("expected JSON integer");
302     }
303     std::int64_t value = 0;
304     while (!this->eof() && this->text_[this->position_] >= '0' &&
305         this->text_[this->position_] <= '9') {
306         const std::int64_t digit = this->text_[this->position_] - '0';
307         if (value >
308             (std::numeric_limits<std::int64_t>::max() - digit) / 10) {
309             throw std::runtime_error("JSON integer overflow");
310         }
311         value = value * 10 + digit;
312         ++this->position_;
313     }
314     return negative ? -value : value;
315 }
316
317 [[nodiscard]] double parse_number() {
318     this->skip_ws();
319     const std::size_t begin = this->position_;
320     if (!this->eof() && this->text_[this->position_] == '-') {
321         ++this->position_;
322     }
323     while (!this->eof() && this->text_[this->position_] >= '0' &&
324         this->text_[this->position_] <= '9') {
325         ++this->position_;
326     }
327     if (!this->eof() && this->text_[this->position_] == '.') {
328         ++this->position_;
329         while (!this->eof() && this->text_[this->position_] >= '0' &&
330             this->text_[this->position_] <= '9') {
331             ++this->position_;
332         }
333     }
334     if (!this->eof() &&
335         (this->text_[this->position_] == 'e' || this->text_[this->position_↵
↵ ] == 'E')) {
336         ++this->position_;
337         if (!this->eof() &&
338             (this->text_[this->position_] == '+' || this->text_[this->↵
↵ position_] == '-')) {
339             ++this->position_;
340         }
341         while (!this->eof() && this->text_[this->position_] >= '0' &&
342             this->text_[this->position_] <= '9') {
343             ++this->position_;
344         }
345     }
346     if (begin == this->position_) {

```

```

347         throw std::runtime_error("expected JSON number");
348     }
349     return std::stod(std::string(this->text_.substr(begin, this->position_ ←
↵ - begin)));
350 }
351
352 void skip_value() {
353     this->skip_ws();
354     if (this->eof()) {
355         throw std::runtime_error("unexpected end of JSON");
356     }
357     const char ch = this->text_[this->position_];
358     if (ch == '"') {
359         static_cast<void>(this->parse_string());
360         return;
361     }
362     if (ch == '{') {
363         this->expect('{');
364         if (this->consume('}')) {
365             return;
366         }
367         while (true) {
368             static_cast<void>(this->parse_string());
369             this->expect(':');
370             this->skip_value();
371             if (this->consume('}')) {
372                 return;
373             }
374             this->expect(',');
375         }
376     }
377     if (ch == '[') {
378         this->expect('[');
379         if (this->consume(']')) {
380             return;
381         }
382         while (true) {
383             this->skip_value();
384             if (this->consume(']')) {
385                 return;
386             }
387             this->expect(',');
388         }
389     }
390     if (ch == 't') {
391         this->expect_literal("true");
392         return;
393     }
394     if (ch == 'f') {
395         this->expect_literal("false");
396         return;
397     }

```

```

398     if (ch == 'n') {
399         this->expect_literal("null");
400         return;
401     }
402     static_cast<void>(this->parse_number());
403 }
404
405 [[nodiscard]] bool eof_after_ws() {
406     this->skip_ws();
407     return this->eof();
408 }
409
410 private:
411     std::string_view text_;
412     std::size_t position_ = 0;
413
414     [[nodiscard]] bool eof() const {
415         return this->position_ >= this->text_.size();
416     }
417
418     void skip_ws() {
419         while (!this->eof() &&
420             (this->text_[this->position_] == ' ' ||
421              this->text_[this->position_] == '\n' ||
422              this->text_[this->position_] == '\r' ||
423              this->text_[this->position_] == '\t')) {
424             ++this->position_;
425         }
426     }
427
428     void expect_literal(std::string_view literal) {
429         for (const char expected : literal) {
430             if (this->eof() || this->text_[this->position_] != expected) {
431                 throw std::runtime_error("invalid JSON literal");
432             }
433             ++this->position_;
434         }
435     }
436
437     [[nodiscard]] std::string parse_escape() {
438         if (this->eof()) {
439             throw std::runtime_error("unterminated JSON escape");
440         }
441         const char escaped = this->text_[this->position_++];
442         switch (escaped) {
443             case '"':
444             case '\\':
445             case '/':
446                 return std::string(1, escaped);
447             case 'b':
448                 return "\b";
449             case 'f':

```

```

450         return "\\f";
451     case 'n':
452         return "\\n";
453     case 'r':
454         return "\\r";
455     case 't':
456         return "\\t";
457     case 'u':
458         return this->parse_unicode_escape();
459     default:
460         throw std::runtime_error("unsupported JSON escape");
461     }
462 }
463
464 [[nodiscard]] std::string parse_unicode_escape() {
465     std::uint32_t value = 0;
466     for (std::int32_t i = 0; i < 4; ++i) {
467         if (this->eof()) {
468             throw std::runtime_error("unterminated JSON unicode escape");
469         }
470         const char ch = this->text_[this->position_++];
471         value <<= 4U;
472         if (ch >= '0' && ch <= '9') {
473             value += static_cast<std::uint32_t>(ch - '0');
474         } else if (ch >= 'a' && ch <= 'f') {
475             value += static_cast<std::uint32_t>(10 + ch - 'a');
476         } else if (ch >= 'A' && ch <= 'F') {
477             value += static_cast<std::uint32_t>(10 + ch - 'A');
478         } else {
479             throw std::runtime_error("invalid JSON unicode escape");
480         }
481     }
482     std::string encoded;
483     if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
484         encoded.push_back(static_cast<char>(value));
485     } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
486         encoded.push_back(static_cast<char>(
487             LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
488         encoded.push_back(static_cast<char>(
489             LCQI_GPT2_UTF8_CONTINUATION_BITS |
490             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
491     } else {
492         encoded.push_back(static_cast<char>(
493             LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
494         encoded.push_back(static_cast<char>(
495             LCQI_GPT2_UTF8_CONTINUATION_BITS |
496             ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
497         encoded.push_back(static_cast<char>(
498             LCQI_GPT2_UTF8_CONTINUATION_BITS |
499             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
500     }
501     return encoded;

```



```

502     }
503 };
504
505 std::string read_text_file(const std::filesystem::path& path) {
506     std::ifstream input(path);
507     if (!input) {
508         throw std::runtime_error("cannot open text file: " + path.string());
509     }
510     std::ostringstream buffer;
511     buffer << input.rdbuf();
512     return buffer.str();
513 }
514
515 std::size_t checked_size(std::int64_t value, const char* name) {
516     if (value < 0) {
517         throw std::runtime_error(std::string(name) + " cannot be negative");
518     }
519     return static_cast<std::size_t>(value);
520 }
521
522 std::string encode_codepoint(std::uint32_t value) {
523     std::string encoded;
524     if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
525         encoded.push_back(static_cast<char>(value));
526     } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
527         encoded.push_back(static_cast<char>(
528             LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
529         encoded.push_back(static_cast<char>(
530             LCQI_GPT2_UTF8_CONTINUATION_BITS |
531             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
532     } else if (value < LCQI_GPT2_UTF8_THREE_BYTE_LIMIT) {
533         encoded.push_back(static_cast<char>(
534             LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
535         encoded.push_back(static_cast<char>(
536             LCQI_GPT2_UTF8_CONTINUATION_BITS |
537             ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
538         encoded.push_back(static_cast<char>(
539             LCQI_GPT2_UTF8_CONTINUATION_BITS |
540             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
541     } else {
542         encoded.push_back(static_cast<char>(
543             LCQI_GPT2_UTF8_FOUR_BYTE_HEAD | (value >> 18U)));
544         encoded.push_back(static_cast<char>(
545             LCQI_GPT2_UTF8_CONTINUATION_BITS |
546             ((value >> 12U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
547         encoded.push_back(static_cast<char>(
548             LCQI_GPT2_UTF8_CONTINUATION_BITS |
549             ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
550         encoded.push_back(static_cast<char>(
551             LCQI_GPT2_UTF8_CONTINUATION_BITS |
552             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
553     }

```

```

554     return encoded;
555 }
556
557 void require_positive(std::int32_t value, const char* name) {
558     if (value <= 0) {
559         throw std::runtime_error(std::string(name) + " must be positive");
560     }
561 }
562
563 void validate_config(const Gpt2Config& config) {
564     require_positive(config.hidden_size, "hidden_size");
565     require_positive(config.head_count, "head_count");
566     require_positive(config.layer_count, "layer_count");
567     require_positive(config.max_positions, "max_positions");
568     require_positive(config.vocab_size, "vocab_size");
569     require_positive(config.intermediate_size, "intermediate_size");
570     if (config.hidden_size % config.head_count != 0) {
571         throw std::runtime_error("hidden_size must be divisible by head_count")↵
↵ ;
572     }
573     if (config.layer_norm_epsilon <= 0.0F) {
574         throw std::runtime_error("layer_norm_epsilon must be positive");
575     }
576 }
577
578 std::int32_t head_dim(const Gpt2Config& config) {
579     return config.hidden_size / config.head_count;
580 }
581
582 std::size_t matrix_size(std::int32_t rows, std::int32_t columns) {
583     return checked_size(rows, "rows") * checked_size(columns, "columns");
584 }
585
586 Gpt2ProfileClock::time_point profile_start(const Gpt2HotspotProfile* profile) {
587     if (profile == nullptr) {
588         return Gpt2ProfileClock::time_point{};
589     }
590     return Gpt2ProfileClock::now();
591 }
592
593 void add_profile_ms(Gpt2HotspotProfile* profile,
594                   double Gpt2HotspotProfile::*field,
595                   Gpt2ProfileClock::time_point start) {
596     if (profile == nullptr) {
597         return;
598     }
599     (*profile).*field +=
600         std::chrono::duration<double, std::milli>(Gpt2ProfileClock::now() - ↵
↵ start).count());
601 }
602
603 void validate_vector_size(std::size_t actual, std::int32_t expected, const char↵

```

```

    ↪ * name) {
604     if (actual != checked_size(expected, name)) {
605         throw std::runtime_error(std::string(name) + " size mismatch");
606     }
607 }
608
609 void validate_linear(const Gpt2LinearF32& linear, const char* name) {
610     require_positive(linear.input_size, "linear input_size");
611     require_positive(linear.output_size, "linear output_size");
612     if (linear.weights.size() != matrix_size(linear.output_size, linear.↪
    ↪ input_size)) {
613         throw std::runtime_error(std::string(name) + " weight size mismatch");
614     }
615     if (!linear.bias.empty() &&
616         linear.bias.size() != checked_size(linear.output_size, "linear ↪
    ↪ output_size")) {
617         throw std::runtime_error(std::string(name) + " bias size mismatch");
618     }
619 }
620
621 void validate_model(const Gpt2ReferenceModel& model) {
622     validate_config(model.config);
623     if (model.layers.size() != checked_size(model.config.layer_count, "↪
    ↪ layer_count")) {
624         throw std::runtime_error("GPT-2 layer count mismatch");
625     }
626     if (model.token_embedding.size() !=
627         matrix_size(model.config.vocab_size, model.config.hidden_size)) {
628         throw std::runtime_error("GPT-2 token embedding size mismatch");
629     }
630     if (model.position_embedding.size() !=
631         matrix_size(model.config.max_positions, model.config.hidden_size)) {
632         throw std::runtime_error("GPT-2 position embedding size mismatch");
633     }
634     validate_vector_size(model.final_ln_weight.size(), model.config.hidden_size↪
    ↪ , "ln_f weight");
635     validate_vector_size(model.final_ln_bias.size(), model.config.hidden_size, ↪
    ↪ "ln_f bias");
636     if (model.tie_lm_head_to_embedding) {
637         if (!model.lm_head_weight.empty()) {
638             throw std::runtime_error("tied GPT-2 lm_head should not carry ↪
    ↪ separate weights");
639         }
640     } else if (model.lm_head_weight.size() !=
641                 matrix_size(model.config.vocab_size, model.config.hidden_size)) ↪
    ↪ {
642         throw std::runtime_error("GPT-2 lm_head size mismatch");
643     }
644     for (const Gpt2LayerWeightsF32& layer : model.layers) {
645         validate_vector_size(layer.ln_1_weight.size(), model.config.hidden_size↪
    ↪ , "ln_1 weight");
646         validate_vector_size(layer.ln_1_bias.size(), model.config.hidden_size, ↪

```

```

↪ "ln_1 bias");
647     validate_linear(layer.c_attn, "c_attn");
648     validate_linear(layer.c_proj, "c_proj");
649     validate_vector_size(layer.ln_2_weight.size(), model.config.hidden_size, ↪
↪ , "ln_2 weight");
650     validate_vector_size(layer.ln_2_bias.size(), model.config.hidden_size, ↪
↪ "ln_2 bias");
651     validate_linear(layer.c_fc, "c_fc");
652     validate_linear(layer.mlp_c_proj, "mlp_c_proj");
653 }
654 }
655
656 std::span<const float> row_span(std::span<const float> values,
657                                std::int32_t row,
658                                std::int32_t row_width) {
659     return values.subspan(checked_size(row, "row") * checked_size(row_width, "↪
↪ row_width"),
660                           checked_size(row_width, "row_width"));
661 }
662
663 void add_inplace(std::span<float> target, std::span<const float> source) {
664     if (target.size() != source.size()) {
665         throw std::runtime_error("GPT-2 residual add size mismatch");
666     }
667     for (std::size_t index = 0; index < target.size(); ++index) {
668         target[index] += source[index];
669     }
670 }
671
672 void layer_norm(std::span<const float> input,
673                std::span<const float> weight,
674                std::span<const float> bias,
675                float epsilon,
676                std::span<float> output) {
677     if (input.empty() || input.size() != weight.size() || input.size() != bias.↪
↪ size() ||
678         input.size() != output.size()) {
679         throw std::runtime_error("GPT-2 LayerNorm size mismatch");
680     }
681     float mean = 0.0F;
682     for (const float value : input) {
683         mean += value;
684     }
685     mean /= static_cast<float>(input.size());
686
687     float variance = 0.0F;
688     for (const float value : input) {
689         const float centered = value - mean;
690         variance += centered * centered;
691     }
692     variance /= static_cast<float>(input.size());
693     const float scale = 1.0F / std::sqrt(variance + epsilon);

```

```

694
695     for (std::size_t index = 0; index < input.size(); ++index) {
696         output[index] = (input[index] - mean) * scale * weight[index] + bias[↵
↵ index];
697     }
698 }
699
700 void linear_f32(const Gpt2LinearF32& linear,
701               std::span<const float> input,
702               std::span<float> output) {
703     validate_linear(linear, "GPT-2 linear");
704     if (input.size() != checked_size(linear.input_size, "linear input") ||
705         output.size() != checked_size(linear.output_size, "linear output")) {
706         throw std::runtime_error("GPT-2 linear input/output size mismatch");
707     }
708     const std::size_t input_size = checked_size(linear.input_size, "linear ↵
↵ input");
709     const std::size_t output_size = checked_size(linear.output_size, "linear ↵
↵ output");
710     for (std::size_t out = 0; out < output_size; ++out) {
711         float sum = linear.bias.empty() ? 0.0F : linear.bias[out];
712         const std::size_t row_base = out * input_size;
713         for (std::size_t in = 0; in < input_size; ++in) {
714             sum += linear.weights[row_base + in] * input[in];
715         }
716         output[out] = sum;
717     }
718 }
719
720 void linear_f32_unchecked(const Gpt2LinearF32& linear,
721                          std::span<const float> input,
722                          std::span<float> output) {
723     const std::size_t input_size = static_cast<std::size_t>(linear.input_size);
724     const std::size_t output_size = static_cast<std::size_t>(linear.output_size↵
↵ );
725     const float* weights = linear.weights.data();
726     const float* bias = linear.bias.empty() ? nullptr : linear.bias.data();
727     const float* input_data = input.data();
728     linear_f32_rows_unchecked(weights, input_data, bias, input_size, 0, ↵
↵ output_size, output.data());
729 }
730
731 [[nodiscard]] bool should_parallelize_rows(const Gpt2ExecutionOptions& options,
732                                           std::size_t rows,
733                                           std::size_t columns,
734                                           const detail::Gpt2ParallelWorkerPool↵
↵ * worker_pool) {
735     if (worker_pool == nullptr || worker_pool->worker_count() <= 1) {
736         return false;
737     }
738     const std::int32_t min_rows =
739         options.parallel_min_rows > 0 ? options.parallel_min_rows

```

```

740         : LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS;
741     if (rows < checked_size(min_rows, "parallel_min_rows")) {
742         return false;
743     }
744     if (options.worker_count > 1) {
745         return true;
746     }
747     return columns >= checked_size(LCQI_GPT2_PARALLEL_MIN_COLUMNS, "↵
↵ parallel_min_columns") ||
748         rows * columns >= static_cast<std::size_t>(↵
↵ LCQI_GPT2_PARALLEL_MIN_OPS);
749 }
750
751 [[nodiscard]] std::size_t parallel_row_grain(std::size_t rows,
752                                             const detail::↵
↵ Gpt2ParallelWorkerPool& worker_pool) {
753     const std::size_t workers = checked_size(worker_pool.worker_count(), "↵
↵ worker_count");
754     const std::size_t target_tasks = workers * ↵
↵ LCQI_GPT2_PARALLEL_ROW_BLOCK_MULTIPLIER;
755     return std::max<std::size_t>(1, (rows + target_tasks - 1) / target_tasks);
756 }
757
758 [[nodiscard]] bool model_has_parallel_work(const Gpt2ReferenceModel& model,
759                                             const Gpt2ExecutionOptions& options)↵
↵ {
760     const std::size_t hidden_size = checked_size(model.config.hidden_size, "↵
↵ hidden_size");
761     const std::size_t vocab_size = checked_size(model.config.vocab_size, "↵
↵ vocab_size");
762     const std::size_t intermediate_size =
763         checked_size(model.config.intermediate_size, "intermediate_size");
764     const std::size_t qkv_rows =
765         checked_size(model.config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS↵
↵ ,
766                     "qkv output rows");
767     const std::int32_t min_rows =
768         options.parallel_min_rows > 0 ? options.parallel_min_rows
769         : LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS;
770     const auto large_enough = [min_rows](std::size_t rows, std::size_t columns)↵
↵ {
771         if (rows < checked_size(min_rows, "parallel_min_rows")) {
772             return false;
773         }
774         return columns >= checked_size(LCQI_GPT2_PARALLEL_MIN_COLUMNS,
775                                         "parallel_min_columns") ||
776             rows * columns >= static_cast<std::size_t>(↵
↵ LCQI_GPT2_PARALLEL_MIN_OPS);
777     };
778     if (options.worker_count > 1) {
779         return vocab_size >= checked_size(min_rows, "parallel_min_rows") ||
780             qkv_rows >= checked_size(min_rows, "parallel_min_rows") ||

```

```

781         intermediate_size >= checked_size(min_rows, "parallel_min_rows")<←
782         ↪ ||
783         hidden_size >= checked_size(min_rows, "parallel_min_rows");
784     }
785     if (options.worker_count == 1) {
786         return false;
787     }
788     return large_enough(vocab_size, hidden_size) ||
789            large_enough(qkv_rows, hidden_size) ||
790            large_enough(hidden_size, hidden_size) ||
791            large_enough(intermediate_size, hidden_size) ||
792            large_enough(hidden_size, intermediate_size);
793 }
794 [[nodiscard]] std::unique_ptr<detail::Gpt2ParallelWorkerPool> make_worker_pool(
795     const Gpt2ReferenceModel& model,
796     const Gpt2ExecutionOptions& options) {
797     if (!model_has_parallel_work(model, options)) {
798         return nullptr;
799     }
800     return std::make_unique<detail::Gpt2ParallelWorkerPool>(options.<←
801     ↪ worker_count);
802 }
803 void linear_f32_unchecked_parallel(const Gpt2LinearF32& linear,
804     std::span<const float> input,
805     std::span<float> output,
806     const Gpt2ExecutionOptions& options,
807     detail::Gpt2ParallelWorkerPool* worker_pool)<←
808     ↪ {
809     const std::size_t input_size = static_cast<std::size_t>(linear.input_size);
810     const std::size_t output_size = static_cast<std::size_t>(linear.output_size)<←
811     ↪ );
812     if (!should_parallelize_rows(options, output_size, input_size, worker_pool)<←
813     ↪ ) {
814         linear_f32_unchecked(linear, input, output);
815         return;
816     }
817     const float* weights = linear.weights.data();
818     const float* bias = linear.bias.empty() ? nullptr : linear.bias.data();
819     const float* input_data = input.data();
820     float* output_data = output.data();
821     const auto rows_fn =
822         [input_size, weights, bias, input_data, output_data](std::size_t begin,
823             std::size_t end) {
824         linear_f32_rows_unchecked(weights,
825             input_data,
826             bias,
827             input_size,
828             begin,
829             end,

```

```

828                                     output_data);
829     };
830     worker_pool->parallel_for_rows(0, output_size, parallel_row_grain(↵
↵ output_size, *worker_pool), rows_fn);
831 }
832
833 float gelu_new(float value) {
834     const float cubic = value * value * value;
835     return 0.5F * value *
836         (1.0F + std::tanh(LCQI_GPT2_GELU_SQRT_2_OVER_PI *
837             (value + LCQI_GPT2_GELU_CUBIC_COEFF * cubic)));
838 }
839
840 std::int32_t argmax(std::span<const float> values) {
841     if (values.empty()) {
842         throw std::runtime_error("cannot take argmax of empty logits");
843     }
844     std::int32_t best_index = 0;
845     float best_value = values.front();
846     for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size(↵
↵ )); ++index) {
847         const float value = values[checked_size(index, "argmax index")];
848         if (value > best_value) {
849             best_value = value;
850             best_index = index;
851         }
852     }
853     return best_index;
854 }
855
856 void project_qkv(const Gpt2Config& config,
857                 const Gpt2LayerWeightsF32& layer,
858                 std::span<const float> hidden,
859                 std::span<float> q,
860                 std::span<float> k,
861                 std::span<float> v) {
862     std::vector<float> packed(matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS,
863                                           config.hidden_size),
864                               0.0F);
865     linear_f32(layer.c_attn, hidden, packed);
866     const std::size_t width = checked_size(config.hidden_size, "hidden_size");
867     std::copy_n(packed.begin(), config.hidden_size, q.begin());
868     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
869               config.hidden_size,
870               k.begin());
871     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
872               config.hidden_size,
873               v.begin());
874 }
875
876 void project_qkv_reuse_workspace(const Gpt2Config& config,
877                                 const Gpt2LayerWeightsF32& layer,

```



```

878         std::span<const float> hidden,
879         std::span<float> packed,
880         std::span<float> q,
881         std::span<float> k,
882         std::span<float> v,
883         const Gpt2ExecutionOptions& options,
884         detail::Gpt2ParallelWorkerPool* worker_pool) {
885     linear_f32_unchecked_parallel(layer.c_attn, hidden, packed, options, ↵
↵ worker_pool);
886     const std::size_t width = checked_size(config.hidden_size, "hidden_size");
887     std::copy_n(packed.begin(), config.hidden_size, q.begin());
888     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
889               config.hidden_size,
890               k.begin());
891     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
892               config.hidden_size,
893               v.begin());
894 }
895
896 void attention_full_sequence(const Gpt2Config& config,
897                             std::span<const float> q_by_pos,
898                             std::span<const float> k_by_pos,
899                             std::span<const float> v_by_pos,
900                             std::int32_t sequence_length,
901                             std::span<float> output_by_pos) {
902     const std::int32_t local_head_dim = head_dim(config);
903     const float score_scale = 1.0F / std::sqrt(static_cast<float>(↵
↵ local_head_dim));
904     std::fill(output_by_pos.begin(), output_by_pos.end(), 0.0F);
905     std::vector<float> scores(checked_size(sequence_length, "sequence_length"),
906                             LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY);
907
908     for (std::int32_t position = 0; position < sequence_length; ++position) {
909         for (std::int32_t head = 0; head < config.head_count; ++head) {
910             float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
911             for (std::int32_t past = 0; past <= position; ++past) {
912                 float dot = 0.0F;
913                 for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
914                     const std::size_t q_index =
915                         matrix_size(position, config.hidden_size) +
916                         matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
917                     const std::size_t k_index =
918                         matrix_size(past, config.hidden_size) +
919                         matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
920                     dot += q_by_pos[q_index] * k_by_pos[k_index];
921                 }
922                 const float score = dot * score_scale;
923                 scores[checked_size(past, "past")] = score;
924                 max_score = std::max(max_score, score);
925             }

```

```

926         float denominator = 0.0F;
927         for (std::int32_t past = 0; past <= position; ++past) {
928             const std::size_t index = checked_size(past, "past");
929             scores[index] = std::exp(scores[index] - max_score);
930             denominator += scores[index];
931         }
932         if (denominator <= 0.0F) {
933             throw std::runtime_error("GPT-2 attention denominator is ↵
↵ invalid");
934         }
935         for (std::int32_t past = 0; past <= position; ++past) {
936             const float probability = scores[checked_size(past, "past")] / ↵
↵ denominator;
937             for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
938                 const std::size_t output_index =
939                     matrix_size(position, config.hidden_size) +
940                     matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
941                 const std::size_t value_index =
942                     matrix_size(past, config.hidden_size) +
943                     matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
944                 output_by_pos[output_index] += probability * v_by_pos[↵
↵ value_index];
945             }
946         }
947     }
948 }
949 }
950 }
951
952 void forward_gpt2_layer(const Gpt2Config& config,
953                         const Gpt2LayerWeightsF32& layer,
954                         std::int32_t sequence_length,
955                         std::vector<float>& hidden_by_pos) {
956     std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), ↵
↵ 0.0F);
957     std::vector<float> q_by_pos(matrix_size(sequence_length, config.hidden_size)↵
↵ ), 0.0F);
958     std::vector<float> k_by_pos(q_by_pos.size(), 0.0F);
959     std::vector<float> v_by_pos(q_by_pos.size(), 0.0F);
960     std::vector<float> attention_by_pos(q_by_pos.size(), 0.0F);
961     std::vector<float> projected(checked_size(config.hidden_size, "hidden_size"↵
↵ ), 0.0F);
962     std::vector<float> mlp_fc(checked_size(config.intermediate_size, "↵
↵ intermediate_size"), 0.0F);
963     std::vector<float> mlp_out(checked_size(config.hidden_size, "hidden_size"),↵
↵ 0.0F);
964
965     for (std::int32_t position = 0; position < sequence_length; ++position) {
966         const std::span<float> hidden = std::span<float>(
967             hidden_by_pos.data() + matrix_size(position, config.hidden_size),

```

```

968         checked_size(config.hidden_size, "hidden_size"));
969     layer_norm(hidden,
970         layer.ln_1_weight,
971         layer.ln_1_bias,
972         config.layer_norm_epsilon,
973         normed);
974     std::span<float> q = std::span<float>(
975         q_by_pos.data() + matrix_size(position, config.hidden_size),
976         checked_size(config.hidden_size, "hidden_size"));
977     std::span<float> k = std::span<float>(
978         k_by_pos.data() + matrix_size(position, config.hidden_size),
979         checked_size(config.hidden_size, "hidden_size"));
980     std::span<float> v = std::span<float>(
981         v_by_pos.data() + matrix_size(position, config.hidden_size),
982         checked_size(config.hidden_size, "hidden_size"));
983     project_qkv(config, layer, normed, q, k, v);
984 }
985
986 attention_full_sequence(config, q_by_pos, k_by_pos, v_by_pos, ←
↪ sequence_length, attention_by_pos);
987
988 for (std::int32_t position = 0; position < sequence_length; ++position) {
989     const std::span<float> hidden = std::span<float>(
990         hidden_by_pos.data() + matrix_size(position, config.hidden_size),
991         checked_size(config.hidden_size, "hidden_size"));
992     const std::span<const float> attention = std::span<const float>(
993         attention_by_pos.data() + matrix_size(position, config.hidden_size)↪
↪ ,
994         checked_size(config.hidden_size, "hidden_size"));
995     linear_f32(layer.c_proj, attention, projected);
996     add_inplace(hidden, projected);
997
998     layer_norm(hidden,
999         layer.ln_2_weight,
1000        layer.ln_2_bias,
1001        config.layer_norm_epsilon,
1002        normed);
1003     linear_f32(layer.c_fc, normed, mlp_fc);
1004     for (float& value : mlp_fc) {
1005         value = gelu_new(value);
1006     }
1007     linear_f32(layer.mlp_c_proj, mlp_fc, mlp_out);
1008     add_inplace(hidden, mlp_out);
1009 }
1010 }
1011
1012 void forward_gpt2_layer_cached(const Gpt2Config& config,
1013     const Gpt2LayerWeightsF32& layer,
1014     std::int32_t layer_id,
1015     std::int32_t model_position,
1016     Gpt2KvCache& cache,
1017     Gpt2ForwardWorkspace& workspace,

```

```

1018         std::span<float> hidden,
1019         Gpt2HotspotProfile* hotspot_profile,
1020         const Gpt2ExecutionOptions& options,
1021         detail::Gpt2ParallelWorkerPool* worker_pool) {
1022     if (hotspot_profile != nullptr) {
1023         ++hotspot_profile->layer_steps;
1024     }
1025     std::span<float> normed = workspace.normed();
1026     std::span<float> query = workspace.query();
1027     std::span<float> key = workspace.key();
1028     std::span<float> value = workspace.value();
1029     std::span<float> attention = workspace.attention();
1030     std::span<float> projected = workspace.projected();
1031     std::span<float> qkv_packed = workspace.qkv_packed();
1032     std::span<float> mlp_fc = workspace.mlp_fc();
1033     std::span<float> mlp_out = workspace.mlp_out();
1034     std::span<float> scores = workspace.scores_prefix(model_position + 1);
1035
1036     Gpt2ProfileClock::time_point start =
1037         profile_start(hotspot_profile);
1038     layer_norm(hidden,
1039               layer.ln_1_weight,
1040               layer.ln_1_bias,
1041               config.layer_norm_epsilon,
1042               normed);
1043     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::layer_norm_ms, start);
1044
1045     start = profile_start(hotspot_profile);
1046     project_qkv_reuse_workspace(config,
1047                                layer,
1048                                normed,
1049                                qkv_packed,
1050                                query,
1051                                key,
1052                                value,
1053                                options,
1054                                worker_pool);
1055     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::qkv_projection_ms, ←
↪ start);
1056
1057     start = profile_start(hotspot_profile);
1058     cache.append(layer_id, model_position, key, value);
1059     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::kv_append_ms, start);
1060
1061     start = profile_start(hotspot_profile);
1062     detail::attend_cached_position(cache,
1063                                   layer_id,
1064                                   model_position,
1065                                   query,
1066                                   scores,
1067                                   attention);
1068     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::attention_ms, start);

```

```

1069
1070     start = profile_start(hotspot_profile);
1071     linear_f32_unchecked_parallel(layer.c_proj, attention, projected, options, ↵
↵ worker_pool);
1072     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::↵
↵ attention_projection_ms, start);
1073
1074     start = profile_start(hotspot_profile);
1075     add_inplace(hidden, projected);
1076     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::residual_add_ms, start↵
↵ );
1077
1078     start = profile_start(hotspot_profile);
1079     layer_norm(hidden,
1080                 layer.ln_2_weight,
1081                 layer.ln_2_bias,
1082                 config.layer_norm_epsilon,
1083                 normed);
1084     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::layer_norm_ms, start);
1085
1086     start = profile_start(hotspot_profile);
1087     linear_f32_unchecked_parallel(layer.c_fc, normed, mlp_fc, options, ↵
↵ worker_pool);
1088     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::mlp_fc_ms, start);
1089
1090     start = profile_start(hotspot_profile);
1091     for (float& value_ref : mlp_fc) {
1092         value_ref = gelu_new(value_ref);
1093     }
1094     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::gelu_ms, start);
1095
1096     start = profile_start(hotspot_profile);
1097     linear_f32_unchecked_parallel(layer.mlp_c_proj, mlp_fc, mlp_out, options, ↵
↵ worker_pool);
1098     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::mlp_projection_ms, ↵
↵ start);
1099
1100     start = profile_start(hotspot_profile);
1101     add_inplace(hidden, mlp_out);
1102     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::residual_add_ms, start↵
↵ );
1103 }
1104
1105 void compute_logits(const Gpt2ReferenceModel& model,
1106                    std::span<const float> hidden,
1107                    std::span<float> logits,
1108                    const Gpt2ExecutionOptions& options,
1109                    detail::Gpt2ParallelWorkerPool* worker_pool) {
1110     if (logits.size() != checked_size(model.config.vocab_size, "vocab_size")) {
1111         throw std::runtime_error("GPT-2 logits size mismatch");
1112     }
1113     const std::span<const float> weight =

```

```

1114     model.tie_lm_head_to_embedding ? std::span<const float>(model.↵
↵ token_embedding)
1115         : std::span<const float>(model.↵
↵ lm_head_weight);
1116     const std::size_t hidden_size = checked_size(model.config.hidden_size, "↵
↵ hidden_size");
1117     const std::size_t vocab_size = checked_size(model.config.vocab_size, "↵
↵ vocab_size");
1118     const float* weight_data = weight.data();
1119     const float* hidden_data = hidden.data();
1120     const auto rows_fn =
1121         [hidden_size, weight_data, hidden_data, logits_data = logits.data()](
1122             std::size_t begin,
1123             std::size_t end) {
1124         linear_f32_rows_unchecked(weight_data,
1125                                   hidden_data,
1126                                   nullptr,
1127                                   hidden_size,
1128                                   begin,
1129                                   end,
1130                                   logits_data);
1131     };
1132     if (should_parallelize_rows(options, vocab_size, hidden_size, worker_pool))↵
↵ {
1133         worker_pool->parallel_for_rows(0,
1134                                         vocab_size,
1135                                         parallel_row_grain(vocab_size, *↵
↵ worker_pool),
1136                                         rows_fn);
1137     } else {
1138         rows_fn(0, vocab_size);
1139     }
1140 }
1141
1142 void compute_logits(const Gpt2ReferenceModel& model,
1143                    std::span<const float> hidden,
1144                    std::span<float> logits) {
1145     Gpt2ExecutionOptions options;
1146     options.worker_count = 1;
1147     compute_logits(model, hidden, logits, options, nullptr);
1148 }
1149
1150 struct Gpt2TokenScore {
1151     std::int32_t token = 0;
1152     float value = -std::numeric_limits<float>::infinity();
1153 };
1154
1155 enum class Gpt2CachedStepMode {
1156     advance_only,
1157     predict_token,
1158     logits,
1159 };

```

```

1160
1161 [[nodiscard]] Gpt2TokenScore compute_predicted_token_range(const float* ↵
    ↵ weight_data,
1162                                                         const float* ↵
    ↵ hidden_data,
1163                                                         std::size_t ↵
    ↵ hidden_size,
1164                                                         std::size_t begin,
1165                                                         std::size_t end) {
1166     Gpt2TokenScore best;
1167     const F32RowMax row_max =
1168         max_dot_f32_rows_unchecked(weight_data, hidden_data, hidden_size, begin↵
    ↵ , end);
1169     best.token = static_cast<std::int32_t>(row_max.row);
1170     best.value = row_max.value;
1171     return best;
1172 }
1173
1174 std::int32_t compute_predicted_token(const Gpt2ReferenceModel& model,
1175                                     std::span<const float> hidden,
1176                                     const Gpt2ExecutionOptions& options,
1177                                     detail::Gpt2ParallelWorkerPool* ↵
    ↵ worker_pool) {
1178     const std::span<const float> weight =
1179         model.tie_lm_head_to_embedding ? std::span<const float>(model.↵
    ↵ token_embedding)
1180                                     : std::span<const float>(model.↵
    ↵ lm_head_weight);
1181     const std::size_t hidden_size = checked_size(model.config.hidden_size, "↵
    ↵ hidden_size");
1182     const std::size_t vocab_size = checked_size(model.config.vocab_size, "↵
    ↵ vocab_size");
1183     const float* weight_data = weight.data();
1184     const float* hidden_data = hidden.data();
1185     if (!should_parallelize_rows(options, vocab_size, hidden_size, worker_pool)↵
    ↵ ) {
1186         return compute_predicted_token_range(weight_data, hidden_data, ↵
    ↵ hidden_size, 0, vocab_size)
1187             .token;
1188     }
1189
1190     const std::size_t worker_count = checked_size(worker_pool->worker_count(), ↵
    ↵ "worker_count");
1191     const std::size_t chunk_count = std::min(worker_count, vocab_size);
1192     std::vector<Gpt2TokenScore> partials(chunk_count);
1193     const auto rows_fn =
1194         [chunk_count, hidden_size, vocab_size, weight_data, hidden_data, &↵
    ↵ partials](
1195         std::size_t begin,
1196         std::size_t end) {
1197         for (std::size_t chunk = begin; chunk < end; ++chunk) {
1198             const std::size_t token_begin = vocab_size * chunk / ↵

```

```

1199     ↪ chunk_count;
1200         const std::size_t token_end = vocab_size * (chunk + 1) / ↪
1201     ↪ chunk_count;
1202         partials[chunk] = compute_predicted_token_range(
1203             weight_data,
1204             hidden_data,
1205             hidden_size,
1206             token_begin,
1207             token_end);
1208     }
1209     };
1210     worker_pool->parallel_for_rows(0, chunk_count, 1, rows_fn);
1211     Gpt2TokenScore best = partials.front();
1212     for (std::size_t index = 1; index < partials.size(); ++index) {
1213         if (partials[index].value > best.value) {
1214             best = partials[index];
1215         }
1216     }
1217     return best.token;
1218 }
1219 Gpt2ForwardResult run_gpt2_cached_step_unchecked(const Gpt2ReferenceModel& ↪
1220     ↪ model,
1221     Gpt2KvCache& cache,
1222     ↪ workspace,
1223     Gpt2ForwardWorkspace& ↪
1224     std::int32_t token_id,
1225     Gpt2CachedStepMode mode,
1226     Gpt2HotspotProfile* ↪
1227     ↪ hotspot_profile,
1228     const Gpt2ExecutionOptions& ↪
1229     ↪ options,
1230     detail::Gpt2ParallelWorkerPool↪
1231     ↪ * worker_pool) {
1232     const Gpt2ProfileClock::time_point step_start = profile_start(↪
1233     ↪ hotspot_profile);
1234     if (hotspot_profile != nullptr) {
1235         ++hotspot_profile->decoder_steps;
1236     }
1237     if (token_id < 0 || token_id >= model.config.vocab_size) {
1238         throw std::runtime_error("GPT-2 token id out of range");
1239     }
1240     const std::int32_t model_position = cache.filled_tokens();
1241     if (model_position >= model.config.max_positions) {
1242         throw std::runtime_error("GPT-2 cached forward would exceed ↪
1243     ↪ max_positions");
1244     }
1245     std::span<float> hidden = workspace.hidden();
1246     const std::span<const float> token =
1247         row_span(model.token_embedding, token_id, model.config.hidden_size);

```



```

1242     const std::span<const float> position_embedding =
1243         row_span(model.position_embedding, model_position, model.config.↵
↵ hidden_size);
1244     Gpt2ProfileClock::time_point start = profile_start(hotspot_profile);
1245     for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
1246         hidden[checked_size(dim, "dim")] =
1247             token[checked_size(dim, "dim")] + position_embedding[checked_size(↵
↵ dim, "dim")];
1248     }
1249     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::embedding_ms, start);
1250
1251     for (std::int32_t layer_id = 0;
1252         layer_id < static_cast<std::int32_t>(model.layers.size());
1253         ++layer_id) {
1254         forward_gpt2_layer_cached(model.config,
1255             model.layers[checked_size(layer_id, "layer_id"↵
↵ ")],
1256             layer_id,
1257             model_position,
1258             cache,
1259             workspace,
1260             hidden,
1261             hotspot_profile,
1262             options,
1263             worker_pool);
1264     }
1265
1266     Gpt2ForwardResult result;
1267     if (mode == Gpt2CachedStepMode::advance_only) {
1268         add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::total_step_ms, ↵
↵ step_start);
1269         return result;
1270     }
1271
1272     std::span<float> normed = workspace.normed();
1273     start = profile_start(hotspot_profile);
1274     layer_norm(hidden,
1275         model.final_ln_weight,
1276         model.final_ln_bias,
1277         model.config.layer_norm_epsilon,
1278         normed);
1279     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::final_norm_ms, start);
1280
1281     if (mode == Gpt2CachedStepMode::logits) {
1282         std::span<float> logits = workspace.logits();
1283         start = profile_start(hotspot_profile);
1284         compute_logits(model, normed, logits, options, worker_pool);
1285         add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::lm_head_ms, start)↵
↵ ;
1286         start = profile_start(hotspot_profile);
1287         result.logits.assign(logits.begin(), logits.end());
1288         result.predicted_token = argmax(result.logits);

```

```

1289     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::logits_result_ms, ←
↪ start);
1290 } else {
1291     start = profile_start(hotspot_profile);
1292     result.predicted_token = compute_predicted_token(model, normed, options↪
↪ , worker_pool);
1293     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::lm_head_ms, start)↪
↪ ;
1294 }
1295 add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::total_step_ms, ←
↪ step_start);
1296 return result;
1297 }
1298
1299 std::vector<std::string> gpt2_bytes_to_unicode() {
1300     std::vector<std::int32_t> bytes;
1301     for (std::int32_t value = LCQI_GPT2_ASCII_EXCLAMATION;
1302         value <= LCQI_GPT2_ASCII_TILDE;
1303         ++value) {
1304         bytes.push_back(value);
1305     }
1306     for (std::int32_t value = LCQI_GPT2_LATIN_INVERTED_EXCLAMATION;
1307         value <= LCQI_GPT2_LATIN_NOT_SIGN;
1308         ++value) {
1309         bytes.push_back(value);
1310     }
1311     for (std::int32_t value = LCQI_GPT2_LATIN_REGISTERED;
1312         value <= LCQI_GPT2_LATIN_Y_DIAERESIS;
1313         ++value) {
1314         bytes.push_back(value);
1315     }
1316
1317     std::unordered_set<std::int32_t> byte_set(bytes.begin(), bytes.end());
1318     std::vector<std::int32_t> chars = bytes;
1319     std::int32_t private_offset = 0;
1320     for (std::int32_t byte = 0; byte < LCQI_GPT2_BYTE_COUNT; ++byte) {
1321         if (!byte_set.contains(byte)) {
1322             bytes.push_back(byte);
1323             chars.push_back(LCQI_GPT2_UNICODE_PRIVATE_BASE + private_offset);
1324             ++private_offset;
1325         }
1326     }
1327
1328     std::vector<std::string> mapping(LCQI_GPT2_BYTE_COUNT);
1329     for (std::size_t index = 0; index < bytes.size(); ++index) {
1330         mapping[checked_size(bytes[index], "byte unicode index")] =
1331             encode_codepoint(static_cast<std::uint32_t>(chars[index]));
1332     }
1333     return mapping;
1334 }
1335
1336 std::unordered_map<std::string, unsigned char> gpt2_unicode_to_bytes(

```

```

1337     const std::vector<std::string>& byte_encoder) {
1338         std::unordered_map<std::string, unsigned char> result;
1339         for (std::size_t byte = 0; byte < byte_encoder.size(); ++byte) {
1340             result.emplace(byte_encoder[byte], static_cast<unsigned char>(byte));
1341         }
1342         return result;
1343     }
1344
1345     std::string bytes_to_bpe_alphabet(std::string_view text) {
1346         static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode↵
↵ ();
1347         std::string result;
1348         for (const unsigned char byte : text) {
1349             result += BYTE_ENCODER[byte];
1350         }
1351         return result;
1352     }
1353
1354     std::vector<std::string> utf8_symbols(std::string_view text) {
1355         std::vector<std::string> symbols;
1356         std::size_t offset = 0;
1357         while (offset < text.size()) {
1358             const unsigned char lead = static_cast<unsigned char>(text[offset]);
1359             std::size_t width = 1;
1360             if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
1361                 width = 2;
1362             } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
1363                 width = 3;
1364             } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
1365                 width = 4;
1366             }
1367             if (offset + width > text.size()) {
1368                 throw std::runtime_error("invalid UTF-8/BPE symbol boundary");
1369             }
1370             symbols.emplace_back(text.substr(offset, width));
1371             offset += width;
1372         }
1373         return symbols;
1374     }
1375
1376     std::string pair_key(std::string_view left, std::string_view right) {
1377         std::string key(left);
1378         key.push_back('\n');
1379         key += right;
1380         return key;
1381     }
1382
1383     std::vector<std::string> bpe_apply(const Gpt2Tokenizer& tokenizer, std::↵
↵ string_view token) {
1384         std::vector<std::string> word = utf8_symbols(token);
1385         if (word.size() <= 1) {
1386             return word;

```

```

1387     }
1388
1389     while (true) {
1390         std::optional<std::int32_t> best_rank;
1391         std::string best_left;
1392         std::string best_right;
1393         for (std::size_t index = 0; index + 1 < word.size(); ++index) {
1394             const auto rank = tokenizer.bpe_ranks.find(pair_key(word[index], ←
↪ word[index + 1]));
1395             if (rank != tokenizer.bpe_ranks.end() &&
1396                 (!best_rank.has_value() || rank->second < *best_rank)) {
1397                 best_rank = rank->second;
1398                 best_left = word[index];
1399                 best_right = word[index + 1];
1400             }
1401         }
1402         if (!best_rank.has_value()) {
1403             return word;
1404         }
1405
1406         std::vector<std::string> merged;
1407         std::size_t index = 0;
1408         while (index < word.size()) {
1409             if (index + 1 < word.size() && word[index] == best_left &&
1410                 word[index + 1] == best_right) {
1411                 merged.push_back(best_left + best_right);
1412                 index += LCQI_GPT2_PAIR_TOKEN_COUNT;
1413             } else {
1414                 merged.push_back(word[index]);
1415                 ++index;
1416             }
1417         }
1418         word = std::move(merged);
1419         if (word.size() == 1) {
1420             return word;
1421         }
1422     }
1423 }
1424
1425 bool is_ascii_letter(unsigned char value) {
1426     return (value >= 'A' && value <= 'Z') || (value >= 'a' && value <= 'z');
1427 }
1428
1429 bool is_ascii_digit(unsigned char value) {
1430     return value >= '0' && value <= '9';
1431 }
1432
1433 bool is_ascii_space(unsigned char value) {
1434     return value == ' ' || value == '\t' || value == '\n' ||
1435            value == '\r' || value == '\f' || value == '\v';
1436 }
1437

```

```

1438 bool is_gpt2_punctuation(unsigned char value) {
1439     return !is_ascii_letter(value) && !is_ascii_digit(value) &&
1440           !is_ascii_space(value);
1441 }
1442
1443 std::size_t consume_utf8_codepoint(std::string_view text, std::size_t offset) {
1444     const unsigned char lead = static_cast<unsigned char>(text[offset]);
1445     std::size_t width = 1;
1446     if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
1447         width = 2;
1448     } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
1449         width = 3;
1450     } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
1451         width = 4;
1452     }
1453     if (offset + width > text.size()) {
1454         throw std::runtime_error("invalid UTF-8 text for GPT-2 tokenizer");
1455     }
1456     return offset + width;
1457 }
1458
1459 bool starts_with_contraction(std::string_view text,
1460                             std::size_t offset,
1461                             std::string_view contraction) {
1462     if (offset + contraction.size() > text.size()) {
1463         return false;
1464     }
1465     for (std::size_t index = 0; index < contraction.size(); ++index) {
1466         char left = text[offset + index];
1467         const char right = contraction[index];
1468         if (left >= 'A' && left <= 'Z') {
1469             left = static_cast<char>(left - 'A' + 'a');
1470         }
1471         if (left != right) {
1472             return false;
1473         }
1474     }
1475     return true;
1476 }
1477
1478 std::vector<std::string> gpt2_pretokenize(std::string_view text) {
1479     std::vector<std::string> pieces;
1480     std::size_t offset = 0;
1481     const std::array<std::string_view, 7> contractions{
1482         "'s", "'t", "'re", "'ve", "'m", "'ll", "'d",
1483     };
1484     while (offset < text.size()) {
1485         bool emitted_contraction = false;
1486         for (const std::string_view contraction : contractions) {
1487             if (starts_with_contraction(text, offset, contraction)) {
1488                 pieces.emplace_back(text.substr(offset, contraction.size()));
1489                 offset += contraction.size();

```

```

1490         emitted_contraction = true;
1491         break;
1492     }
1493 }
1494 if (emitted_contraction) {
1495     continue;
1496 }
1497
1498 const std::size_t begin = offset;
1499 bool has_prefix_space = false;
1500 if (text[offset] == ' ' && offset + 1 < text.size() &&
1501     !is_ascii_space(static_cast<unsigned char>(text[offset + 1]))) {
1502     has_prefix_space = true;
1503     ++offset;
1504 }
1505
1506 if (offset >= text.size()) {
1507     pieces.emplace_back(text.substr(begin, offset - begin));
1508     continue;
1509 }
1510
1511 const unsigned char ch = static_cast<unsigned char>(text[offset]);
1512 if (is_ascii_letter(ch)) {
1513     while (offset < text.size() &&
1514         is_ascii_letter(static_cast<unsigned char>(text[offset]))) {
1515         ++offset;
1516     }
1517 } else if (is_ascii_digit(ch)) {
1518     while (offset < text.size() &&
1519         is_ascii_digit(static_cast<unsigned char>(text[offset]))) {
1520         ++offset;
1521     }
1522 } else if (is_gpt2_punctuation(ch) && ch != LCQI_GPT2_ASCII_APOSTROPHE) ↵
↵ {
1523     while (offset < text.size() &&
1524         is_gpt2_punctuation(static_cast<unsigned char>(text[offset])) ↵
↵ ) &&
1525         static_cast<unsigned char>(text[offset]) != ↵
↵ LCQI_GPT2_ASCII_APOSTROPHE) {
1526         offset = consume_utf8_codepoint(text, offset);
1527     }
1528 } else if (is_ascii_space(ch)) {
1529     while (offset < text.size() &&
1530         is_ascii_space(static_cast<unsigned char>(text[offset]))) {
1531         ++offset;
1532     }
1533 } else {
1534     offset = consume_utf8_codepoint(text, offset);
1535 }
1536
1537 if (has_prefix_space || offset > begin) {
1538     pieces.emplace_back(text.substr(begin, offset - begin));

```

```

1539     } else {
1540         ++offset;
1541     }
1542 }
1543 return pieces;
1544 }
1545
1546 std::int32_t parse_i32_field(JsonCursor& cursor) {
1547     const std::int64_t value = cursor.parse_i64();
1548     if (value < std::numeric_limits<std::int32_t>::min() ||
1549         value > std::numeric_limits<std::int32_t>::max()) {
1550         throw std::runtime_error("JSON int32 field out of range");
1551     }
1552     return static_cast<std::int32_t>(value);
1553 }
1554
1555 std::unordered_map<std::string, std::int32_t> parse_vocab_json(std::string_view↵
↵ text) {
1556     JsonCursor cursor(text);
1557     std::unordered_map<std::string, std::int32_t> vocab;
1558     cursor.expect('{');
1559     if (cursor.consume('}')) {
1560         return vocab;
1561     }
1562     while (true) {
1563         const std::string key = cursor.parse_string();
1564         cursor.expect(':');
1565         const std::int32_t id = parse_i32_field(cursor);
1566         vocab.emplace(key, id);
1567         if (cursor.consume('}')) {
1568             break;
1569         }
1570         cursor.expect(',');
1571     }
1572     if (!cursor.eof_after_ws()) {
1573         throw std::runtime_error("trailing data after vocab JSON");
1574     }
1575     return vocab;
1576 }
1577
1578 std::vector<std::string> build_id_to_token(
1579     const std::unordered_map<std::string, std::int32_t>& vocab) {
1580     std::int32_t max_id = -1;
1581     for (const auto& [token, id] : vocab) {
1582         if (id < 0) {
1583             throw std::runtime_error("GPT-2 vocab id cannot be negative");
1584         }
1585         max_id = std::max(max_id, id);
1586     }
1587     std::vector<std::string> result(check_size(max_id + 1, "max vocab id"));
1588     for (const auto& [token, id] : vocab) {
1589         std::string& slot = result[checked_size(id, "vocab id")];

```

```

1590     if (!slot.empty()) {
1591         throw std::runtime_error("duplicate GPT-2 vocab id");
1592     }
1593     slot = token;
1594 }
1595 return result;
1596 }
1597
1598 std::unordered_map<std::string, std::int32_t> parse_merges(std::string_view ↵
↵ text) {
1599     std::unordered_map<std::string, std::int32_t> ranks;
1600     std::istringstream input{std::string(text)};
1601     std::string line;
1602     std::int32_t rank = 0;
1603     while (std::getline(input, line)) {
1604         if (line.empty() || line[0] == '#') {
1605             continue;
1606         }
1607         std::istringstream line_stream(line);
1608         std::string left;
1609         std::string right;
1610         if (!(line_stream >> left >> right)) {
1611             throw std::runtime_error("invalid GPT-2 merges line");
1612         }
1613         ranks.emplace(pair_key(left, right), rank);
1614         ++rank;
1615     }
1616     return ranks;
1617 }
1618
1619 void transpose_hf_conv1d(std::vector<float>& values,
1620                         std::int32_t input_size,
1621                         std::int32_t output_size) {
1622     if (values.size() != matrix_size(input_size, output_size)) {
1623         throw std::runtime_error("GPT-2 Conv1D tensor size mismatch");
1624     }
1625     std::vector<float> transposed(matrix_size(output_size, input_size), 0.0F);
1626     for (std::int32_t input = 0; input < input_size; ++input) {
1627         for (std::int32_t output = 0; output < output_size; ++output) {
1628             transposed[matrix_size(output, input_size) + checked_size(input, "↵
↵ input")] =
1629                 values[matrix_size(input, output_size) + checked_size(output, "↵
↵ output")];
1630         }
1631     }
1632     values = std::move(transposed);
1633 }
1634
1635 std::vector<float> require_tensor(const SafeTensorFile& file,
1636                                  std::string_view name,
1637                                  std::span<const std::int64_t> expected_shape)↵
↵ {

```



```

1638     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
1639     if (entry == nullptr) {
1640         throw std::runtime_error("missing GPT-2 tensor: " + std::string(name));
1641     }
1642     if (entry->shape.size() != expected_shape.size()) {
1643         throw std::runtime_error("GPT-2 tensor rank mismatch: " + std::string(↵
↵ name));
1644     }
1645     for (std::size_t index = 0; index < expected_shape.size(); ++index) {
1646         if (entry->shape[index] != expected_shape[index]) {
1647             throw std::runtime_error("GPT-2 tensor shape mismatch: " + std::↵
↵ string(name));
1648         }
1649     }
1650     return read_safetensor_f32_tensor(file, name);
1651 }
1652
1653 std::vector<float> require_first_tensor(const SafeTensorFile& file,
1654                                         std::span<const std::string_view> names↵
↵ ,
1655                                         std::span<const std::int64_t> ↵
↵ expected_shape) {
1656     for (const std::string_view name : names) {
1657         if (file.manifest.find_tensor(name) != nullptr) {
1658             return require_tensor(file, name, expected_shape);
1659         }
1660     }
1661     throw std::runtime_error("missing GPT-2 tensor: " + std::string(names.front↵
↵ ()));
1662 }
1663
1664 Gpt2LinearF32 make_linear(std::int32_t input_size,
1665                           std::int32_t output_size,
1666                           std::vector<float> weights,
1667                           std::vector<float> bias = {}) {
1668     Gpt2LinearF32 linear;
1669     linear.input_size = input_size;
1670     linear.output_size = output_size;
1671     linear.weights = std::move(weights);
1672     linear.bias = std::move(bias);
1673     validate_linear(linear, "make GPT-2 linear");
1674     return linear;
1675 }
1676
1677 std::vector<float> zeros(std::int32_t count) {
1678     return std::vector<float>(checked_size(count, "zero count"), 0.0F);
1679 }
1680
1681 std::vector<float> ones(std::int32_t count) {
1682     return std::vector<float>(checked_size(count, "one count"), 1.0F);
1683 }
1684

```

```

1685 std::string tensor_name(std::int32_t layer, std::string_view suffix) {
1686     return "transformer.h." + std::to_string(layer) + "." + std::string(suffix) ←
    ↪ ;
1687 }
1688
1689 std::string bare_tensor_name(std::int32_t layer, std::string_view suffix) {
1690     return "h." + std::to_string(layer) + "." + std::string(suffix);
1691 }
1692
1693 std::array<std::string_view, 2> embedding_names(std::string_view bare) {
1694     if (bare == "wte.weight") {
1695         return {"transformer.wte.weight", "wte.weight"};
1696     }
1697     if (bare == "wpe.weight") {
1698         return {"transformer.wpe.weight", "wpe.weight"};
1699     }
1700     if (bare == "ln_f.weight") {
1701         return {"transformer.ln_f.weight", "ln_f.weight"};
1702     }
1703     if (bare == "ln_f.bias") {
1704         return {"transformer.ln_f.bias", "ln_f.bias"};
1705     }
1706     throw std::runtime_error("unsupported GPT-2 embedding tensor alias");
1707 }
1708
1709 std::array<std::string, 2> layer_names(std::int32_t layer, std::string_view ←
    ↪ suffix) {
1710     return {tensor_name(layer, suffix), bare_tensor_name(layer, suffix)};
1711 }
1712
1713 std::vector<float> require_layer_tensor(const SafeTensorFile& file,
1714                                         std::int32_t layer,
1715                                         std::string_view suffix,
1716                                         std::span<const std::int64_t> ←
    ↪ expected_shape) {
1717     const std::array<std::string, 2> names = layer_names(layer, suffix);
1718     const std::array<std::string_view, 2> views{names[0], names[1]};
1719     return require_first_tensor(file, views, expected_shape);
1720 }
1721
1722 std::filesystem::path find_gpt2_weight_file(const std::filesystem::path& ←
    ↪ model_directory) {
1723     const std::array<const char*, 2> candidates{
1724         "model.safetensors",
1725         "pytorch_model.safetensors",
1726     };
1727     for (const char* candidate : candidates) {
1728         const std::filesystem::path path = model_directory / candidate;
1729         if (std::filesystem::exists(path)) {
1730             return path;
1731         }
1732     }

```

```

1733     throw std::runtime_error("cannot find model.safetensors in GPT-2 directory"↵
↵ );
1734 }
1735
1736 } // namespace
1737
1738 Gpt2Tokenizer load_gpt2_tokenizer(const std::filesystem::path& vocab_json,
1739                                  const std::filesystem::path& merges_txt,
1740                                  std::int32_t bos_token_id,
1741                                  std::int32_t eos_token_id) {
1742     Gpt2Tokenizer tokenizer;
1743     tokenizer.bos_token_id = bos_token_id;
1744     tokenizer.eos_token_id = eos_token_id;
1745     tokenizer.vocab = parse_vocab_json(read_text_file(vocab_json));
1746     tokenizer.id_to_token = build_id_to_token(tokenizer.vocab);
1747     tokenizer.bpe_ranks = parse_merges(read_text_file(merges_txt));
1748     if (tokenizer.vocab.empty() || tokenizer.bpe_ranks.empty()) {
1749         throw std::runtime_error("GPT-2 tokenizer assets are empty");
1750     }
1751     return tokenizer;
1752 }
1753
1754 std::vector<std::int32_t> gpt2_encode(const Gpt2Tokenizer& tokenizer,
1755                                       std::string_view text) {
1756     std::vector<std::int32_t> ids;
1757     for (const std::string& piece : gpt2_pretokenize(text)) {
1758         const std::string byte_piece = bytes_to_bpe_alphabet(piece);
1759         const std::vector<std::string> bpe_tokens = bpe_apply(tokenizer, ↵
↵ byte_piece);
1760         for (const std::string& token : bpe_tokens) {
1761             const auto found = tokenizer.vocab.find(token);
1762             if (found == tokenizer.vocab.end()) {
1763                 throw std::runtime_error("GPT-2 BPE token is missing from vocab"↵
↵ );
1764             }
1765             ids.push_back(found->second);
1766         }
1767     }
1768     return ids;
1769 }
1770
1771 std::string gpt2_decode(const Gpt2Tokenizer& tokenizer,
1772                        std::span<const std::int32_t> token_ids) {
1773     static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode↵
↵ ();
1774     static const std::unordered_map<std::string, unsigned char> BYTE_DECODER =
1775         gpt2_unicode_to_bytes(BYTE_ENCODER);
1776
1777     std::string token_text;
1778     for (const std::int32_t token_id : token_ids) {
1779         if (token_id < 0 || token_id >= static_cast<std::int32_t>(tokenizer.↵
↵ id_to_token.size())) {

```

```

1780         throw std::runtime_error("GPT-2 token id out of range");
1781     }
1782     token_text += tokenizer.id_to_token[checked_size(token_id, "token id")↵
↵ ];
1783 }
1784
1785 std::string result;
1786 const std::vector<std::string> symbols = utf8_symbols(token_text);
1787 for (const std::string& symbol : symbols) {
1788     const auto found = BYTE_DECODER.find(symbol);
1789     if (found == BYTE_DECODER.end()) {
1790         throw std::runtime_error("GPT-2 token cannot be decoded to byte");
1791     }
1792     result.push_back(static_cast<char>(found->second));
1793 }
1794 return result;
1795 }
1796
1797 Gpt2Config load_gpt2_config(const std::filesystem::path& config_json) {
1798     const std::string config_text = read_text_file(config_json);
1799     JsonCursor cursor(config_text);
1800     Gpt2Config config;
1801     config.bos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;
1802     config.eos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;
1803
1804     cursor.expect('{');
1805     if (cursor.consume('}')) {
1806         throw std::runtime_error("GPT-2 config is empty");
1807     }
1808     while (true) {
1809         const std::string key = cursor.parse_string();
1810         cursor.expect(':');
1811         if (key == "n_embd") {
1812             config.hidden_size = parse_i32_field(cursor);
1813         } else if (key == "n_head") {
1814             config.head_count = parse_i32_field(cursor);
1815         } else if (key == "n_layer") {
1816             config.layer_count = parse_i32_field(cursor);
1817         } else if (key == "n_positions" || key == "n_ctx") {
1818             const std::int32_t value = parse_i32_field(cursor);
1819             config.max_positions = std::max(config.max_positions, value);
1820         } else if (key == "vocab_size") {
1821             config.vocab_size = parse_i32_field(cursor);
1822         } else if (key == "n_inner") {
1823             config.intermediate_size = parse_i32_field(cursor);
1824         } else if (key == "layer_norm_epsilon") {
1825             config.layer_norm_epsilon = static_cast<float>(cursor.parse_number↵
↵ ());
1826         } else if (key == "activation_function") {
1827             config.activation_function = cursor.parse_string();
1828         } else if (key == "bos_token_id") {
1829             config.bos_token_id = parse_i32_field(cursor);

```

```

1830     } else if (key == "eos_token_id") {
1831         config.eos_token_id = parse_i32_field(cursor);
1832     } else {
1833         cursor.skip_value();
1834     }
1835     if (cursor.consume('')) {
1836         break;
1837     }
1838     cursor.expect(',');
1839 }
1840 if (!cursor.eof_after_ws()) {
1841     throw std::runtime_error("trailing data after GPT-2 config JSON");
1842 }
1843 if (config.intermediate_size == 0 && config.hidden_size > 0) {
1844     config.intermediate_size = config.hidden_size * 4;
1845 }
1846 validate_config(config);
1847 return config;
1848 }
1849
1850 Gpt2ReferenceModel load_gpt2_reference_model(const std::filesystem::path& ↵
    ↵ config_json,
1851                                             const std::filesystem::path& ↵
    ↵ safetensors_path) {
1852     Gpt2ReferenceModel model;
1853     model.config = load_gpt2_config(config_json);
1854     const SafeTensorFile file = load_safetensors_file(safetensors_path);
1855     const Gpt2Config& config = model.config;
1856     const std::array<std::int64_t, 2> wte_shape{config.vocab_size, config.↵
    ↵ hidden_size};
1857     const std::array<std::int64_t, 2> wpe_shape{config.max_positions, config.↵
    ↵ hidden_size};
1858     const std::array<std::int64_t, 1> hidden_shape{config.hidden_size};
1859
1860     const std::array<std::string_view, 2> wte_names = embedding_names("wte.↵
    ↵ weight");
1861     const std::array<std::string_view, 2> wpe_names = embedding_names("wpe.↵
    ↵ weight");
1862     model.token_embedding = require_first_tensor(file, wte_names, wte_shape);
1863     model.position_embedding = require_first_tensor(file, wpe_names, wpe_shape)↵
    ↵ ;
1864     model.layers.resize(check_size(config.layer_count, "layer_count"));
1865
1866     for (std::int32_t layer_id = 0; layer_id < config.layer_count; ++layer_id) ↵
    ↵ {
1867         Gpt2LayerWeightsF32& layer = model.layers[check_size(layer_id, "↵
    ↵ layer_id")];
1868         layer.ln_1_weight = require_layer_tensor(file, layer_id, "ln_1.weight",↵
    ↵ hidden_shape);
1869         layer.ln_1_bias = require_layer_tensor(file, layer_id, "ln_1.bias", ↵
    ↵ hidden_shape);
1870         layer.ln_2_weight = require_layer_tensor(file, layer_id, "ln_2.weight",↵

```

```

1871     layer.ln_2_bias = require_layer_tensor(file, layer_id, "ln_2.bias", ↵
↵ hidden_shape);
1872
1873     const std::array<std::int64_t, 2> c_attn_shape{
1874         config.hidden_size,
1875         config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
1876     std::vector<float> c_attn_weight =
1877         require_layer_tensor(file, layer_id, "attn.c_attn.weight", ↵
↵ c_attn_shape);
1878     transpose_hf_conv1d(c_attn_weight,
1879         config.hidden_size,
1880         config.hidden_size * ↵
↵ LCQI_GPT2_ATTENTION_PROJECTIONS);
1881     const std::array<std::int64_t, 1> c_attn_bias_shape{
1882         config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
1883     layer.c_attn = make_linear(config.hidden_size,
1884         config.hidden_size * ↵
↵ LCQI_GPT2_ATTENTION_PROJECTIONS,
1885         std::move(c_attn_weight),
1886         require_layer_tensor(file,
1887             layer_id,
1888             "attn.c_attn.bias",
1889             c_attn_bias_shape));
1890
1891     const std::array<std::int64_t, 2> c_proj_shape{config.hidden_size, ↵
↵ config.hidden_size};
1892     std::vector<float> c_proj_weight =
1893         require_layer_tensor(file, layer_id, "attn.c_proj.weight", ↵
↵ c_proj_shape);
1894     transpose_hf_conv1d(c_proj_weight, config.hidden_size, config.↵
↵ hidden_size);
1895     layer.c_proj = make_linear(config.hidden_size,
1896         config.hidden_size,
1897         std::move(c_proj_weight),
1898         require_layer_tensor(file,
1899             layer_id,
1900             "attn.c_proj.bias",
1901             hidden_shape));
1902
1903     const std::array<std::int64_t, 2> c_fc_shape{
1904         config.hidden_size,
1905         config.intermediate_size};
1906     std::vector<float> c_fc_weight =
1907         require_layer_tensor(file, layer_id, "mlp.c_fc.weight", c_fc_shape)↵
↵ ;
1908     transpose_hf_conv1d(c_fc_weight, config.hidden_size, config.↵
↵ intermediate_size);
1909     const std::array<std::int64_t, 1> intermediate_shape{config.↵
↵ intermediate_size};
1910     layer.c_fc = make_linear(config.hidden_size,
1911         config.intermediate_size,

```

```

1912         std::move(c_fc_weight),
1913         require_layer_tensor(file,
1914                               layer_id,
1915                               "mlp.c_fc.bias",
1916                               intermediate_shape));
1917
1918         const std::array<std::int64_t, 2> mlp_proj_shape{
1919             config.intermediate_size,
1920             config.hidden_size};
1921         std::vector<float> mlp_proj_weight =
1922             require_layer_tensor(file, layer_id, "mlp.c_proj.weight", ↵
↵ mlp_proj_shape);
1923         transpose_hf_conv1d(mlp_proj_weight, config.intermediate_size, config.↵
↵ hidden_size);
1924         layer.mlp_c_proj = make_linear(config.intermediate_size,
1925                                       config.hidden_size,
1926                                       std::move(mlp_proj_weight),
1927                                       require_layer_tensor(file,
1928                                                             layer_id,
1929                                                             "mlp.c_proj.bias",
1930                                                             hidden_shape));
1931     }
1932
1933     const std::array<std::string_view, 2> ln_weight_names = embedding_names("↵
↵ ln_f.weight");
1934     const std::array<std::string_view, 2> ln_bias_names = embedding_names("ln_f↵
↵ .bias");
1935     model.final_ln_weight = require_first_tensor(file, ln_weight_names, ↵
↵ hidden_shape);
1936     model.final_ln_bias = require_first_tensor(file, ln_bias_names, ↵
↵ hidden_shape);
1937     if (file.manifest.find_tensor("lm_head.weight") != nullptr) {
1938         model.tie_lm_head_to_embedding = false;
1939         model.lm_head_weight = require_tensor(file, "lm_head.weight", wte_shape↵
↵ );
1940     } else {
1941         model.tie_lm_head_to_embedding = true;
1942     }
1943     validate_model(model);
1944     return model;
1945 }
1946
1947 Gpt2ReferenceModel load_gpt2_from_directory(const std::filesystem::path& ↵
↵ model_directory) {
1948     return load_gpt2_reference_model(model_directory / "config.json",
1949                                     find_gpt2_weight_file(model_directory));
1950 }
1951
1952 Gpt2ForwardWorkspace::Gpt2ForwardWorkspace(const Gpt2Config& config) : config_(↵
↵ config) {
1953     this->reset_for_config(config);
1954 }

```

```

1955
1956 void Gpt2ForwardWorkspace::reset_for_config(const Gpt2Config& config) {
1957     validate_config(config);
1958     this->config_ = config;
1959     const std::size_t hidden_size = checked_size(this->config_.hidden_size, "↵
↵ hidden_size");
1960     const std::size_t intermediate_size =
1961         checked_size(this->config_.intermediate_size, "intermediate_size");
1962     this->hidden_.assign(hidden_size, 0.0F);
1963     this->normed_.assign(hidden_size, 0.0F);
1964     this->query_.assign(hidden_size, 0.0F);
1965     this->key_.assign(hidden_size, 0.0F);
1966     this->value_.assign(hidden_size, 0.0F);
1967     this->attention_.assign(hidden_size, 0.0F);
1968     this->projected_.assign(hidden_size, 0.0F);
1969     this->qkv_packed_.assign(
1970         matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS, this->config_.hidden_size)↵
↵ ,
1971         0.0F);
1972     this->mlp_fc_.assign(intermediate_size, 0.0F);
1973     this->mlp_out_.assign(hidden_size, 0.0F);
1974     this->logits_.assign(checked_size(this->config_.vocab_size, "vocab_size"), ↵
↵ 0.0F);
1975     this->scores_.assign(checked_size(this->config_.max_positions, "↵
↵ max_positions"), 0.0F);
1976 }
1977
1978 std::span<float> Gpt2ForwardWorkspace::hidden() noexcept {
1979     return this->hidden_;
1980 }
1981
1982 std::span<float> Gpt2ForwardWorkspace::normed() noexcept {
1983     return this->normed_;
1984 }
1985
1986 std::span<float> Gpt2ForwardWorkspace::query() noexcept {
1987     return this->query_;
1988 }
1989
1990 std::span<float> Gpt2ForwardWorkspace::key() noexcept {
1991     return this->key_;
1992 }
1993
1994 std::span<float> Gpt2ForwardWorkspace::value() noexcept {
1995     return this->value_;
1996 }
1997
1998 std::span<float> Gpt2ForwardWorkspace::attention() noexcept {
1999     return this->attention_;
2000 }
2001
2002 std::span<float> Gpt2ForwardWorkspace::projected() noexcept {

```



```

2003     return this->projected_;
2004 }
2005
2006 std::span<float> Gpt2ForwardWorkspace::qkv_packed() noexcept {
2007     return this->qkv_packed_;
2008 }
2009
2010 std::span<float> Gpt2ForwardWorkspace::mlp_fc() noexcept {
2011     return this->mlp_fc_;
2012 }
2013
2014 std::span<float> Gpt2ForwardWorkspace::mlp_out() noexcept {
2015     return this->mlp_out_;
2016 }
2017
2018 std::span<float> Gpt2ForwardWorkspace::logits() noexcept {
2019     return this->logits_;
2020 }
2021
2022 std::span<float> Gpt2ForwardWorkspace::scores_prefix(std::int32_t count) {
2023     if (count < 0 || count > this->config_.max_positions) {
2024         throw std::runtime_error("GPT-2 attention scores count out of range");
2025     }
2026     return std::span<float>(this->scores_.data(), checked_size(count, "scores ←
↪ count"));
2027 }
2028
2029 Gpt2KvCache::Gpt2KvCache(const Gpt2Config& config) : config_(config) {
2030     validate_config(this->config_);
2031     const std::size_t element_count =
2032         checked_size(this->config_.layer_count, "layer_count") *
2033         checked_size(this->config_.max_positions, "max_positions") *
2034         checked_size(this->config_.head_count, "head_count") *
2035         checked_size(head_dim(this->config_), "head_dim");
2036     this->keys_.assign(element_count, 0.0F);
2037     this->values_.assign(element_count, 0.0F);
2038     this->written_.assign(
2039         checked_size(this->config_.layer_count, "layer_count") *
2040         checked_size(this->config_.max_positions, "max_positions"),
2041         0);
2042 }
2043
2044 void Gpt2KvCache::append(std::int32_t layer_id,
2045                          std::int32_t model_position,
2046                          std::span<const float> key,
2047                          std::span<const float> value) {
2048     const std::size_t hidden_size = checked_size(this->config_.hidden_size, "↪
↪ hidden_size");
2049     if (key.size() != hidden_size || value.size() != hidden_size) {
2050         throw std::runtime_error("GPT-2 KV key/value size mismatch");
2051     }
2052     this->validate_address(layer_id, model_position, 0);

```

```

2053     const std::int32_t local_head_dim = head_dim(this->config_);
2054     for (std::int32_t head = 0; head < this->config_.head_count; ++head) {
2055         const std::size_t source =
2056             checked_size(head, "head") * checked_size(local_head_dim, "head_dim"
↵ ");
2057         const std::size_t target = this->base_offset(layer_id, model_position,
↵ head);
2058         std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
2059                     local_head_dim,
2060                     this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
2061         std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
2062                     local_head_dim,
2063                     this->values_.begin() + static_cast<std::ptrdiff_t>(target)
↵ );
2064     }
2065     const std::size_t written_index =
2066         checked_size(layer_id, "layer_id") *
2067         checked_size(this->config_.max_positions, "max_positions") +
2068         checked_size(model_position, "model_position");
2069     this->written_[written_index] = 1;
2070     this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
2071 }
2072
2073 std::span<const float> Gpt2KvCache::key(std::int32_t layer_id,
2074                                         std::int32_t model_position,
2075                                         std::int32_t head) const {
2076     this->validate_address(layer_id, model_position, head);
2077     this->validate_written(layer_id, model_position);
2078     return this->key_unchecked(layer_id, model_position, head);
2079 }
2080
2081 std::span<const float> Gpt2KvCache::value(std::int32_t layer_id,
2082                                           std::int32_t model_position,
2083                                           std::int32_t head) const {
2084     this->validate_address(layer_id, model_position, head);
2085     this->validate_written(layer_id, model_position);
2086     return this->value_unchecked(layer_id, model_position, head);
2087 }
2088
2089 void detail::attend_cached_position(const Gpt2KvCache& cache,
2090                                   std::int32_t layer_id,
2091                                   std::int32_t model_position,
2092                                   std::span<const float> query,
2093                                   std::span<float> scores,
2094                                   std::span<float> output) {
2095     cache.validate_address(layer_id, model_position, 0);
2096     cache.validate_written(layer_id, model_position);
2097     const std::int32_t local_head_dim = head_dim(cache.config_);
2098     const std::size_t hidden_size = checked_size(cache.config_.hidden_size, "
↵ hidden_size");
2099     const std::size_t local_head_dim_size = checked_size(local_head_dim, "
↵ head_dim");

```

[illegible]

```

2149                                     std::int32_t head) const ↵
    ↵ noexcept {
2150     const std::size_t offset = this->base_offset_unchecked(layer_id, ↵
    ↵ model_position, head);
2151     return std::span<const float>(
2152         this->keys_.data() + offset,
2153         static_cast<std::size_t>(head_dim(this->config_)));
2154 }
2155
2156 std::span<const float> Gpt2KvCache::value_unchecked(std::int32_t layer_id,
2157                                                     std::int32_t model_position↵
    ↵ ,
2158                                                     std::int32_t head) const ↵
    ↵ noexcept {
2159     const std::size_t offset = this->base_offset_unchecked(layer_id, ↵
    ↵ model_position, head);
2160     return std::span<const float>(
2161         this->values_.data() + offset,
2162         static_cast<std::size_t>(head_dim(this->config_)));
2163 }
2164
2165 std::int32_t Gpt2KvCache::filled_tokens() const noexcept {
2166     return this->filled_tokens_;
2167 }
2168
2169 std::size_t Gpt2KvCache::byte_size() const noexcept {
2170     return (this->keys_.size() + this->values_.size()) * sizeof(float) +
2171         this->written_.size() * sizeof(std::uint8_t);
2172 }
2173
2174 std::size_t Gpt2KvCache::base_offset(std::int32_t layer_id,
2175                                       std::int32_t model_position,
2176                                       std::int32_t head) const {
2177     return (((checked_size(layer_id, "layer_id") *
2178         checked_size(this->config_.max_positions, "max_positions")) +
2179         checked_size(model_position, "model_position")) *
2180         checked_size(this->config_.head_count, "head_count") +
2181         checked_size(head, "head")) *
2182         checked_size(head_dim(this->config_), "head_dim");
2183 }
2184
2185 std::size_t Gpt2KvCache::base_offset_unchecked(std::int32_t layer_id,
2186                                                 std::int32_t model_position,
2187                                                 std::int32_t head) const ↵
    ↵ noexcept {
2188     return (((static_cast<std::size_t>(layer_id) *
2189         static_cast<std::size_t>(this->config_.max_positions)) +
2190         static_cast<std::size_t>(model_position)) *
2191         static_cast<std::size_t>(this->config_.head_count) +
2192         static_cast<std::size_t>(head)) *
2193         static_cast<std::size_t>(head_dim(this->config_));
2194 }

```

```

2195
2196 void Gpt2KvCache::validate_address(std::int32_t layer_id,
2197                                   std::int32_t model_position,
2198                                   std::int32_t head) const {
2199     if (layer_id < 0 || layer_id >= this->config_.layer_count) {
2200         throw std::runtime_error("GPT-2 KV layer_id out of range");
2201     }
2202     if (model_position < 0 || model_position >= this->config_.max_positions) {
2203         throw std::runtime_error("GPT-2 KV model_position out of range");
2204     }
2205     if (head < 0 || head >= this->config_.head_count) {
2206         throw std::runtime_error("GPT-2 KV head out of range");
2207     }
2208 }
2209
2210 void Gpt2KvCache::validate_written(std::int32_t layer_id,
2211                                   std::int32_t model_position) const {
2212     const std::size_t written_index =
2213         checked_size(layer_id, "layer_id") *
2214         checked_size(this->config_.max_positions, "max_positions") +
2215         checked_size(model_position, "model_position");
2216     if (written_index >= this->written_.size() || this->written_[written_index] ←
    ↪ == 0) {
2217         throw std::runtime_error("GPT-2 KV slot has not been written");
2218     }
2219 }
2220
2221 Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& ←
    ↪ model)
2222     : Gpt2CachedGreedyDecoder(model, nullptr) {}
2223
2224 Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(
2225     const Gpt2ReferenceModel& model,
2226     Gpt2HotspotProfile* hotspot_profile)
2227     : Gpt2CachedGreedyDecoder(model, hotspot_profile, Gpt2ExecutionOptions{}) ←
    ↪ {}
2228
2229 Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(
2230     const Gpt2ReferenceModel& model,
2231     Gpt2HotspotProfile* hotspot_profile,
2232     const Gpt2ExecutionOptions& execution_options)
2233     : model_(&model),
2234       hotspot_profile_(hotspot_profile),
2235       execution_options_(execution_options),
2236       cache_(model.config),
2237       workspace_(model.config),
2238       worker_pool_(make_worker_pool(model, execution_options)) {
2239     validate_model(*this->model_);
2240 }
2241
2242 Gpt2CachedGreedyDecoder::~Gpt2CachedGreedyDecoder() = default;
2243

```

```

2244 Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(Gpt2CachedGreedyDecoder&&) ←
    ↪ noexcept =
2245     default;
2246
2247 Gpt2CachedGreedyDecoder& Gpt2CachedGreedyDecoder::operator=(
2248     Gpt2CachedGreedyDecoder&&) noexcept = default;
2249
2250 void Gpt2CachedGreedyDecoder::advance_without_prediction(std::int32_t token_id)↪
    ↪ {
2251     static_cast<void>(run_gpt2_cached_step_unchecked(*this->model_,
2252                                                         this->cache_,
2253                                                         this->workspace_,
2254                                                         token_id,
2255                                                         Gpt2CachedStepMode::↪
    ↪ advance_only,
2256                                                         this->hotspot_profile_,
2257                                                         this->execution_options_,
2258                                                         this->worker_pool_.get()));
2259 }
2260
2261 std::int32_t Gpt2CachedGreedyDecoder::step(std::int32_t token_id) {
2262     return run_gpt2_cached_step_unchecked(*this->model_,
2263                                             this->cache_,
2264                                             this->workspace_,
2265                                             token_id,
2266                                             Gpt2CachedStepMode::predict_token,
2267                                             this->hotspot_profile_,
2268                                             this->execution_options_,
2269                                             this->worker_pool_.get())
2270     .predicted_token;
2271 }
2272
2273 Gpt2ForwardResult Gpt2CachedGreedyDecoder::step_with_logits(std::int32_t ↪
    ↪ token_id) {
2274     return run_gpt2_cached_step_unchecked(*this->model_,
2275                                             this->cache_,
2276                                             this->workspace_,
2277                                             token_id,
2278                                             Gpt2CachedStepMode::logits,
2279                                             this->hotspot_profile_,
2280                                             this->execution_options_,
2281                                             this->worker_pool_.get());
2282 }
2283
2284 const Gpt2KvCache& Gpt2CachedGreedyDecoder::cache() const noexcept {
2285     return this->cache_;
2286 }
2287
2288 std::int32_t Gpt2CachedGreedyDecoder::filled_tokens() const noexcept {
2289     return this->cache_.filled_tokens();
2290 }
2291

```

```

2292 std::size_t Gpt2CachedGreedyDecoder::kv_cache_bytes() const noexcept {
2293     return this->cache_.byte_size();
2294 }
2295
2296 std::int32_t Gpt2CachedGreedyDecoder::worker_count() const noexcept {
2297     return this->worker_pool_ == nullptr ? 1 : this->worker_pool_->worker_count()
↵ ();
2298 }
2299
2300 Gpt2ForwardResult run_gpt2_forward(const Gpt2ReferenceModel& model,
2301                                     std::span<const std::int32_t> token_ids) {
2302     validate_model(model);
2303     if (token_ids.empty()) {
2304         throw std::runtime_error("GPT-2 forward needs at least one token");
2305     }
2306     if (token_ids.size() > checked_size(model.config.max_positions, "↵
↵ max_positions")) {
2307         throw std::runtime_error("GPT-2 token_ids exceed max_positions");
2308     }
2309
2310     const std::int32_t sequence_length = static_cast<std::int32_t>(token_ids.↵
↵ size());
2311     std::vector<float> hidden_by_pos(matrix_size(sequence_length, model.config.↵
↵ hidden_size), 0.0F);
2312     for (std::int32_t position = 0; position < sequence_length; ++position) {
2313         const std::int32_t token_id = token_ids[checked_size(position, "↵
↵ position")];
2314         if (token_id < 0 || token_id >= model.config.vocab_size) {
2315             throw std::runtime_error("GPT-2 token id out of range");
2316         }
2317         const std::span<const float> token =
2318             row_span(model.token_embedding, token_id, model.config.hidden_size)↵
↵ ;
2319         const std::span<const float> position_embedding =
2320             row_span(model.position_embedding, position, model.config.↵
↵ hidden_size);
2321         std::span<float> hidden = std::span<float>(
2322             hidden_by_pos.data() + matrix_size(position, model.config.↵
↵ hidden_size),
2323             checked_size(model.config.hidden_size, "hidden_size"));
2324         for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
2325             hidden[checked_size(dim, "dim")] =
2326                 token[checked_size(dim, "dim")] + position_embedding[↵
↵ checked_size(dim, "dim")];
2327         }
2328     }
2329
2330     for (const Gpt2LayerWeightsF32& layer : model.layers) {
2331         forward_gpt2_layer(model.config, layer, sequence_length, hidden_by_pos)↵
↵ ;
2332     }
2333

```



```

2334     std::vector<float> normed(check_size(model.config.hidden_size, "↵
↵ hidden_size"), 0.0F);
2335     const std::span<const float> final_hidden = std::span<const float>(
2336         hidden_by_pos.data() + matrix_size(sequence_length - 1, model.config.↵
↵ hidden_size),
2337         checked_size(model.config.hidden_size, "hidden_size"));
2338     layer_norm(final_hidden,
2339                 model.final_ln_weight,
2340                 model.final_ln_bias,
2341                 model.config.layer_norm_epsilon,
2342                 normed);
2343
2344     Gpt2ForwardResult result;
2345     result.logits.assign(check_size(model.config.vocab_size, "vocab_size"), ↵
↵ 0.0F);
2346     compute_logits(model, normed, result.logits);
2347     result.predicted_token = argmax(result.logits);
2348     return result;
2349 }
2350
2351 Gpt2ForwardResult run_gpt2_forward_cached(const Gpt2ReferenceModel& model,
2352                                           Gpt2KvCache& cache,
2353                                           std::int32_t token_id) {
2354     validate_model(model);
2355     Gpt2ForwardWorkspace workspace(model.config);
2356     Gpt2ExecutionOptions options;
2357     options.worker_count = 1;
2358     return run_gpt2_cached_step_unchecked(
2359         model,
2360         cache,
2361         workspace,
2362         token_id,
2363         Gpt2CachedStepMode::logits,
2364         nullptr,
2365         options,
2366         nullptr);
2367 }
2368
2369 std::vector<std::int32_t> gpt2_generate_greedy(const Gpt2ReferenceModel& model,
2370                                               std::span<const std::int32_t> ↵
↵ prompt_token_ids,
2371                                               std::int32_t max_new_tokens) {
2372     if (max_new_tokens < 0) {
2373         throw std::runtime_error("max_new_tokens cannot be negative");
2374     }
2375     std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids↵
↵ .end());
2376     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
2377         if (tokens.size() >= checked_size(model.config.max_positions, "↵
↵ max_positions")) {
2378             throw std::runtime_error("GPT-2 generation would exceed ↵
↵ max_positions");

```



```

2379     }
2380     const Gpt2ForwardResult result = run_gpt2_forward(model, tokens);
2381     tokens.push_back(result.predicted_token);
2382     if (model.config.eos_token_id >= 0 && result.predicted_token == model.↵
↵ config.eos_token_id) {
2383         break;
2384     }
2385 }
2386 return tokens;
2387 }
2388
2389 std::vector<std::int32_t> gpt2_generate_greedy_cached(
2390     const Gpt2ReferenceModel& model,
2391     std::span<const std::int32_t> prompt_token_ids,
2392     std::int32_t max_new_tokens) {
2393     if (prompt_token_ids.empty()) {
2394         throw std::runtime_error("GPT-2 generation needs at least one prompt ↵
↵ token");
2395     }
2396     if (max_new_tokens < 0) {
2397         throw std::runtime_error("max_new_tokens cannot be negative");
2398     }
2399     if (prompt_token_ids.size() > checked_size(model.config.max_positions, "↵
↵ max_positions")) {
2400         throw std::runtime_error("GPT-2 prompt exceeds max_positions");
2401     }
2402
2403     Gpt2CachedGreedyDecoder decoder(model);
2404     std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids.↵
↵ .end());
2405     if (max_new_tokens == 0) {
2406         return tokens;
2407     }
2408     for (std::size_t index = 0; index + 1 < prompt_token_ids.size(); ++index) {
2409         decoder.advance_without_prediction(prompt_token_ids[index]);
2410     }
2411     std::int32_t predicted_token = decoder.step(prompt_token_ids.back());
2412     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
2413         if (tokens.size() >= checked_size(model.config.max_positions, "↵
↵ max_positions")) {
2414             throw std::runtime_error("GPT-2 generation would exceed ↵
↵ max_positions");
2415         }
2416         tokens.push_back(predicted_token);
2417         if (model.config.eos_token_id >= 0 && predicted_token == model.config.↵
↵ eos_token_id) {
2418             break;
2419         }
2420         if (step + 1 < max_new_tokens) {
2421             predicted_token = decoder.step(predicted_token);
2422         }
2423     }

```

```

2424     return tokens;
2425 }
2426
2427 Gpt2ReferenceModel make_tiny_gpt2_reference_model() {
2428     Gpt2ReferenceModel model;
2429     model.config.hidden_size = 4;
2430     model.config.head_count = 2;
2431     model.config.layer_count = 1;
2432     model.config.max_positions = 8;
2433     model.config.vocab_size = 6;
2434     model.config.intermediate_size = 8;
2435     model.config.bos_token_id = 0;
2436     model.config.eos_token_id = 5;
2437     model.config.layer_norm_epsilon = 1.0e-5F;
2438     model.config.activation_function = "gelu_new";
2439
2440     model.token_embedding = {
2441         0.20F, -0.10F, 0.00F, 0.10F,
2442         -0.10F, 0.30F, 0.20F, -0.20F,
2443         0.05F, 0.10F, -0.30F, 0.25F,
2444         0.40F, -0.20F, 0.15F, 0.05F,
2445         -0.30F, 0.05F, 0.35F, -0.15F,
2446         0.10F, 0.20F, -0.10F, 0.30F,
2447     };
2448     model.position_embedding = {
2449         0.01F, 0.02F, 0.03F, 0.04F,
2450         -0.02F, 0.01F, 0.00F, 0.03F,
2451         0.03F, -0.01F, 0.02F, 0.00F,
2452         0.00F, 0.02F, -0.02F, 0.01F,
2453         0.02F, 0.00F, 0.01F, -0.01F,
2454         -0.01F, 0.03F, 0.00F, 0.02F,
2455         0.01F, -0.02F, 0.03F, 0.00F,
2456         0.00F, 0.01F, -0.01F, 0.02F,
2457     };
2458
2459     Gpt2LayerWeightsF32 layer;
2460     layer.ln_1_weight = ones(4);
2461     layer.ln_1_bias = zeros(4);
2462     layer.ln_2_weight = {0.9F, 1.1F, 1.0F, 0.95F};
2463     layer.ln_2_bias = {0.01F, -0.02F, 0.00F, 0.03F};
2464     layer.c_attn = make_linear(
2465         4,
2466         12,
2467         {
2468             0.10F, -0.20F, 0.05F, 0.15F,
2469             -0.05F, 0.12F, 0.20F, -0.10F,
2470             0.18F, 0.02F, -0.08F, 0.11F,
2471             -0.12F, 0.06F, 0.14F, 0.04F,
2472             0.07F, 0.16F, -0.09F, 0.03F,
2473             -0.15F, 0.05F, 0.13F, 0.10F,
2474             0.04F, -0.11F, 0.19F, -0.02F,
2475             0.09F, 0.08F, -0.06F, 0.17F,

```

```

2476         0.20F, -0.04F, 0.01F, 0.06F,
2477         -0.08F, 0.15F, 0.07F, -0.03F,
2478         0.05F, 0.10F, -0.14F, 0.12F,
2479         0.11F, -0.07F, 0.16F, 0.02F,
2480     },
2481     {
2482         0.01F, -0.02F, 0.00F, 0.03F,
2483         -0.01F, 0.02F, 0.01F, -0.02F,
2484         0.00F, 0.01F, -0.01F, 0.02F,
2485     });
2486     layer.c_proj = make_linear(
2487         4,
2488         4,
2489         {
2490             0.10F, 0.05F, -0.08F, 0.12F,
2491             -0.04F, 0.11F, 0.09F, -0.02F,
2492             0.07F, -0.10F, 0.13F, 0.05F,
2493             0.02F, 0.14F, -0.06F, 0.08F,
2494         },
2495         {0.01F, 0.00F, -0.01F, 0.02F});
2496     layer.c_fc = make_linear(
2497         4,
2498         8,
2499         {
2500             0.20F, -0.10F, 0.05F, 0.03F,
2501             -0.05F, 0.14F, 0.12F, -0.08F,
2502             0.09F, 0.07F, -0.11F, 0.16F,
2503             -0.12F, 0.05F, 0.18F, 0.02F,
2504             0.04F, -0.09F, 0.13F, 0.10F,
2505             0.11F, 0.03F, -0.07F, 0.15F,
2506             -0.02F, 0.17F, 0.06F, -0.04F,
2507             0.08F, -0.06F, 0.10F, 0.12F,
2508         },
2509         {0.01F, -0.02F, 0.00F, 0.03F, -0.01F, 0.02F, 0.01F, 0.00F});
2510     layer.mlp_c_proj = make_linear(
2511         8,
2512         4,
2513         {
2514             0.10F, -0.08F, 0.06F, 0.12F, -0.04F, 0.07F, 0.03F, 0.11F,
2515             -0.05F, 0.13F, 0.02F, -0.09F, 0.15F, 0.04F, -0.03F, 0.08F,
2516             0.07F, 0.01F, -0.12F, 0.10F, 0.06F, -0.05F, 0.14F, 0.02F,
2517             0.03F, 0.09F, 0.11F, -0.06F, 0.05F, 0.12F, -0.08F, 0.04F,
2518         },
2519         {0.00F, 0.01F, -0.01F, 0.02F});
2520     model.layers.push_back(std::move(layer));
2521     model.final_ln_weight = {1.0F, 0.95F, 1.05F, 1.0F};
2522     model.final_ln_bias = {0.0F, 0.01F, -0.01F, 0.02F};
2523     model.tie_lm_head_to_embedding = true;
2524     validate_model(model);
2525     return model;
2526 }
2527

```

```
2528 } // namespace lcqi
```

`gpt2_cli.cpp` 负责把模型目录、prompt、`max_new_tokens` 和 benchmark 选项接到 GPT-2 reference path。无参数时运行 tiny GPT-2 fixture；传入 HuggingFace 模型目录时加载真实资产。

Listing 5.5 `src/gpt2_cli.cpp`

```
1 #include <lcqi/gpt2_reference.hpp>
2
3 #include <chrono>
4 #include <cstdlib>
5 #include <exception>
6 #include <filesystem>
7 #include <iostream>
8 #include <span>
9 #include <stdexcept>
10 #include <string>
11 #include <string_view>
12 #include <vector>
13
14 namespace {
15
16 constexpr int LCQI_GPT2_TINY_ARGC = 1;
17 constexpr int LCQI_GPT2_MIN_REAL_ARGC = 3;
18 constexpr std::int32_t LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS = 8;
19 constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_0 = 1;
20 constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_1 = 2;
21 constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_2 = 3;
22
23 enum class Gpt2EngineMode {
24     full_prefix,
25     cached_kv,
26 };
27
28 struct Gpt2CliOptions {
29     Gpt2EngineMode engine = Gpt2EngineMode::cached_kv;
30     bool benchmark = false;
31     bool profile_hotspots = false;
32     std::filesystem::path model_dir;
33     std::string prompt;
34     std::int32_t max_new_tokens = LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS;
35     std::int32_t worker_count = 0;
36 };
37
38 struct Gpt2BenchmarkTimings {
39     double load_ms = 0.0;
40     double tokenize_ms = 0.0;
41     double prefill_ms = 0.0;
42     double decode_ms = 0.0;
43     double generate_ms = 0.0;
44     double total_ms = 0.0;
45     std::size_t prefill_steps = 0;
```

```

46     std::size_t decode_steps = 0;
47     std::size_t kv_cache_bytes = 0;
48     std::int32_t worker_count = 1;
49 };
50
51 using Clock = std::chrono::steady_clock;
52
53 void print_ids(std::span<const std::int32_t> ids) {
54     for (std::size_t index = 0; index < ids.size(); ++index) {
55         if (index != 0) {
56             std::cout << ' ';
57         }
58         std::cout << ids[index];
59     }
60 }
61
62 [[nodiscard]] double elapsed_ms(Clock::time_point start, Clock::time_point end)↵
    ↵ {
63     return std::chrono::duration<double, std::milli>(end - start).count();
64 }
65
66 [[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
67     if (value < 0) {
68         throw std::runtime_error(std::string(name) + " cannot be negative");
69     }
70     return static_cast<std::size_t>(value);
71 }
72
73 [[nodiscard]] const char* engine_name(Gpt2EngineMode engine) {
74     switch (engine) {
75         case Gpt2EngineMode::full_prefix:
76             return "full_prefix";
77         case Gpt2EngineMode::cached_kv:
78             return "cached_kv";
79     }
80     return "unknown";
81 }
82
83 [[nodiscard]] Gpt2EngineMode parse_engine(std::string_view value) {
84     if (value == "full" || value == "full_prefix") {
85         return Gpt2EngineMode::full_prefix;
86     }
87     if (value == "cached" || value == "cached_kv") {
88         return Gpt2EngineMode::cached_kv;
89     }
90     throw std::runtime_error("unknown --engine value, expected cached or full")↵
    ↵ ;
91 }
92
93 [[nodiscard]] std::int32_t parse_non_negative_i32(const std::string& text,
94                                                    const char* name) {
95     const std::int32_t value = static_cast<std::int32_t>(std::stoi(text));

```

```

96     if (value < 0) {
97         throw std::runtime_error(std::string(name) + " cannot be negative");
98     }
99     return value;
100 }
101
102 [[nodiscard]] Gpt2CliOptions parse_options(int argc, char** argv) {
103     Gpt2CliOptions options;
104     std::vector<std::string> positional;
105     for (int index = 1; index < argc; ++index) {
106         const std::string arg = argv[index];
107         if (arg == "--benchmark") {
108             options.benchmark = true;
109         } else if (arg == "--profile-hotspots") {
110             options.profile_hotspots = true;
111             options.benchmark = true;
112         } else if (arg == "--engine") {
113             if (index + 1 >= argc) {
114                 throw std::runtime_error("--engine requires a value");
115             }
116             ++index;
117             options.engine = parse_engine(argv[index]);
118         } else if (arg.rfind("--engine=", 0) == 0) {
119             options.engine = parse_engine(std::string_view(arg).substr(std::string("--engine=").size()));
120         } else if (arg == "--threads") {
121             if (index + 1 >= argc) {
122                 throw std::runtime_error("--threads requires a value");
123             }
124             ++index;
125             options.worker_count = parse_non_negative_i32(argv[index], "--threads");
126         } else if (arg.rfind("--threads=", 0) == 0) {
127             options.worker_count =
128                 parse_non_negative_i32(arg.substr(std::string("--threads=").size()), "--threads");
129         } else if (arg == "--full") {
130             options.engine = Gpt2EngineMode::full_prefix;
131         } else if (arg == "--cached") {
132             options.engine = Gpt2EngineMode::cached_kv;
133         } else if (!arg.empty() && arg.front() == '-') {
134             throw std::runtime_error("unknown option: " + arg);
135         } else {
136             positional.push_back(arg);
137         }
138     }
139
140     if (positional.size() < 2 || positional.size() > 3) {
141         throw std::runtime_error(
142             "usage: lcqi_gpt2 [--engine cached|full] [--threads N] [--benchmark] "
143             "model_dir prompt [max_new_tokens]");

```

```

144     }
145     options.model_dir = positional[0];
146     options.prompt = positional[1];
147     if (positional.size() == 3) {
148         options.max_new_tokens = parse_non_negative_i32(positional[2], "↵
↵ max_new_tokens");
149     }
150     return options;
151 }
152
153 [[nodiscard]] std::vector<std::int32_t> generate_full_prefix(
154     const lcqi::Gpt2ReferenceModel& model,
155     std::span<const std::int32_t> prompt_ids,
156     std::int32_t max_new_tokens,
157     Gpt2BenchmarkTimings& timings) {
158     const Clock::time_point generate_start = Clock::now();
159     std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
160     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
161         if (tokens.size() >= checked_size(model.config.max_positions, "↵
↵ max_positions")) {
162             throw std::runtime_error("GPT-2 generation would exceed ↵
↵ max_positions");
163         }
164         const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, ↵
↵ tokens);
165         ++timings.decode_steps;
166         tokens.push_back(result.predicted_token);
167         if (model.config.eos_token_id >= 0 && result.predicted_token == model.↵
↵ config.eos_token_id) {
168             break;
169         }
170     }
171     const Clock::time_point generate_end = Clock::now();
172     timings.generate_ms = elapsed_ms(generate_start, generate_end);
173     timings.decode_ms = timings.generate_ms;
174     return tokens;
175 }
176
177 [[nodiscard]] std::vector<std::int32_t> generate_cached(
178     const lcqi::Gpt2ReferenceModel& model,
179     std::span<const std::int32_t> prompt_ids,
180     std::int32_t max_new_tokens,
181     Gpt2BenchmarkTimings& timings,
182     lcqi::Gpt2HotspotProfile* hotspot_profile,
183     std::int32_t worker_count) {
184     if (prompt_ids.empty()) {
185         throw std::runtime_error("GPT-2 generation needs at least one prompt ↵
↵ token");
186     }
187     lcqi::Gpt2ExecutionOptions execution_options;
188     execution_options.worker_count = worker_count;
189     lcqi::Gpt2CachedGreedyDecoder decoder(model, hotspot_profile, ↵

```

```

    ↪ execution_options);
190 timings.kv_cache_bytes = decoder.kv_cache_bytes();
191 timings.worker_count = decoder.worker_count();
192 std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
193 if (max_new_tokens == 0) {
194     return tokens;
195 }
196
197 const Clock::time_point generate_start = Clock::now();
198 const Clock::time_point prefill_start = Clock::now();
199 for (std::size_t index = 0; index + 1 < prompt_ids.size(); ++index) {
200     decoder.advance_without_prediction(prompt_ids[index]);
201     ++timings.prefill_steps;
202 }
203 std::int32_t predicted_token = decoder.step(prompt_ids.back());
204 ++timings.prefill_steps;
205 const Clock::time_point prefill_end = Clock::now();
206 timings.prefill_ms = elapsed_ms(prefill_start, prefill_end);
207
208 const Clock::time_point decode_start = Clock::now();
209 for (std::int32_t step = 0; step < max_new_tokens; ++step) {
210     if (tokens.size() >= checked_size(model.config.max_positions, "↪
    ↪ max_positions")) {
211         throw std::runtime_error("GPT-2 generation would exceed ↪
    ↪ max_positions");
212     }
213     tokens.push_back(predicted_token);
214     if (model.config.eos_token_id >= 0 && predicted_token == model.config.↪
    ↪ eos_token_id) {
215         break;
216     }
217     if (step + 1 < max_new_tokens) {
218         predicted_token = decoder.step(predicted_token);
219         ++timings.decode_steps;
220     }
221 }
222 const Clock::time_point decode_end = Clock::now();
223 const Clock::time_point generate_end = Clock::now();
224 timings.decode_ms = elapsed_ms(decode_start, decode_end);
225 timings.generate_ms = elapsed_ms(generate_start, generate_end);
226 return tokens;
227 }
228
229 void print_benchmark(const Gpt2BenchmarkTimings& timings,
230                     std::size_t prompt_tokens,
231                     std::size_t generated_tokens) {
232     const double new_tokens = static_cast<double>(generated_tokens);
233     const double generated_tokens_per_second =
234         timings.generate_ms > 0.0 ? new_tokens * 1000.0 / timings.generate_ms :↪
    ↪ 0.0;
235     const double decode_tokens_per_second =
236         timings.decode_ms > 0.0

```



```

237         ? static_cast<double>(timings.decode_steps) * 1000.0 / timings.↵
↵ decode_ms
238         : 0.0;
239
240     std::cout << "benchmark_load_ms " << timings.load_ms << "\n";
241     std::cout << "benchmark_tokenize_ms " << timings.tokenize_ms << "\n";
242     std::cout << "benchmark_prefill_ms " << timings.prefill_ms << "\n";
243     std::cout << "benchmark_decode_ms " << timings.decode_ms << "\n";
244     std::cout << "benchmark_generate_ms " << timings.generate_ms << "\n";
245     std::cout << "benchmark_total_ms " << timings.total_ms << "\n";
246     std::cout << "benchmark_prompt_tokens " << prompt_tokens << "\n";
247     std::cout << "benchmark_generated_tokens " << generated_tokens << "\n";
248     std::cout << "benchmark_prefill_steps " << timings.prefill_steps << "\n";
249     std::cout << "benchmark_decode_steps " << timings.decode_steps << "\n";
250     std::cout << "benchmark_worker_count " << timings.worker_count << "\n";
251     std::cout << "benchmark_generate_tokens_per_second "
252         << generated_tokens_per_second << "\n";
253     std::cout << "benchmark_decode_tokens_per_second "
254         << decode_tokens_per_second << "\n";
255     std::cout << "benchmark_kv_cache_bytes " << timings.kv_cache_bytes << "\n";
256 }
257
258 void print_hotspot_profile(const lcqi::Gpt2HotspotProfile& profile) {
259     const double total = profile.total_step_ms > 0.0 ? profile.total_step_ms : ↵
↵ 1.0;
260     const auto print_ms = [total](std::string_view name, double value) {
261         std::cout << "hotspot_" << name << "_ms " << value << "\n";
262         std::cout << "hotspot_" << name << "_pct " << (value * 100.0 / total) ↵
↵ << "\n";
263     };
264     std::cout << "hotspot_decoder_steps " << profile.decoder_steps << "\n";
265     std::cout << "hotspot_layer_steps " << profile.layer_steps << "\n";
266     print_ms("total_step", profile.total_step_ms);
267     print_ms("embedding", profile.embedding_ms);
268     print_ms("layer_norm", profile.layer_norm_ms);
269     print_ms("final_norm", profile.final_norm_ms);
270     print_ms("qkv_projection", profile.qkv_projection_ms);
271     print_ms("kv_append", profile.kv_append_ms);
272     print_ms("attention", profile.attention_ms);
273     print_ms("attention_projection", profile.attention_projection_ms);
274     print_ms("residual_add", profile.residual_add_ms);
275     print_ms("mlp_fc", profile.mlp_fc_ms);
276     print_ms("gelu", profile.gelu_ms);
277     print_ms("mlp_projection", profile.mlp_projection_ms);
278     print_ms("lm_head", profile.lm_head_ms);
279     print_ms("logits_result", profile.logits_result_ms);
280 }
281
282 int run_tiny_mode() {
283     const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model↵
↵ ();
284     const std::vector<std::int32_t> prompt{

```

```

285     LCQI_GPT2_TINY_PROMPT_0,
286     LCQI_GPT2_TINY_PROMPT_1,
287     LCQI_GPT2_TINY_PROMPT_2,
288 };
289 const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, prompt↵
↵ );
290 const std::vector<std::int32_t> generated =
291     lcqi::gpt2_generate_greedy(model, prompt, 2);
292 std::cout << "mode tiny\n";
293 std::cout << "prompt_ids ";
294 print_ids(prompt);
295 std::cout << "\n";
296 std::cout << "predicted_token " << result.predicted_token << "\n";
297 std::cout << "logits";
298 for (const float value : result.logits) {
299     std::cout << ' ' << value;
300 }
301 std::cout << "\n";
302 std::cout << "generated_ids ";
303 print_ids(generated);
304 std::cout << "\n";
305 return EXIT_SUCCESS;
306 }
307
308 int run_real_model_mode(int argc, char** argv) {
309     if (argc < LCQI_GPT2_MIN_REAL_ARGC) {
310         throw std::runtime_error(
311             "usage: lcqi_gpt2 [--engine cached|full] [--threads N] [--benchmark↵
↵ ] "
312             "model_dir prompt [max_new_tokens]");
313     }
314     const Clock::time_point total_start = Clock::now();
315     const Gpt2CliOptions options = parse_options(argc, argv);
316
317     Gpt2BenchmarkTimings timings;
318     const Clock::time_point load_start = Clock::now();
319     const lcqi::Gpt2ReferenceModel model = lcqi::load_gpt2_from_directory(↵
↵ options.model_dir);
320     const lcqi::Gpt2Tokenizer tokenizer = lcqi::load_gpt2_tokenizer(
321         options.model_dir / "vocab.json",
322         options.model_dir / "merges.txt",
323         model.config.bos_token_id,
324         model.config.eos_token_id);
325     timings.load_ms = elapsed_ms(load_start, Clock::now());
326
327     const Clock::time_point tokenize_start = Clock::now();
328     const std::vector<std::int32_t> prompt_ids = lcqi::gpt2_encode(tokenizer, ↵
↵ options.prompt);
329     timings.tokenize_ms = elapsed_ms(tokenize_start, Clock::now());
330
331     std::vector<std::int32_t> generated;
332     lcqi::Gpt2HotspotProfile hotspot_profile;

```

```

333     if (options.engine == Gpt2EngineMode::full_prefix) {
334         if (options.profile_hotspots) {
335             throw std::runtime_error("--profile-hotspots currently supports ←
↪ cached engine only");
336         }
337         generated = generate_full_prefix(model, prompt_ids, options.↪
↪ max_new_tokens, timings);
338     } else {
339         generated = generate_cached(model,
340                                     prompt_ids,
341                                     options.max_new_tokens,
342                                     timings,
343                                     options.profile_hotspots ? &hotspot_profile↪
↪ : nullptr,
344                                     options.worker_count);
345     }
346     timings.total_ms = elapsed_ms(total_start, Clock::now());
347
348     std::cout << "mode gpt2_directory\n";
349     std::cout << "engine " << engine_name(options.engine) << "\n";
350     std::cout << "model_dir " << options.model_dir.string() << "\n";
351     std::cout << "prompt_ids ";
352     print_ids(prompt_ids);
353     std::cout << "\n";
354     std::cout << "generated_ids ";
355     print_ids(generated);
356     std::cout << "\n";
357     std::cout << "generated_text " << lcqi::gpt2_decode(tokenizer, generated) ←
↪ << "\n";
358     if (options.benchmark) {
359         const std::size_t generated_tokens = generated.size() - prompt_ids.size↪
↪ ();
360         print_benchmark(timings, prompt_ids.size(), generated_tokens);
361     }
362     if (options.profile_hotspots) {
363         print_hotspot_profile(hotspot_profile);
364     }
365     return EXIT_SUCCESS;
366 }
367
368 } // namespace
369
370 int main(int argc, char** argv) {
371     try {
372         if (argc == LCQI_GPT2_TINY_ARGC) {
373             return run_tiny_mode();
374         }
375         return run_real_model_mode(argc, argv);
376     } catch (const std::exception& error) {
377         std::cerr << "error: " << error.what() << "\n";
378         return EXIT_FAILURE;
379     }

```

```
380 }
```

第 6 章

工具、报告、Benchmark 与回归测试

6.1 工具不是附属品

LCQI 的工程证据不只在 C++ 源码里。smoke 脚本决定真实模型怎样下载、校验和运行；benchmark 程序决定性能口径；trace、ledger 和 serving probe 决定报告字段；CTest 决定哪些行为能长期回归。读者必须把这些文件当作工程合同的一部分。

6.2 Benchmark、Trace、Ledger 与 Serving Probe

bench.cpp 输出 int8 kernel benchmark。它服务性能观察，但必须和 max diff 对照一起看。

Listing 6.1 src/bench.cpp

```
1 #include <lcqi/int8_kernels.hpp>
2
3 #include <algorithm>
4 #include <cstdlib>
5 #include <iostream>
6 #include <stdexcept>
7 #include <string>
8 #include <vector>
9
10 namespace {
11
12 constexpr const char* LCQI_TARGET_ARCH_X86_64 = "x86_64";
13 constexpr const char* LCQI_TARGET_ARCH_I386 = "i386";
14 constexpr const char* LCQI_TARGET_ARCH_UNKNOWN = "unknown";
15 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 200;
16 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
17 constexpr std::int32_t LCQI_LARGE_CASE_REPEAT_DIVISOR = 4;
18 constexpr std::int32_t LCQI_MIN_LARGE_CASE_REPEAT = 1;
19 constexpr std::int32_t LCQI_DECODE_SHAPE_SIZE = 4096;
20
21 const char* target_arch() noexcept {
22     #if defined(__x86_64__) || defined(_M_X64)
23         return LCQI_TARGET_ARCH_X86_64;
24     #elif defined(__i386__) || defined(_M_IX86)
25         return LCQI_TARGET_ARCH_I386;
26     #else
27         return LCQI_TARGET_ARCH_UNKNOWN;
28     #endif
29 }
30
31 std::int32_t parse_repeat(int argc, char** argv) {
32     if (argc <= LCQI_REPEAT_ARG_INDEX) {
33         return LCQI_DEFAULT_REPEAT;
```

```

34     }
35     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ LCQI_REPEAT_ARG_INDEX]));
36     if (repeat <= 0) {
37         throw std::runtime_error("repeat must be positive");
38     }
39     return repeat;
40 }
41
42 std::vector<lcqi::KernelBenchmarkCase> make_cases(std::int32_t repeat) {
43     return {
44         {128, 128, 16, repeat},
45         {256, 256, 16, repeat},
46         {512, 512, 16, repeat},
47         {513, 257, 16, repeat},
48         {1024, 4096, 16, std::max(LCQI_MIN_LARGE_CASE_REPEAT,
49                                   repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
50         {LCQI_DECODE_SHAPE_SIZE, LCQI_DECODE_SHAPE_SIZE, 16,
51          std::max(LCQI_MIN_LARGE_CASE_REPEAT, repeat / ↵
↵ LCQI_LARGE_CASE_REPEAT_DIVISOR)},
52     };
53 }
54
55 } // namespace
56
57 int main(int argc, char** argv) {
58     try {
59         const std::int32_t repeat = parse_repeat(argc, argv);
60         std::cout
61             << "target_arch,input_size,output_size,output_block,repeat,backend,↵
↵ available,"
62             << "average_us,max_abs_diff,checksum\n";
63         for (const lcqi::KernelBenchmarkCase& benchmark_case : make_cases(↵
↵ repeat)) {
64             const lcqi::KernelBenchmarkResult result =
65                 lcqi::benchmark_linear_i8_case(benchmark_case);
66             for (const lcqi::KernelBackendBenchmark& backend : result.backends)↵
↵ {
67                 std::cout << target_arch() << ','
68                     << result.benchmark_case.input_size << ','
69                     << result.benchmark_case.output_size << ','
70                     << result.benchmark_case.output_block_size << ','
71                     << result.benchmark_case.repeat << ','
72                     << backend.name << ','
73                     << (backend.available ? 1 : 0) << ','
74                     << backend.average_us << ','
75                     << backend.max_abs_diff << ','
76                     << backend.checksum << '\n';
77             }
78         }
79         return EXIT_SUCCESS;
80     } catch (const std::exception& error) {

```

```

81         std::cerr << "error: " << error.what() << '\n';
82         return EXIT_FAILURE;
83     }
84 }

```

`trace.cpp` 输出 reference decoder checkpoint。它用于定位错误边界，不应该被简化成只打印最终 token。

Listing 6.2 src/trace.cpp

```

1  #include <lcqi/reference_decoder.hpp>
2  #include <lcqi/tokenizer.hpp>
3
4  #include <algorithm>
5  #include <array>
6  #include <cstdlib>
7  #include <exception>
8  #include <filesystem>
9  #include <iostream>
10 #include <span>
11 #include <sstream>
12 #include <string>
13 #include <string_view>
14
15 namespace {
16
17 constexpr std::array<std::int32_t, 3> LCQI_TRACE_TOKEN_IDS{1, 2, 3};
18 constexpr std::string_view LCQI_TRACE_PROMPT = "tok1 tok2 tok3";
19 constexpr std::int32_t LCQI_TRACE_METADATA_LAYER = -1;
20 constexpr std::int32_t LCQI_TRACE_BOS_ID = 0;
21 constexpr std::int32_t LCQI_TRACE_EOS_ID = 4;
22 constexpr std::size_t LCQI_TRACE_VALUE_LIMIT = 8;
23
24 std::filesystem::path tokenizer_fixture_path() {
25     return std::filesystem::path(__FILE__).parent_path().parent_path() /
26         "models" / "tiny_tokenizer.txt";
27 }
28
29 std::vector<std::int32_t> decoder_tokens_from_prompt(const lcqi::TokenizerModel& ←
    ↪ & tokenizer,
30                                                     std::string_view prompt) {
31     std::vector<std::int32_t> full_tokens = lcqi::encode_prompt(tokenizer, ←
    ↪ prompt);
32     if (full_tokens.size() != LCQI_TRACE_TOKEN_IDS.size() + 2 ||
33         full_tokens.front() != LCQI_TRACE_BOS_ID ||
34         full_tokens.back() != LCQI_TRACE_EOS_ID) {
35         throw std::runtime_error("unexpected tokenizer fixture ids");
36     }
37
38     std::vector<std::int32_t> decoder_tokens(
39         full_tokens.begin() + 1,
40         full_tokens.end() - 1);
41     if (!std::equal(decoder_tokens.begin(),

```

```

42         decoder_tokens.end(),
43         LCQI_TRACE_TOKEN_IDS.begin(),
44         LCQI_TRACE_TOKEN_IDS.end())) {
45     throw std::runtime_error("unexpected decoder token ids");
46 }
47 return decoder_tokens;
48 }
49
50 std::string join_ints(std::span<const std::int32_t> values) {
51     std::ostringstream stream;
52     for (std::size_t index = 0; index < values.size(); ++index) {
53         if (index != 0) {
54             stream << 'x';
55         }
56         stream << values[index];
57     }
58     return stream.str();
59 }
60
61 std::string join_int_values(std::span<const std::int32_t> values) {
62     std::ostringstream stream;
63     for (std::size_t index = 0; index < values.size(); ++index) {
64         if (index != 0) {
65             stream << ';';
66         }
67         stream << values[index];
68     }
69     return stream.str();
70 }
71
72 std::int32_t checksum_ints(std::span<const std::int32_t> values) {
73     std::int32_t result = 0;
74     for (const std::int32_t value : values) {
75         result += value;
76     }
77     return result;
78 }
79
80 std::int32_t max_abs_ints(std::span<const std::int32_t> values) {
81     std::int32_t result = 0;
82     for (const std::int32_t value : values) {
83         result = std::max(result, std::abs(value));
84     }
85     return result;
86 }
87
88 std::string join_floats(std::span<const float> values) {
89     std::ostringstream stream;
90     const std::size_t count = std::min(values.size(), LCQI_TRACE_VALUE_LIMIT);
91     for (std::size_t index = 0; index < count; ++index) {
92         if (index != 0) {
93             stream << ';';

```



```

94     }
95     stream << values[index];
96 }
97 if (values.size() > LCQI_TRACE_VALUE_LIMIT) {
98     stream << "...";
99 }
100 return stream.str();
101 }
102
103 void print_trace_csv(const lcqi::ReferenceDecodeTraceResult& trace,
104                    const lcqi::TokenizerModel& tokenizer,
105                    std::span<const std::int32_t> full_tokens) {
106     std::cout << "name,token_position,layer_id,shape,stride,dtype,layout,↵
↵ checksum,max_abs,values\n";
107     std::cout << "prompt,0," << LCQI_TRACE_METADATA_LAYER
108         << ",scalar,scalar,utf8,text,0,0," << LCQI_TRACE_PROMPT << '\n';
109     std::cout << "tokenizer,0," << LCQI_TRACE_METADATA_LAYER
110         << ',' << full_tokens.size() << ",1,i32,contiguous,"
111         << checksum_ints(full_tokens) << ','
112         << max_abs_ints(full_tokens) << ','
113         << join_int_values(full_tokens) << '\n';
114     std::cout << "tokenizer_contract,0," << LCQI_TRACE_METADATA_LAYER
115         << ",scalar,scalar,u64,fnv1a,"
116         << tokenizer_contract_hash(tokenizer) << ",0,"
117         << tokenizer.chat_template_hash << '\n';
118     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↵
↵ {
119         std::cout << checkpoint.name << ','
120             << checkpoint.token_position << ','
121             << checkpoint.layer_id << ','
122             << join_ints(checkpoint.shape) << ','
123             << join_ints(checkpoint.stride) << ','
124             << checkpoint.dtype << ','
125             << checkpoint.layout << ','
126             << checkpoint.checksum << ','
127             << checkpoint.max_abs << ','
128             << join_floats(checkpoint.values) << '\n';
129     }
130     std::cout << "sampler_argmax,"
131         << LCQI_TRACE_TOKEN_IDS.size() - 1 << ','
132         << LCQI_TRACE_METADATA_LAYER
133         << ',' << trace.result.logits.size() << ",1,f32,argmax,"
134         << "0,0,token=" << trace.result.predicted_token << '\n';
135     std::cout << "generated_token,"
136         << LCQI_TRACE_TOKEN_IDS.size() << ','
137         << LCQI_TRACE_METADATA_LAYER
138         << ",scalar,scalar,i32,generated,"
139         << trace.result.predicted_token << ','
140         << trace.result.predicted_token << ','
141         << trace.result.predicted_token << '\n';
142 }
143

```

```

144 } // namespace
145
146 int main() {
147     try {
148         const lcqi::TokenizerModel tokenizer =
149             lcqi::load_tokenizer(tokenizer_fixture_path());
150         const std::vector<std::int32_t> full_tokens =
151             lcqi::encode_prompt(tokenizer, LCQI_TRACE_PROMPT);
152         const std::vector<std::int32_t> decoder_tokens =
153             decoder_tokens_from_prompt(tokenizer, LCQI_TRACE_PROMPT);
154         const lcqi::ReferenceDecoderModel model = lcqi::↵
↵ make_tiny_reference_decoder_model();
155         const lcqi::ReferenceDecodeTraceResult trace =
156             lcqi::run_reference_decode_with_trace(model, decoder_tokens);
157         print_trace_csv(trace, tokenizer, full_tokens);
158         return EXIT_SUCCESS;
159     } catch (const std::exception& error) {
160         std::cerr << "error: " << error.what() << '\n';
161         return EXIT_FAILURE;
162     }
163 }

```

`ledger.cpp` 输出 7B 规模账本。它是估算，不是实测吞吐；读者要区分 FLOPs、KV 容量、权重/KV 字节和 bandwidth-only tokens/s 上限。

Listing 6.3 src/ledger.cpp

```

1 #include <cstdlib>
2 #include <cstdint>
3 #include <exception>
4 #include <iomanip>
5 #include <iostream>
6 #include <string_view>
7
8 namespace {
9
10 constexpr double LCQI_BYTES_PER_GB = 1000.0 * 1000.0 * 1000.0;
11 constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
12 constexpr double LCQI_FLOPS_PER_MAC = 2.0;
13 constexpr std::int32_t LCQI_SEVENB_LAYER_COUNT = 32;
14 constexpr std::int32_t LCQI_SEVENB_HIDDEN_SIZE = 4096;
15 constexpr std::int32_t LCQI_SEVENB_QUERY_HEADS = 32;
16 constexpr std::int32_t LCQI_SEVENB_KV_HEADS = 8;
17 constexpr std::int32_t LCQI_SEVENB_HEAD_DIM = 128;
18 constexpr std::int32_t LCQI_SEVENB_INTERMEDIATE_SIZE = 11008;
19 constexpr std::int32_t LCQI_SEVENB_CONTEXT_LENGTH = 4096;
20
21 struct DTypeLedger {
22     std::string_view name;
23     double weight_gb_per_token = 0.0;
24     double kv_element_bytes = 0.0;
25 };
26

```

```

27 constexpr DTypeLedger LCQI_DTYPE_LEDGERS[] = {
28     {"fp16", 14.0, 2.0},
29     {"int8", 7.0, 2.0},
30     {"int4", 3.7, 2.0},
31 };
32
33 constexpr double LCQI_BANDWIDTH_GB_PER_S[] = {
34     50.0,
35     100.0,
36     200.0,
37     600.0,
38     1000.0,
39     1600.0,
40 };
41
42 double linear_macs_per_layer() noexcept {
43     const double hidden = LCQI_SEVENB_HIDDEN_SIZE;
44     const double kv_width = LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM;
45     const double qkv = hidden * (hidden + 2.0 * kv_width);
46     const double output_projection = hidden * hidden;
47     const double mlp = 3.0 * hidden * LCQI_SEVENB_INTERMEDIATE_SIZE;
48     return qkv + output_projection + mlp;
49 }
50
51 double attention_macs_per_layer() noexcept {
52     return 2.0 * LCQI_SEVENB_QUERY_HEADS * LCQI_SEVENB_CONTEXT_LENGTH *
53           LCQI_SEVENB_HEAD_DIM;
54 }
55
56 double kv_bytes_per_token_all_layers(double element_bytes) noexcept {
57     return static_cast<double>(LCQI_SEVENB_LAYER_COUNT) * 2.0 *
58           LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM * element_bytes;
59 }
60
61 double kv_read_gb_per_decode_token(double element_bytes) noexcept {
62     const double bytes = static_cast<double>(LCQI_SEVENB_CONTEXT_LENGTH) *
63           kv_bytes_per_token_all_layers(element_bytes);
64     return bytes / LCQI_BYTES_PER_GB;
65 }
66
67 double kv_capacity_mib(double element_bytes, std::int32_t context_length) ↵
68     ↵ noexcept {
69     const double bytes = static_cast<double>(context_length) *
70           kv_bytes_per_token_all_layers(element_bytes);
71     return bytes / LCQI_BYTES_PER_MIB;
72 }
73
74 void print_model_rows() {
75     const double linear_macs = linear_macs_per_layer();
76     const double attention_macs = attention_macs_per_layer();
77     const double total_flops =
78         (linear_macs + attention_macs) * LCQI_SEVENB_LAYER_COUNT * ↵

```

```

↪ LCQI_FLOPS_PER_MAC;
78 std::cout << "section,item,value,unit,notes\n";
79 std::cout << "model,layers," << LCQI_SEVENB_LAYER_COUNT << ",count,7B ↪
↪ fixture\n";
80 std::cout << "model,hidden," << LCQI_SEVENB_HIDDEN_SIZE << ",elements,H\n";
81 std::cout << "model,query_heads," << LCQI_SEVENB_QUERY_HEADS << ",count,6QA↪
↪ query heads\n";
82 std::cout << "model,kv_heads," << LCQI_SEVENB_KV_HEADS << ",count,6QA KV ↪
↪ heads\n";
83 std::cout << "model,head_dim," << LCQI_SEVENB_HEAD_DIM << ",elements,per ↪
↪ head\n";
84 std::cout << "compute,linear_macs_per_layer," << linear_macs << ",macs,↪
↪ decode token\n";
85 std::cout << "compute,attention_macs_per_layer," << attention_macs
86 << ",macs,context=4096\n";
87 std::cout << "compute,total_decode_flops_per_token," << total_flops
88 << ",flops,linear plus attention\n";
89 }
90
91 void print_kv_rows() {
92     for (const std::int32_t context : {1024, 2048, 4096, 8192}) {
93         std::cout << "kv,fp16_capacity_context_" << context << ', '
94 << kv_capacity_mib(2.0, context)
95 << ",MiB,single session\n";
96     }
97 }
98
99 void print_dtype_rows() {
100     for (const DTypeLedger& dtype : LCQI_DTYPE_LEDGERS) {
101         const double kv_read_gb = kv_read_gb_per_decode_token(dtype.↪
↪ kv_element_bytes);
102         const double total_gb = dtype.weight_gb_per_token + kv_read_gb;
103         std::cout << "decode_bytes," << dtype.name << "_weight_gb,"
104 << dtype.weight_gb_per_token << ",GB/token,weight stream\n";
105         std::cout << "decode_bytes," << dtype.name << "_kv_read_gb,"
106 << kv_read_gb << ",GB/token,context=4096\n";
107         std::cout << "decode_bytes," << dtype.name << "_total_gb,"
108 << total_gb << ",GB/token,weight plus KV\n";
109         for (const double bandwidth : LCQI_BANDWIDTH_GB_PER_S) {
110             std::cout << "bandwidth_limit," << dtype.name << "_at_"
111 << static_cast<std::int32_t>(bandwidth) << "_gbps,"
112 << bandwidth / total_gb
113 << ",tokens/s,bandwidth-only upper bound\n";
114         }
115     }
116 }
117
118 } // namespace
119
120 int main() {
121     try {
122         std::cout << std::fixed << std::setprecision(3);

```

```

123     print_model_rows();
124     print_kv_rows();
125     print_dtype_rows();
126     return EXIT_SUCCESS;
127 } catch (const std::exception& error) {
128     std::cerr << "error: " << error.what() << '\n';
129     return EXIT_FAILURE;
130 }
131 }

```

`serving_probe.cpp` 固定短请求、RAG 请求、取消请求和超长请求。它把 TTFT、TPOT、拒绝和取消回收变成可观察字段。

Listing 6.4 src/serving_probe.cpp

```

1  #include <algorithm>
2  #include <cstdlib>
3  #include <stdint>
4  #include <exception>
5  #include <cmath>
6  #include <iomanip>
7  #include <iostream>
8  #include <string_view>
9  #include <vector>
10
11 namespace {
12
13 constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
14 constexpr std::int32_t LCQI_PROBE_LAYER_COUNT = 32;
15 constexpr std::int32_t LCQI_PROBE_KV_HEADS = 8;
16 constexpr std::int32_t LCQI_PROBE_HEAD_DIM = 128;
17 constexpr double LCQI_PROBE_KV_ELEMENT_BYTES = 2.0;
18 constexpr double LCQI_PROBE_KV_BUDGET_MIB = 512.0;
19 constexpr double LCQI_PROBE_WORKSPACE_MIB = 64.0;
20 constexpr double LCQI_PROBE_ARENA_MIB = 32.0;
21 constexpr double LCQI_PROBE_TRACE_MIB = 4.0;
22 constexpr double LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS = 8.0;
23 constexpr double LCQI_PROBE_PREFILL_MS_PER_TOKEN = 0.08;
24 constexpr double LCQI_PROBE_DECODE_STEP_MS = 14.0;
25 constexpr double LCQI_PROBE_CANCEL_RECLAIM_MS = 2.0;
26 constexpr std::size_t LCQI_PERCENTILE_MIN_INDEX = 1;
27
28 struct ProbeRequest {
29     std::string_view request_id;
30     std::int32_t prompt_tokens = 0;
31     std::int32_t max_new_tokens = 0;
32     bool cancel_after_prefill = false;
33 };
34
35 struct ProbeResult {
36     std::string_view request_id;
37     std::int32_t accepted = 0;
38     std::string_view rejection_reason;

```

```

39     std::int32_t prompt_tokens = 0;
40     std::int32_t reserved_tokens = 0;
41     double kv_reserved_mib = 0.0;
42     double queue_wait_ms = 0.0;
43     double prefill_ms = 0.0;
44     double ttft_ms = 0.0;
45     double decode_step_p95_ms = 0.0;
46     double tpot_ms = 0.0;
47     std::int32_t completion_tokens = 0;
48     std::string_view finish_reason;
49     double cancel_reclaim_ms = 0.0;
50 };
51
52 const std::vector<ProbeRequest> LCQI_PROBE_REQUESTS{
53     {"req_short", 32, 4, false},
54     {"req_rag", 512, 8, false},
55     {"req_cancel", 128, 16, true},
56     {"req_too_long", 4096, 512, false},
57 };
58
59 double kv_mib_for_tokens(std::int32_t tokens) noexcept {
60     const double bytes = static_cast<double>(tokens) * LCQI_PROBE_LAYER_COUNT *
        ↪ 2.0 *
61         LCQI_PROBE_KV_HEADS * LCQI_PROBE_HEAD_DIM *
62         LCQI_PROBE_KV_ELEMENT_BYTES;
63     return bytes / LCQI_BYTES_PER_MIB;
64 }
65
66 double percentile(std::vector<double> values, double fraction) {
67     if (values.empty()) {
68         return 0.0;
69     }
70     std::sort(values.begin(), values.end());
71     const std::size_t max_index = values.size() - LCQI_PERCENTILE_MIN_INDEX;
72     const std::size_t index =
73         std::min(max_index,
74             static_cast<std::size_t>(
75                 std::ceil(fraction * static_cast<double>(max_index))));
76     return values[index];
77 }
78
79 std::vector<ProbeResult> run_probe() {
80     std::vector<ProbeResult> results;
81     double reserved_kv_mib = 0.0;
82     std::int32_t accepted_before = 0;
83     for (const ProbeRequest& request : LCQI_PROBE_REQUESTS) {
84         ProbeResult result;
85         result.request_id = request.request_id;
86         result.prompt_tokens = request.prompt_tokens;
87         result.reserved_tokens = request.prompt_tokens + request.max_new_tokens
        ↪ ;
88         result.kv_reserved_mib = kv_mib_for_tokens(result.reserved_tokens);

```

```

89     const double static_session_mib =
90         result.kv_reserved_mib + LCQI_PROBE_WORKSPACE_MIB +
91         LCQI_PROBE_ARENA_MIB + LCQI_PROBE_TRACE_MIB;
92     if (reserved_kv_mib + result.kv_reserved_mib > LCQI_PROBE_KV_BUDGET_MIB) {
93         result.accepted = 0;
94         result.rejection_reason = "memory_budget_exceeded";
95         result.finish_reason = "rejected";
96         results.push_back(result);
97         continue;
98     }
99
100     reserved_kv_mib += result.kv_reserved_mib;
101     result.accepted = 1;
102     result.rejection_reason = "none";
103     result.queue_wait_ms = static_cast<double>(accepted_before) *
104         LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS;
105     result.prefill_ms = static_cast<double>(request.prompt_tokens) *
106         LCQI_PROBE_PREFILL_MS_PER_TOKEN;
107     result.ttft_ms = result.queue_wait_ms + result.prefill_ms +
108     LCQI_PROBE_DECODE_STEP_MS;
109     result.decode_step_p95_ms = LCQI_PROBE_DECODE_STEP_MS +
110         static_session_mib / 512.0;
111     result.tpot_ms = result.decode_step_p95_ms;
112     if (request.cancel_after_prefill) {
113         result.completion_tokens = 0;
114         result.finish_reason = "cancelled";
115         result.cancel_reclaim_ms = LCQI_PROBE_CANCEL_RECLAIM_MS;
116         reserved_kv_mib -= result.kv_reserved_mib;
117     } else {
118         result.completion_tokens = request.max_new_tokens;
119         result.finish_reason = "max_tokens";
120     }
121     ++accepted_before;
122     results.push_back(result);
123 }
124 return results;
125 }
126
127 void print_request_rows(const std::vector<ProbeResult>& results) {
128     std::cout << "row_type,request_id,accepted,rejection_reason,prompt_tokens,
129     reserved_tokens,"
130     "kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,
131     decode_step_p95_ms,"
132     "tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,
133     metric_name,"
134     "metric_value\n";
135     for (const ProbeResult& result : results) {
136         std::cout << "request," << result.request_id << ',' <<
137             result.accepted << ',' <<
138             result.rejection_reason << ',' <<
139             result.prompt_tokens << ',' <<

```

```

136         << result.reserved_tokens << ','
137         << result.kv_reserved_mib << ','
138         << result.queue_wait_ms << ','
139         << result.prefill_ms << ','
140         << result.ttft_ms << ','
141         << result.decode_step_p95_ms << ','
142         << result.tpot_ms << ','
143         << result.completion_tokens << ','
144         << result.finish_reason << ','
145         << result.cancel_reclaim_ms << ",,\n";
146     }
147 }
148
149 void print_summary_row(const std::vector<ProbeResult>& results) {
150     std::vector<double> ttft_values;
151     std::vector<double> queue_values;
152     double accepted = 0.0;
153     double rejected = 0.0;
154     double cancelled = 0.0;
155     double reserved_peak_mib = 0.0;
156     double running_reserved_mib = 0.0;
157     for (const ProbeResult& result : results) {
158         if (result.accepted == 0) {
159             rejected += 1.0;
160             continue;
161         }
162         accepted += 1.0;
163         ttft_values.push_back(result.ttft_ms);
164         queue_values.push_back(result.queue_wait_ms);
165         running_reserved_mib += result.kv_reserved_mib;
166         reserved_peak_mib = std::max(reserved_peak_mib, running_reserved_mib);
167         if (result.finish_reason == "cancelled") {
168             cancelled += 1.0;
169             running_reserved_mib -= result.kv_reserved_mib;
170         }
171     }
172     const double total = accepted + rejected;
173     const double rejection_rate = total == 0.0 ? 0.0 : rejected / total;
174     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
175         << "accepted_count," << accepted << '\n';
176     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,"
177         << LCQI_PROBE_CANCEL_RECLAIM_MS << ",cancel_count," << cancelled <<
178         << '\n';
179     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
180         << "rejected_count," << rejected << '\n';
181     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
182         << "rejection_rate," << rejection_rate << '\n';
183     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
184         << "kv_reserved_peak_mib," << reserved_peak_mib << '\n';
185     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
186         << "queue_wait_p95_ms," << percentile(queue_values, 0.95) << '\n' <<
187         ;

```



```

186     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
187               << "ttft_p95_ms," << percentile(ttft_values, 0.95) << '\n';
188 }
189
190 } // namespace
191
192 int main() {
193     try {
194         std::cout << std::fixed << std::setprecision(3);
195         const std::vector<ProbeResult> results = run_probe();
196         print_request_rows(results);
197         print_summary_row(results);
198         return EXIT_SUCCESS;
199     } catch (const std::exception& error) {
200         std::cerr << "error: " << error.what() << '\n';
201         return EXIT_FAILURE;
202     }
203 }

```

`onednn_bench.cpp` 是可选外部库对标入口。它只在环境安装 oneDNN 时构建，不能把不可用平台写成已验证性能。

Listing 6.5 src/onednn_bench.cpp

```

1  #include <dnndl.hpp>
2
3  #include <chrono>
4  #include <cstdint>
5  #include <cstdlib>
6  #include <iostream>
7  #include <numeric>
8  #include <stdexcept>
9  #include <string>
10 #include <unordered_map>
11 #include <vector>
12
13 namespace {
14
15 constexpr std::int64_t LCQI_ONEDNN_BATCH = 1;
16 constexpr std::int64_t LCQI_ONEDNN_K = 4096;
17 constexpr std::int64_t LCQI_ONEDNN_O = 4096;
18 constexpr std::int32_t LCQI_ONEDNN_DEFAULT_REPEAT = 20;
19 constexpr std::int32_t LCQI_ONEDNN_WARMUP = 3;
20 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
21 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
22 constexpr float LCQI_INPUT_SCALE = 0.125F;
23 constexpr std::int32_t LCQI_INPUT_MODULUS = 17;
24 constexpr std::int32_t LCQI_INPUT_CENTER = 8;
25 constexpr std::int32_t LCQI_WEIGHT_MODULUS = 23;
26 constexpr std::int32_t LCQI_WEIGHT_CENTER = 11;
27 constexpr float LCQI_WEIGHT_SCALE = 0.03125F;
28
29 std::int32_t parse_repeat(int argc, char** argv) {

```

```

30     if (argc <= LCQI_REPEAT_ARG_INDEX) {
31         return LCQI_ONEDNN_DEFAULT_REPEAT;
32     }
33     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ LCQI_REPEAT_ARG_INDEX]));
34     if (repeat <= 0) {
35         throw std::runtime_error("repeat must be positive");
36     }
37     return repeat;
38 }
39
40 std::vector<float> make_input() {
41     std::vector<float> input(static_cast<std::size_t>(LCQI_ONEDNN_K));
42     for (std::int64_t index = 0; index < LCQI_ONEDNN_K; ++index) {
43         input[static_cast<std::size_t>(index)] =
44             static_cast<float>((index % LCQI_INPUT_MODULUS) - LCQI_INPUT_CENTER↵
↵ ) *
45             LCQI_INPUT_SCALE;
46     }
47     return input;
48 }
49
50 std::vector<float> make_weights_kn() {
51     std::vector<float> weights(static_cast<std::size_t>(LCQI_ONEDNN_K * ↵
↵ LCQI_ONEDNN_0));
52     for (std::int64_t k = 0; k < LCQI_ONEDNN_K; ++k) {
53         for (std::int64_t out = 0; out < LCQI_ONEDNN_0; ++out) {
54             const std::int64_t row_major_out_k = out * LCQI_ONEDNN_K + k;
55             const float value =
56                 static_cast<float>((row_major_out_k % LCQI_WEIGHT_MODULUS) -
57                                     LCQI_WEIGHT_CENTER) *
58                 LCQI_WEIGHT_SCALE;
59             weights[static_cast<std::size_t>(k * LCQI_ONEDNN_0 + out)] = value;
60         }
61     }
62     return weights;
63 }
64
65 float checksum(const std::vector<float>& values) {
66     return std::accumulate(values.begin(), values.end(), 0.0F);
67 }
68
69 } // namespace
70
71 int main(int argc, char** argv) {
72     try {
73         const std::int32_t repeat = parse_repeat(argc, argv);
74         std::vector<float> input = make_input();
75         std::vector<float> weights = make_weights_kn();
76         std::vector<float> output(static_cast<std::size_t>(LCQI_ONEDNN_BATCH * ↵
↵ LCQI_ONEDNN_0),
77                                 0.0F);

```

```

78
79     dnnl::engine engine(dnnl::engine::kind::cpu, 0);
80     dnnl::stream stream(engine);
81
82     const dnnl::memory::dims source_dims = {LCQI_ONEDNN_BATCH, ↵
↵ LCQI_ONEDNN_K};
83     const dnnl::memory::dims weight_dims = {LCQI_ONEDNN_K, LCQI_ONEDNN_0};
84     const dnnl::memory::dims output_dims = {LCQI_ONEDNN_BATCH, ↵
↵ LCQI_ONEDNN_0};
85
86     const dnnl::memory::desc source_desc(
87         source_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↵
↵ ::ab);
88     const dnnl::memory::desc weight_desc(
89         weight_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↵
↵ ::ab);
90     const dnnl::memory::desc output_desc(
91         output_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↵
↵ ::ab);
92
93     dnnl::memory source_memory(source_desc, engine, input.data());
94     dnnl::memory weight_memory(weight_desc, engine, weights.data());
95     dnnl::memory output_memory(output_desc, engine, output.data());
96
97     dnnl::matmul::primitive_desc matmul_desc(engine, source_desc, ↵
↵ weight_desc, output_desc);
98     dnnl::matmul matmul(matmul_desc);
99     const std::unordered_map<int, dnnl::memory> arguments = {
100         {DNNL_ARG_SRC, source_memory},
101         {DNNL_ARG_WEIGHTS, weight_memory},
102         {DNNL_ARG_DST, output_memory},
103     };
104
105     for (std::int32_t index = 0; index < LCQI_ONEDNN_WARMUP; ++index) {
106         matmul.execute(stream, arguments);
107         stream.wait();
108     }
109
110     const auto begin = std::chrono::steady_clock::now();
111     for (std::int32_t index = 0; index < repeat; ++index) {
112         matmul.execute(stream, arguments);
113         stream.wait();
114     }
115     const auto end = std::chrono::steady_clock::now();
116     const std::chrono::duration<double> elapsed = end - begin;
117     const double average_us =
118         elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double↵
↵ >(repeat);
119
120     std::cout << "library,input_size,output_size,dtype,repeat,average_us,↵
↵ checksum\n";
121     std::cout << "onednn," << LCQI_ONEDNN_K << ',' << LCQI_ONEDNN_0

```

```

122         << ",f32," << repeat << ',' << average_us << ','
123         << checksum(output) << '\n';
124     return EXIT_SUCCESS;
125 } catch (const std::exception& error) {
126     std::cerr << "error: " << error.what() << '\n';
127     return EXIT_FAILURE;
128 }
129 }

```

6.3 外部模型 Smoke 与对比脚本

`run_smollm2_small_smoke.py` 下载并校验 SmoLLM2-135M-Instruct 的 GGUF 量化文件，构建固定 commit 的 llama-simple-chat，在 CPU 上运行一次真实开源 small 模型生成，并输出可复查报告。它是外部真实模型 smoke，不替代 LCQI tiny golden trace。

Listing 6.6 tools/run_smollm2_small_smoke.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import hashlib
7  import os
8  import re
9  import shlex
10 import shutil
11 import subprocess
12 import sys
13 from dataclasses import dataclass
14 from pathlib import Path
15
16
17 MODEL_SOURCE_ID = "HuggingFaceTB/SmolLM2-135M-Instruct"
18 MODEL_GGUF_REPO_ID = "bartowski/SmolLM2-135M-Instruct-GGUF"
19 MODEL_FILE_NAME = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
20 MODEL_URL = (
21     "https://huggingface.co/bartowski/SmolLM2-135M-Instruct-GGUF"
22     f"/resolve/main/{MODEL_FILE_NAME}"
23 )
24 MODEL_EXPECTED_BYTES = 105454432
25 MODEL_EXPECTED_SHA256 = (
26     "2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d"
27 )
28 LLAMA_CPP_REPO_URL = "https://github.com/ggml-org/llama.cpp.git"
29 LLAMA_CPP_COMMIT = "b3fed31b99f9bd37725833674252bccb429bb183"
30 LLAMA_TARGET = "llama-simple-chat"
31 DEFAULT_PROMPT = "What is 2+3? Answer in one short sentence."
32 DEFAULT_CONTEXT_TOKENS = 512
33 DEFAULT_TIMEOUT_SECONDS = 180
34 DEFAULT_BUILD_JOBS = 2

```

```

35
36
37 @dataclass(frozen=True)
38 class CommandResult:
39     args: list[str]
40     returncode: int
41     stdout: str
42     stderr: str
43
44
45 def script_path() -> Path:
46     return Path(__file__).resolve()
47
48
49 def volume_dir() -> Path:
50     return script_path().parents[1]
51
52
53 def repo_dir() -> Path:
54     return script_path().parents[3]
55
56
57 def default_cache_dir() -> Path:
58     explicit_cache = os.environ.get("LCQI_SMOLLM2_CACHE_DIR")
59     if explicit_cache:
60         return Path(explicit_cache).expanduser()
61     xdg_cache = os.environ.get("XDG_CACHE_HOME")
62     cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "←
↪ .cache"
63     return cache_root / "lcqi-smolllm2-small-smoke"
64
65
66 def default_report_path() -> Path:
67     return volume_dir() / "results" / "lcqi-smolllm2-small-model-smoke.txt"
68
69
70 def command_line(args: list[str]) -> str:
71     return " ".join(shlex.quote(arg) for arg in args)
72
73
74 def require_tool(name: str) -> str:
75     path = shutil.which(name)
76     if path is None:
77         raise RuntimeError(f"required tool not found in PATH: {name}")
78     return path
79
80
81 def run_command(
82     args: list[str],
83     *,
84     cwd: Path | None = None,
85     input_text: str | None = None,

```

```

86     timeout_seconds: int | None = None,
87 ) -> CommandResult:
88     try:
89         completed = subprocess.run(
90             args,
91             cwd=str(cwd) if cwd else None,
92             input=input_text,
93             text=True,
94             capture_output=True,
95             timeout=timeout_seconds,
96             check=False,
97         )
98     except subprocess.TimeoutExpired as exc:
99         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
100         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
101         return CommandResult(args=args, returncode=124, stdout=stdout, stderr=
↵ stderr)
102
103     return CommandResult(
104         args=args,
105         returncode=completed.returncode,
106         stdout=completed.stdout,
107         stderr=completed.stderr,
108     )
109
110
111 def ensure_success(result: CommandResult, action: str) -> None:
112     if result.returncode == 0:
113         return
114     raise RuntimeError(
115         f"{action} failed with exit code {result.returncode}\n"
116         f"command: {command_line(result.args)}\n"
117         f"stdout tail:\n{tail(result.stdout)}\n"
118         f"stderr tail:\n{tail(result.stderr)}"
119     )
120
121
122 def tail(text: str, max_chars: int = 4000) -> str:
123     if len(text) <= max_chars:
124         return text
125     return text[-max_chars:]
126
127
128 def sha256_file(path: Path) -> str:
129     digest = hashlib.sha256()
130     with path.open("rb") as handle:
131         for block in iter(lambda: handle.read(1024 * 1024), b""):
132             digest.update(block)
133     return digest.hexdigest()
134
135
136 def verify_model_file(path: Path) -> tuple[bool, str, int]:

```

```

137     if not path.exists():
138         return False, "", 0
139     actual_bytes = path.stat().st_size
140     actual_sha = sha256_file(path)
141     return (
142         actual_bytes == MODEL_EXPECTED_BYTES
143         and actual_sha == MODEL_EXPECTED_SHA256,
144         actual_sha,
145         actual_bytes,
146     )
147
148
149 def download_model(model_path: Path, *, force_download: bool, no_download: bool ↵
    ↵ ) -> None:
150     model_path.parent.mkdir(parents=True, exist_ok=True)
151
152     if force_download and model_path.exists():
153         model_path.unlink()
154
155     is_valid, actual_sha, actual_bytes = verify_model_file(model_path)
156     if is_valid:
157         print(f"[lcqi-smoke] model cache hit: {model_path}")
158         return
159
160     if model_path.exists():
161         print(
162             "[lcqi-smoke] cached model hash mismatch, redownloading: "
163             f"bytes={actual_bytes}, sha256={actual_sha}"
164         )
165         model_path.unlink()
166
167     if no_download:
168         raise RuntimeError(
169             f"model file is missing or invalid and --no-download was set: {↵
    ↵ model_path}"
170         )
171
172     require_tool("curl")
173     part_path = model_path.with_suffix(model_path.suffix + ".part")
174     if part_path.exists():
175         part_path.unlink()
176
177     print(f"[lcqi-smoke] downloading small GGUF model: {MODEL_GGUF_REPO_ID}/{↵
    ↵ MODEL_FILE_NAME}")
178     result = run_command(
179         [
180             "curl",
181             "-L",
182             "--fail",
183             "--retry",
184             "3",
185             "-o",

```

```

186         str(part_path),
187         MODEL_URL,
188     ]
189 )
190 ensure_success(result, "download SmolLM2 GGUF")
191
192 downloaded_ok, downloaded_sha, downloaded_bytes = verify_model_file(↵
↵ part_path)
193 if not downloaded_ok:
194     raise RuntimeError(
195         "downloaded model did not match expected identity: "
196         f"bytes={downloaded_bytes}, sha256={downloaded_sha}"
197     )
198 part_path.replace(model_path)
199 print(f"[lcqi-smoke] model verified: sha256={MODEL_EXPECTED_SHA256}")
200
201
202 def ensure_llama_simple_chat(cache_dir: Path, *, jobs: int, skip_build: bool) ↵
↵ -> Path:
203     cached_binary = cache_dir / "llama.cpp" / "build" / "bin" / LLAMA_TARGET
204     source_dir = cache_dir / "llama.cpp"
205     build_dir = source_dir / "build"
206     if cached_binary.exists() and os.access(cached_binary, os.X_OK) and ↵
↵ is_pinned_llama_checkout(source_dir):
207         print(f"[lcqi-smoke] llama.cpp binary cache hit: {cached_binary}")
208         return cached_binary
209
210     if skip_build:
211         raise RuntimeError(f"llama.cpp binary is missing and --skip-build was ↵
↵ set: {cached_binary}")
212
213     require_tool("git")
214     require_tool("cmake")
215
216     cache_dir.mkdir(parents=True, exist_ok=True)
217
218     if not (source_dir / ".git").exists():
219         print(f"[lcqi-smoke] cloning llama.cpp into cache: {source_dir}")
220         result = run_command(
221             [
222                 "git",
223                 "clone",
224                 "--filter=blob:none",
225                 "--no-checkout",
226                 LLAMA_CPP_REPO_URL,
227                 str(source_dir),
228             ]
229         )
230         ensure_success(result, "clone llama.cpp")
231
232     print(f"[lcqi-smoke] checking out pinned llama.cpp commit: {↵
↵ LLAMA_CPP_COMMIT}")

```



```

233     fetch = run_command(
234         ["git", "-C", str(source_dir), "fetch", "--depth", "1", "origin", ↵
↵ LLAMA_CPP_COMMIT]
235     )
236     ensure_success(fetch, "fetch pinned llama.cpp commit")
237     checkout = run_command(["git", "-C", str(source_dir), "checkout", "--detach↵
↵ ", LLAMA_CPP_COMMIT])
238     ensure_success(checkout, "checkout pinned llama.cpp commit")
239
240     print(f"[lcqi-smoke] configuring llama.cpp build: {build_dir}")
241     configure = run_command(
242         [
243             "cmake",
244             "-S",
245             str(source_dir),
246             "-B",
247             str(build_dir),
248             "-DCMAKE_BUILD_TYPE=Release",
249             "-DLLAMA_BUILD_TESTS=OFF",
250             "-DLLAMA_BUILD_EXAMPLES=ON",
251             "-DLLAMA_BUILD_SERVER=OFF",
252             "-DGGML_NATIVE=OFF",
253         ]
254     )
255     ensure_success(configure, "configure llama.cpp")
256
257     print(f"[lcqi-smoke] building {LLAMA_TARGET} with -j{jobs}")
258     build = run_command(
259         ["cmake", "--build", str(build_dir), "--target", LLAMA_TARGET, "-j", ↵
↵ str(jobs)]
260     )
261     ensure_success(build, f"build {LLAMA_TARGET}")
262
263     if not cached_binary.exists():
264         raise RuntimeError(f"expected binary was not produced: {cached_binary}"↵
↵ )
265     return cached_binary
266
267
268 def is_pinned_llama_checkout(source_dir: Path) -> bool:
269     if not (source_dir / ".git").exists():
270         return False
271     result = run_command(["git", "-C", str(source_dir), "rev-parse", "HEAD"])
272     if result.returncode != 0:
273         return False
274     return result.stdout.strip() == LLAMA_CPP_COMMIT
275
276
277 def strip_ansi(text: str) -> str:
278     return re.sub(r"\x1b\[([0-9]?[0-9]?[* -/]*[@~])", "", text)
279
280

```

```

281 def extract_response(stdout: str) -> str:
282     cleaned = strip_ansi(stdout).replace("\r", "")
283     chunks = [chunk.strip() for chunk in re.split(r"(?:^|\n)> ?", cleaned) if ←
    ↪ chunk.strip()]
284     if not chunks:
285         return ""
286     return chunks[0]
287
288
289 def run_model_smoke(
290     llama_binary: Path,
291     model_path: Path,
292     *,
293     prompt: str,
294     context_tokens: int,
295     timeout_seconds: int,
296 ) -> tuple[CommandResult, str]:
297     command = [
298         str(llvm_binary),
299         "-m",
300         str(model_path),
301         "-ngl",
302         "0",
303         "-c",
304         str(context_tokens),
305     ]
306     print("[lcqi-smoke] running real small model generation")
307     result = run_command(
308         command,
309         input_text=prompt.rstrip("\n") + "\n\n",
310         timeout_seconds=timeout_seconds,
311     )
312     ensure_success(result, "run small model smoke")
313
314     response = extract_response(result.stdout)
315     if not response:
316         raise RuntimeError(
317             "small model smoke exited successfully but produced no assistant ←
    ↪ text\n"
318             f"stdout:\n{result.stdout}\n"
319             f"stderr:\n{result.stderr}"
320         )
321     return result, response
322
323
324 def current_repo_commit() -> str:
325     result = run_command(["git", "-C", str(repo_dir()), "rev-parse", "--short", ←
    ↪ "HEAD"])
326     if result.returncode != 0:
327         return "unknown"
328     return result.stdout.strip()
329

```

```

330
331 def current_uname() -> str:
332     result = run_command(["uname", "-a"])
333     if result.returncode != 0:
334         return "unknown"
335     return result.stdout.strip()
336
337
338 def write_report(
339     report_path: Path,
340     *,
341     cache_dir: Path,
342     llama_binary: Path,
343     model_path: Path,
344     prompt: str,
345     result: CommandResult,
346     response: str,
347 ) -> None:
348     report_path.parent.mkdir(parents=True, exist_ok=True)
349     clean_stdout = strip_ansi(result.stdout).strip()
350     report = "\n".join(
351         [
352             "LCQI SmolLM2 Small Model Smoke",
353             "",
354             f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
355             f"repo_commit={current_repo_commit()}",
356             f"host={current_uname()}",
357             f"python={sys.version.split()[0]}",
358             f"cache_dir={cache_dir}",
359             "",
360             f"model_source_id={MODEL_SOURCE_ID}",
361             f"model_gguf_repo_id={MODEL_GGUF_REPO_ID}",
362             f"model_file={MODEL_FILE_NAME}",
363             f"model_url={MODEL_URL}",
364             f"model_expected_bytes={MODEL_EXPECTED_BYTES}",
365             f"model_expected_sha256={MODEL_EXPECTED_SHA256}",
366             f"model_path={model_path}",
367             "",
368             f"llama_cpp_repo={LLAMA_CPP_REPO_URL}",
369             f"llama_cpp_commit={LLAMA_CPP_COMMIT}",
370             f"llama_target={LLAMA_TARGET}",
371             f"llama_binary={llama_binary}",
372             "",
373             f"prompt={prompt}",
374             f"command={command_line(result.args)}",
375             f"exit_code={result.returncode}",
376             "",
377             "validation=PASS: model hash matched, llama-simple-chat exited 0, "
378             "assistant response was non-empty",
379             "",
380             "assistant_response:",
381             response,

```

```

382         "",
383         "stdout_clean_tail:",
384         tail(clean_stdout, 2000),
385         "",
386         "stderr_tail:",
387         tail(result.stderr.strip(), 2000),
388         "",
389     ]
390 )
391 report_path.write_text(report, encoding="utf-8")
392
393
394 def parse_args() -> argparse.Namespace:
395     parser = argparse.ArgumentParser(
396         description=(
397             "Run a real local small open-source model smoke for CPU Volume 3. "
398             "The script caches llama.cpp and the GGUF model outside the Git ↵
↵ repository."
399         )
400     )
401     parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
402     parser.add_argument("--model-path", type=Path, default=None)
403     parser.add_argument("--llama-bin", type=Path, default=None)
404     parser.add_argument("--report", type=Path, default=default_report_path())
405     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
406     parser.add_argument("--ctx", type=int, default=DEFAULT_CONTEXT_TOKENS)
407     parser.add_argument("--timeout", type=int, default=DEFAULT_TIMEOUT_SECONDS)
408     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
409     parser.add_argument("--force-download", action="store_true")
410     parser.add_argument("--no-download", action="store_true")
411     parser.add_argument("--skip-build", action="store_true")
412     return parser.parse_args()
413
414
415 def main() -> int:
416     args = parse_args()
417     cache_dir = args.cache_dir.expanduser().resolve()
418     model_path = (
419         args.model_path.expanduser().resolve()
420         if args.model_path
421         else cache_dir / "models" / MODEL_FILE_NAME
422     )
423     report_path = args.report.expanduser().resolve()
424
425     try:
426         download_model(
427             model_path,
428             force_download=args.force_download,
429             no_download=args.no_download,
430         )
431     if args.llama_bin:
432         llama_binary = args.llama_bin.expanduser().resolve()

```

```

433         if not llama_binary.exists() or not os.access(llvm_binary, os.X_OK):
434             raise RuntimeError(f"--llama-bin is not executable: {llama_binary}")
435         else:
436             llama_binary = ensure_llama_simple_chat(
437                 cache_dir,
438                 jobs=args.jobs,
439                 skip_build=args.skip_build,
440             )
441
442             result, response = run_model_smoke(
443                 llama_binary,
444                 model_path,
445                 prompt=args.prompt,
446                 context_tokens=args.ctx,
447                 timeout_seconds=args.timeout,
448             )
449             write_report(
450                 report_path,
451                 cache_dir=cache_dir,
452                 llama_binary=llama_binary,
453                 model_path=model_path,
454                 prompt=args.prompt,
455                 result=result,
456                 response=response,
457             )
458         except RuntimeError as exc:
459             print(f"[lcqi-smoke] ERROR: {exc}", file=sys.stderr)
460             return 1
461
462         print(f"report={report_path}")
463         print(f"model_sha256={MODEL_EXPECTED_SHA256}")
464         print(f"assistant_response={response}")
465         return 0
466
467     if __name__ == "__main__":
468         raise SystemExit(main())
469 
```

run_gpt2_smoke.py 下载标准 GPT-2 的 config.json、vocab.json、merges.txt 和 model.safetensors，再调用 LCQI 自研 lcqi_gpt2 reference path 生成 token。它证明自研 safetensors、BPE、GPT-2 forward、KV cache decode 和 greedy generation 已连成闭环。

Listing 6.7 tools/run_gpt2_smoke.py

```

1 #!/usr/bin/env python3
2 from __future__ import annotations
3
4 import argparse
5 import datetime as dt
6 import hashlib

```

```

7 import os
8 import shlex
9 import subprocess
10 import sys
11 import urllib.request
12 from dataclasses import dataclass
13 from pathlib import Path
14
15
16 MODEL_REPO_ID = "openai-community/gpt2"
17 MODEL_REVISION = "main"
18 MODEL_FILE_IDENTITIES = {
19     "config.json": (
20         665,
21         "0daed7749b4f02b8f76240d5444551d7b08712dab4d0adb8239c56ba823bb7b4",
22     ),
23     "vocab.json": (
24         1042301,
25         "196139668be63f3b5d6574427317ae82f612a97c5d1cdf36ed2256dbf636783",
26     ),
27     "merges.txt": (
28         456318,
29         "1ce1664773c50f3e0cc8842619a93edc4624525b728b188a9e0be33b7726adc5",
30     ),
31     "model.safetensors": (
32         548105171,
33         "248dfc3911869ec493c76e65bf2fcf7f615828b0254c12b473182f0f81d3a707",
34     ),
35 }
36 DEFAULT_PROMPT = "Hello, my name is"
37 DEFAULT_MAX_NEW_TOKENS = 1
38 DEFAULT_TIMEOUT_SECONDS = 300
39 DEFAULT_BUILD_JOBS = 2
40
41
42 @dataclass(frozen=True)
43 class CommandResult:
44     args: list[str]
45     returncode: int
46     stdout: str
47     stderr: str
48
49
50 def script_path() -> Path:
51     return Path(__file__).resolve()
52
53
54 def volume_dir() -> Path:
55     return script_path().parents[1]
56
57
58 def repo_dir() -> Path:

```

```

59     return script_path().parents[3]
60
61
62 def default_cache_dir() -> Path:
63     explicit_cache = os.environ.get("LCQI_GPT2_CACHE_DIR")
64     if explicit_cache:
65         return Path(explicit_cache).expanduser()
66     xdg_cache = os.environ.get("XDG_CACHE_HOME")
67     cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "←
    ↪ .cache"
68     return cache_root / "lcqi-gpt2-smoke" / MODEL_REPO_ID.replace("/", "--")
69
70
71 def default_build_dir() -> Path:
72     return volume_dir() / "build" / "lcqi-release"
73
74
75 def default_report_path() -> Path:
76     return volume_dir() / "results" / "lcqi-gpt2-smoke.txt"
77
78
79 def command_line(args: list[str]) -> str:
80     return " ".join(shlex.quote(arg) for arg in args)
81
82
83 def run_command(
84     args: list[str],
85     *,
86     cwd: Path | None = None,
87     timeout_seconds: int | None = None,
88 ) -> CommandResult:
89     try:
90         completed = subprocess.run(
91             args,
92             cwd=cwd if cwd else None,
93             text=True,
94             capture_output=True,
95             timeout=timeout_seconds,
96             check=False,
97         )
98     except subprocess.TimeoutExpired as exc:
99         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
100         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
101         return CommandResult(args=args, returncode=124, stdout=stdout, stderr=←
    ↪ stderr)
102     return CommandResult(
103         args=args,
104         returncode=completed.returncode,
105         stdout=completed.stdout,
106         stderr=completed.stderr,
107     )
108

```

```

109
110 def ensure_success(result: CommandResult, action: str) -> None:
111     if result.returncode == 0:
112         return
113     raise RuntimeError(
114         f"{action} failed with exit code {result.returncode}\n"
115         f"command: {command_line(result.args)}\n"
116         f"stdout tail:\n{tail(result.stdout)}\n"
117         f"stderr tail:\n{tail(result.stderr)}"
118     )
119
120
121 def tail(text: str, max_chars: int = 4000) -> str:
122     if len(text) <= max_chars:
123         return text
124     return text[-max_chars:]
125
126
127 def sha256_file(path: Path) -> str:
128     digest = hashlib.sha256()
129     with path.open("rb") as handle:
130         for block in iter(lambda: handle.read(1024 * 1024), b""):
131             digest.update(block)
132     return digest.hexdigest()
133
134
135 def model_url(file_name: str) -> str:
136     return (
137         f"https://huggingface.co/{MODEL_REPO_ID}/resolve/"
138         f"{MODEL_REVISION}/{file_name}"
139     )
140
141
142 def download_file(url: str, target: Path, timeout_seconds: int) -> None:
143     part_path = target.with_suffix(target.suffix + ".part")
144     if part_path.exists():
145         part_path.unlink()
146     request = urllib.request.Request(url, headers={"User-Agent": "lcqi-gpt2-↵
↵ smoke"})
147     with urllib.request.urlopen(request, timeout=timeout_seconds) as response:
148         with part_path.open("wb") as output:
149             while True:
150                 block = response.read(1024 * 1024)
151                 if not block:
152                     break
153                 output.write(block)
154     part_path.replace(target)
155
156
157 def file_identity_matches(path: Path, expected_bytes: int, expected_sha256: str↵
↵ ) -> bool:
158     return (

```



```

159     path.exists()
160     and path.stat().st_size == expected_bytes
161     and sha256_file(path) == expected_sha256
162 )
163
164
165 def ensure_model_files(cache_dir: Path, *, no_download: bool, timeout_seconds: ←
    ↪ int) -> None:
166     cache_dir.mkdir(parents=True, exist_ok=True)
167     missing = [
168         name
169         for name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.←
    ↪ items()
170         if not file_identity_matches(cache_dir / name, expected_bytes, ←
    ↪ expected_sha256)
171     ]
172     if not missing:
173         print(f"[lcqi-gpt2] model cache hit: {cache_dir}")
174         return
175     if no_download:
176         raise RuntimeError(
177             f"missing GPT-2 model files and --no-download was set: {missing}"
178         )
179     for file_name in missing:
180         target = cache_dir / file_name
181         if target.exists():
182             target.unlink()
183         print(f"[lcqi-gpt2] downloading {MODEL_REPO_ID}/{file_name}")
184         download_file(model_url(file_name), target, timeout_seconds)
185         expected_bytes, expected_sha256 = MODEL_FILE_IDENTITIES[file_name]
186         if not file_identity_matches(target, expected_bytes, expected_sha256):
187             raise RuntimeError(
188                 f"downloaded GPT-2 file identity mismatch: {file_name}"
189             )
190
191
192 def build_lcqi_gpt2(build_dir: Path, jobs: int) -> Path:
193     configure = run_command(
194         [
195             "cmake",
196             "-S",
197             str(volume_dir()),
198             "-B",
199             str(build_dir),
200             "-DCMAKE_BUILD_TYPE=Release",
201         ],
202         timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
203     )
204     ensure_success(configure, "configure LCQI")
205     build = run_command(
206         [
207             "cmake",

```

```

208         "--build",
209         str(build_dir),
210         "--target",
211         "lcqi_gpt2",
212         "-j",
213         str(jobs),
214     ],
215     timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
216 )
217 ensure_success(build, "build lcqi_gpt2")
218 binary = build_dir / "labs" / "linux_cpu_inference" / "lcqi_gpt2"
219 if not binary.exists():
220     raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
221 return binary
222
223
224 def write_report(
225     report_path: Path,
226     *,
227     cache_dir: Path,
228     binary: Path,
229     prompt: str,
230     max_new_tokens: int,
231     engine: str,
232     result: CommandResult,
233 ) -> None:
234     report_path.parent.mkdir(parents=True, exist_ok=True)
235     lines = [
236         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
237         f"model_repo_id={MODEL_REPO_ID}",
238         f"model_revision={MODEL_REVISION}",
239         f"model_cache_dir={cache_dir}",
240     ]
241     for file_name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.items():
242         path = cache_dir / file_name
243         lines.append(f"file.{file_name}.bytes={path.stat().st_size}")
244         lines.append(f"file.{file_name}.sha256={sha256_file(path)}")
245         lines.append(f"file.{file_name}.expected_bytes={expected_bytes}")
246         lines.append(f"file.{file_name}.expected_sha256={expected_sha256}")
247     lines.extend(
248         [
249             f"binary={binary}",
250             f"prompt={prompt}",
251             f"max_new_tokens={max_new_tokens}",
252             f"engine={engine}",
253             f"command={command_line(result.args)}",
254             f"exit_code={result.returncode}",
255             "stdout_begin",
256             result.stdout.rstrip(),
257             "stdout_end",
258             "stderr_begin",

```

```

259         tail(result.stderr).rstrip(),
260         "stderr_end",
261     ]
262 )
263 passed = result.returncode == 0 and "generated_text " in result.stdout
264 lines.append(f"validation={'PASS' if passed else 'FAIL'}")
265 report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
266
267
268 def parse_args() -> argparse.Namespace:
269     parser = argparse.ArgumentParser(
270         description="Run LCQI self-owned GPT-2 smoke on a HuggingFace ↵
↵ safetensors directory."
271     )
272     parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
273     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
274     parser.add_argument("--report", type=Path, default=default_report_path())
275     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
276     parser.add_argument("--max-new-tokens", type=int, default=↵
↵ DEFAULT_MAX_NEW_TOKENS)
277     parser.add_argument("--engine", choices=("cached", "full"), default="cached↵
↵ ")
278     parser.add_argument("--benchmark", action="store_true")
279     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
280     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
281     parser.add_argument("--no-download", action="store_true")
282     return parser.parse_args()
283
284
285 def main() -> int:
286     args = parse_args()
287     try:
288         if args.max_new_tokens < 0:
289             raise RuntimeError("--max-new-tokens cannot be negative")
290         ensure_model_files(
291             args.cache_dir,
292             no_download=args.no_download,
293             timeout_seconds=args.timeout_seconds,
294         )
295         binary = build_lcqi_gpt2(args.build_dir, args.jobs)
296         command = [
297             str(binary),
298             "--engine",
299             args.engine,
300             str(args.cache_dir),
301             args.prompt,
302             str(args.max_new_tokens),
303         ]
304         if args.benchmark:
305             command.insert(1, "--benchmark")
306         result = run_command(command, cwd=repo_dir(), timeout_seconds=args.↵

```

```

↪ timeout_seconds)
307     write_report(
308         args.report,
309         cache_dir=args.cache_dir,
310         binary=binary,
311         prompt=args.prompt,
312         max_new_tokens=args.max_new_tokens,
313         engine=args.engine,
314         result=result,
315     )
316     print(f"[lcqi-gpt2] report: {args.report}")
317     print(tail(result.stdout, max_chars=2000))
318     return 0 if result.returncode == 0 else result.returncode
319 except Exception as error:
320     print(f"[lcqi-gpt2] error: {error}", file=sys.stderr)
321     return 1
322
323
324 if __name__ == "__main__":
325     raise SystemExit(main())

```

`run_gpt2_benchmark_compare.py` 用同一个 GPT-2 F32 safetensors 模型分别运行 LCQI full-prefix 与 cached-KV 路径,并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考。它的报告说明算法差距、kernel 差距和模型口径差异。

Listing 6.8 tools/run_gpt2_benchmark_compare.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import os
7  import re
8  import shlex
9  import subprocess
10 import sys
11 from dataclasses import dataclass
12 from pathlib import Path
13
14 import run_gpt2_smoke
15
16
17 DEFAULT_PROMPT = "Hello, my name is"
18 DEFAULT_MAX_NEW_TOKENS = 4
19 DEFAULT_BUILD_JOBS = 2
20 DEFAULT_TIMEOUT_SECONDS = 300
21 DEFAULT_SMOLLM2_CACHE_DIR = Path.home() / ".cache" / "lcqi-smolllm2-small-smoke"
22 SMOLLM2_GGUF = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
23
24
25 @dataclass(frozen=True)
26 class EngineRun:

```

```

27     name: str
28     command: list[str]
29     returncode: int
30     stdout: str
31     stderr: str
32     metrics: dict[str, str]
33
34
35 def volume_dir() -> Path:
36     return Path(__file__).resolve().parents[1]
37
38
39 def repo_dir() -> Path:
40     return Path(__file__).resolve().parents[3]
41
42
43 def default_report_path() -> Path:
44     return volume_dir() / "results" / "lcqi-gpt2-benchmark-compare.txt"
45
46
47 def default_build_dir() -> Path:
48     return volume_dir() / "build" / "lcqi-release"
49
50
51 def command_line(args: list[str]) -> str:
52     return " ".join(shlex.quote(arg) for arg in args)
53
54
55 def tail(text: str, max_chars: int = 4000) -> str:
56     return text if len(text) <= max_chars else text[-max_chars:]
57
58
59 def run_command(args: list[str], *, timeout_seconds: int) -> tuple[int, str, ←
    ↪ str]:
60     try:
61         completed = subprocess.run(
62             args,
63             cwd=repo_dir(),
64             text=True,
65             capture_output=True,
66             timeout=timeout_seconds,
67             check=False,
68         )
69     except subprocess.TimeoutExpired as exc:
70         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
71         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
72         return 124, stdout, stderr
73     return completed.returncode, completed.stdout, completed.stderr
74
75
76 def parse_lcqi_metrics(stdout: str) -> dict[str, str]:
77     metrics: dict[str, str] = {}

```

```

78     for line in stdout.splitlines():
79         if not line:
80             continue
81         key, _, value = line.partition(" ")
82         if key in {
83             "engine",
84             "generated_text",
85             "benchmark_load_ms",
86             "benchmark_tokenize_ms",
87             "benchmark_prefill_ms",
88             "benchmark_decode_ms",
89             "benchmark_generate_ms",
90             "benchmark_total_ms",
91             "benchmark_prompt_tokens",
92             "benchmark_generated_tokens",
93             "benchmark_prefill_steps",
94             "benchmark_decode_steps",
95             "benchmark_generate_tokens_per_second",
96             "benchmark_decode_tokens_per_second",
97             "benchmark_kv_cache_bytes",
98         }:
99             metrics[key] = value
100     return metrics
101
102
103 def run_lcqi(
104     binary: Path,
105     model_dir: Path,
106     *,
107     engine: str,
108     prompt: str,
109     max_new_tokens: int,
110     timeout_seconds: int,
111 ) -> EngineRun:
112     command = [
113         str(binary),
114         "--benchmark",
115         "--engine",
116         engine,
117         str(model_dir),
118         prompt,
119         str(max_new_tokens),
120     ]
121     returncode, stdout, stderr = run_command(command, timeout_seconds=↵
↵ timeout_seconds)
122     return EngineRun(
123         name=f"lcqi_{engine}",
124         command=command,
125         returncode=returncode,
126         stdout=stdout,
127         stderr=stderr,
128         metrics=parse_lcqi_metrics(stdout),

```

```

129     )
130
131
132 def llama_bench_binary(cache_dir: Path) -> Path:
133     return cache_dir / "llama.cpp" / "build" / "bin" / "llama-bench"
134
135
136 def smollm2_model_path(cache_dir: Path) -> Path:
137     return cache_dir / "models" / SMOLLM2_GGUF
138
139
140 def parse_llama_bench_metrics(stdout: str) -> dict[str, str]:
141     metrics: dict[str, str] = {}
142     lines = [line.strip() for line in stdout.splitlines() if line.strip()]
143     for line in lines:
144         if not line.startswith("|"):
145             continue
146         columns = [column.strip() for column in line.strip("|").split("|")]
147         if len(columns) != 7 or columns[0] == "model" or columns[0].startswith(←
↪ "-"):
148             continue
149         metrics["llama_model"] = columns[0]
150         metrics["llama_size"] = columns[1]
151         metrics["llama_params"] = columns[2]
152         metrics["llama_backend"] = columns[3]
153         metrics["llama_threads"] = columns[4]
154         test_name = columns[5]
155         tokens_per_second = re.sub(r"\s+.*$", "", columns[6])
156         if test_name.startswith("pp"):
157             metrics["llama_prefill_test"] = test_name
158             metrics["llama_prefill_tps"] = tokens_per_second
159         elif test_name.startswith("tg"):
160             metrics["llama_decode_test"] = test_name
161             metrics["llama_decode_tps"] = tokens_per_second
162     return metrics
163
164
165 def run_llama_bench(cache_dir: Path, *, timeout_seconds: int) -> EngineRun | ←
↪ None:
166     binary = llama_bench_binary(cache_dir)
167     model = smollm2_model_path(cache_dir)
168     if not binary.exists() or not os.access(binary, os.X_OK) or not model.←
↪ exists():
169         return None
170     command = [
171         str(binary),
172         "-m",
173         str(model),
174         "-ngl",
175         "0",
176         "-p",
177         "5",

```

```

178     "-n",
179     "4",
180     "-r",
181     "1",
182 ]
183 returncode, stdout, stderr = run_command(command, timeout_seconds=↵
↵ timeout_seconds)
184 return EngineRun(
185     name="llama_cpp_smolllm2_q4_k_m",
186     command=command,
187     returncode=returncode,
188     stdout=stdout,
189     stderr=stderr,
190     metrics=parse_llama_bench_metrics(stdout),
191 )
192
193
194 def write_report(path: Path, runs: list[EngineRun], *, prompt: str, ↵
↵ max_new_tokens: int) -> None:
195     path.parent.mkdir(parents=True, exist_ok=True)
196     lines = [
197         "LCQI GPT-2 Benchmark Compare",
198         "",
199         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
200         f"repo_commit={current_commit()}",
201         f"working_tree_dirty={working_tree_dirty()}",
202         f"python={sys.version.split()[0]}",
203         f"prompt={prompt}",
204         f"max_new_tokens={max_new_tokens}",
205         "",
206         "scope=LCQI full_prefix and cached_kv run the same GPT-2 F32 ↵
↵ safetensors model. "
207         "llama.cpp uses SmolLM2-135M-Instruct Q4_K_M as an external mature↵
↵ engine reference, "
208         "so its numbers are engineering context, not same-model quality ranking↵
↵ .",
209         "",
210     ]
211     for run in runs:
212         lines.extend(
213             [
214                 f"[{run.name}]",
215                 f"returncode={run.returncode}",
216                 f"command={command_line(run.command)}",
217             ]
218         )
219         for key in sorted(run.metrics):
220             lines.append(f"{key}={run.metrics[key]}")
221         lines.extend(
222             [
223                 "stdout_tail:",
224                 tail(run.stdout.strip(), 2000),

```



```

225         "stderr_tail:",
226         tail(run.stderr.strip(), 2000),
227         "",
228     ]
229 )
230 while lines and lines[-1] == "":
231     lines.pop()
232 path.write_text("\n".join(lines) + "\n", encoding="utf-8")
233
234
235 def current_commit() -> str:
236     result = subprocess.run(
237         ["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"],
238         text=True,
239         capture_output=True,
240         check=False,
241     )
242     return result.stdout.strip() if result.returncode == 0 else "unknown"
243
244
245 def working_tree_dirty() -> str:
246     result = subprocess.run(
247         ["git", "-C", str(repo_dir()), "status", "--porcelain"],
248         text=True,
249         capture_output=True,
250         check=False,
251     )
252     if result.returncode != 0:
253         return "unknown"
254     return "true" if result.stdout.strip() else "false"
255
256
257 def parse_args() -> argparse.Namespace:
258     parser = argparse.ArgumentParser(
259         description="Compare LCQI GPT-2 full-prefix and KV-cache paths, with ↵
↵ optional llama.cpp context."
260     )
261     parser.add_argument("--cache-dir", type=Path, default=run_gpt2_smoke.↵
↵ default_cache_dir())
262     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
263     parser.add_argument("--report", type=Path, default=default_report_path())
264     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
265     parser.add_argument("--max-new-tokens", type=int, default=↵
↵ DEFAULT_MAX_NEW_TOKENS)
266     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
267     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
268     parser.add_argument("--no-download", action="store_true")
269     parser.add_argument("--skip-llama", action="store_true")
270     parser.add_argument("--smollm2-cache-dir", type=Path, default=↵
↵ DEFAULT_SMOLLM2_CACHE_DIR)
271     return parser.parse_args()

```

```

272
273
274 def main() -> int:
275     args = parse_args()
276     if args.max_new_tokens < 0:
277         print("[lcqi-compare] --max-new-tokens cannot be negative", file=sys.↵
↵ stderr)
278         return 1
279     try:
280         run_gpt2_smoke.ensure_model_files(
281             args.cache_dir,
282             no_download=args.no_download,
283             timeout_seconds=args.timeout_seconds,
284         )
285         binary = run_gpt2_smoke.build_lcqi_gpt2(args.build_dir, args.jobs)
286         runs = [
287             run_lcqi(
288                 binary,
289                 args.cache_dir,
290                 engine="full",
291                 prompt=args.prompt,
292                 max_new_tokens=args.max_new_tokens,
293                 timeout_seconds=args.timeout_seconds,
294             ),
295             run_lcqi(
296                 binary,
297                 args.cache_dir,
298                 engine="cached",
299                 prompt=args.prompt,
300                 max_new_tokens=args.max_new_tokens,
301                 timeout_seconds=args.timeout_seconds,
302             ),
303         ]
304         if not args.skip_llama:
305             llama_run = run_llama_bench(
306                 args.smollm2_cache_dir.expanduser().resolve(),
307                 timeout_seconds=args.timeout_seconds,
308             )
309             if llama_run is not None:
310                 runs.append(llama_run)
311         write_report(args.report, runs, prompt=args.prompt, max_new_tokens=args.↵
↵ .max_new_tokens)
312     except Exception as error:
313         print(f"[lcqi-compare] error: {error}", file=sys.stderr)
314         return 1
315
316     for run in runs:
317         print(f"{run.name}: returncode={run.returncode}")
318         for key in sorted(run.metrics):
319             print(f"  {key}={run.metrics[key]}")
320     print(f"report={args.report}")
321     return 0 if all(run.returncode == 0 for run in runs) else 1

```

```

322
323
324 if __name__ == "__main__":
325     raise SystemExit(main())

```

`run_lcqi_benchmark.sh` 是本地 benchmark 收集脚本。它适合在固定机器上反复运行，把同一组 shape 和 backend 的结果落到报告文件。

Listing 6.9 tools/run_lcqi_benchmark.sh

```

1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5 VOLUME_DIR="$(cd "${SCRIPT_DIR}/.." && pwd)"
6 REPO_DIR="$(cd "${VOLUME_DIR}/../.." && pwd)"
7 BUILD_DIR="${VOLUME_DIR}/build/lcqi-release"
8 RESULT_DIR="${VOLUME_DIR}/results"
9 REPEAT="${1:-200}"
10 STAMP="$(date -u +%Y%m%dT%H%M%S)"
11 CSV_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.csv"
12 META_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.meta.txt"
13 PERF_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.perf.txt"
14
15 mkdir -p "${RESULT_DIR}"
16
17 cmake -S "${VOLUME_DIR}" -B "${BUILD_DIR}" -DCMAKE_BUILD_TYPE=Release
18 cmake --build "${BUILD_DIR}"
19 ctest --test-dir "${BUILD_DIR}" --output-on-failure
20
21 {
22     echo "timestamp_utc=${STAMP}"
23     echo "repeat=${REPEAT}"
24     echo "commit=$(git -C "${REPO_DIR}" rev-parse --short HEAD)"
25     echo "uname=$(uname -a)"
26     echo "compiler=$((${CXX:-c++} --version | sed -n '1p')"
27     echo "cmake=$(cmake --version | sed -n '1p')"
28     echo "bench=${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench"
29 } > "${META_PATH}"
30
31 "${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" > "${CSV_PATH}"
32
33 if command -v perf >/dev/null 2>&1; then
34     perf stat \
35         -e cycles,instructions,cache-misses,branches,branch-misses \
36         "${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" \
37         > /dev/null 2> "${PERF_PATH}" || true
38 else
39     echo "perf_unavailable=1" > "${PERF_PATH}"
40 fi
41
42 echo "csv=${CSV_PATH}"
43 echo "meta=${META_PATH}"

```

```
44 echo "perf=${PERF_PATH}"
```

6.4 完整测试

lcqi_tests.cpp 把 safetensors、tokenizer、tiny MLP、int8 kernel、reference decoder、GPT-2 tiny path、trace、ledger 和 serving probe 串成验收网。新增优化或 shape 时，必须先扩展这里的 golden 和边界样例。

Listing 6.10 tests/lcqi_tests.cpp

```
1 #include <lcqi/gpt2_reference.hpp>
2 #include <lcqi/f32_kernels.hpp>
3 #include <lcqi/ggml_matvec.hpp>
4 #include <lcqi/ggml_tensors.hpp>
5 #include <lcqi/gguf.hpp>
6 #include <lcqi/inference.hpp>
7 #include <lcqi/int8_kernels.hpp>
8 #include <lcqi/llama_gguf.hpp>
9 #include <lcqi/model.hpp>
10 #include <lcqi/q4_k.hpp>
11 #include <lcqi/reference_decoder.hpp>
12 #include <lcqi/safetensors.hpp>
13 #include <lcqi/tokenizer.hpp>
14
15 #include <array>
16 #include <cmath>
17 #include <cstring>
18 #include <cstdlib>
19 #include <filesystem>
20 #include <fstream>
21 #include <iostream>
22 #include <span>
23 #include <sstream>
24 #include <stdexcept>
25 #include <string>
26 #include <utility>
27 #include <vector>
28
29 namespace {
30
31 constexpr float LCQI_TEST_EPSILON = 1.0e-5F;
32 constexpr std::int32_t LCQI_TEST_INPUT_SIZE = 4;
33 constexpr std::int32_t LCQI_TEST_HIDDEN_SIZE = 3;
34 constexpr std::int32_t LCQI_TEST_OUTPUT_SIZE = 2;
35 constexpr std::int32_t LCQI_TEST_PREDICTED_CLASS = 1;
36 constexpr float LCQI_TEST_LOGIT_0 = -0.48F;
37 constexpr float LCQI_TEST_LOGIT_1 = 1.06F;
38 constexpr float LCQI_TEST_HIDDEN_0 = 0.0F;
39 constexpr float LCQI_TEST_HIDDEN_1 = 0.4F;
40 constexpr float LCQI_TEST_HIDDEN_2 = 1.4F;
41 constexpr float LCQI_RMS_EXPECTED_0 = 0.848528F;
42 constexpr float LCQI_RMS_EXPECTED_1 = 0.565685F;
```

```

43 constexpr float LCQI_ATTENTION_EXPECTED_0 = 2.0F;
44 constexpr float LCQI_ATTENTION_EXPECTED_1 = 3.0F;
45 constexpr float LCQI_KERNEL_MAX_DIFF = 1.0e-5F;
46 constexpr float LCQI_F32_KERNEL_MAX_DIFF = 1.0e-4F;
47 constexpr std::int32_t LCQI_SIMD_TEST_BLOCK = 8;
48 constexpr std::size_t LCQI_F32_DOT_TEST_SIZE = 257;
49 constexpr std::size_t LCQI_F32_ROW_TEST_INPUT_SIZE = 65;
50 constexpr std::size_t LCQI_F32_ROW_TEST_OUTPUT_SIZE = 17;
51 constexpr std::int32_t LCQI_REFERENCE_DECODER_PREDICTED_TOKEN = 0;
52 constexpr std::size_t LCQI_REFERENCE_DECODER_VOCAB_SIZE = 5;
53 constexpr std::array<float, LCQI_REFERENCE_DECODER_VOCAB_SIZE> ↵
    ↵ LCQI_REFERENCE_DECODER_LOGITS{
54     0.352516979F,
55     -0.126884326F,
56     0.0797837451F,
57     -0.29797405F,
58     0.127030417F,
59 };
60 constexpr std::int32_t LCQI_REFERENCE_TRACE_TOKEN_COUNT = 3;
61 constexpr std::int32_t LCQI_REFERENCE_TRACE_HIDDEN_SIZE = 4;
62 constexpr std::int32_t LCQI_REFERENCE_TRACE_FIRST_TOKEN = 0;
63 constexpr std::uint64_t LCQI_TOKENIZER_HASH_EXPECTED = 13452902845388333734ULL;
64 constexpr std::int32_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
65 constexpr std::int32_t LCQI_TEST_BYTE_BITS = 8;
66 constexpr std::uint64_t LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES = 48;
67 constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_BEGIN = 0;
68 constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_END = 24;
69 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_BEGIN = 24;
70 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_END = 48;
71 constexpr unsigned int LCQI_BYTE_MASK = 0xFFU;
72 constexpr std::int32_t LCQI_GPT2_PREDICTED_TOKEN = 3;
73 constexpr std::size_t LCQI_GPT2_VOCAB_SIZE = 6;
74 constexpr std::array<float, LCQI_GPT2_VOCAB_SIZE> LCQI_GPT2_LOGITS{
75     0.407358F,
76     -0.504049F,
77     -0.107834F,
78     0.840337F,
79     -0.450123F,
80     -0.156763F,
81 };
82 constexpr std::int32_t LCQI_GPT2_GENERATED_LENGTH = 5;
83 constexpr std::int32_t LCQI_GPT2_TOKENIZER_HELLO_ID = 7;
84 constexpr std::uint16_t LCQI_TEST_F16_ONE = 0x3C00U;
85 constexpr std::uint16_t LCQI_TEST_F16_HALF = 0x3800U;
86 constexpr std::uint16_t LCQI_TEST_F16_NEGATIVE_TWO = 0xC000U;
87 constexpr std::uint16_t LCQI_TEST_BF16_ONE_POINT_FIVE = 0x3FC0U;
88 constexpr std::uint16_t LCQI_TEST_BF16_NEGATIVE_HALF = 0xBF00U;
89 constexpr std::uint32_t LCQI_GGUF_TEST_VERSION = 3;
90 constexpr std::uint32_t LCQI_GGUF_TEST_ALIGNMENT = 32;
91 constexpr std::int64_t LCQI_GGUF_TEST_TENSOR_COUNT = 1;
92 constexpr std::int64_t LCQI_GGUF_TEST_METADATA_COUNT = 3;
93 constexpr std::int32_t LCQI_GGUF_TYPE_UINT32 = 4;

```

```

94 constexpr std::int32_t LCQI_GGUF_TYPE_FLOAT32 = 6;
95 constexpr std::int32_t LCQI_GGUF_TYPE_STRING = 8;
96 constexpr std::int32_t LCQI_GGUF_TYPE_ARRAY = 9;
97 constexpr std::int32_t LCQI_GGUF_TENSOR_TYPE_Q4_K = 12;
98 constexpr std::uint64_t LCQI_GGUF_TEST_TENSOR_OFFSET = 0;
99 constexpr float LCQI_Q4K_TEST_BLOCK_SCALE = 0.5F;
100 constexpr float LCQI_Q4K_TEST_BLOCK_MIN = 0.25F;
101 constexpr float LCQI_Q4K_Q8_DIFF_LIMIT = 0.08F;
102 constexpr float LCQI_GGML_MATVEC_MAX_DIFF = 1.0e-4F;
103 constexpr float LCQI_GGML_Q8_MATVEC_MAX_DIFF = 0.2F;
104 constexpr std::int32_t LCQI_GGML_TEST_Q5_HALF_VALUES = 16;
105 constexpr std::int32_t LCQI_GGML_TEST_Q5_ZERO_POINT = 16;
106 constexpr std::int32_t LCQI_GGML_TEST_Q6_LANE = 5;
107 constexpr std::int32_t LCQI_LLAMA_TEST_HIDDEN = 4;
108 constexpr std::int32_t LCQI_LLAMA_TEST_HEADS = 2;
109 constexpr std::int32_t LCQI_LLAMA_TEST_KV_HEADS = 1;
110 constexpr std::int32_t LCQI_LLAMA_TEST_HEAD_DIM = 2;
111 constexpr std::int32_t LCQI_LLAMA_TEST_INTERMEDIATE = 4;
112 constexpr std::int32_t LCQI_LLAMA_TEST_VOCAB = 4;
113 constexpr std::int32_t LCQI_LLAMA_TEST_CONTEXT = 8;
114 constexpr std::int32_t LCQI_LLAMA_TEST_LAYER_COUNT = 1;
115 constexpr std::int32_t LCQI_LLAMA_TEST_EXPECTED_TOKEN = 2;
116 constexpr float LCQI_LLAMA_TEST_SMALL_ATTENTION_VALUE = 0.1F;
117 constexpr float LCQI_LLAMA_TEST_SMALL_OUTPUT_SCALE = 0.1F;
118 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_HIDDEN = 256;
119 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_HEADS = 1;
120 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_KV_HEADS = 1;
121 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_HEAD_DIM = 256;
122 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_INTERMEDIATE = 256;
123 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_VOCAB = 2;
124 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_CONTEXT = 4;
125 constexpr std::int32_t LCQI_LLAMA_Q4_TEST_LAYER_COUNT = 1;
126 constexpr const char* LCQI_TEST_LLAMA_GGML_DIRECT_ENV = "LCQI_LLAMA_GGML_DIRECT↵
↵ ";
127 constexpr const char* LCQI_TEST_TRUE_ENV = "1";
128
129 struct KernelShapeCase {
130     std::int32_t input_size = 0;
131     std::int32_t output_size = 0;
132     std::int32_t output_block_size = 0;
133 };
134
135 struct RawTensorFixture {
136     std::string name;
137     std::string dtype;
138     std::vector<std::int64_t> shape;
139     std::vector<std::uint8_t> bytes;
140 };
141
142 struct FloatTensorFixture {
143     std::string name;
144     std::vector<std::int64_t> shape;

```

```

145     std::vector<float> values;
146 };
147
148 struct GgufTensorFixture {
149     std::string name;
150     std::vector<std::int64_t> shape;
151     lcqi::GgmlType type = lcqi::GgmlType::f32;
152     std::vector<std::uint8_t> bytes;
153     std::uint64_t relative_offset = 0;
154 };
155
156 struct ScopedEnvVar {
157     std::string name;
158     std::string old_value;
159     bool had_old_value = false;
160
161     ScopedEnvVar(const char* env_name, const char* value) : name(env_name) {
162         const char* old = std::getenv(env_name);
163         if (old != nullptr) {
164             this->old_value = old;
165             this->had_old_value = true;
166         }
167         if (setenv(env_name, value, 1) != 0) {
168             throw std::runtime_error("failed to set test environment variable");
169         }
170     }
171
172     ScopedEnvVar(const ScopedEnvVar&) = delete;
173     ScopedEnvVar& operator=(const ScopedEnvVar&) = delete;
174
175     ~ScopedEnvVar() {
176         if (this->had_old_value) {
177             static_cast<void>(setenv(this->name.c_str(), this->old_value.c_str(),
178             ↪ (), 1));
179         } else {
180             static_cast<void>(unsetenv(this->name.c_str()));
181         }
182     }
183 };
184
185 constexpr std::array<KernelShapeCase, 4> LCQI_KERNEL_SHAPE_CASES{{
186     {17, 19, 8},
187     {33, 7, 8},
188     {64, 32, 16},
189     {65, 31, 16},
190 }};
191
192 void require(bool condition, const char* message) {
193     if (!condition) {
194         throw std::runtime_error(message);
195     }
196 }

```

```

195 }
196
197 void require_close(float actual, float expected, const char* message) {
198     if (std::fabs(actual - expected) > LCQI_TEST_EPSILON) {
199         throw std::runtime_error(message);
200     }
201 }
202
203 std::filesystem::path test_model_path() {
204     return std::filesystem::path(__FILE__).parent_path().parent_path() /
205         "models" / "tiny_mlp_i8.txt";
206 }
207
208 std::filesystem::path test_tokenizer_path() {
209     return std::filesystem::path(__FILE__).parent_path().parent_path() /
210         "models" / "tiny_tokenizer.txt";
211 }
212
213 std::filesystem::path temp_file_path(const std::string& name) {
214     return std::filesystem::temp_directory_path() / name;
215 }
216
217 void write_le_u64(std::ofstream& output, std::uint64_t value) {
218     for (std::int32_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
219         const char byte = static_cast<char>(
220             (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK);
221         output.write(&byte, 1);
222     }
223 }
224
225 void append_le_u16(std::vector<std::uint8_t>& output, std::uint16_t value) {
226     output.push_back(static_cast<std::uint8_t>(value & LCQI_BYTE_MASK));
227     output.push_back(static_cast<std::uint8_t>((value >> LCQI_TEST_BYTE_BITS) &
↵ LCQI_BYTE_MASK));
228 }
229
230 void append_le_u32(std::vector<std::uint8_t>& output, std::uint32_t value) {
231     for (std::int32_t i = 0; i < 4; ++i) {
232         output.push_back(static_cast<std::uint8_t>(
233             (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
234     }
235 }
236
237 void append_le_i32(std::vector<std::uint8_t>& output, std::int32_t value) {
238     std::uint32_t bits = 0;
239     std::memcpy(&bits, &value, sizeof(bits));
240     append_le_u32(output, bits);
241 }
242
243 void append_le_u64(std::vector<std::uint8_t>& output, std::uint64_t value) {
244     for (std::int32_t i = 0; i < 8; ++i) {
245         output.push_back(static_cast<std::uint8_t>(

```



```

246         (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
247     }
248 }
249
250 void append_le_i64(std::vector<std::uint8_t>& output, std::int64_t value) {
251     std::uint64_t bits = 0;
252     std::memcpy(&bits, &value, sizeof(bits));
253     append_le_u64(output, bits);
254 }
255
256 void append_gguf_string(std::vector<std::uint8_t>& output, const std::string& ↵
↵ text) {
257     append_le_u64(output, static_cast<std::uint64_t>(text.size()));
258     output.insert(output.end(), text.begin(), text.end());
259 }
260
261 void append_le_f32(std::vector<std::uint8_t>& output, float value) {
262     std::uint32_t bits = 0;
263     std::memcpy(&bits, &value, sizeof(bits));
264     for (std::int32_t i = 0; i < 4; ++i) {
265         output.push_back(static_cast<std::uint8_t>(
266             (bits >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
267     }
268 }
269
270 void write_safetensors_fixture(const std::filesystem::path& path,
271                                const std::string& header,
272                                std::uint64_t payload_bytes) {
273     std::ofstream output(path, std::ios::binary);
274     if (!output) {
275         throw std::runtime_error("cannot create safetensors test fixture");
276     }
277     write_le_u64(output, static_cast<std::uint64_t>(header.size()));
278     output.write(header.data(), static_cast<std::streamsize>(header.size()));
279     for (std::uint64_t i = 0; i < payload_bytes; ++i) {
280         const char byte = static_cast<char>(i & 0xFFU);
281         output.write(&byte, 1);
282     }
283 }
284
285 std::string json_shape(std::span<const std::int64_t> shape) {
286     std::ostringstream stream;
287     stream << '[';
288     for (std::size_t index = 0; index < shape.size(); ++index) {
289         if (index != 0) {
290             stream << ',';
291         }
292         stream << shape[index];
293     }
294     stream << ']';
295     return stream.str();
296 }

```

```

297
298 void write_safetensors_raw_fixture(const std::filesystem::path& path,
299                                   std::span<const RawTensorFixture> tensors) {
300     std::ostringstream header;
301     header << "{\"__metadata__\":{\"format\":\"lcqi-test\"}}";
302     std::uint64_t offset = 0;
303     for (const RawTensorFixture& tensor : tensors) {
304         header << ",\"" << tensor.name << "\":{\"dtype\":\"" << tensor.dtype
305             << "\",\"shape\":\"" << json_shape(tensor.shape)
306             << "\",\"data_offsets\":[\"" << offset << ', '
307             << offset + tensor.bytes.size() << "]}\"";
308         offset += tensor.bytes.size();
309     }
310     header << '}'';
311
312     std::ofstream output(path, std::ios::binary);
313     if (!output) {
314         throw std::runtime_error("cannot create safetensors raw test fixture");
315     }
316     const std::string header_text = header.str();
317     write_le_u64(output, static_cast<std::uint64_t>(header_text.size()));
318     output.write(header_text.data(), static_cast<std::streamsize>(header_text.↵
↵ size()));
319     for (const RawTensorFixture& tensor : tensors) {
320         output.write(reinterpret_cast<const char*>(tensor.bytes.data()),
321                     static_cast<std::streamsize>(tensor.bytes.size()));
322     }
323 }
324
325 std::vector<std::uint8_t> f32_payload(std::span<const float> values) {
326     std::vector<std::uint8_t> bytes;
327     bytes.reserve(values.size() * 4);
328     for (const float value : values) {
329         append_le_f32(bytes, value);
330     }
331     return bytes;
332 }
333
334 std::vector<float> transpose_linear_to_hf_conv1d(const lcqi::Gpt2LinearF32& ↵
↵ linear) {
335     std::vector<float> transposed(
336         static_cast<std::size_t>(linear.input_size) *
337         static_cast<std::size_t>(linear.output_size),
338         0.0F);
339     for (std::int32_t out = 0; out < linear.output_size; ++out) {
340         for (std::int32_t in = 0; in < linear.input_size; ++in) {
341             transposed[static_cast<std::size_t>(in) *
342                 static_cast<std::size_t>(linear.output_size) +
343                 static_cast<std::size_t>(out)] =
344                 linear.weights[static_cast<std::size_t>(out) *
345                     static_cast<std::size_t>(linear.input_size) ↵

```

↵ +

```

346         static_cast<std::size_t>(in)];
347     }
348 }
349 return transposed;
350 }
351
352 void add_f32_tensor(std::vector<RawTensorFixture>& tensors,
353                    std::string name,
354                    std::vector<std::int64_t> shape,
355                    std::span<const float> values) {
356     tensors.push_back(RawTensorFixture{
357         std::move(name),
358         "F32",
359         std::move(shape),
360         f32_payload(values),
361     });
362 }
363
364 void write_text_file(const std::filesystem::path& path, const std::string& text←
    ↪ ) {
365     std::ofstream output(path);
366     if (!output) {
367         throw std::runtime_error("cannot create text fixture");
368     }
369     output << text;
370 }
371
372 std::vector<std::uint8_t> make_q4_k_test_block() {
373     std::vector<std::uint8_t> block(
374         static_cast<std::size_t>(lcqi::LCQI_Q4_K_BLOCK_BYTES),
375         0);
376     block[0] = 0x00U;
377     block[1] = 0x38U;
378     block[2] = 0x00U;
379     block[3] = 0x34U;
380
381     std::array<std::uint8_t, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS←
    ↪ )> scales{
382         1, 2, 3, 4, 5, 6, 7, 8,
383     };
384     std::array<std::uint8_t, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS←
    ↪ )> mins{
385         1, 1, 2, 2, 3, 3, 4, 4,
386     };
387     for (std::size_t index = 0; index < 4; ++index) {
388         block[4 + index] = static_cast<std::uint8_t>(
389             (scales[index] & 0x3FU) | ((scales[index + 4] >> 4U) << 6U));
390         block[8 + index] = static_cast<std::uint8_t>(
391             (mins[index] & 0x3FU) | ((mins[index + 4] >> 4U) << 6U));
392         block[12 + index] = static_cast<std::uint8_t>(
393             (scales[index + 4] & 0x0FU) | ((mins[index + 4] & 0x0FU) << 4U));
394     }

```

```

395
396     for (std::int32_t pair = 0; pair < 4; ++pair) {
397         const std::int32_t low_subblock = pair * 2;
398         const std::int32_t high_subblock = low_subblock + 1;
399         for (std::int32_t index = 0; index < lcqi::LCQI_Q4_K_SUBBLOCK_VALUES; ←
↪ ++index) {
400             const std::uint8_t low = static_cast<std::uint8_t>(
401                 (low_subblock + index) & 0x0F);
402             const std::uint8_t high = static_cast<std::uint8_t>(
403                 (high_subblock + index + 1) & 0x0F);
404             block[static_cast<std::size_t>(16 + pair * 32 + index)] =
405                 static_cast<std::uint8_t>(low | (high << 4U));
406         }
407     }
408     return block;
409 }
410
411 std::vector<float> expected_q4_k_test_values() {
412     const std::array<float, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS)↪
↪ > scales{
413         1, 2, 3, 4, 5, 6, 7, 8,
414     };
415     const std::array<float, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS)↪
↪ > mins{
416         1, 1, 2, 2, 3, 3, 4, 4,
417     };
418     std::vector<float> values(
419         static_cast<std::size_t>(lcqi::LCQI_QK_K_BLOCK_VALUES),
420         0.0F);
421     for (std::int32_t subblock = 0; subblock < lcqi::LCQI_Q4_K_SUBBLOCKS; ++↪
↪ subblock) {
422         for (std::int32_t index = 0; index < lcqi::LCQI_Q4_K_SUBBLOCK_VALUES; ←
↪ ++index) {
423             const std::int32_t quantized =
424                 (subblock % 2 == 0)
425                 ? ((subblock + index) & 0x0F)
426                 : ((subblock + index + 1) & 0x0F);
427             values[static_cast<std::size_t>(
428                 subblock * lcqi::LCQI_Q4_K_SUBBLOCK_VALUES + index)] =
429                 LCQI_Q4K_TEST_BLOCK_SCALE * scales[static_cast<std::size_t>(↪
↪ subblock)] *
430                 static_cast<float>(quantized) -
431                 LCQI_Q4K_TEST_BLOCK_MIN * mins[static_cast<std::size_t>(↪
↪ subblock)];
432         }
433     }
434     return values;
435 }
436
437 std::vector<std::uint8_t> make_q8_0_test_block() {
438     std::vector<std::uint8_t> block(
439         static_cast<std::size_t>(lcqi::LCQI_Q8_0_BLOCK_BYTES),

```

```

440     0);
441     block[0] = static_cast<std::uint8_t>(LCQI_TEST_F16_HALF & LCQI_BYTE_MASK);
442     block[1] = static_cast<std::uint8_t>((LCQI_TEST_F16_HALF >> ←
    ↪ LCQI_TEST_BYTE_BITS) &
    ↪
    ↪ LCQI_BYTE_MASK);
443
444     for (std::int32_t index = 0; index < lcqi::LCQI_Q8_0_BLOCK_VALUES; ++index)↪
    ↪ {
445         block[static_cast<std::size_t>(2 + index)] =
446             static_cast<std::uint8_t>(static_cast<std::int8_t>(index - 16));
447     }
448     return block;
449 }
450
451 std::vector<std::uint8_t> make_q5_0_test_block() {
452     std::vector<std::uint8_t> block(
453         static_cast<std::size_t>(lcqi::LCQI_Q5_0_BLOCK_BYTES),
454         0);
455     block[0] = static_cast<std::uint8_t>(LCQI_TEST_F16_HALF & LCQI_BYTE_MASK);
456     block[1] = static_cast<std::uint8_t>((LCQI_TEST_F16_HALF >> ←
    ↪ LCQI_TEST_BYTE_BITS) &
    ↪
    ↪ LCQI_BYTE_MASK);
457
458
459     std::uint32_t qh = 0;
460     for (std::int32_t index = 0; index < lcqi::LCQI_Q5_0_BLOCK_VALUES / 2; ++↪
    ↪ index) {
461         const std::int32_t first_quantized = index - ←
    ↪ LCQI_GGML_TEST_Q5_ZERO_POINT;
462         const std::int32_t second_quantized = index;
463         const std::uint8_t first_encoded =
464             static_cast<std::uint8_t>(first_quantized + ←
    ↪ LCQI_GGML_TEST_Q5_ZERO_POINT);
465         const std::uint8_t second_encoded =
466             static_cast<std::uint8_t>(second_quantized + ←
    ↪ LCQI_GGML_TEST_Q5_ZERO_POINT);
467         if ((first_encoded & 0x10U) != 0U) {
468             qh |= (1U << index);
469         }
470         if ((second_encoded & 0x10U) != 0U) {
471             qh |= (1U << (index + LCQI_GGML_TEST_Q5_HALF_VALUES));
472         }
473         const std::uint8_t low = static_cast<std::uint8_t>(first_encoded & 0↪
    ↪ x0FU);
474         const std::uint8_t high = static_cast<std::uint8_t>(second_encoded & 0↪
    ↪ x0FU);
475         block[static_cast<std::size_t>(6 + index)] =
476             static_cast<std::uint8_t>(low | (high << 4U));
477     }
478     block[2] = static_cast<std::uint8_t>(qh & LCQI_BYTE_MASK);
479     block[3] = static_cast<std::uint8_t>((qh >> LCQI_TEST_BYTE_BITS) & ←
    ↪ LCQI_BYTE_MASK);
480     block[4] = static_cast<std::uint8_t>((qh >> (LCQI_TEST_BYTE_BITS * 2)) & ←
    ↪ LCQI_BYTE_MASK);

```

```

481     block[5] = static_cast<std::uint8_t>((qh >> (LCQI_TEST_BYTE_BITS * 3)) &
    ↪ LCQI_BYTE_MASK);
482     return block;
483 }
484
485 std::vector<std::uint8_t> make_q6_k_test_block() {
486     std::vector<std::uint8_t> block(
487         static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES),
488         0);
489     const std::size_t ql_offset = 0;
490     const std::size_t qh_offset = static_cast<std::size_t>(lcqi::
    ↪ LCQI_Q6_K_BLOCK_VALUES / 2);
491     const std::size_t scales_offset =
492         qh_offset + static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_VALUES / 4);
493     for (std::int32_t index = 0; index < 16; ++index) {
494         block[scales_offset + static_cast<std::size_t>(index)] = 1;
495     }
496     block[static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES - 2)] =
497         static_cast<std::uint8_t>(LCQI_TEST_F16_ONE & LCQI_BYTE_MASK);
498     block[static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES - 1)] =
499         static_cast<std::uint8_t>((LCQI_TEST_F16_ONE >> LCQI_TEST_BYTE_BITS) &
500             LCQI_BYTE_MASK);
501
502     const auto store_q6 = [&block, ql_offset, qh_offset](std::int32_t position,
503         std::uint8_t encoded) {
504         const std::int32_t half = position / 128;
505         const std::int32_t local = position % 128;
506         const std::size_t ql_base = ql_offset + static_cast<std::size_t>(half *
    ↪ 64);
507         const std::size_t qh_base = qh_offset + static_cast<std::size_t>(half *
    ↪ 32);
508         const std::uint8_t low = encoded & 0x0FU;
509         const std::uint8_t high = static_cast<std::uint8_t>((encoded >> 4U) & 0
    ↪ x03U);
510         if (local < 32) {
511             block[ql_base + static_cast<std::size_t>(local)] =
512                 static_cast<std::uint8_t>(
513                     (block[ql_base + static_cast<std::size_t>(local)] & 0xF0U)
    ↪ | low);
514             block[qh_base + static_cast<std::size_t>(local)] =
515                 static_cast<std::uint8_t>(
516                     (block[qh_base + static_cast<std::size_t>(local)] & 0xFCU)
    ↪ | high);
517         } else if (local < 64) {
518             const std::size_t lane = static_cast<std::size_t>(local - 32);
519             block[ql_base + static_cast<std::size_t>(32) + lane] =
520                 static_cast<std::uint8_t>(
521                     (block[ql_base + static_cast<std::size_t>(32) + lane] & 0
    ↪ xF0U) | low);
522             block[qh_base + lane] =
523                 static_cast<std::uint8_t>((block[qh_base + lane] & 0xF3U) | (
    ↪ high << 2U));

```

```

524     } else if (local < 96) {
525         const std::size_t lane = static_cast<std::size_t>(local - 64);
526         block[ql_base + lane] =
527             static_cast<std::uint8_t>(
528                 (block[ql_base + lane] & 0x0FU) | (low << 4U));
529         block[qh_base + lane] =
530             static_cast<std::uint8_t>((block[qh_base + lane] & 0xCFU) | (↵
↵ high << 4U));
531     } else {
532         const std::size_t lane = static_cast<std::size_t>(local - 96);
533         block[ql_base + static_cast<std::size_t>(32) + lane] =
534             static_cast<std::uint8_t>(
535                 (block[ql_base + static_cast<std::size_t>(32) + lane] & 0↵
↵ x0FU) |
536                 (low << 4U));
537         block[qh_base + lane] =
538             static_cast<std::uint8_t>((block[qh_base + lane] & 0x3FU) | (↵
↵ high << 6U));
539     }
540 };
541
542 store_q6(LCQI_GGML_TEST_Q6_LANE, 35);
543 store_q6(32 + LCQI_GGML_TEST_Q6_LANE, 31);
544 store_q6(64 + LCQI_GGML_TEST_Q6_LANE, 63);
545 store_q6(96 + LCQI_GGML_TEST_Q6_LANE, 0);
546 store_q6(128 + LCQI_GGML_TEST_Q6_LANE, 36);
547 return block;
548 }
549
550 void write_gguf_q4_k_fixture(const std::filesystem::path& path) {
551     std::vector<std::uint8_t> bytes;
552     bytes.push_back('G');
553     bytes.push_back('G');
554     bytes.push_back('U');
555     bytes.push_back('F');
556     append_le_u32(bytes, LCQI_GGUF_TEST_VERSION);
557     append_le_i64(bytes, LCQI_GGUF_TEST_TENSOR_COUNT);
558     append_le_i64(bytes, LCQI_GGUF_TEST_METADATA_COUNT);
559
560     append_gguf_string(bytes, "general.alignment");
561     append_le_i32(bytes, LCQI_GGUF_TYPE_UINT32);
562     append_le_u32(bytes, LCQI_GGUF_TEST_ALIGNMENT);
563     append_gguf_string(bytes, "general.architecture");
564     append_le_i32(bytes, LCQI_GGUF_TYPE_STRING);
565     append_gguf_string(bytes, "lcqi-test");
566     append_gguf_string(bytes, "tokenizer.ggml.tokens");
567     append_le_i32(bytes, LCQI_GGUF_TYPE_ARRAY);
568     append_le_i32(bytes, LCQI_GGUF_TYPE_STRING);
569     append_le_u64(bytes, 2);
570     append_gguf_string(bytes, "<bos>");
571     append_gguf_string(bytes, "hello");
572

```



```

573     append_gguf_string(bytes, "blk.0.ffn_up.weight");
574     append_le_u32(bytes, 2);
575     append_le_i64(bytes, lcqi::LCQI_QK_K_BLOCK_VALUES);
576     append_le_i64(bytes, 1);
577     append_le_i32(bytes, LCQI_GGUF_TENSOR_TYPE_Q4_K);
578     append_le_u64(bytes, LCQI_GGUF_TEST_TENSOR_OFFSET);
579     while (bytes.size() % LCQI_GGUF_TEST_ALIGNMENT != 0) {
580         bytes.push_back(0);
581     }
582     const std::vector<std::uint8_t> block = make_q4_k_test_block();
583     bytes.insert(bytes.end(), block.begin(), block.end());
584
585     std::ofstream output(path, std::ios::binary);
586     if (!output) {
587         throw std::runtime_error("cannot create GGUF fixture");
588     }
589     output.write(reinterpret_cast<const char*>(bytes.data()),
590                 static_cast<std::streamsize>(bytes.size()));
591 }
592
593 void append_gguf_uint32_metadata(std::vector<std::uint8_t>& bytes,
594                                 const std::string& key,
595                                 std::uint32_t value) {
596     append_gguf_string(bytes, key);
597     append_le_i32(bytes, LCQI_GGUF_TYPE_UINT32);
598     append_le_u32(bytes, value);
599 }
600
601 void append_gguf_float32_metadata(std::vector<std::uint8_t>& bytes,
602                                   const std::string& key,
603                                   float value) {
604     append_gguf_string(bytes, key);
605     append_le_i32(bytes, LCQI_GGUF_TYPE_FLOAT32);
606     append_le_f32(bytes, value);
607 }
608
609 void append_gguf_string_metadata(std::vector<std::uint8_t>& bytes,
610                                  const std::string& key,
611                                  const std::string& value) {
612     append_gguf_string(bytes, key);
613     append_le_i32(bytes, LCQI_GGUF_TYPE_STRING);
614     append_gguf_string(bytes, value);
615 }
616
617 void write_gguf_fixture(const std::filesystem::path& path,
618                        std::span<GgufTensorFixture> tensors,
619                        const std::vector<std::uint8_t>& metadata_bytes) {
620     std::uint64_t relative_offset = 0;
621     for (GgufTensorFixture& tensor : tensors) {
622         tensor.relative_offset = relative_offset;
623         relative_offset += tensor.bytes.size();
624         while (relative_offset % LCQI_GGUF_TEST_ALIGNMENT != 0) {

```



```

625         ++relative_offset;
626     }
627 }
628
629 std::vector<std::uint8_t> bytes;
630 bytes.push_back('G');
631 bytes.push_back('G');
632 bytes.push_back('U');
633 bytes.push_back('F');
634 append_le_u32(bytes, LCQI_GGUF_TEST_VERSION);
635 append_le_u64(bytes, static_cast<std::uint64_t>(tensors.size()));
636 append_le_u64(bytes, 14);
637 bytes.insert(bytes.end(), metadata_bytes.begin(), metadata_bytes.end());
638
639 for (const GgufTensorFixture& tensor : tensors) {
640     append_gguf_string(bytes, tensor.name);
641     append_le_u32(bytes, static_cast<std::uint32_t>(tensor.shape.size()));
642     for (const std::int64_t extent : tensor.shape) {
643         append_le_i64(bytes, extent);
644     }
645     append_le_i32(bytes, static_cast<std::int32_t>(tensor.type));
646     append_le_u64(bytes, tensor.relative_offset);
647 }
648 while (bytes.size() % LCQI_GGUF_TEST_ALIGNMENT != 0) {
649     bytes.push_back(0);
650 }
651 std::uint64_t written_relative = 0;
652 for (const GgufTensorFixture& tensor : tensors) {
653     while (written_relative < tensor.relative_offset) {
654         bytes.push_back(0);
655         ++written_relative;
656     }
657     bytes.insert(bytes.end(), tensor.bytes.begin(), tensor.bytes.end());
658     written_relative += tensor.bytes.size();
659 }
660
661 std::ofstream output(path, std::ios::binary);
662 if (!output) {
663     throw std::runtime_error("cannot create GGUF fixture");
664 }
665 output.write(reinterpret_cast<const char*>(bytes.data()),
666             static_cast<std::streamsize>(bytes.size()));
667 }
668
669 void add_gguf_f32_tensor(std::vector<GgufTensorFixture>& tensors,
670                         std::string name,
671                         std::vector<std::int64_t> shape,
672                         std::span<const float> values) {
673     tensors.push_back(GgufTensorFixture{
674         std::move(name),
675         std::move(shape),
676         lcqi::GgmlType::f32,

```

```

677         f32_payload(values),
678         0,
679     });
680 }
681
682 void add_gguf_q4_k_tensor(std::vector<GgufTensorFixture>& tensors,
683                          std::string name,
684                          std::vector<std::int64_t> shape,
685                          const std::vector<std::uint8_t>& bytes) {
686     tensors.push_back(GgufTensorFixture{
687         std::move(name),
688         std::move(shape),
689         lcqi::GgmlType::q4_k,
690         bytes,
691         0,
692     });
693 }
694
695 void add_gguf_quantized_tensor(std::vector<GgufTensorFixture>& tensors,
696                               std::string name,
697                               std::vector<std::int64_t> shape,
698                               lcqi::GgmlType type,
699                               const std::vector<std::uint8_t>& bytes) {
700     tensors.push_back(GgufTensorFixture{
701         std::move(name),
702         std::move(shape),
703         type,
704         bytes,
705         0,
706     });
707 }
708
709 std::vector<std::uint8_t> repeated_ggml_block(const std::vector<std::uint8_t>& ←
    ↪ block,
710                                              std::int32_t input_size,
711                                              std::int32_t output_size,
712                                              std::int64_t block_values) {
713     const std::size_t block_count =
714         static_cast<std::size_t>((input_size / block_values) * output_size);
715     std::vector<std::uint8_t> bytes;
716     bytes.reserve(block.size() * block_count);
717     for (std::size_t index = 0; index < block_count; ++index) {
718         bytes.insert(bytes.end(), block.begin(), block.end());
719     }
720     return bytes;
721 }
722
723 void write_tiny_llama_gguf_fixture(const std::filesystem::path& path) {
724     std::vector<std::uint8_t> metadata;
725     append_gguf_uint32_metadata(metadata, "general.alignment", ←
    ↪ LCQI_GGUF_TEST_ALIGNMENT);
726     append_gguf_string_metadata(metadata, "general.architecture", "llama");

```

```

727     append_gguf_string_metadata(metadata, "general.name", "lcqi tiny llama");
728     append_gguf_uint32_metadata(metadata, "llama.embedding_length", ←
    ↪ LCQI_LLAMA_TEST_HIDDEN);
729     append_gguf_uint32_metadata(metadata, "llama.block_count", ←
    ↪ LCQI_LLAMA_TEST_LAYER_COUNT);
730     append_gguf_uint32_metadata(metadata, "llama.feed_forward_length", ←
    ↪ LCQI_LLAMA_TEST_INTERMEDIATE);
731     append_gguf_uint32_metadata(metadata, "llama.attention.head_count", ←
    ↪ LCQI_LLAMA_TEST_HEADS);
732     append_gguf_uint32_metadata(metadata, "llama.attention.head_count_kv", ←
    ↪ LCQI_LLAMA_TEST_KV_HEADS);
733     append_gguf_uint32_metadata(metadata, "llama.rope.dimension_count", ←
    ↪ LCQI_LLAMA_TEST_HEAD_DIM);
734     append_gguf_float32_metadata(metadata, "llama.rope.freq_base", 10000.0F);
735     append_gguf_uint32_metadata(metadata, "llama.context_length", ←
    ↪ LCQI_LLAMA_TEST_CONTEXT);
736     append_gguf_uint32_metadata(metadata, "llama.vocab_size", ←
    ↪ LCQI_LLAMA_TEST_VOCAB);
737     append_gguf_float32_metadata(metadata, "llama.attention.←
    ↪ layer_norm_rms_epsilon", 1.0e-5F);
738     append_gguf_uint32_metadata(metadata, "tokenizer.ggml.eos_token_id", 3);
739
740     std::vector<GgufTensorFixture> tensors;
741     const std::vector<float> embedding{
742         1.0F, 0.0F, 0.0F, 0.0F,
743         0.0F, 1.0F, 0.0F, 0.0F,
744         0.0F, 2.0F, 0.0F, 0.0F,
745         0.0F, 0.0F, 1.0F, 0.0F,
746     };
747     add_gguf_f32_tensor(tensors,
748         "token_embd.weight",
749         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_VOCAB},
750         embedding);
751     add_gguf_f32_tensor(tensors,
752         "output_norm.weight",
753         {LCQI_LLAMA_TEST_HIDDEN},
754         std::vector<float>(LCQI_LLAMA_TEST_HIDDEN, 1.0F));
755     add_gguf_f32_tensor(tensors,
756         "blk.0.attn_norm.weight",
757         {LCQI_LLAMA_TEST_HIDDEN},
758         std::vector<float>(LCQI_LLAMA_TEST_HIDDEN, 1.0F));
759     add_gguf_f32_tensor(tensors,
760         "blk.0.ffn_norm.weight",
761         {LCQI_LLAMA_TEST_HIDDEN},
762         std::vector<float>(LCQI_LLAMA_TEST_HIDDEN, 1.0F));
763     const std::vector<float> hidden_to_hidden(
764         static_cast<std::size_t>(LCQI_LLAMA_TEST_HIDDEN * ←
    ↪ LCQI_LLAMA_TEST_HIDDEN),
765         0.0F);
766     std::vector<float> hidden_to_kv(
767         static_cast<std::size_t>(LCQI_LLAMA_TEST_HIDDEN * ←
    ↪ LCQI_LLAMA_TEST_HEAD_DIM),

```

```

768     0.0F);
769     hidden_to_kv[1] = LCQI_LLAMA_TEST_SMALL_ATTENTION_VALUE;
770     std::vector<float> attention_output = hidden_to_hidden;
771     attention_output[0] = LCQI_LLAMA_TEST_SMALL_OUTPUT_SCALE;
772     attention_output[10] = LCQI_LLAMA_TEST_SMALL_OUTPUT_SCALE;
773     const std::vector<float> hidden_to_intermediate(
774         static_cast<std::size_t>(LCQI_LLAMA_TEST_HIDDEN * ←
↪ LCQI_LLAMA_TEST_INTERMEDIATE),
775         0.0F);
776     const std::vector<float> intermediate_to_hidden(
777         static_cast<std::size_t>(LCQI_LLAMA_TEST_INTERMEDIATE * ←
↪ LCQI_LLAMA_TEST_HIDDEN),
778         0.0F);
779     add_gguf_f32_tensor(tensors,
780         "blk.0.attn.q.weight",
781         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HIDDEN},
782         hidden_to_hidden);
783     add_gguf_f32_tensor(tensors,
784         "blk.0.attn.k.weight",
785         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HEAD_DIM},
786         hidden_to_kv);
787     add_gguf_f32_tensor(tensors,
788         "blk.0.attn.v.weight",
789         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HEAD_DIM},
790         hidden_to_kv);
791     add_gguf_f32_tensor(tensors,
792         "blk.0.attn_output.weight",
793         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HIDDEN},
794         attention_output);
795     add_gguf_f32_tensor(tensors,
796         "blk.0.ffn_gate.weight",
797         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_INTERMEDIATE},
798         hidden_to_intermediate);
799     add_gguf_f32_tensor(tensors,
800         "blk.0.ffn_up.weight",
801         {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_INTERMEDIATE},
802         hidden_to_intermediate);
803     add_gguf_f32_tensor(tensors,
804         "blk.0.ffn_down.weight",
805         {LCQI_LLAMA_TEST_INTERMEDIATE, LCQI_LLAMA_TEST_HIDDEN},
806         intermediate_to_hidden);
807     write_gguf_fixture(path, tensors, metadata);
808 }
809
810 void write_tiny_llama_ggml_direct_gguf_fixture(const std::filesystem::path& ←
↪ path) {
811     std::vector<std::uint8_t> metadata;
812     append_gguf_uint32_metadata(metadata, "general.alignment", ←
↪ LCQI_GGUF_TEST_ALIGNMENT);
813     append_gguf_string_metadata(metadata, "general.architecture", "llama");
814     append_gguf_string_metadata(metadata, "general.name", "lcqi ggml direct ←
↪ tiny llama");

```

```

815     append_gguf_uint32_metadata(metadata, "llama.embedding_length", ←
    ↪ LCQI_LLAMA_Q4_TEST_HIDDEN);
816     append_gguf_uint32_metadata(metadata, "llama.block_count", ←
    ↪ LCQI_LLAMA_Q4_TEST_LAYER_COUNT);
817     append_gguf_uint32_metadata(metadata,
818                                "llama.feed_forward_length",
819                                LCQI_LLAMA_Q4_TEST_INTERMEDIATE);
820     append_gguf_uint32_metadata(metadata, "llama.attention.head_count", ←
    ↪ LCQI_LLAMA_Q4_TEST_HEADS);
821     append_gguf_uint32_metadata(metadata,
822                                "llama.attention.head_count_kv",
823                                LCQI_LLAMA_Q4_TEST_KV_HEADS);
824     append_gguf_uint32_metadata(metadata,
825                                "llama.rope.dimension_count",
826                                LCQI_LLAMA_Q4_TEST_HEAD_DIM);
827     append_gguf_float32_metadata(metadata, "llama.rope.freq_base", 10000.0F);
828     append_gguf_uint32_metadata(metadata, "llama.context_length", ←
    ↪ LCQI_LLAMA_Q4_TEST_CONTEXT);
829     append_gguf_uint32_metadata(metadata, "llama.vocab_size", ←
    ↪ LCQI_LLAMA_Q4_TEST_VOCAB);
830     append_gguf_float32_metadata(metadata, "llama.attention.←
    ↪ layer_norm_rms_epsilon", 1.0e-5F);
831     append_gguf_uint32_metadata(metadata, "tokenizer.ggml.eos_token_id", 1);
832
833     std::vector<GgufTensorFixture> tensors;
834     std::vector<float> embedding(
835         static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN * ←
    ↪ LCQI_LLAMA_Q4_TEST_VOCAB),
836         0.0F);
837     embedding[0] = 1.0F;
838     embedding[static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN)] = 0.5F;
839     add_gguf_f32_tensor(tensors,
840                        "token_embd.weight",
841                        {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_VOCAB},
842                        embedding);
843     add_gguf_f32_tensor(tensors,
844                        "output_norm.weight",
845                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
846                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
847     add_gguf_f32_tensor(tensors,
848                        "blk.0.attn_norm.weight",
849                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
850                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
851     add_gguf_f32_tensor(tensors,
852                        "blk.0.ffn_norm.weight",
853                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
854                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
855
856     const std::vector<std::uint8_t> q5_matrix = repeated_ggml_block(
857         std::vector<std::uint8_t>(static_cast<std::size_t>(lcqi::←
    ↪ LCQI_Q5_0_BLOCK_BYTES), 0),
858         LCQI_LLAMA_Q4_TEST_HIDDEN,

```

```

859     LCQI_LLAMA_Q4_TEST_HIDDEN,
860     lcqi::LCQI_Q5_0_BLOCK_VALUES);
861     const std::vector<std::uint8_t> q8_matrix = repeated_ggml_block(
862         std::vector<std::uint8_t>(static_cast<std::size_t>(lcqi::↵
↵ LCQI_Q8_0_BLOCK_BYTES), 0),
863         LCQI_LLAMA_Q4_TEST_HIDDEN,
864         LCQI_LLAMA_Q4_TEST_HIDDEN,
865         lcqi::LCQI_Q8_0_BLOCK_VALUES);
866     const std::vector<std::uint8_t> q6_matrix = repeated_ggml_block(
867         std::vector<std::uint8_t>(static_cast<std::size_t>(lcqi::↵
↵ LCQI_Q6_K_BLOCK_BYTES), 0),
868         LCQI_LLAMA_Q4_TEST_INTERMEDIATE,
869         LCQI_LLAMA_Q4_TEST_HIDDEN,
870         lcqi::LCQI_Q6_K_BLOCK_VALUES);
871
872     add_gguf_quantized_tensor(tensors,
873                             "blk.0.attn_q.weight",
874                             {LCQI_LLAMA_Q4_TEST_HIDDEN, ↵
↵ LCQI_LLAMA_Q4_TEST_HIDDEN},
875                             lcqi::GgmlType::q5_0,
876                             q5_matrix);
877     add_gguf_quantized_tensor(tensors,
878                             "blk.0.attn_k.weight",
879                             {LCQI_LLAMA_Q4_TEST_HIDDEN, ↵
↵ LCQI_LLAMA_Q4_TEST_HEAD_DIM},
880                             lcqi::GgmlType::q5_0,
881                             q5_matrix);
882     add_gguf_quantized_tensor(tensors,
883                             "blk.0.attn_v.weight",
884                             {LCQI_LLAMA_Q4_TEST_HIDDEN, ↵
↵ LCQI_LLAMA_Q4_TEST_HEAD_DIM},
885                             lcqi::GgmlType::q8_0,
886                             q8_matrix);
887     add_gguf_quantized_tensor(tensors,
888                             "blk.0.attn_output.weight",
889                             {LCQI_LLAMA_Q4_TEST_HIDDEN, ↵
↵ LCQI_LLAMA_Q4_TEST_HIDDEN},
890                             lcqi::GgmlType::q5_0,
891                             q5_matrix);
892     add_gguf_quantized_tensor(tensors,
893                             "blk.0.ffn_gate.weight",
894                             {LCQI_LLAMA_Q4_TEST_HIDDEN, ↵
↵ LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
895                             lcqi::GgmlType::q5_0,
896                             q5_matrix);
897     add_gguf_quantized_tensor(tensors,
898                             "blk.0.ffn_up.weight",
899                             {LCQI_LLAMA_Q4_TEST_HIDDEN, ↵
↵ LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
900                             lcqi::GgmlType::q8_0,
901                             q8_matrix);
902     add_gguf_quantized_tensor(tensors,

```

```

903         "blk.0.ffn_down.weight",
904         {LCQI_LLAMA_Q4_TEST_INTERMEDIATE, ←
    ↪ LCQI_LLAMA_Q4_TEST_HIDDEN},
905         lcqi::GgmlType::q6_k,
906         q6_matrix);
907     write_gguf_fixture(path, tensors, metadata);
908 }
909
910 void write_tiny_llama_q4_direct_gguf_fixture(const std::filesystem::path& path)←
    ↪ {
911     std::vector<std::uint8_t> metadata;
912     append_gguf_uint32_metadata(metadata, "general.alignment", ←
    ↪ LCQI_GGUF_TEST_ALIGNMENT);
913     append_gguf_string_metadata(metadata, "general.architecture", "llama");
914     append_gguf_string_metadata(metadata, "general.name", "lcqi q4 direct tiny ←
    ↪ llama");
915     append_gguf_uint32_metadata(metadata, "llama.embedding_length", ←
    ↪ LCQI_LLAMA_Q4_TEST_HIDDEN);
916     append_gguf_uint32_metadata(metadata, "llama.block_count", ←
    ↪ LCQI_LLAMA_Q4_TEST_LAYER_COUNT);
917     append_gguf_uint32_metadata(metadata, "llama.feed_forward_length", ←
    ↪ LCQI_LLAMA_Q4_TEST_INTERMEDIATE);
918     append_gguf_uint32_metadata(metadata, "llama.attention.head_count", ←
    ↪ LCQI_LLAMA_Q4_TEST_HEADS);
919     append_gguf_uint32_metadata(metadata, "llama.attention.head_count_kv", ←
    ↪ LCQI_LLAMA_Q4_TEST_KV_HEADS);
920     append_gguf_uint32_metadata(metadata, "llama.rope.dimension_count", ←
    ↪ LCQI_LLAMA_Q4_TEST_HEAD_DIM);
921     append_gguf_float32_metadata(metadata, "llama.rope.freq_base", 10000.0F);
922     append_gguf_uint32_metadata(metadata, "llama.context_length", ←
    ↪ LCQI_LLAMA_Q4_TEST_CONTEXT);
923     append_gguf_uint32_metadata(metadata, "llama.vocab_size", ←
    ↪ LCQI_LLAMA_Q4_TEST_VOCAB);
924     append_gguf_float32_metadata(metadata, "llama.attention.←
    ↪ layer_norm_rms_epsilon", 1.0e-5F);
925     append_gguf_uint32_metadata(metadata, "tokenizer.ggml.eos_token_id", 1);
926
927     std::vector<GgufTensorFixture> tensors;
928     std::vector<float> embedding(
929         static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN * ←
    ↪ LCQI_LLAMA_Q4_TEST_VOCAB),
930         0.0F);
931     embedding[0] = 1.0F;
932     embedding[static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN)] = 0.5F;
933     add_gguf_f32_tensor(tensors,
934         "token_embd.weight",
935         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_VOCAB},
936         embedding);
937     add_gguf_f32_tensor(tensors,
938         "output_norm.weight",
939         {LCQI_LLAMA_Q4_TEST_HIDDEN},
940         std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));

```



```

941 add_gguf_f32_tensor(tensors,
942                     "blk.0.attn_norm.weight",
943                     {LCQI_LLAMA_Q4_TEST_HIDDEN},
944                     std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
945 add_gguf_f32_tensor(tensors,
946                     "blk.0.ffn_norm.weight",
947                     {LCQI_LLAMA_Q4_TEST_HIDDEN},
948                     std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
949
950 const std::vector<std::uint8_t> zero_q4_matrix(
951     static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN) *
952     static_cast<std::size_t>(lcqi::LCQI_Q4_K_BLOCK_BYTES),
953     0);
954 add_gguf_q4_k_tensor(tensors,
955                     "blk.0.attn_q.weight",
956                     {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HIDDEN},
957                     zero_q4_matrix);
958 add_gguf_q4_k_tensor(tensors,
959                     "blk.0.attn_k.weight",
960                     {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HEAD_DIM},
961                     zero_q4_matrix);
962 add_gguf_q4_k_tensor(tensors,
963                     "blk.0.attn_v.weight",
964                     {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HEAD_DIM},
965                     zero_q4_matrix);
966 add_gguf_q4_k_tensor(tensors,
967                     "blk.0.attn_output.weight",
968                     {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HIDDEN},
969                     zero_q4_matrix);
970 add_gguf_q4_k_tensor(tensors,
971                     "blk.0.ffn_gate.weight",
972                     {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
973                     zero_q4_matrix);
974 add_gguf_q4_k_tensor(tensors,
975                     "blk.0.ffn_up.weight",
976                     {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
977                     zero_q4_matrix);
978 add_gguf_q4_k_tensor(tensors,
979                     "blk.0.ffn_down.weight",
980                     {LCQI_LLAMA_Q4_TEST_INTERMEDIATE, LCQI_LLAMA_Q4_TEST_HIDDEN},
981                     zero_q4_matrix);
982 write_gguf_fixture(path, tensors, metadata);
983 }
984
985 void write_tiny_gpt2_safetensors(const std::filesystem::path& path,

```



```

986         const lcqi::Gpt2ReferenceModel& model) {
987     std::vector<RawTensorFixture> tensors;
988     add_f32_tensor(tensors,
989         "transformer.wte.weight",
990         {model.config.vocab_size, model.config.hidden_size},
991         model.token_embedding);
992     add_f32_tensor(tensors,
993         "transformer.wpe.weight",
994         {model.config.max_positions, model.config.hidden_size},
995         model.position_embedding);
996     for (std::int32_t layer_id = 0; layer_id < model.config.layer_count; ++
↵ layer_id) {
997         const lcqi::Gpt2LayerWeightsF32& layer =
998             model.layers[static_cast<std::size_t>(layer_id)];
999         const std::string prefix = "transformer.h." + std::to_string(layer_id)
↵ + ".";
1000         add_f32_tensor(tensors, prefix + "ln_1.weight", {model.config.
↵ hidden_size}, layer.ln_1_weight);
1001         add_f32_tensor(tensors, prefix + "ln_1.bias", {model.config.hidden_size
↵ }, layer.ln_1_bias);
1002         const std::vector<float> c_attn_hf = transpose_linear_to_hf_conv1d(
↵ layer.c_attn);
1003         add_f32_tensor(tensors,
1004             prefix + "attn.c_attn.weight",
1005             {model.config.hidden_size,
1006              model.config.hidden_size * 3},
1007             c_attn_hf);
1008         add_f32_tensor(tensors,
1009             prefix + "attn.c_attn.bias",
1010             {model.config.hidden_size * 3},
1011             layer.c_attn.bias);
1012         const std::vector<float> c_proj_hf = transpose_linear_to_hf_conv1d(
↵ layer.c_proj);
1013         add_f32_tensor(tensors,
1014             prefix + "attn.c_proj.weight",
1015             {model.config.hidden_size, model.config.hidden_size},
1016             c_proj_hf);
1017         add_f32_tensor(tensors,
1018             prefix + "attn.c_proj.bias",
1019             {model.config.hidden_size},
1020             layer.c_proj.bias);
1021         add_f32_tensor(tensors, prefix + "ln_2.weight", {model.config.
↵ hidden_size}, layer.ln_2_weight);
1022         add_f32_tensor(tensors, prefix + "ln_2.bias", {model.config.hidden_size
↵ }, layer.ln_2_bias);
1023         const std::vector<float> c_fc_hf = transpose_linear_to_hf_conv1d(layer.
↵ c_fc);
1024         add_f32_tensor(tensors,
1025             prefix + "mlp.c_fc.weight",
1026             {model.config.hidden_size, model.config.
↵ intermediate_size},
1027             c_fc_hf);

```

```

1028     add_f32_tensor(tensors,
1029                     prefix + "mlp.c_fc.bias",
1030                     {model.config.intermediate_size},
1031                     layer.c_fc.bias);
1032     const std::vector<float> mlp_proj_hf =
1033         transpose_linear_to_hf_conv1d(layer.mlp_c_proj);
1034     add_f32_tensor(tensors,
1035                     prefix + "mlp.c_proj.weight",
1036                     {model.config.intermediate_size, model.config.↵
↵ hidden_size},
1037                     mlp_proj_hf);
1038     add_f32_tensor(tensors,
1039                     prefix + "mlp.c_proj.bias",
1040                     {model.config.hidden_size},
1041                     layer.mlp_c_proj.bias);
1042 }
1043 add_f32_tensor(tensors,
1044                 "transformer.ln_f.weight",
1045                 {model.config.hidden_size},
1046                 model.final_ln_weight);
1047 add_f32_tensor(tensors,
1048                 "transformer.ln_f.bias",
1049                 {model.config.hidden_size},
1050                 model.final_ln_bias);
1051 write_safetensors_raw_fixture(path, tensors);
1052 }
1053
1054 void test_tiny_mlp() {
1055     const lcqi::TinyMlpModel model = lcqi::load_model(test_model_path());
1056     require(model.input_size == LCQI_TEST_INPUT_SIZE, "input size mismatch");
1057     require(model.hidden_size == LCQI_TEST_HIDDEN_SIZE, "hidden size mismatch")↵
↵ ;
1058     require(model.output_size == LCQI_TEST_OUTPUT_SIZE, "output size mismatch")↵
↵ ;
1059
1060     const std::vector<float> input{1.0F, 2.0F, -1.0F, 0.5F};
1061     const lcqi::InferenceResult result = lcqi::run_inference(model, input);
1062
1063     require(result.predicted_class == LCQI_TEST_PREDICTED_CLASS, "unexpected ↵
↵ predicted class");
1064     require(result.logits.size() == static_cast<std::size_t>(↵
↵ LCQI_TEST_OUTPUT_SIZE),
1065             "unexpected logits size");
1066     require_close(result.logits[0], LCQI_TEST_LOGIT_0, "logit 0 mismatch");
1067     require_close(result.logits[1], LCQI_TEST_LOGIT_1, "logit 1 mismatch");
1068
1069     std::vector<float> hidden(static_cast<std::size_t>(LCQI_TEST_HIDDEN_SIZE), ↵
↵ 0.0F);
1070     lcqi::linear_i8(model.hidden, input, hidden);
1071     lcqi::relu_inplace(hidden);
1072     require_close(hidden[0], LCQI_TEST_HIDDEN_0, "hidden 0 mismatch");
1073     require_close(hidden[1], LCQI_TEST_HIDDEN_1, "hidden 1 mismatch");

```

```

1074     require_close(hidden[2], LCQI_TEST_HIDDEN_2, "hidden 2 mismatch");
1075 }
1076
1077 void test_tokenizer_contract() {
1078     const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(↵
↵ test_tokenizer_path());
1079     require(tokenizer.bos_id == 0, "tokenizer BOS id mismatch");
1080     require(tokenizer.eos_id == 4, "tokenizer EOS id mismatch");
1081     require(tokenizer.unk_id == -1, "tokenizer UNK id mismatch");
1082     require(tokenizer.chat_template_hash == "tiny-chat-template-v1",
1083             "tokenizer template hash mismatch");
1084
1085     const std::vector<std::int32_t> token_ids =
1086         lcqi::encode_prompt(tokenizer, "tok1 tok2 tok3");
1087     const std::vector<std::int32_t> expected{0, 1, 2, 3, 4};
1088     require(token_ids == expected, "tokenizer prompt ids mismatch");
1089     require(lcqi::tokenizer_contract_hash(tokenizer) == ↵
↵ LCQI_TOKENIZER_HASH_EXPECTED,
1090             "tokenizer contract hash mismatch");
1091 }
1092
1093 void test_tokenizer_rejects_unknown_without_unk() {
1094     const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(↵
↵ test_tokenizer_path());
1095     bool threw = false;
1096     try {
1097         static_cast<void>(lcqi::encode_prompt(tokenizer, "tok1 missing_token"))↵
↵ ;
1098     } catch (const std::runtime_error&) {
1099         threw = true;
1100     }
1101     require(threw, "tokenizer should reject unknown token when unk id is absent↵
↵ ");
1102 }
1103
1104 void test_safetensors_manifest_contract() {
1105     const std::filesystem::path path =
1106         temp_file_path("lcqi_safetensors_manifest_contract.safetensors");
1107     const std::string header =
1108         R("{ "__metadata__": {"format": "lcqi-fixture"}, "model.embed_tokens.weight"↵
↵ ": {"dtype": "F32", "shape": [2, 3], "data_offsets": [0, 24]}, "model.layers.0."↵
↵ "attn.q_proj.weight": {"dtype": "F16", "shape": [4, 3], "data_offsets"↵
↵ ": [24, 48]}}");
1109     write_safetensors_fixture(path, header, ↵
↵ LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES);
1110
1111     const lcqi::SafeTensorManifest manifest = lcqi::load_safetensors_manifest(↵
↵ path);
1112     std::filesystem::remove(path);
1113
1114     require(manifest.header_size == static_cast<std::uint64_t>(header.size()),
1115             "safetensors header size mismatch");

```

```

1116     require(manifest.data_start_offset ==
1117             static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) +
1118             static_cast<std::uint64_t>(header.size()),
1119             "safetensors data start mismatch");
1120     require(manifest.metadata.size() == 1, "safetensors metadata count mismatch"
1121     ↪ ");
1122     require(manifest.metadata[0].first == "format" &&
1123             manifest.metadata[0].second == "lcqi-fixture",
1124             "safetensors metadata mismatch");
1125     require(manifest.tensors.size() == 2, "safetensors tensor count mismatch");
1126
1127     const lcqi::SafeTensorEntry* embedding =
1128         manifest.find_tensor("model.embed_tokens.weight");
1129     require(embedding != nullptr, "safetensors embedding tensor missing");
1130     require(embedding->dtype == "F32", "safetensors embedding dtype mismatch");
1131     require(embedding->shape == std::vector<std::int64_t>({2, 3}),
1132             "safetensors embedding shape mismatch");
1133     require(embedding->data_begin == LCQI_SAFETENSORS_EMBED_BEGIN &&
1134             embedding->data_end == LCQI_SAFETENSORS_EMBED_END,
1135             "safetensors embedding offset mismatch");
1136     require(embedding->byte_size() ==
1137             LCQI_SAFETENSORS_EMBED_END - LCQI_SAFETENSORS_EMBED_BEGIN,
1138             "safetensors embedding byte size mismatch");
1139
1140     const lcqi::SafeTensorEntry* q_proj =
1141         manifest.find_tensor("model.layers.0.attn.q_proj.weight");
1142     require(q_proj != nullptr, "safetensors q projection tensor missing");
1143     require(q_proj->dtype == "F16", "safetensors q projection dtype mismatch");
1144     require(q_proj->data_begin == LCQI_SAFETENSORS_QPROJ_BEGIN &&
1145             q_proj->data_end == LCQI_SAFETENSORS_QPROJ_END,
1146             "safetensors q projection offset mismatch");
1147 }
1148
1149 void test_safetensors_rejects_bad_offsets() {
1150     const std::filesystem::path path =
1151         temp_file_path("lcqi_safetensors_bad_offsets.safetensors");
1152     const std::string header =
1153         R("{"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,32]}}");
1154     write_safetensors_fixture(path, header, 4);
1155
1156     bool threw = false;
1157     try {
1158         static_cast<void>(lcqi::load_safetensors_manifest(path));
1159     } catch (const std::runtime_error&) {
1160         threw = true;
1161     }
1162     std::filesystem::remove(path);
1163     require(threw, "safetensors parser should reject offsets beyond payload");
1164 }
1165
1166 void test_safetensors_rejects_bad_dtype_shape_size() {
1167     const std::filesystem::path path =

```

```

1167     temp_file_path("lcqi_safetensors_bad_dtype_shape_size.safetensors");
1168     const std::string header =
1169         R("{\"tensor\":{\"dtype\":\"F32\",\"shape\":[4],\"data_offsets\":[0,8]}}");
1170     write_safetensors_fixture(path, header, 8);
1171
1172     bool threw = false;
1173     try {
1174         static_cast<void>(lcqi::load_safetensors_manifest(path));
1175     } catch (const std::runtime_error&) {
1176         threw = true;
1177     }
1178     std::filesystem::remove(path);
1179     require(threw, "safetensors parser should reject dtype/shape byte mismatch"↵
↵ );
1180 }
1181
1182 void test_safetensors_reads_float_tensors() {
1183     const std::filesystem::path path =
1184         temp_file_path("lcqi_safetensors_float_read.safetensors");
1185     std::vector<std::uint8_t> f16_bytes;
1186     append_le_u16(f16_bytes, LCQI_TEST_F16_ONE);
1187     append_le_u16(f16_bytes, LCQI_TEST_F16_NEGATIVE_TWO);
1188     std::vector<std::uint8_t> bf16_bytes;
1189     append_le_u16(bf16_bytes, LCQI_TEST_BF16_ONE_POINT_FIVE);
1190     append_le_u16(bf16_bytes, LCQI_TEST_BF16_NEGATIVE_HALF);
1191
1192     const std::array<RawTensorFixture, 3> tensors{{
1193         {"f32", "F32", {2}, f32_payload(std::array<float, 2>{1.25F, -2.5F})},
1194         {"f16", "F16", {2}, f16_bytes},
1195         {"bf16", "BF16", {2}, bf16_bytes},
1196     }};
1197     write_safetensors_raw_fixture(path, tensors);
1198
1199     const lcqi::SafeTensorFile file = lcqi::load_safetensors_file(path);
1200     std::filesystem::remove(path);
1201     const std::vector<float> f32 = lcqi::read_safetensor_f32_tensor(file, "f32"↵
↵ );
1202     const std::vector<float> f16 = lcqi::read_safetensor_f32_tensor(file, "f16"↵
↵ );
1203     const std::vector<float> bf16 = lcqi::read_safetensor_f32_tensor(file, "↵
↵ bf16");
1204
1205     require_close(f32[0], 1.25F, "safetensors F32 value 0 mismatch");
1206     require_close(f32[1], -2.5F, "safetensors F32 value 1 mismatch");
1207     require_close(f16[0], 1.0F, "safetensors F16 value 0 mismatch");
1208     require_close(f16[1], -2.0F, "safetensors F16 value 1 mismatch");
1209     require_close(bf16[0], 1.5F, "safetensors BF16 value 0 mismatch");
1210     require_close(bf16[1], -0.5F, "safetensors BF16 value 1 mismatch");
1211 }
1212
1213 void test_gguf_manifest_reads_q4_k_tensor() {
1214     const std::filesystem::path path = temp_file_path("lcqi_q4_k_fixture.gguf")↵

```

```

1215     ↵ ;
1216     write_gguf_q4_k_fixture(path);
1217
1218     const lcqi::GgufManifest manifest = lcqi::load_gguf_manifest(path);
1219     const lcqi::GgufTensorInfo* tensor =
1220         manifest.find_tensor("blk.0.ffn_up.weight");
1221     require(tensor != nullptr, "GGUF Q4_K tensor missing");
1222     const std::vector<std::uint8_t> bytes =
1223         lcqi::read_gguf_tensor_bytes(path, *tensor);
1224     std::filesystem::remove(path);
1225
1226     require(manifest.version == LCQI_GGUF_TEST_VERSION, "GGUF version mismatch" ↵
1227     ↵ );
1228     require(manifest.alignment == LCQI_GGUF_TEST_ALIGNMENT, "GGUF alignment ↵
1229     ↵ mismatch");
1230     require(manifest.metadata.size() == LCQI_GGUF_TEST_METADATA_COUNT,
1231         "GGUF metadata count mismatch");
1232     const lcqi::GgufMetadataEntry* tokens = manifest.find_metadata("tokenizer. ↵
1233     ↵ ggml.tokens");
1234     require(tokens != nullptr, "GGUF tokenizer tokens metadata missing");
1235     require(tokens->string_values == std::vector<std::string>({"<bos>", "hello" ↵
1236     ↵ })),
1237         "GGUF tokenizer string array mismatch");
1238     require(manifest.tensors.size() == LCQI_GGUF_TEST_TENSOR_COUNT,
1239         "GGUF tensor count mismatch");
1240     require(tensor->type == lcqi::GgmlType::q4_k, "GGUF tensor type mismatch");
1241     require(tensor->shape == std::vector<std::int64_t>({lcqi:: ↵
1242     ↵ LCQI_QK_K_BLOCK_VALUES, 1})),
1243         "GGUF tensor shape mismatch");
1244     require(tensor->byte_size == lcqi::LCQI_Q4_K_BLOCK_BYTES,
1245         "GGUF tensor byte size mismatch");
1246     require(bytes == make_q4_k_test_block(), "GGUF tensor payload mismatch");
1247 }
1248
1249 void test_q4_k_dequant_and_dot_paths() {
1250     const std::vector<std::uint8_t> block = make_q4_k_test_block();
1251     const std::vector<float> expected = expected_q4_k_test_values();
1252     const std::vector<float> decoded =
1253         lcqi::dequantize_q4_k(block, lcqi::LCQI_QK_K_BLOCK_VALUES);
1254
1255     require(decoded.size() == expected.size(), "Q4_K decoded size mismatch");
1256     for (std::size_t index = 0; index < expected.size(); ++index) {
1257         require_close(decoded[index], expected[index], "Q4_K decoded value ↵
1258     ↵ mismatch");
1259     }
1260
1261     std::vector<float> input(expected.size(), 0.0F);
1262     for (std::size_t index = 0; index < input.size(); ++index) {
1263         input[index] = static_cast<float>(static_cast<std::int32_t>(index % 110 ↵
1264     ↵ ) - 5) *
1265             0.125F;
1266     }

```



```

1259     float expected_dot = 0.0F;
1260     for (std::size_t index = 0; index < expected.size(); ++index) {
1261         expected_dot += expected[index] * input[index];
1262     }
1263     const float q4_f32_dot = lcqi::dot_q4_k_f32(block, input);
1264     require_close(q4_f32_dot, expected_dot, "Q4_K F32 dot mismatch");
1265
1266     const std::vector<lcqi::Q8KBlock> q8_input = lcqi::quantize_q8_k_input(↵
↵ input);
1267     const float q4_q8_dot = lcqi::dot_q4_k_q8(block, q8_input);
1268     require(std::fabs(q4_q8_dot - q4_f32_dot) <= LCQI_Q4K_Q8_DIFF_LIMIT,
1269             "Q4_K Q8 dot drift is too large");
1270
1271     std::vector<float> matvec_output(1, 0.0F);
1272     lcqi::matvec_q4_k_q8(block, 1, lcqi::LCQI_QK_K_BLOCK_VALUES, q8_input, ↵
↵ matvec_output);
1273     require_close(matvec_output[0], q4_q8_dot, "Q4_K Q8 matvec mismatch");
1274 }
1275
1276 void test_ggml_mixed_dequantizers() {
1277     std::vector<std::uint8_t> f32_bytes;
1278     append_le_f32(f32_bytes, 1.25F);
1279     append_le_f32(f32_bytes, -2.5F);
1280     const std::vector<float> f32 =
1281         lcqi::dequantize_ggml_tensor(lcqi::GgmlType::f32, f32_bytes, 2);
1282     require_close(f32[0], 1.25F, "GGML F32 decoded value 0 mismatch");
1283     require_close(f32[1], -2.5F, "GGML F32 decoded value 1 mismatch");
1284
1285     const std::vector<std::uint8_t> q8 = make_q8_0_test_block();
1286     const std::vector<float> q8_values =
1287         lcqi::dequantize_ggml_tensor(lcqi::GgmlType::q8_0,
1288                                     q8,
1289                                     lcqi::LCQI_Q8_0_BLOCK_VALUES);
1290     require_close(q8_values[0], -8.0F, "Q8_0 decoded value 0 mismatch");
1291     require_close(q8_values[16], 0.0F, "Q8_0 decoded value 16 mismatch");
1292     require_close(q8_values[31], 7.5F, "Q8_0 decoded value 31 mismatch");
1293
1294     const std::vector<std::uint8_t> q5 = make_q5_0_test_block();
1295     const std::vector<float> q5_values =
1296         lcqi::dequantize_ggml_tensor(lcqi::GgmlType::q5_0,
1297                                     q5,
1298                                     lcqi::LCQI_Q5_0_BLOCK_VALUES);
1299     require_close(q5_values[0], -8.0F, "Q5_0 decoded value 0 mismatch");
1300     require_close(q5_values[1], -7.5F, "Q5_0 decoded value 1 mismatch");
1301     require_close(q5_values[15], -0.5F, "Q5_0 decoded value 15 mismatch");
1302     require_close(q5_values[16], 0.0F, "Q5_0 decoded value 16 mismatch");
1303     require_close(q5_values[31], 7.5F, "Q5_0 decoded value 31 mismatch");
1304
1305     const std::vector<std::uint8_t> q6 = make_q6_k_test_block();
1306     const std::vector<float> q6_values =
1307         lcqi::dequantize_ggml_tensor(lcqi::GgmlType::q6_k,
1308                                     q6,

```

```

1309         lcqi::LCQI_Q6_K_BLOCK_VALUES);
1310     require_close(q6_values[LCQI_GGML_TEST_Q6_LANE], 3.0F, "Q6_K q1 mismatch");
1311     require_close(q6_values[32 + LCQI_GGML_TEST_Q6_LANE], -1.0F, "Q6_K q2 ↵
↵ mismatch");
1312     require_close(q6_values[64 + LCQI_GGML_TEST_Q6_LANE], 31.0F, "Q6_K q3 ↵
↵ mismatch");
1313     require_close(q6_values[96 + LCQI_GGML_TEST_Q6_LANE], -32.0F, "Q6_K q4 ↵
↵ mismatch");
1314     require_close(q6_values[128 + LCQI_GGML_TEST_Q6_LANE], 4.0F, "Q6_K second ↵
↵ half mismatch");
1315
1316     bool rejected_bad_size = false;
1317     try {
1318         static_cast<void>(lcqi::dequantize_ggml_tensor(
1319             lcqi::GgmlType::q8_0,
1320             std::span<const std::uint8_t>(q8.data(), q8.size() - 1U),
1321             lcqi::LCQI_Q8_0_BLOCK_VALUES));
1322     } catch (const std::runtime_error&) {
1323         rejected_bad_size = true;
1324     }
1325     require(rejected_bad_size, "GGML dequantizer accepted a truncated Q8_0 ↵
↵ block");
1326 }
1327
1328 void test_ggml_quantized_f32_matvec_paths() {
1329     const std::vector<float> input32 = [] {
1330         std::vector<float> values(static_cast<std::size_t>(lcqi::↵
↵ LCQI_Q8_0_BLOCK_VALUES), 0.0F);
1331         for (std::size_t index = 0; index < values.size(); ++index) {
1332             values[index] =
1333                 static_cast<float>(static_cast<std::int32_t>(index % 9U) - 4) *↵
↵ 0.125F;
1334         }
1335         return values;
1336     }();
1337     const std::vector<float> input256 = [] {
1338         std::vector<float> values(static_cast<std::size_t>(lcqi::↵
↵ LCQI_Q6_K_BLOCK_VALUES), 0.0F);
1339         for (std::size_t index = 0; index < values.size(); ++index) {
1340             values[index] =
1341                 static_cast<float>(static_cast<std::int32_t>(index % 13U) - 6) ↵
↵ * 0.0625F;
1342         }
1343         return values;
1344     }();
1345
1346     const auto assert_matvec_matches_dequantized =
1347         [](lcqi::GgmlType type,
1348            const std::vector<std::uint8_t>& block,
1349            std::span<const float> input,
1350            const char* message) {
1351         const std::int64_t columns = static_cast<std::int64_t>(input.size())↵

```



```

↪ );
1352     const std::vector<std::uint8_t> matrix = [&] {
1353         std::vector<std::uint8_t> bytes;
1354         bytes.reserve(block.size() * 2U);
1355         bytes.insert(bytes.end(), block.begin(), block.end());
1356         bytes.insert(bytes.end(), block.begin(), block.end());
1357         return bytes;
1358     }();
1359     std::vector<float> direct(2, 0.0F);
1360     lcqi::matvec_ggml_quantized_f32(type, matrix, 2, columns, input, ↪
↪ direct);
1361     const std::vector<lcqi::Q8_0InputBlock> q8_0_input =
1362         lcqi::quantize_q8_0_input(input);
1363     std::vector<float> q8_0_direct(2, 0.0F);
1364     lcqi::matvec_ggml_quantized_q8_0(type,
1365                                     matrix,
1366                                     2,
1367                                     columns,
1368                                     q8_0_input,
1369                                     q8_0_direct);
1370     std::vector<float> q8_0_row_range_direct(2, 0.0F);
1371     lcqi::matvec_ggml_quantized_q8_0_rows_unchecked(type,
1372                                                         matrix,
1373                                                         2,
1374                                                         columns,
1375                                                         q8_0_input,
1376                                                         ↪
↪ q8_0_row_range_direct,
1377                                                         0,
1378                                                         1);
1379     lcqi::matvec_ggml_quantized_q8_0_rows_unchecked(type,
1380                                                         matrix,
1381                                                         2,
1382                                                         columns,
1383                                                         q8_0_input,
1384                                                         ↪
↪ q8_0_row_range_direct,
1385                                                         1,
1386                                                         2);
1387     const std::vector<float> weights =
1388         lcqi::dequantize_ggml_tensor(type, matrix, columns * 2);
1389     std::array<float, 2> reference{0.0F, 0.0F};
1390     for (std::size_t row = 0; row < reference.size(); ++row) {
1391         for (std::size_t column = 0; column < input.size(); ++column) {
1392             reference[row] +=
1393                 weights[row * input.size() + column] * input[column];
1394         }
1395         if (std::fabs(reference[row] - direct[row]) > ↪
↪ LCQI_GGML_MATVEC_MAX_DIFF) {
1396             throw std::runtime_error(message);
1397         }
1398         if (std::fabs(reference[row] - q8_0_direct[row]) >

```

```

1399         LCQI_GGML_Q8_MATVEC_MAX_DIFF) {
1400             throw std::runtime_error(message);
1401         }
1402         if (std::fabs(q8_0_direct[row] - q8_0_row_range_direct[row]) >
1403             LCQI_GGML_Q8_MATVEC_MAX_DIFF) {
1404             throw std::runtime_error(message);
1405         }
1406     }
1407 };
1408
1409 require(lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::q5_0),
1410         "Q5_0 direct matvec support flag mismatch");
1411 require(lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::q6_k),
1412         "Q6_K direct matvec support flag mismatch");
1413 require(lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::q8_0),
1414         "Q8_0 direct matvec support flag mismatch");
1415 require(!lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::f32),
1416         "F32 should not be reported as quantized direct matvec");
1417 require(lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::q5_0),
1418         "Q5_0 Q8_0 direct matvec support flag mismatch");
1419 require(lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::q6_k),
1420         "Q6_K Q8_0 direct matvec support flag mismatch");
1421 require(lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::q8_0),
1422         "Q8_0 Q8_0 direct matvec support flag mismatch");
1423 require(!lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::f32),
1424         "F32 should not be reported as Q8_0 quantized direct matvec");
1425
1426 assert_matvec_matches_dequantized(lcqi::GgmlType::q8_0,
1427                                     make_q8_0_test_block(),
1428                                     input32,
1429                                     "Q8_0 direct matvec mismatch");
1430 assert_matvec_matches_dequantized(lcqi::GgmlType::q5_0,
1431                                     make_q5_0_test_block(),
1432                                     input32,
1433                                     "Q5_0 direct matvec mismatch");
1434 assert_matvec_matches_dequantized(lcqi::GgmlType::q6_k,
1435                                     make_q6_k_test_block(),
1436                                     input256,
1437                                     "Q6_K direct matvec mismatch");
1438 }
1439
1440 void test_llama_gguf_loader_and_generation() {
1441     const std::filesystem::path path = temp_file_path("lcqi_tiny_llama_fixture.↵
1442     ↪ gguf");
1443     write_tiny_llama_gguf_fixture(path);
1444
1445     const lcqi::LlamaGgufLoadedModel loaded =
1446         lcqi::load_llama_gguf_reference_model(path);
1447     const std::vector<std::int32_t> prompt{1};
1448     const lcqi::LlamaGgufGenerationResult result =
1449         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1450     std::filesystem::remove(path);

```

```

1450
1451     require(loader.model.architecture == "llama", "LLaMA GGUF architecture ↵
↵ mismatch");
1452     require(loader.model.decoder.config.hidden_size == LCQI_LLAMA_TEST_HIDDEN,
1453         "LLaMA GGUF hidden size mismatch");
1454     require(loader.model.decoder.layers.size() == LCQI_LLAMA_TEST_LAYER_COUNT,
1455         "LLaMA GGUF layer count mismatch");
1456     require(loader.model.lm_head_tied_to_embedding,
1457         "LLaMA GGUF tiny fixture should tie lm head to embeddings");
1458     require(result.generated_ids == std::vector<std::int32_t>({1, ↵
↵ LCQI_LLAMA_TEST_EXPECTED_TOKEN}),
1459         "LLaMA GGUF generated ids mismatch");
1460     require(result.predicted_first_token == LCQI_LLAMA_TEST_EXPECTED_TOKEN,
1461         "LLaMA GGUF predicted token mismatch");
1462     require(loader.report.tensors_loaded == 11, "LLaMA GGUF tensor load count ↵
↵ mismatch");
1463     require(loader.report.f32_weight_bytes == loader.report.↵
↵ quantized_weight_bytes,
1464         "LLaMA GGUF F32 fixture byte accounting mismatch");
1465     require(loader.report.q4_k_direct_tensors == 0,
1466         "LLaMA GGUF F32 fixture should not use direct Q4_K tensors");
1467     require(loader.report.ggml_direct_tensors == 0,
1468         "LLaMA GGUF F32 fixture should not use direct GGML tensors");
1469     require(result.hotspots.f32_fallback_calls == 7,
1470         "LLaMA GGUF F32 fixture linear fallback call count mismatch");
1471     require(result.hotspots.q4_k_direct_calls == 0,
1472         "LLaMA GGUF F32 fixture should not call Q4_K direct matvec");
1473     require(result.hotspots.ggml_direct_calls == 0,
1474         "LLaMA GGUF F32 fixture should not call GGML direct matvec");
1475 }
1476
1477 void test_llama_gguf_parallel_generation_matches_serial() {
1478     const std::filesystem::path path = temp_file_path("lcqi_tiny_llama_parallel↵
↵ .gguf");
1479     write_tiny_llama_gguf_fixture(path);
1480
1481     const lcqi::LlamaGgufLoadedModel loader =
1482         lcqi::load_llama_gguf_reference_model(path);
1483     const std::vector<std::int32_t> prompt{1};
1484     const lcqi::LlamaGgufGenerationResult serial =
1485         lcqi::llama_gguf_generate_greedy(loader.model, prompt, 1);
1486     lcqi::LlamaGgufExecutionOptions parallel_options;
1487     parallel_options.worker_count = 2;
1488     parallel_options.parallel_min_rows = 1;
1489     const lcqi::LlamaGgufGenerationResult parallel =
1490         lcqi::llama_gguf_generate_greedy(loader.model, prompt, 1, ↵
↵ parallel_options);
1491     std::filesystem::remove(path);
1492
1493     require(parallel.worker_count == parallel_options.worker_count,
1494         "parallel LLaMA worker count mismatch");
1495     require(parallel.generated_ids == serial.generated_ids,

```

```

1496         "parallel LLaMA generated ids mismatch");
1497     require(parallel.predicted_first_token == serial.predicted_first_token,
1498         "parallel LLaMA predicted token mismatch");
1499     require(parallel.hotspots.f32_fallback_calls == serial.hotspots.↵
↵ f32_fallback_calls,
1500         "parallel LLaMA fallback call count mismatch");
1501     require(parallel.hotspots.q4_k_direct_calls == serial.hotspots.↵
↵ q4_k_direct_calls,
1502         "parallel LLaMA Q4 direct call count mismatch");
1503     require(parallel.hotspots.ggml_direct_calls == serial.hotspots.↵
↵ ggml_direct_calls,
1504         "parallel LLaMA GGML direct call count mismatch");
1505 }
1506
1507 void test_llama_gguf_q4_direct_generation() {
1508     const std::filesystem::path path = temp_file_path("↵
↵ lcqi_tiny_llama_q4_direct.gguf");
1509     write_tiny_llama_q4_direct_gguf_fixture(path);
1510
1511     const lcqi::LlamaGgufLoadedModel loaded =
1512         lcqi::load_llama_gguf_reference_model(path);
1513     const std::vector<std::int32_t> prompt{0};
1514     const lcqi::LlamaGgufGenerationResult result =
1515         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1516     std::filesystem::remove(path);
1517
1518     require(loaded.model.decoder.config.hidden_size == ↵
↵ LCQI_LLAMA_Q4_TEST_HIDDEN,
1519         "LLaMA Q4_K fixture hidden size mismatch");
1520     require(loaded.report.tensors_loaded == 11, "LLaMA Q4_K fixture tensor load↵
↵ count mismatch");
1521     require(loaded.report.q4_k_direct_tensors == 7,
1522         "LLaMA Q4_K fixture direct tensor count mismatch");
1523     require(loaded.report.ggml_direct_tensors == 0,
1524         "LLaMA Q4_K fixture should not use GGML direct tensors");
1525     require(loaded.report.f32_fallback_tensors == 4,
1526         "LLaMA Q4_K fixture fallback tensor count mismatch");
1527     require(loaded.report.direct_quantized_weight_bytes >
1528         loaded.report.fallback_dequantized_weight_bytes,
1529         "LLaMA Q4_K fixture should keep direct quantized matrix bytes");
1530     require(result.hotspots.q4_k_direct_calls == 7,
1531         "LLaMA Q4_K fixture direct matvec call count mismatch");
1532     require(result.hotspots.ggml_direct_calls == 0,
1533         "LLaMA Q4_K fixture should not call GGML direct matvec");
1534     require(result.hotspots.f32_fallback_calls == 0,
1535         "LLaMA Q4_K fixture should not call fallback linear matvec");
1536     require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
1537         "LLaMA Q4_K fixture generated ids mismatch");
1538 }
1539
1540 void test_llama_gguf_ggml_direct_generation() {
1541     const ScopedEnvVar enable_ggml_direct(LCQI_TEST_LLAMA_GGML_DIRECT_ENV,

```

```

1542                                     LCQI_TEST_TRUE_ENV);
1543     const std::filesystem::path path = temp_file_path("↵
↵ lcqi_tiny_llama_ggml_direct.gguf");
1544     write_tiny_llama_ggml_direct_gguf_fixture(path);
1545
1546     const lcqi::LlamaGgufLoadedModel loaded =
1547         lcqi::load_llama_gguf_reference_model(path);
1548     const std::vector<std::int32_t> prompt{0};
1549     const lcqi::LlamaGgufGenerationResult result =
1550         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1551     std::filesystem::remove(path);
1552
1553     require(loaded.model.decoder.config.hidden_size == ↵
↵ LCQI_LLAMA_Q4_TEST_HIDDEN,
1554         "LLaMA GGML fixture hidden size mismatch");
1555     require(loaded.report.tensors_loaded == 11, "LLaMA GGML fixture tensor load↵
↵ count mismatch");
1556     require(loaded.report.q4_k_direct_tensors == 0,
1557         "LLaMA GGML fixture should not use Q4_K direct tensors");
1558     require(loaded.report.ggml_direct_tensors == 7,
1559         "LLaMA GGML fixture direct tensor count mismatch");
1560     require(loaded.report.f32_fallback_tensors == 4,
1561         "LLaMA GGML fixture fallback tensor count mismatch");
1562     require(result.hotspots.ggml_direct_calls == 7,
1563         "LLaMA GGML fixture direct matvec call count mismatch");
1564     require(result.hotspots.q4_k_direct_calls == 0,
1565         "LLaMA GGML fixture should not call Q4_K direct matvec");
1566     require(result.hotspots.f32_fallback_calls == 0,
1567         "LLaMA GGML fixture should not call fallback linear matvec");
1568     require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
1569         "LLaMA GGML fixture generated ids mismatch");
1570 }
1571
1572 void test_gpt2_tiny_forward_and_generation() {
1573     const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model↵
↵ ();
1574     const std::vector<std::int32_t> tokens{1, 2, 3};
1575     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, tokens↵
↵ );
1576
1577     require(result.logits.size() == LCQI_GPT2_VOCAB_SIZE, "GPT-2 logits size ↵
↵ mismatch");
1578     require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
1579         "GPT-2 predicted token mismatch");
1580     for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
1581         require_close(result.logits[index], LCQI_GPT2_LOGITS[index], "GPT-2 ↵
↵ logit mismatch");
1582     }
1583
1584     const std::vector<std::int32_t> generated =
1585         lcqi::gpt2_generate_greedy(model, tokens, 2);
1586     const std::vector<std::int32_t> expected{1, 2, 3, 3, 3};

```

```

1587     require(generated == expected, "GPT-2 greedy generation mismatch");
1588     require(static_cast<std::int32_t>(generated.size()) == ←
    ↪ LCQI_GPT2_GENERATED_LENGTH,
1589         "GPT-2 generated length mismatch");
1590
1591     lcqi::Gpt2KvCache cache(model.config);
1592     lcqi::Gpt2ForwardResult cached_result;
1593     for (const std::int32_t token : tokens) {
1594         cached_result = lcqi::run_gpt2_forward_cached(model, cache, token);
1595     }
1596     require(cache.filled_tokens() == static_cast<std::int32_t>(tokens.size()),
1597         "GPT-2 KV cache filled token count mismatch");
1598     require(cache.byte_size() > 0, "GPT-2 KV cache byte size should be non-zero↪
    ↪ ");
1599     require(cached_result.predicted_token == result.predicted_token,
1600         "cached GPT-2 predicted token mismatch");
1601     require(cached_result.logits.size() == result.logits.size(),
1602         "cached GPT-2 logits size mismatch");
1603     for (std::size_t index = 0; index < result.logits.size(); ++index) {
1604         require_close(cached_result.logits[index],
1605             result.logits[index],
1606             "cached GPT-2 logit mismatch");
1607     }
1608
1609     lcqi::Gpt2CachedGreedyDecoder decoder_with_logits(model);
1610     lcqi::Gpt2ForwardResult decoder_result;
1611     for (const std::int32_t token : tokens) {
1612         decoder_result = decoder_with_logits.step_with_logits(token);
1613     }
1614     require(decoder_with_logits.filled_tokens() == static_cast<std::int32_t>(←
    ↪ tokens.size()),
1615         "GPT-2 optimized decoder filled token count mismatch");
1616     require(decoder_with_logits.kv_cache_bytes() == cache.byte_size(),
1617         "GPT-2 optimized decoder KV byte size mismatch");
1618     require(decoder_result.predicted_token == result.predicted_token,
1619         "optimized cached GPT-2 predicted token mismatch");
1620     require(decoder_result.logits.size() == result.logits.size(),
1621         "optimized cached GPT-2 logits size mismatch");
1622     for (std::size_t index = 0; index < result.logits.size(); ++index) {
1623         require_close(decoder_result.logits[index],
1624             result.logits[index],
1625             "optimized cached GPT-2 logit mismatch");
1626     }
1627
1628     lcqi::Gpt2ExecutionOptions parallel_options;
1629     parallel_options.worker_count = 2;
1630     parallel_options.parallel_min_rows = 1;
1631     lcqi::Gpt2CachedGreedyDecoder parallel_decoder(model, nullptr, ←
    ↪ parallel_options);
1632     lcqi::Gpt2ForwardResult parallel_result;
1633     for (const std::int32_t token : tokens) {
1634         parallel_result = parallel_decoder.step_with_logits(token);

```



```

1635     }
1636     require(parallel_decoder.worker_count() == parallel_options.worker_count,
1637             "parallel GPT-2 decoder worker count mismatch");
1638     require(parallel_result.predicted_token == result.predicted_token,
1639             "parallel cached GPT-2 predicted token mismatch");
1640     require(parallel_result.logits.size() == result.logits.size(),
1641             "parallel cached GPT-2 logits size mismatch");
1642     for (std::size_t index = 0; index < result.logits.size(); ++index) {
1643         require_close(parallel_result.logits[index],
1644                       result.logits[index],
1645                       "parallel cached GPT-2 logit mismatch");
1646     }
1647
1648     lcqi::Gpt2CachedGreedyDecoder greedy_decoder(model);
1649     std::int32_t greedy_predicted = 0;
1650     for (const std::int32_t token : tokens) {
1651         greedy_predicted = greedy_decoder.step(token);
1652     }
1653     require(greedy_predicted == result.predicted_token,
1654             "optimized logits-free GPT-2 predicted token mismatch");
1655
1656     lcqi::Gpt2CachedGreedyDecoder prefill_decoder(model);
1657     for (std::size_t index = 0; index + 1 < tokens.size(); ++index) {
1658         prefill_decoder.advance_without_prediction(tokens[index]);
1659     }
1660     require(prefill_decoder.filled_tokens() ==
1661             static_cast<std::int32_t>(tokens.size() - 1U),
1662             "GPT-2 prefill-only decoder filled token count mismatch");
1663     const std::int32_t prefill_predicted = prefill_decoder.step(tokens.back());
1664     require(prefill_decoder.filled_tokens() == static_cast<std::int32_t>(tokens.size()),
1665             "GPT-2 prefill decoder final filled token count mismatch");
1666     require(prefill_predicted == result.predicted_token,
1667             "GPT-2 prefill-skipped predicted token mismatch");
1668
1669     lcqi::Gpt2HotspotProfile hotspot_profile;
1670     lcqi::Gpt2CachedGreedyDecoder profiled_decoder(model, &hotspot_profile);
1671     std::int32_t profiled_predicted = 0;
1672     for (const std::int32_t token : tokens) {
1673         profiled_predicted = profiled_decoder.step(token);
1674     }
1675     require(profiled_predicted == result.predicted_token,
1676             "profiled GPT-2 predicted token mismatch");
1677     require(hotspot_profile.decoder_steps == static_cast<std::int64_t>(tokens.size()),
1678             "profiled GPT-2 decoder step count mismatch");
1679     require(hotspot_profile.layer_steps ==
1680             static_cast<std::int64_t>(tokens.size()) * model.config.layer_count,
1681             "profiled GPT-2 layer step count mismatch");
1682     require(hotspot_profile.total_step_ms >= hotspot_profile.lm_head_ms,
1683             "profiled GPT-2 total time should include lm head time");

```

```

1684
1685     const std::vector<std::int32_t> cached_generated =
1686         lcqi::gpt2_generate_greedy_cached(model, tokens, 2);
1687     require(cached_generated == expected, "cached GPT-2 greedy generation ↵
↵ mismatch");
1688 }
1689
1690 void test_gpt2_byte_bpe_tokenizer() {
1691     const std::filesystem::path vocab_path = temp_file_path("lcqi_gpt2_vocab.↵
↵ json");
1692     const std::filesystem::path merges_path = temp_file_path("lcqi_gpt2_merges.↵
↵ txt");
1693     const std::string space_marker = "\xC4\xA0";
1694     const std::string vocab =
1695         "{ \"Hello\":7, \" \":8, \"\" + space_marker + "world\":9, \"!\":10}";
1696     const std::string merges =
1697         "#version: 0.2\n"
1698         "H e\n"
1699         "He l\n"
1700         "Hel l\n"
1701         "Hell o\n" +
1702         space_marker + " w\n" +
1703         space_marker + "w o\n" +
1704         space_marker + "wo r\n" +
1705         space_marker + "wor l\n" +
1706         space_marker + "worl d\n";
1707     write_text_file(vocab_path, vocab);
1708     write_text_file(merges_path, merges);
1709
1710     const lcqi::Gpt2Tokenizer tokenizer =
1711         lcqi::load_gpt2_tokenizer(vocab_path, merges_path, -1, -1);
1712     std::filesystem::remove(vocab_path);
1713     std::filesystem::remove(merges_path);
1714
1715     const std::vector<std::int32_t> ids =
1716         lcqi::gpt2_encode(tokenizer, "Hello, world!");
1717     const std::vector<std::int32_t> expected{
1718         LCQI_GPT2_TOKENIZER_HELLO_ID,
1719         8,
1720         9,
1721         10,
1722     };
1723     require(ids == expected, "GPT-2 BPE ids mismatch");
1724     require(lcqi::gpt2_decode(tokenizer, ids) == "Hello, world!",
1725         "GPT-2 BPE decode mismatch");
1726 }
1727
1728 void test_gpt2_loads_hf_style_tiny_checkpoint() {
1729     const std::filesystem::path directory =
1730         temp_file_path("lcqi_gpt2_tiny_checkpoint_dir");
1731     std::filesystem::remove_all(directory);
1732     std::filesystem::create_directories(directory);

```



```

1733
1734     const lcqi::Gpt2ReferenceModel original = lcqi::↵
↵ make_tiny_gpt2_reference_model();
1735     write_text_file(
1736         directory / "config.json",
1737         R("{\"model_type\":\"gpt2\",\"n_embd\":4,\"n_head\":2,\"n_layer\":1,\"n_positions\"↵
↵ :8,\"n_ctx\":8,\"vocab_size\":6,\"n_inner\":8,\"layer_norm_epsilon\":0.00001,\"↵
↵ activation_function\":\"gelu_new\",\"bos_token_id\":0,\"eos_token_id\":5})");
1738     write_tiny_gpt2_safetensors(directory / "model.safetensors", original);
1739
1740     const lcqi::Gpt2ReferenceModel loaded = lcqi::load_gpt2_from_directory(↵
↵ directory);
1741     const std::vector<std::int32_t> tokens{1, 2, 3};
1742     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(loaded, ↵
↵ tokens);
1743     std::filesystem::remove_all(directory);
1744
1745     require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
1746         "loaded GPT-2 predicted token mismatch");
1747     for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
1748         require_close(result.logits[index],
1749             LCQI_GPT2_LOGITS[index],
1750             "loaded GPT-2 logit mismatch");
1751     }
1752 }
1753
1754 void test_gpt2_loader_rejects_bad_shape() {
1755     const std::filesystem::path directory =
1756         temp_file_path("lcqi_gpt2_bad_shape_dir");
1757     std::filesystem::remove_all(directory);
1758     std::filesystem::create_directories(directory);
1759
1760     const lcqi::Gpt2ReferenceModel original = lcqi::↵
↵ make_tiny_gpt2_reference_model();
1761     write_text_file(
1762         directory / "config.json",
1763         R("{\"model_type\":\"gpt2\",\"n_embd\":5,\"n_head\":1,\"n_layer\":1,\"n_positions\"↵
↵ :8,\"n_ctx\":8,\"vocab_size\":6,\"n_inner\":8,\"layer_norm_epsilon\":0.00001,\"↵
↵ activation_function\":\"gelu_new\",\"bos_token_id\":0,\"eos_token_id\":5})");
1764     write_tiny_gpt2_safetensors(directory / "model.safetensors", original);
1765
1766     bool threw = false;
1767     try {
1768         static_cast<void>(lcqi::load_gpt2_from_directory(directory));
1769     } catch (const std::runtime_error&) {
1770         threw = true;
1771     }
1772     std::filesystem::remove_all(directory);
1773     require(threw, "GPT-2 loader should reject bad tensor shape");
1774 }
1775
1776 void test_reference_rms_norm() {

```

```

1777     const std::vector<float> input{3.0F, 4.0F};
1778     const std::vector<float> weight{1.0F, 0.5F};
1779     std::vector<float> output(2, 0.0F);
1780     lcqi::rms_norm(input, weight, 0.0F, output);
1781     require_close(output[0], LCQI_RMS_EXPECTED_0, "RMSNorm output 0 mismatch");
1782     require_close(output[1], LCQI_RMS_EXPECTED_1, "RMSNorm output 1 mismatch");
1783 }
1784
1785 void test_reference_rope() {
1786     std::vector<float> heads{1.0F, 0.0F, 0.0F, 1.0F};
1787     lcqi::apply_rope(heads, 1, 4, 1, 10000.0F);
1788     require_close(heads[0], std::cos(1.0F), "RoPE pair 0 cosine mismatch");
1789     require_close(heads[1], std::sin(1.0F), "RoPE pair 0 sine mismatch");
1790     require_close(heads[2], -std::sin(0.01F), "RoPE pair 1 negative sine ←
    ↪ mismatch");
1791     require_close(heads[3], std::cos(0.01F), "RoPE pair 1 cosine mismatch");
1792 }
1793
1794 void test_reference_kv_attention() {
1795     lcqi::DecoderConfig config;
1796     config.hidden_size = 4;
1797     config.query_heads = 2;
1798     config.kv_heads = 1;
1799     config.head_dim = 2;
1800     config.intermediate_size = 4;
1801     config.vocab_size = 4;
1802     config.max_context = 4;
1803
1804     lcqi::ReferenceKVCache cache(config, 1);
1805     const std::vector<float> key{1.0F, 0.0F};
1806     const std::vector<float> value{LCQI_ATTENTION_EXPECTED_0, ←
    ↪ LCQI_ATTENTION_EXPECTED_1};
1807     cache.append(0, 0, key, value);
1808     require(cache.filled_tokens() == 1, "KV filled token count mismatch");
1809     require_close(cache.key(0, 0, 0)[0], 1.0F, "KV key read mismatch");
1810
1811     const std::vector<float> query{1.0F, 0.0F, 0.0F, 1.0F};
1812     std::vector<float> output(4, 0.0F);
1813     lcqi::attention_decode(config, cache, 0, 0, query, output);
1814     require_close(output[0], LCQI_ATTENTION_EXPECTED_0, "attention head 0 dim 0←
    ↪ mismatch");
1815     require_close(output[1], LCQI_ATTENTION_EXPECTED_1, "attention head 0 dim 1←
    ↪ mismatch");
1816     require_close(output[2], LCQI_ATTENTION_EXPECTED_0, "attention head 1 dim 0←
    ↪ mismatch");
1817     require_close(output[3], LCQI_ATTENTION_EXPECTED_1, "attention head 1 dim 1←
    ↪ mismatch");
1818 }
1819
1820 void test_reference_decoder_end_to_end() {
1821     const lcqi::ReferenceDecoderModel model = lcqi::←
    ↪ make_tiny_reference_decoder_model();

```

```

1822     const std::vector<std::int32_t> tokens{1, 2, 3};
1823     const lcqi::ReferenceDecodeResult result = lcqi::run_reference_decode(model,
1824     ↪ , tokens);
1824     require(result.logits.size() == LCQI_REFERENCE_DECODER_VOCAB_SIZE,
1825     ↪ "reference decoder logits size mismatch");
1826     require(result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
1827     ↪ "reference decoder predicted token mismatch");
1828     for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ↪
1829     ↪ ++index) {
1829         require_close(result.logits[index],
1830         ↪ LCQI_REFERENCE_DECODER_LOGITS[index],
1831         ↪ "reference decoder golden logit mismatch");
1832     }
1833 }
1834
1835 void test_reference_decoder_trace_contract() {
1836     const lcqi::ReferenceDecoderModel model = lcqi::
1837     ↪ make_tiny_reference_decoder_model();
1838     const std::vector<std::int32_t> tokens{1, 2, 3};
1839     const lcqi::ReferenceDecodeTraceResult trace =
1840     ↪ lcqi::run_reference_decode_with_trace(model, tokens);
1841
1842     require(trace.result.predicted_token == ↪
1843     ↪ LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
1844     ↪ "reference trace predicted token mismatch");
1845     require(!trace.checkpoints.empty(), "reference trace checkpoints missing");
1846
1847     const lcqi::ReferenceTensorCheckpoint& first = trace.checkpoints.front();
1848     require(first.name == "embedding", "first checkpoint should be embedding");
1849     require(first.token_position == LCQI_REFERENCE_TRACE_FIRST_TOKEN,
1850     ↪ "first checkpoint token position mismatch");
1851     require(first.dtype == "f32", "first checkpoint dtype mismatch");
1852     require(first.layout == "row_major", "first checkpoint layout mismatch");
1853     require(first.shape.size() == 1 &&
1854     ↪ first.shape[0] == LCQI_REFERENCE_TRACE_HIDDEN_SIZE,
1855     ↪ "first checkpoint shape mismatch");
1856     require(first.stride.size() == 1 && first.stride[0] == 1,
1857     ↪ "first checkpoint stride mismatch");
1858
1859     bool saw_logits = false;
1860     bool saw_kv_slot = false;
1861     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints) ↪
1862     ↪ {
1863         if (checkpoint.name == "kv_cache_key_slot") {
1864             saw_kv_slot = true;
1865             require(checkpoint.shape.size() == 4, "KV checkpoint rank mismatch" ↪
1866             ↪ );
1867             require(checkpoint.layout == "kv_contiguous", "KV checkpoint layout ↪
1868             ↪ mismatch");
1869         }
1870         if (checkpoint.name == "logits") {
1871             saw_logits = true;

```

```

1867         require(checkpoint.token_position == ↵
↵ LCQI_REFERENCE_TRACE_TOKEN_COUNT - 1,
1868             "logits checkpoint token position mismatch");
1869         require(checkpoint.shape.size() == 1 &&
1870             checkpoint.shape[0] ==
1871                 static_cast<std::int32_t>(↵
↵ LCQI_REFERENCE_DECODER_VOCAB_SIZE),
1872             "logits checkpoint shape mismatch");
1873         require(checkpoint.values.size() == LCQI_REFERENCE_DECODER_LOGITS.↵
↵ size(),
1874             "logits checkpoint value size mismatch");
1875         for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.↵
↵ size(); ++index) {
1876             require_close(checkpoint.values[index],
1877                 LCQI_REFERENCE_DECODER_LOGITS[index],
1878                 "trace logits checkpoint mismatch");
1879         }
1880     }
1881 }
1882 require(saw_kv_slot, "KV checkpoint missing");
1883 require(saw_logits, "logits checkpoint missing");
1884 }
1885
1886 void test_packed_i8_kernel_matches_scalar() {
1887     for (const KernelShapeCase& shape_case : LCQI_KERNEL_SHAPE_CASES) {
1888         const lcqi::QuantizedLinearLayer layer =
1889             lcqi::make_deterministic_i8_layer(shape_case.input_size,
1890                 shape_case.output_size,
1891                 0.03125F);
1892         const lcqi::PackedLinearI8 packed =
1893             lcqi::pack_linear_i8(layer, shape_case.output_block_size);
1894         const std::vector<float> input = lcqi::make_deterministic_input(layer.↵
↵ input_size);
1895         std::vector<float> scalar_output(static_cast<std::size_t>(layer.↵
↵ output_size), 0.0F);
1896         std::vector<float> packed_output(static_cast<std::size_t>(layer.↵
↵ output_size), 0.0F);
1897         lcqi::linear_i8(layer, input, scalar_output);
1898         lcqi::linear_i8_packed(packed, input, packed_output);
1899         require(lcqi::max_abs_diff(scalar_output, packed_output) <= ↵
↵ LCQI_KERNEL_MAX_DIFF,
1900             "packed int8 kernel output differs from scalar");
1901
1902         const lcqi::PackedLinearI8 simd_packed =
1903             lcqi::pack_linear_i8(layer, LCQI_SIMD_TEST_BLOCK);
1904         if (lcqi::linear_i8_packed_avx2_available()) {
1905             std::vector<float> avx2_output(static_cast<std::size_t>(layer.↵
↵ output_size), 0.0F);
1906             lcqi::linear_i8_packed_avx2(simd_packed, input, avx2_output);
1907             require(lcqi::max_abs_diff(scalar_output, avx2_output) <= ↵
↵ LCQI_KERNEL_MAX_DIFF,
1908                 "AVX2 int8 kernel output differs from scalar");

```

```

1909     }
1910 }
1911 }
1912
1913 void test_f32_dot_kernel_matches_scalar() {
1914     std::vector<float> lhs(LCQI_F32_DOT_TEST_SIZE, 0.0F);
1915     std::vector<float> rhs(LCQI_F32_DOT_TEST_SIZE, 0.0F);
1916     for (std::size_t index = 0; index < LCQI_F32_DOT_TEST_SIZE; ++index) {
1917         lhs[index] = static_cast<float>(static_cast<std::int32_t>(index % 19U) ←
↵ + 1) *
1918             0.03125F;
1919         rhs[index] = static_cast<float>(static_cast<std::int32_t>(index % 23U) ←
↵ - 11) *
1920             0.0625F;
1921     }
1922
1923     const float scalar =
1924         lcqi::dot_f32_scalar_unchecked(lhs.data(), rhs.data(), lhs.size());
1925     const float dispatched = lcqi::dot_f32(lhs, rhs);
1926     require(std::fabs(scalar - dispatched) <= LCQI_F32_KERNEL_MAX_DIFF,
1927         "F32 dot kernel output differs from scalar");
1928
1929     if (lcqi::dot_f32_avx2_available()) {
1930         const float avx2 =
1931             lcqi::dot_f32_avx2_unchecked(lhs.data(), rhs.data(), lhs.size());
1932         require(std::fabs(scalar - avx2) <= LCQI_F32_KERNEL_MAX_DIFF,
1933             "AVX2 F32 dot kernel output differs from scalar");
1934     }
1935
1936     bool rejected_mismatch = false;
1937     try {
1938         (void)lcqi::dot_f32(std::span<const float>(lhs.data(), lhs.size() - 1U) ←
↵ , rhs);
1939     } catch (const std::runtime_error&) {
1940         rejected_mismatch = true;
1941     }
1942     require(rejected_mismatch, "F32 dot kernel accepted mismatched sizes");
1943 }
1944
1945 void test_f32_row_kernels_match_scalar() {
1946     std::vector<float> weights(LCQI_F32_ROW_TEST_INPUT_SIZE *
1947         LCQI_F32_ROW_TEST_OUTPUT_SIZE,
1948         0.0F);
1949     std::vector<float> input(LCQI_F32_ROW_TEST_INPUT_SIZE, 0.0F);
1950     std::vector<float> bias(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
1951     for (std::size_t index = 0; index < weights.size(); ++index) {
1952         weights[index] = static_cast<float>(static_cast<std::int32_t>(index % ←
↵ 29U) - 14) *
1953             0.015625F;
1954     }
1955     for (std::size_t index = 0; index < input.size(); ++index) {
1956         input[index] = static_cast<float>(static_cast<std::int32_t>(index % 13U) ←

```

```

↪ ) - 6) *
1957         0.03125F;
1958     }
1959     for (std::size_t index = 0; index < bias.size(); ++index) {
1960         bias[index] = static_cast<float>(static_cast<std::int32_t>(index % 7U) ↪
↪ - 3) *
1961         0.125F;
1962     }
1963
1964     std::vector<float> scalar_output(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
1965     std::vector<float> dispatched_output(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
1966     lcqi::linear_f32_rows_scalar_unchecked(weights.data(),
1967                                           input.data(),
1968                                           bias.data(),
1969                                           input.size(),
1970                                           0,
1971                                           LCQI_F32_ROW_TEST_OUTPUT_SIZE,
1972                                           scalar_output.data());
1973     lcqi::linear_f32_rows_unchecked(weights.data(),
1974                                     input.data(),
1975                                     bias.data(),
1976                                     input.size(),
1977                                     0,
1978                                     LCQI_F32_ROW_TEST_OUTPUT_SIZE,
1979                                     dispatched_output.data());
1980     for (std::size_t index = 0; index < scalar_output.size(); ++index) {
1981         require(std::fabs(scalar_output[index] - dispatched_output[index]) <=
1982                 LCQI_F32_KERNEL_MAX_DIFF,
1983                 "F32 linear row kernel output differs from scalar");
1984     }
1985
1986     const lcqi::F32RowMax scalar_max =
1987         lcqi::max_dot_f32_rows_scalar_unchecked(weights.data(),
1988                                                  input.data(),
1989                                                  input.size(),
1990                                                  0,
1991                                                  LCQI_F32_ROW_TEST_OUTPUT_SIZE);
1992     const lcqi::F32RowMax dispatched_max =
1993         lcqi::max_dot_f32_rows_unchecked(weights.data(),
1994                                          input.data(),
1995                                          input.size(),
1996                                          0,
1997                                          LCQI_F32_ROW_TEST_OUTPUT_SIZE);
1998     require(scalar_max.row == dispatched_max.row,
1999             "F32 row max kernel selected a different row");
2000     require(std::fabs(scalar_max.value - dispatched_max.value) <= ↪
↪ LCQI_F32_KERNEL_MAX_DIFF,
2001             "F32 row max kernel value differs from scalar");
2002
2003     if (lcqi::dot_f32_avx2_available()) {
2004         std::vector<float> avx2_output(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
2005         lcqi::linear_f32_rows_avx2_unchecked(weights.data(),

```

```

2006         input.data(),
2007         bias.data(),
2008         input.size(),
2009         0,
2010         LCQI_F32_ROW_TEST_OUTPUT_SIZE,
2011         avx2_output.data());
2012     for (std::size_t index = 0; index < scalar_output.size(); ++index) {
2013         require(std::fabs(scalar_output[index] - avx2_output[index]) <=
2014             LCQI_F32_KERNEL_MAX_DIFF,
2015             "AVX2 F32 linear row kernel output differs from scalar");
2016     }
2017
2018     std::vector<float> scalar_subrange(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F)↵
↵ ;
2019     std::vector<float> avx2_subrange(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
2020     constexpr std::size_t LCQI_F32_ROW_TEST_SUBRANGE_BEGIN = 1;
2021     constexpr std::size_t LCQI_F32_ROW_TEST_SUBRANGE_END = 7;
2022     lcqi::linear_f32_rows_scalar_unchecked(weights.data(),
2023         input.data(),
2024         bias.data(),
2025         input.size(),
2026         LCQI_F32_ROW_TEST_SUBRANGE_BEGIN↵
↵ ,
2027         LCQI_F32_ROW_TEST_SUBRANGE_END,
2028         scalar_subrange.data());
2029     lcqi::linear_f32_rows_avx2_unchecked(weights.data(),
2030         input.data(),
2031         bias.data(),
2032         input.size(),
2033         LCQI_F32_ROW_TEST_SUBRANGE_BEGIN,
2034         LCQI_F32_ROW_TEST_SUBRANGE_END,
2035         avx2_subrange.data());
2036     for (std::size_t index = LCQI_F32_ROW_TEST_SUBRANGE_BEGIN;
2037         index < LCQI_F32_ROW_TEST_SUBRANGE_END;
2038         ++index) {
2039         require(std::fabs(scalar_subrange[index] - avx2_subrange[index]) <=
2040             LCQI_F32_KERNEL_MAX_DIFF,
2041             "AVX2 F32 subrange row kernel output differs from scalar");
2042     }
2043
2044     const lcqi::F32RowMax avx2_max =
2045         lcqi::max_dot_f32_rows_avx2_unchecked(weights.data(),
2046             input.data(),
2047             input.size(),
2048             0,
2049             LCQI_F32_ROW_TEST_OUTPUT_SIZE↵
↵ );
2050     require(scalar_max.row == avx2_max.row,
2051         "AVX2 F32 row max kernel selected a different row");
2052     require(std::fabs(scalar_max.value - avx2_max.value) <= ↵
↵ LCQI_F32_KERNEL_MAX_DIFF,
2053         "AVX2 F32 row max kernel value differs from scalar");

```



```

2054     }
2055 }
2056
2057 } // namespace
2058
2059 int main() {
2060     try {
2061         test_tiny_mlp();
2062         test_tokenizer_contract();
2063         test_tokenizer_rejects_unknown_without_unk();
2064         test_safetensors_manifest_contract();
2065         test_safetensors_rejects_bad_offsets();
2066         test_safetensors_rejects_bad_dtype_shape_size();
2067         test_safetensors_reads_float_tensors();
2068         test_gguf_manifest_reads_q4_k_tensor();
2069         test_q4_k_dequant_and_dot_paths();
2070         test_ggml_mixed_dequantizers();
2071         test_ggml_quantized_f32_matvec_paths();
2072         test_llama_gguf_loader_and_generation();
2073         test_llama_gguf_parallel_generation_matches_serial();
2074         test_llama_gguf_q4_direct_generation();
2075         test_llama_gguf_ggml_direct_generation();
2076         test_gpt2_tiny_forward_and_generation();
2077         test_gpt2_byte_bpe_tokenizer();
2078         test_gpt2_loads_hf_style_tiny_checkpoint();
2079         test_gpt2_loader_rejects_bad_shape();
2080         test_reference_rms_norm();
2081         test_reference_rope();
2082         test_reference_kv_attention();
2083         test_reference_decoder_end_to_end();
2084         test_reference_decoder_trace_contract();
2085         test_packed_i8_kernel_matches_scalar();
2086         test_f32_dot_kernel_matches_scalar();
2087         test_f32_row_kernels_match_scalar();
2088
2089         std::cout << "lcqi tests passed\n";
2090         return EXIT_SUCCESS;
2091     } catch (const std::exception& error) {
2092         std::cerr << "lcqi test failure: " << error.what() << "\n";
2093         return EXIT_FAILURE;
2094     }
2095 }

```


读码检查清单

构建入口 是否能从 CMake 文件看出每个可执行目标依赖哪些源码，哪些目标属于默认 CTest，哪些目标依赖外部环境。

资产边界 模型文件、tokenizer、safetensors header 和外部下载模型是否都有校验或拒绝路径。

公共合同 头文件是否把 shape、dtype、token、trace checkpoint、KV cache 地址和返回字段说清楚。

正确性路径 tiny MLP、reference decoder 和 GPT-2 tiny fixture 是否有可复查 golden。

性能口径 benchmark 是否同时报告 backend、available、耗时、误差和 checksum，避免只有速度没有正确性。

真实模型证据 GPT-2 smoke 和 SmolLM2 smoke 是否记录模型文件 hash、命令、输出和验证结论。

回归门禁 CTest 是否覆盖坏资产、坏 shape、tokenizer 合同、trace 关键字段、ledger 固定公式和 serving probe 拒绝路径。

如果某段新代码不能回答上面的检查项，它就还不能进入第三册主线工程。源码卷的作用是把这条标准显式化：代码不只是能编译，还要能解释、能测量、能拒绝坏输入、能被长期回归。